

# Claude Code 2026 — 에이전트 페어 프로그래밍 실전

---

## 목차

---

### 1장. 첫걸음: Claude Code 설치와 첫 세션 [입문]

- 도입: 자동완성을 넘어선 에이전트형 코딩
- 정의: Claude Code란 무엇인가(터미널·IDE·데스크톱·웹)
- 동작 원리: 관찰→계획→도구 호출→결과 확인 루프
- 설치와 인증(npm 전역·네이티브)
- worked example: 빈 폴더에서 첫 파일 생성·실행
- 대화형 REPL과 1회성 실행(-p/print 모드)
- worked example: 기존 저장소를 열어 요약시키기
- 세션 관리: 중단·재개(--continue/--resume)·종료
- 즉시 연습(2종 1세트): 첫 세션 시나리오 채우기
- 흔한 오류와 오개념: 자동완성 오해·붙여넣기 습관
- 자주 묻는 질문(FAQ)

### 2장. 에이전트 다루기: 효과적 프롬프팅과 작업 지시 [입문]

- 도입: 결과는 지시의 명확성에 달렸다(선수 상기)
- 좋은 지시의 네 요소(목표·맥락·제약·완료 조건)
- 동작 원리: 에이전트 루프와 잘못된 가정
- worked example: 모호한 지시 vs 구체 지시 비교
- 작업 분해와 단계적 지시(한 번에 하나)
- @파일 멘션·참조로 맥락 주입하기
- worked example: 3단계(테스트→구현→리팩터) 순차 지시
- 반복 교정(iterate): 검토→피드백→재시도
- 즉시 연습(2종 1세트): 지시문 4요소 채우기
- 흔한 오류와 오개념: 과대 요청·모호한 형용사·검증 누락
- 단원 미니 프로젝트: 첫 세션으로 미니 도구 만들기(2종 1세트)

### 3장. 핵심 작업 루프: 파일 탐색과 외과적 편집 [입문]

- 도입: 탐색→편집→확인의 일상 루프
- 코드베이스 탐색: 검색(Glob·Grep)·읽기(Read)
- 동작 원리: 외과적 편집(Edit) vs 신규 작성(Write)
- worked example: 함수 이름을 전체에서 일관되게 바꾸기

- diff 검토와 변경 수용·거부
- worked example: 버그 한 줄을 외과적으로 고치고 실행 확인
- 다중 파일 변경과 일관성·되돌리기
- 작은 단위 편집 원칙과 결과 실행·확인
- 즉시 연습(2종 1세트): 탐색→편집 시나리오 채우기
- 흔한 오류와 오개념: 전체 재작성·일괄 변경·무검토 수용

#### 4장. 통제: 권한과 도구 허용목록(allowlist) [기초]

- 도입: 편집·실행에는 권한이 따른다(선수 상기)
- 부수효과 도구와 읽기 전용 도구
- 동작 원리: allow·deny·ask 판정 흐름
- worked example: 최소 권한 settings.json 작성
- 도구·명령 패턴 매칭(Bash(git:\*) 등)
- 권한 범위: 프로젝트·사용자·로컬
- worked example: 위험 명령 deny로 사고 예방
- 자동 승인(권한 위임)과 그 위험
- 즉시 연습(2종 1세트): allow·deny 규칙 채우기
- 흔한 오류와 오개념: 전체 bypass·이름만 허용·로컬 오남용
- 단원 미니 프로젝트: 안전한 리팩터링 세션 구성(2종 1세트)

#### 5장. 프로젝트 헌법: CLAUDE.md와 컨텍스트 관리 [기초]

- 도입: 매번 설명하지 않으려면 영속 컨텍스트가 필요하다
- CLAUDE.md의 역할: 세션마다 읽히는 헌법
- 동작 원리: 컨텍스트 계층(사용자·프로젝트·하위)
- worked example: 빌드·테스트 명령과 컨벤션 담은 CLAUDE.md
- 무엇을 담는가: 명령·컨벤션·아키텍처·금지사항
- 메모리 모듈화: import·@참조
- worked example: 사용자/프로젝트 규칙 분리 배치
- 컨텍스트 원도와 압축(/compact)·초기화(/clear)
- 즉시 연습(2종 1세트): CLAUDE.md 골격 채우기
- 흔한 오류와 오개념: 방치된 낡은 규칙·과적·무한 컨텍스트 가정

#### 6장. 먼저 계획하기: 플랜 모드 [기초]

- 도입: 바꾸기 전에 계획을 먼저 본다(선수 상기)
- 플랜 모드란: 변경 전 읽기 전용 계획 단계
- 동작 원리: 계획→승인→실행 분리 흐름
- worked example: 다중 파일 리팩터링 계획받아 실행
- 계획 교정: 단계 추가·삭제·재정렬
- 언제 쓰는가: 복잡·고위험·다중 파일 변경
- worked example: 잘못된 가정을 교정 지시로 바로잡기
- 플랜 모드와 CLAUDE.md·서브에이전트 연계
- 즉시 연습(2종 1세트): 계획 검토·교정 채우기
- 흔한 오류와 오개념: 단순작업 과용·무비판 실행·읽기전용 오해

- 단원 미니 프로젝트: 프로젝트 헌법과 구현 계획 세우기(2종 1세트)

## 7장. 버전관리: Git과 커밋 통합 [기초]

- 도입: 버전관리를 워크플로우의 일부로
- git 상태·diff 확인과 변경 맥락 파악
- 동작 원리: 스테이징→커밋 단위 통제
- worked example: 변경을 feat: 메시지로 커밋
- Conventional Commits 메시지 작성 위임
- 브랜치 생성·전환과 PR 초안(gh 연계)
- worked example: 변경을 두 논리 커밋으로 분리
- 커밋 관련 권한(Bash(git commit:\*)) 설계
- 즉시 연습(2종 1세트): 커밋 단위·메시지 채우기
- 흔한 오류와 오개념: 통제 없는 커밋·몰아 커밋·무엇만 담은 메시지

## 8장. 반복 표준화: 슬래시 커맨드 [중급]

- 도입: 자주 쓰는 절차를 /이름으로 부르기
- 내장 커맨드 개요(/clear-/compact-/review-/help)
- 동작 원리: 커맨드는 맥락을 주입하는 프롬프트 템플릿
- worked example: /test 커스텀 커맨드 만들기
- 인자(\$ARGUMENTS)와 파일·맥락 참조
- 프로젝트 커맨드와 사용자 커맨드의 범위
- worked example: 인자 받는 /review-module 커맨드
- 커맨드·스킬·서브에이전트의 관계(2026 통합)
- 즉시 연습(2종 1세트): 커스텀 커맨드 본문 채우기
- 흔한 오류와 오개념: 매크로 오해·텍스트만의 한계·범위 혼동
- 단원 미니 프로젝트: 커밋 워크플로우와 /커맨드 표준화(2종 1세트)

## 9장. 위임: 서브에이전트로 일 나누기 [중급]

- 도입: 한정된 일을 전문가에게 맡기기(선수 상기)
- 서브에이전트란: 독립 컨텍스트·프롬프트·도구
- 동작 원리: 컨텍스트 격리와 병렬 실행
- worked example: reviewer 서브에이전트 정의·호출
- 정의 파일: .claude/agents/이름.md(역할·tools·model)
- 위임에 적합한 작업(리뷰·테스트·조사·QA)
- worked example: 병렬 조사 서브에이전트로 동시 수행
- 메인 에이전트의 계획·통합 책임
- 즉시 연습(2종 1세트): 서브에이전트 정의 채우기
- 흔한 오류와 오개념: 맥락 공유 가정·과잉 위임·넓은 권한

## 10장. 연결: MCP 서버로 외부 도구·데이터 연동 [중급]

- 도입: 에이전트를 바깥 세상과 잇기
- MCP란: 외부 통합을 위한 표준 프로토콜

- 동작 원리: 서버·도구·리소스·프롬프트 구조
- worked example: 파일시스템/문서 MCP 등록·참조
- 서버 등록: .mcp.json과 범위(로컬·프로젝트·사용자)
- 대표 MCP(파일시스템·GitHub·DB·브라우저)
- worked example: GitHub MCP로 이슈·PR 다루기
- MCP 도구의 권한 통제와 비밀 관리
- 즉시 연습(2종 1세트): .mcp.json 등록 채우기
- 흔한 오류와 오개념: 플러그인 난립 오해·권한 예외 오해·토큰 노출
- 단원 미니 프로젝트: 전문 에이전트와 외부 연동 구성(2종 1세트)

## 11장. 결정론적 게이트: 흑으로 자동화 강제하기 [실무]

- 도입: 확률이 아니라 '항상' 강제하기(선수 상기)
- 흑이란: 생애주기 이벤트에 실행되는 스크립트
- 동작 원리: 이벤트와 차단·허용(exit code) 제어
- worked example: 편집 후 자동 포맷 PostToolUse 흑
- 이벤트 종류(PreToolUse·PostToolUse·Stop·SessionStart·SubagentStop)
- 흑 설정과 매치(도구·패턴 매칭)
- worked example: 위험 경로 수정 막는 PreToolUse 차단 흑
- 안전 설계: fail-open vs fail-closed·로깅
- 즉시 연습(2종 1세트): 차단 흑 조건 채우기
- 흔한 오류와 오개념: 확률적 오해·과잉 차단·권한과 혼동

## 12장. 엮기: 멀티에이전트 워크플로우 오케스트레이션 [실무]

- 도입: 단일 에이전트로 어려운 일 나눠 정복하기
- 오케스트레이션이란: 다중 에이전트·도구 워크플로우
- 동작 원리: 오케스트레이터·전문가 파이프라인
- worked example: 조사→구현→리뷰 파이프라인 구성
- 에이전트 팀: 계획·분배·통합 분업
- 커맨드로 워크플로우 절차 표준화(/feature)
- worked example: 병렬 결과를 메인이 통합·정리
- 한계: 비용·복잡도와 단일 에이전트가 나온 경우
- 즉시 연습(2종 1세트): 파이프라인 단계 채우기
- 흔한 오류와 오개념: 만능 오해·직접 처리·통합 누락

## 13장. 무인 자동화: 헤드리스·CI·스케줄 [실무]

- 도입: 대화형을 넘어 사람 없이 도는 자동화로
- 헤드리스(-p) 모드와 종료 코드·구조화 출력
- 동작 원리: 트리거→헤드리스 실행→판정 파이프라인
- worked example: -p 결과를 셸 스크립트에 통합
- CI 통합: GitHub Actions 무인 실행
- 스케줄·주기 작업(의존성 감사·릴리스 노트)
- worked example: PR 자동 코드 리뷰 Actions
- 무인 환경의 권한·비밀·안전과 흑·오케스트레이션 결합

- 즉시 연습(2종 1세트): CI 워크플로우 채우기
- 흔한 오류와 오개념: 넓은 권한·안전장치 생략·무검증 신뢰
- 단원 미니 프로젝트: 자동화 게이트와 CI 파이프라인(2종 1세트)

#### 14장. 종합 프로젝트: repo-audit — 분석(요구사항·범위) [실무]

- 드라이빙 퀘스천과 POV 다시 보기
- 기능 요구사항(FR) 정의
- 비기능 요구사항(NFR) 정의
- 사용자 스토리와 유스케이스
- 범위 In/Out 확정
- 추적성: 요구사항 ↔ 단원 개념 매핑
- 작성형 연습(2종 1세트): 요구사항 표 채우기·자가체크

#### 15장. 종합 프로젝트: repo-audit — 설계(데이터·모듈·ADR·에이전트 워크플로우) [실무]

- 설계 개요: 무엇을 어떻게 나눌까
- 데이터 구조 설계: finding-report-config
- 모듈·함수 분해(repo\_audit-checks-report-tests)
- 설계 결정 기록: ADR 5건
- 에이전트 오케스트레이션·CI 파이프라인 흐름도
- 작성형 연습(2종 1세트): ADR 작성·자가체크

#### 16장. 종합 프로젝트: repo-audit — 구현(마일스톤 증분) [실무]

- 구현 전략과 마일스톤 개요
- M1 CLAUDE.md·플랜 모드로 셋업·계획(체크포인트)
- M2 CLI 골격과 TODO 수집·첫 커밋
- M3 링크·파일크기·문서 점검과 /audit 커맨드
- M4 리포트·종료 코드와 reviewer 서브에이전트 리뷰
- M5 PreToolUse 혹·권한 게이트와 MCP 연동
- M6 GitHub Actions CI·스케줄 무인화와 오케스트레이션
- 통합 실행과 결과 확인(텍스트·JSON·CI 로그)
- 작성형 연습(2종 1세트): 빈칸 코드 채우기·자가체크

#### 17장. 종합 프로젝트: repo-audit — 발표·평가·성찰 [실무]

- 산출물 시연: repo-audit 데모와 CI 녹색
- 평가 루브릭
- 동료 피드백(갤러리 워크)
- 성찰 회고: 13개 개념의 적용
- 작성형 연습(2종 1세트): 성찰 템플릿·자가체크



## 첫걸음: Claude Code 설치와 첫 세션

**학습 목표** - Claude Code를 설치·인증해 첫 대화형 세션을 시작할 수 있습니다. - 에이전트형 코딩 도구가 자동 완성과 무엇이 다른지 설명할 수 있습니다. - REPL의 기본 조작(프롬프트 입력·중단·세션 재개·종료)을 할 수 있습니다. - 작업 디렉터리·모델·기본 설정을 확인하고 조정할 수 있습니다.

이 장은 이 책 전체를 관통하는 종합 프로젝트 **저장소 건강검진 CLI(repo-audit)** 를 만들기 위한 첫 벽돌을 놓는 장입니다. 어떤 자동화 워크플로우도 결국 "도구를 설치하고, 세션을 열고, 작업을 지시하고, 결과를 확인하는" 일의 반복입니다. 그래서 첫 단원에서는 **설치와 첫 세션**(이 장)과 **효과적 프롬프팅**(다음 장)을 충분히 익힙니다. 이 둘만 손에 익혀도 "빈 폴더에서 작은 스크립트를 만들어 실행시키기", "남이 만든 저장소를 열어 요약 받기" 같은 실전 작업을 스스로 해낼 수 있게 됩니다.

이 장의 선수 지식: **없습니다**. 터미널에서 명령을 입력하고 Git 저장소를 다뤄 본 실무 개발자라면 충분합니다. 도구 자체는 처음이라는 전제로, 모든 동작을 단계로 풀어 설명합니다.

이 장에서 가르치는 개념은 하나입니다.

- **개념 c01 — 설치와 첫 세션**: Claude Code를 설치·인증해 대화형 REPL을 열고, 에이전트가 도구를 호출해 변경을 만들어 내는 루프를 직접 관찰하는 법.

## 1. 도입: 자동완성을 넘어선 에이전트형 코딩

지난 10여 년간 우리가 쓰던 AI 코딩 보조 도구는 대부분 **자동완성(autocomplete)** 이었습니다. 커서가 놓인 자리에서 다음 몇 줄을 제안하면, 사람이 Tab을 눌러 받아들이는 방식입니다. 자동완성은 빠르지만 "지금 내가 타이핑하는 한 지점"의 문맥만 봅니다. 파일을 직접 열거나, 테스트를 돌리거나, 여러 파일에 걸친 변경을 스스로 적용하지는 못합니다. 결정과 실행은 전부 사람의 손에 남습니다.

**에이전트형 코딩(agentic coding)** 은 여기서 한 단계 나아갑니다. 사람이 "무엇을 원하는지"를 자연어로 말하면, 도구가 스스로 **코드베이스를 읽고(관찰)**, **무엇을 어떻게 바꿀지 계획하고**, **파일을 편집하거나 명령을 실행(도구 호출)** 한 뒤, **결과를 확인**하고 필요하면 다시 시도합니다. 사람은 한 줄 한 줄을 받아 적는 대신, 목표를 주고 결과를 검토하는 역할로 올라섭니다. 자동완성이 "타이핑 도우미"라면, 에이전트형 코딩은 "옆자리에 앉은 페어 프로그래머"에 가깝습니다.

**한눈에 보는 이 장의 지도** - 옆자리 페어 프로그래머를 설치하기 → **네이티브 설치(공식 권장)** 또는 **npm 전역 설치(대안)** - 페어와 대화 시작하기 → **대화형 REPL(c<sub>laude</sub>)** - 한 번만 물어보고 답만 받기 → **print 모드(c<sub>laude</sub> -p)** - 페어가 일하는 방식 이해하기 → **에이전트 루프(관찰→계획→도구 호출→결과 확인)** - 자리를 비웠다 돌아오기 → **세션 재개(--continue·--resume)**

실무에서 이 차이는 크게 다가옵니다. 자동완성은 "이 함수의 다음 줄"을 도와주지만, 에이전트는 "이 모듈 전체에서 함수 이름을 바꾸고 테스트까지 통과시켜 줘" 같은 **완결된 작업**을 받아 갑니다. repo-audit 같은 작은 CLI를 처음부터 끝까지 함께 만들 수 있는 것도 이 때문입니다.

## 2. 정의: Claude Code란 무엇인가(터미널·IDE·데스크톱·웹)

### 정의

먼저 이 장의 핵심 개념인 **설치와 첫 세션**을 한 문장으로 정의하겠습니다.

**Claude Code**는 터미널·IDE·데스크톱·웹에서 동작하는 Anthropic 공식 **에이전트형 코딩 CLI(agent coding CLI)**로, 단순 자동완성이 아니라 파일을 직접 읽고 편집하고 명령을 실행하며 커밋까지 수행하는 AI 페어 프로그래머입니다. 그리고 **첫 세션(first session)**이란 설치·인증·작업 디렉터리 선택으로 시작해, 대화형 REPL에서 자연어 지시에 에이전트가 도구를 호출해 변경을 만들어 내는 루프를 처음으로 관찰하는 출발점입니다.

용어를 정리하겠습니다.

- **CLI(Command-Line Interface, 명령줄 인터페이스)**: 터미널에 명령을 입력해 쓰는 프로그램입니다. Claude Code는 터미널에서 `claude`를 입력하는 것이 기본 진입점입니다.
- **에이전트(agent)**: 목표를 받아 스스로 도구를 골라 쓰며 여러 단계를 거쳐 작업을 완수하는 프로그램입니다.
- **REPL(Read-Eval-Print Loop, 읽고-실행하고-출력하는 반복 루프)**: 입력을 받아 처리하고 결과를 보여 주기를 반복하는 대화형 환경입니다. Claude Code의 대화 화면이 바로 REPL입니다.

### 같은 도구, 네 가지 표면

Claude Code는 한 가지 엔진을 여러 **표면(surface)**에서 쓸 수 있게 제공합니다. 어디서 쓰든 "관찰→계획→도구 호출→결과 확인"이라는 핵심은 같습니다.

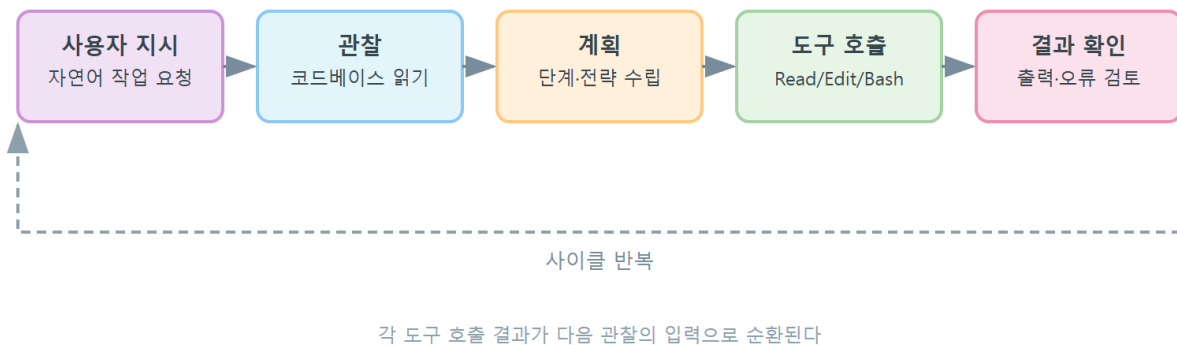
| 표면       | 진입 방법                      | 잘 맞는 상황                            |
|----------|----------------------------|------------------------------------|
| 터미널(CLI) | 셸에서 <code>claude</code> 실행 | 대부분의 개발 작업, 스크립트·CI 통합(이 책의 기본 무대) |
| IDE 확장   | VS Code·JetBrains 확장에서 호출  | 편집기 안에서 diff를 보며 작업할 때             |
| 데스크톱 앱   | Claude 데스크톱에서 코딩 세션        | GUI로 여러 세션을 시각적으로 관리할 때            |
| 웹        | 브라우저에서 클라우드 작업공간 실행        | 로컬 설치 없이 원격에서 작업할 때                |

이 책은 가장 이식성이 좋고 자동화에 직결되는 **터미널(CLI)** 을 기본 무대로 삼습니다. CLI에서 익힌 개념은 IDE·데스크톱·웹에 그대로 옮겨집니다.

**왜 터미널부터인가** 종합 프로젝트 repo-audit는 마지막에 GitHub Actions에서 사람 없이 자동 실행됩니다. CI 러너에는 GUI가 없으므로, 결국 터미널에서 도는 `claude -p` 형태가 자동화의 종착점입니다. 터미널을 먼저 익히면 13개 개념이 하나의 자동화로 모이는 그림이 자연스럽게 이어집니다.

### 3. 동작 원리: 관찰→계획→도구 호출→결과 확인 루프

#### 에이전트 루프 — 관찰 · 계획 · 도구 호출 · 결과 확인



에이전트형 코딩이 자동완성과 근본적으로 다른 이유는 **루프(loop)** 로 동작하기 때문입니다. 사람이 한 번 지시 하면, 에이전트는 다음 네 단계를 한 번 또는 여러 번 돕니다.

1. **관찰(observe)**: 작업에 필요한 코드베이스를 스스로 읽습니다. 폴더 구조를 훑고(Glob), 키워드를 검색하고(Grep), 관련 파일을 엽니다(Read). 사람이 코드를 붙여넣어 줄 필요가 없습니다.
2. **계획(plan)**: 관찰한 내용을 바탕으로 "어떤 파일의 어디를 어떻게 바꿀지" 순서를 세웁니다. 복잡한 작업이면 사람에게 계획을 먼저 보여 주기도 합니다(플랜 모드, 6장).
3. **도구 호출(act)**: 계획대로 부수효과가 있는 도구를 씁니다. 파일을 외과적으로 고치고(Edit), 새 파일을 만들고(Write), 명령을 실행합니다(Bash). 부수효과가 있는 도구는 권한 확인을 거칩니다(4장).
4. **결과 확인(verify)**: 변경을 적용한 뒤 테스트나 실행으로 결과를 확인합니다. 기대와 다르면 1번으로 돌아가 다시 관찰·교정합니다.

이 네 단계가 사람의 피드백을 받아 반복되는 것이 핵심입니다. 자동완성은 1회성 제안이지만, 에이전트는 "고쳤다 → 실행했다 → 실패했다 → 원인을 찾아 다시 고쳤다"를 스스로 돌 수 있습니다.

#### 자동완성과의 대비

| 구분      | 자동완성(autocomplete) | 에이전트형 코딩(Claude Code) |
|---------|--------------------|-----------------------|
| 보는 범위   | 커서 주변 한 지점         | 프로젝트 전체(스스로 탐색)       |
| 하는 일    | 다음 몇 줄 제안          | 읽기·편집·실행·커밋까지         |
| 누가 실행하나 | 사람이 직접 타이핑·실행      | 에이전트가 도구로 실행          |
| 작업 단위   | 한 줄~한 블록           | 완결된 작업(여러 파일·여러 단계)   |
| 검증      | 사람이 따로             | 에이전트가 실행해 확인, 사람은 검토  |

기억할 한 가지 사람의 역할이 "코드를 받아 적는 사람"에서 "목표를 주고 결과를 검토·교정하는 사람"으로 바뀝니다. 그래서 이 책의 다음 장이 곧바로 "어떻게 지시할 것인가(프롬프팅)"인 것입니다. 좋은 지시가 좋은 루프를 만듭니다.

## 4. 설치와 인증(네이티브 권장·npm 대안)

이제 직접 페어 프로그래머를 자리에 앉혀 보겠습니다. 이 책의 대상 환경은 **Windows 11**이므로, Windows를 기준으로 설명하고 macOS·Linux는 함께 표기합니다. 설치 방법은 두 갈래지만, **공식 권장은 네이티브 설치**입니다. 둘 중 하나만 고르면 됩니다.

**Windows 11 사전 점검 - 관리자 권한은 필요 없습니다.** 네이티브 설치의 사용자 폴더(`C:\Users\사용자\local\bin`)에 바이너리를 두므로 일반 권한으로 충분합니다. - **Git for Windows**가 설치돼 있으면 Claude Code가 그 Bash를 도구(셸)로 사용합니다(권장). 없으면 PowerShell을 셸로 씁니다. Git을 함께 설치해 두면 7장의 버전관리 통합이 매끄럽습니다. - **WSL2**를 쓰는 분은 WSL 배포판 안에서 아래 macOS/Linux용 `install.sh`로 설치할 수 있습니다. Windows 11은 완전 지원됩니다.

### 방법 A — 네이티브 설치(공식 권장)

Anthropic이 **권장(Recommended)**하는 방식입니다. Node.js 의존이 전혀 없고, 백그라운드에서 자동으로 최신 버전으로 업데이트되며, 우선 지원을 받습니다. 패키지 매니저 충돌도 없어 가장 안정적입니다. 먼저 이 방법을 시도하십시오.

#### Windows(PowerShell) — 권장 환경

```
irm https://claude.ai/install.ps1 | iex
```

`irm` (Invoke-RestMethod)이 공식 설치 스크립트를 내려받고, `iex` (Invoke-Expression)가 그것을 실행합니다. 설치가 끝나면 바이너리가 `C:\Users\사용자\local\bin`에 놓이고 PATH가 갱신됩니다. **PATH 갱신은 새 터미널에만 반영**되므로, 설치 직후에는 PowerShell 창을 닫고 새로 연 뒤 확인합니다.

#### macOS / Linux(또는 WSL2 내부)

```
curl -fsSL https://claude.ai/install.sh | bash
```

## 방법 B — npm 전역 설치(대안)

Node.js 생태계를 이미 쓰고 있거나, **CI에서 버전을 고정**하거나 **팀 표준을 npm으로 통일**하고 싶을 때의 대안입니다. Node.js 18 이상이 전제이며, 업데이트는 자동이 아니라 `@latest` 로 직접 다시 설치해야 합니다.

```
npm install -g @anthropic-ai/claude-code
```

정리: 특별한 이유가 없으면 **방법 A(네이티브)**를 쓰십시오. npm은 "Node를 이미 쓴다 / CI 버전 고정 / 팀 표준화"처럼 명확한 이유가 있을 때의 선택지입니다. 어느 쪽이든 결과적으로 `claude` 명령이 PATH에 등록됩니다.

## 설치 확인과 업데이트

설치가 끝났다면(새 터미널에서) 다음 두 줄로 환경을 점검합니다.

```
claude --version
claude doctor
```

`--version` 은 설치된 버전을, `doctor` 는 인증·PATH·권한 등 환경 상태를 한 번에 진단합니다. 업데이트 방법은 설치 방식에 따라 다릅니다.

| 설치 방식           | 업데이트 방법                                                          |
|-----------------|------------------------------------------------------------------|
| 네이티브(방법 A)      | 백그라운드 자동 업데이트(수동은 <code>claude update</code> )                   |
| npm(방법 B)       | <code>npm install -g @anthropic-ai/claude-code@latest</code> 재설치 |
| WinGet으로 설치한 경우 | <code>winget upgrade Anthropic.ClaudeCode</code>                 |

## 인증(authentication)

설치 후 처음 `claude` 를 실행하면 인증을 안내합니다. **(a) 구독 로그인**과 **(b) API 키** 두 경로가 있습니다. 개인·팀이라면 (a)가 간단하고, 팀 공용·CI에서 사용량 과금으로 운용하려면 (b)를 씁니다.

### (a) 구독 로그인 — 권장(개인/팀)

- 작업 폴더에서 `claude` 를 처음 실행하면 브라우저가 자동으로 열립니다.
- Claude.ai 계정(Pro·Max·Teams·Enterprise)으로 OAuth 로그인합니다.
- 인증이 끝나면 자격 증명이 자동으로 저장되어, 이후에는 다시 로그인할 필요가 없습니다.
- 로그아웃은 REPL 안에서 `/logout` 으로 합니다.

### (b) Anthropic API 키 방식 — 팀/CI·사용량 과금

1. **키 발급**: 브라우저로 `console.anthropic.com` 에 접속 → **Settings** → **API Keys** → **Create Key** → 발급된 키를 안전한 곳에 보관합니다. 키는 발급 직후 한 번만 보이고 **다시 표시되지 않습니다**.
2. **환경 변수 등록**: 키를 `ANTHROPIC_API_KEY` 환경 변수로 둡니다.
3. Windows PowerShell(현재 세션): `text $env:ANTHROPIC_API_KEY="sk-ant-..."` 영구 등록은 "시스템 환경 변수 편집"에서 사용자 변수로 추가합니다(새 터미널부터 반영).
4. macOS / Linux: `text export ANTHROPIC_API_KEY="sk-ant-..."`
5. **실행**: `claude` 를 실행하면 키가 자동 감지되고, 처음 한 번 사용 승인을 거친 뒤 이후로는 자동으로 적용됩니다.

구독과 API 키를 둘 다 가진 경우, **API 키가 우선** 적용됩니다. 둘을 오가려면 REPL 안에서 `/config` 로 토글합니다.

### 구독 vs API 키 — 무엇을 고를까

| 구분    | (a) 구독 로그인          | (b) API 키               |
|-------|---------------------|-------------------------|
| 요금 체계 | 월 정액(Pro·Max·Teams) | 사용량 기반(토큰 단위 과금)        |
| 청구 대상 | 개인·팀 통합 청구          | Anthropic 콘솔 워크스페이스     |
| 권장 대상 | 개인·소규모 팀의 일상 개발     | 기업·CI/CD·사용량을 분리 정산하는 팀 |
| 설정 난도 | 브라우저 로그인 한 번        | 키 발급 + 환경 변수 등록         |

**비밀 관리 주의** API 키는 **코드나 저장소에 절대 넣지 않습니다**. 환경 변수(`ANTHROPIC_API_KEY`)나 도구가 안전하게 보관하는 인증 흐름을 쓰고, `.env` 파일을 쓴다면 `.gitignore` 에 반드시 추가합니다. 실수로 커밋한 키는 즉시 콘솔에서 폐기(revoke)하고 새로 발급하십시오. 이 원칙은 뒤에서 MCP 토큰·CI 비밀을 다룰 때(10·13장) 그대로 이어집니다.

### 첫 실행과 신뢰(trust)

작업할 폴더로 이동해 `claude` 를 실행하면, 그 폴더를 신뢰할지(trust) 한 번 묻습니다. 신뢰는 "이 폴더에서 에이전트가 파일을 읽고 도구를 쓰는 것을 허용하겠다"는 확인입니다. 신뢰하지 않은 폴더에서는 동작이 제한됩니다.

```
cd C:\work\playground
claude
```

이제 대화형 REPL이 열립니다.

### 흔한 설치 문제

처음 설치에서 자주 마주치는 문제와 원인은 다음과 같습니다(Windows 11 기준 포함).

| 증상                                       | 원인                    | 해결                                        |
|------------------------------------------|-----------------------|-------------------------------------------|
| claude : 'claude' 용어가 cmdlet으로 인식되지 않습니다 | 새 터미널을 열지 않아 PATH 미반영 | PowerShell 창을 닫고 새로 연 뒤 다시 실행             |
| command not found:<br>claude (mac/Linux) | PATH에 설치 경로가 없음       | 셸을 새로 열거나 PATH를 다시 읽기 (source 또는 터미널 재시작) |
| 권한 오류로 npm 설치 실패                         | 전역 설치 권한 부족           | 네이티브 설치(방법 A)로 대체                         |
| 실행했는데 인증을 반복해서 물어                        | 인증 정보가 저장되지 않음        | claude doctor 로 인증 상태 확인 후 재 로그인          |
| 폴더에서 동작이 제한됨                             | 폴더를 신뢰(trust)하지 않음    | 첫 실행 시 신뢰 확인에 동의                          |

**막히면 doctor부터** 설치·인증 문제로 막히면 추측으로 이것저것 바꾸기 전에 claude doctor 를 먼저 돌려 진단 결과를 보십시오. 무엇이 어긋났는지(PATH·인증·권한)를 도구가 짚어 주므로, 2장에서 배울 "추측하지 말고 실제 자료를 보기"의 원칙이 설치 단계부터 적용됩니다.

## 5. worked example: 첫 작업 — 소스에서 TODO/FIXME 주석 수집하기

말로만 설명하면 감이 오지 않으니, 완전히 풀어 보이는 첫 세션을 따라가 보겠습니다. "Hello World" 대신 **실무에서 바로 써먹을 첫 작업**을 시켜 보겠습니다. 목표는 **현재 폴더의 소스 파일을 훑어 TODO·FIXME 주석을 찾아, 파일 경로와 줄 번호로 출력하는 작은 스크립트를 만들고 실행하는 것**입니다. 이 작업은 우연이 아니라 의도된 것입니다. 이 책의 종합 프로젝트 repo-audit가 저장소를 점검할 때 가장 먼저 하는 일 중 하나가 바로 "훑어진 TODO를 모아 보여 주기"이기 때문입니다(16장에서 다시 만납니다).

### 세션 기록

점검할 프로젝트 폴더에서 세션을 엽니다(빈 폴더가 아니라, 이미 소스가 몇 개 있는 폴더를 가정합니다).

```
cd C:\work\sample-service
claude
```

REPL이 열리면 자연어로 지시합니다(아래 > 가 우리가 입력한 프롬프트입니다).

> todo\_scan.py 스크립트를 만들어 줘. 현재 폴더와 하위의 .py 파일을 훑어서 "TODO" 또는 "FIXME"가 들어간 줄을 찾아, "파일경로:줄번호: 내용" 형식으로 출력하면 돼. 표준 라이브러리만 쓰고, 다 만들면 실제로 실행해서 결과를 보여 줘.

[에이전트] 작업을 진행하겠습니다.

- 1) todo\_scan.py 생성 (Write)
- 2) python todo\_scan.py 실행으로 동작 확인 (Bash)

Write todo\_scan.py  
+28 -0

Bash python todo\_scan.py  
app\service.py:14: # TODO: 타임아웃을 설정값으로 분리  
app\service.py:31: # FIXME: 재시도 로직이 예외를 삼킨다  
app\cli.py:9: # TODO: --json 출력 옵션 추가

todo\_scan.py를 만들고 실행했습니다. 소스 3곳에서 TODO/FIXME 표시를 찾았습니다.

세션을 마치고 만들어진 파일을 보면 다음과 같습니다.

```
# todo_scan.py
# 현재 폴더와 하위의 파이썬 소스에서 TODO/FIXME 주석을 모아 출력합니다.
import os

# 찾을 표시어 — 대소문자를 구분해 코드 주석 관례를 따릅니다.
MARKERS = ("TODO", "FIXME")

def scan_file(path: str) -> None:
    """한 파일을 한 줄씩 읽어 표시어가 든 줄을 출력합니다."""
    with open(path, encoding="utf-8") as f:
        for lineno, line in enumerate(f, start=1):
            if any(marker in line for marker in MARKERS):
                print(f"{path}:{lineno}: {line.strip()}")

def main() -> None:
    """현재 폴더 이하를 걸어가며 .py 파일만 검사합니다."""
    for root, _dirs, files in os.walk("."):
        for name in files:
            if name.endswith(".py"):
                scan_file(os.path.join(root, name))

if __name__ == "__main__":
    main()
```

직접 실행해 보면 결과는 다음과 같습니다.

```
$ python todo_scan.py
.\app\service.py:14: # TODO: 타임아웃을 설정값으로 분리
.\app\service.py:31: # FIXME: 재시도 로직이 예외를 삼킨다
.\app\cli.py:9: # TODO: --json 출력 옵션 추가
```

## 줄별 해설

- **기존 폴더에서 시작:** 코드를 붙여넣지 않았습니다. 에이전트는 폴더에 이미 소스가 있다는 맥락을 관찰하고, "주석을 훑는 스크립트가 필요하다"고 **계획**한 뒤 `Write` 도구로 파일을 만들었습니다.
- `Write todo_scan.py / +28 -0`: 에이전트가 부수효과 도구(Write)를 호출했음을 보여 줍니다. 몇 줄이 추가·삭제됐는지(+28 -0)가 함께 표시됩니다.
- `Bash python todo_scan.py`: 만들기만 한 것이 아니라 **실제로 실행해 결과까지** 확인했습니다. 이것이 자동 완성과의 결정적 차이입니다.
- **출력 형식:** "파일경로:줄번호: 내용" 형식을 지시에 명시했더니 그대로 따랐습니다. 다음 장에서 배울 "형식 제약을 명시하기"의 작은 예고편입니다.

방금 한 세션 안에서 3장의 루프 네 단계가 그대로 돌았습니다. 관찰(폴더의 소스 파악) → 계획(주석 수집 스크립트 작성·실행) → 도구 호출(Write·Bash) → 결과 확인(실제 출력 검토). 그리고 이 결과물은 toy가 아니라, 코드 리뷰 전에 "남은 일이 어디에 표시돼 있나"를 한눈에 보는 실무 도구입니다.

## 6. 대화형 REPL과 1회성 실행(-p/print 모드)

Claude Code를 쓰는 방식은 크게 둘입니다.

### 대화형 REPL

`claude` 만 입력하면 대화형 REPL이 열립니다. 한 작업을 마친 뒤에도 같은 화면에서 이어 지시할 수 있고, 에이전트는 앞선 대화 맥락을 기억합니다. 탐색·구현·검토를 주고받는 **개발 작업의 기본 모드**입니다.

### 1회성 print 모드(-p)

질문 하나에 답만 받고 끝내고 싶다면 `-p` (print) 모드를 씁니다. 대화 화면을 열지 않고, 결과를 표준 출력으로 내보낸 뒤 즉시 종료합니다.

```
claude -p "이 폴더에서 가장 큰 파이썬 파일의 이름만 알려 줘"
```

print 모드의 진짜 가치는 **셸 파이프라인·스크립트·CI에 끼워 넣을 수 있다는 점**입니다. 출력이 표준 출력으로 나오므로 다른 명령으로 넘길 수 있고, 종료 코드로 성공·실패를 판정할 수 있습니다.

```
claude -p "테스트가 모두 통과하면 OK, 아니면 FAIL만 출력해" > result.txt
```

구조화된 출력이 필요하면 JSON 형식을 요청할 수 있습니다.

```
claude -p "발견한 TODO 개수를 JSON으로 알려 줘" --output-format json
```

**미리 보는 종착점** 13장에서 repo-audit를 GitHub Actions로 무인 실행할 때, CI 러너가 호출하는 것이 바로 이 `claude -p`입니다. 대화형으로 익힌 작업을 print 모드로 옮기면 사람 없이 도는 자동화가 됩니다. 지금은 "대화형은 개발용, print는 자동화용"이라는 두 갈래만 기억하면 됩니다.

| 구분    | 대화형 REPL( <code>claude</code> ) | <code>print</code> 모드( <code>claude -p</code> ) |
|-------|---------------------------------|-------------------------------------------------|
| 화면    | 대화창 유지                          | 출력 후 즉시 종료                                      |
| 맥락    | 앞 대화를 이어감                       | 1회성(단발)                                         |
| 쓰임새   | 탐색·구현·검토                        | 스크립트·파이프라인·CI                                   |
| 결과 활용 | 사람이 화면에서 검토                     | 표준 출력·종료 코드로 자동 판정                              |

## 7. worked example: 처음 보는 저장소를 열어 파악하기

첫 세션이 항상 새 파일을 만드는 일은 아닙니다. 실무에서 더 잦은 첫 세션은 **인수인계받은, 또는 처음 합류한 저장소를 열어 구조를 파악**하는 일입니다. README가 부실하거나 만든 사람이 자리에 없을 때, 에이전트에게 "진입점·의존성·실행법·구조"를 물어 빠르게 지도를 그릴 수 있습니다. 이번에는 여러 파일과 설정이 섞인 작은 저장소를 열어 파악해 보겠습니다.

### 예시 저장소 구조

다음과 같이 소스·설정·테스트가 섞인 작은 파이썬 서비스가 있다고 합시다(실제로 흔히 만나는 형태입니다).

```
sample-service/
  pyproject.toml      # 패키지·의존성 정의
  README.md          # (한 줄짜리 빈약한 설명)
  app/
    __init__.py
    cli.py            # 명령줄 진입점
    service.py        # 핵심 로직(외부 API 호출)
  tests/
    test_service.py
```

`pyproject.toml` 에는 의존성과 실행 진입점이 적혀 있습니다.

```
[project]
name = "sample-service"
dependencies = ["httpx>=0.27"]

[project.scripts]
sample = "app.cli:main"
```

진입점 `app/cli.py` 는 입력을 받아 핵심 로직을 호출합니다.

```
# app/cli.py
# 명령줄에서 받은 도시 이름으로 서비스 로직을 호출하는 진입점입니다.
import sys

from app.service import fetch_weather

def main() -> None:
    """첫 번째 인자를 도시 이름으로 받아 날씨 요약물 출력합니다."""
    city = sys.argv[1] if len(sys.argv) > 1 else "Seoul"
    print(fetch_weather(city))

if __name__ == "__main__":
    main()
```

핵심 로직 `app/service.py` 는 외부 API(httpx)를 호출합니다.

```
# app/service.py
# 외부 날씨 API를 호출해 한 줄 요약 문자열을 만드는 핵심 로직입니다.
import httpx

API_URL = "https://api.example.com/weather"

def fetch_weather(city: str) -> str:
    """도시 이름으로 외부 API를 호출해 요약 문자열을 반환합니다."""
    # TODO: 타임아웃을 설정값으로 분리
    response = httpx.get(API_URL, params={"city": city}, timeout=5.0)
    data = response.json()
    return f"{city}: {data['summary']}, {data['temp']}C"
```

## 세션 기록

저장소 폴더에서 세션을 열고, 코드를 붙여넣지 않은 채 파악을 지시합니다.

```
cd C:\work\sample-service
claude
```

> 이 저장소를 처음 봐. 코드를 붙여넣지 않을 테니 네가 직접 파일을 읽고 파악해 줘.  
(1) 진입점, (2) 외부 의존성, (3) 실행·테스트 방법, (4) 폴더 구조 한 줄 요약  
이 네 가지를 항목별로 정리해 줘.

[에이전트]

```
Glob **/*.py
Read pyproject.toml
Read app\cli.py
Read app\service.py
```

정리:

- (1) 진입점: pyproject.toml의 [project.scripts]에 sample = "app.cli:main"로 등록돼 있고, 실제 시작 함수는 app\cli.py의 main()입니다.
- (2) 외부 의존성: httpx(>=0.27) 한 개. service.py가 HTTP 호출에 사용합니다.
- (3) 실행: 설치 후 `sample` 명령 또는 `python -m app.cli`. 테스트는 `pytest`로 tests\test\_service.py가 돕니다.
- (4) 구조: app/(소스: cli=입력처리, service=핵심 로직) + tests/(검증), 설정은 pyproject.toml 한 곳에 모여 있습니다.

## 줄별 해설

- `Glob **/*.py`: 에이전트가 먼저 폴더 전체에서 파이썬 파일을 찾았습니다(관찰). 우리가 파일 목록을 알려주지 않았습니다.
- `Read pyproject.toml`: 진입점 의존성은 소스가 아니라 **설정 파일**에 있다는 것을 알고 그쪽을 먼저 열었습니다. 사람이 저장소를 파악하는 순서와 같습니다.
- `Read app\cli.py` / `Read app\service.py`: 설정이 가리킨 진입점과 핵심 로직 파일을 차례로 직접 열어 읽었습니다. **코드를 채팅창에 붙여넣을 필요가 없다는 점**이 핵심입니다.
- **네 항목 정리**: 지시에 "(1)~(4) 항목별로"라는 형식 제약을 넣었더니 그 골격으로 답했습니다. 모호하게 "설명해 줘"라고 했을 때보다 훨씬 쓸 만한 지도가 나옵니다(2장에서 깊이 다룹니다).

5절의 "새로 만들기" 예제와 이 절의 "기존 코드 파악하기" 예제를 합치면, 첫 세션의 두 전형이 모두 손에 들어옵니다. **새로 만들기(Write 중심)**와 **기존 코드 파악하기(Glob-Grep-Read 중심)**입니다. 실무에서는 후자, 즉 "처음 보는 코드를 빠르게 파악하기"가 특히 자주 쓰입니다.

## 8. 세션 관리: 중단·재개(--continue/--resume)·종료

페어 프로그래머와 일하다 보면 잠깐 멈추거나, 자리를 비웠다 이어서 일하고 싶을 때가 있습니다. Claude Code는 이를 위한 조작을 제공합니다.

### 중단(interrupt)

에이전트가 엉뚱한 방향으로 가고 있다고 느끼면 즉시 멈출 수 있습니다. REPL에서 **Esc** 키를 누르면 현재 작업을 중단합니다. 잘못된 대규모 변경이 끝까지 진행되기 전에 끊을 수 있는 안전장치입니다.

### 재개(resume): `--continue` 와 `--resume`

세션을 닫아도 대화 맥락이 사라지지 않습니다. 두 가지 재개 방법이 있습니다.

- `claude --continue` (또는 `-c`): 가장 최근 세션을 곧바로 이어 엽니다. "방금 하던 작업"으로 바로 돌아갈 때 씁니다.
- `claude --resume` (또는 `-r`): 지난 세션 목록을 보여 주고, 그중 하나를 골라 재개합니다. 여러 작업을 오가며 특정 세션으로 돌아갈 때 씁니다.

```
claude --continue
```

```
claude --resume
```

print 모드와도 결합할 수 있어, 직전 맥락을 이어 한 줄 질의만 던지는 식의 자동화도 가능합니다.

### 종료(exit)

세션을 끝낼 때는 REPL에서 `/exit` 를 입력하거나 Ctrl-C를 두 번 누릅니다. 종료해도 위에서 본 것처럼 다음에 `-continue` 로 이어 갈 수 있습니다.

### 작업 디렉터리·모델·기본 설정 확인

세션 안에서 내장 슬래시 커맨드로 환경을 확인·조정할 수 있습니다(슬래시 커맨드는 8장에서 본격적으로 다룹니다).

| 커맨드                   | 역할                         |
|-----------------------|----------------------------|
| <code>/help</code>    | 사용 가능한 커맨드·단축키 안내          |
| <code>/model</code>   | 현재 모델 확인·변경(작업 난이도에 맞게 선택) |
| <code>/clear</code>   | 대화 맥락 초기화(새 작업으로 깨끗이 시작)   |
| <code>/compact</code> | 긴 대화를 핵심만 남겨 압축            |

모델은 명령행에서 `--model` 옵션으로도 고를 수 있습니다. 작업 디렉터리는 `claude` 를 실행한 폴더가 기본이며, 그 폴더와 하위가 에이전트의 작업 범위가 됩니다.

**중단을 두려워하지 마십시오** 초보자가 가장 자주 하는 실수는 "에이전트가 이상하게 가는데도 끝까지 지켜보는 것"입니다. 방향이 틀렸다고 느끼면 Esc로 즉시 멈추고 다시 지시하는 편이 훨씬 빠릅니다. 잘못된 루프를 끝까지 돌리는 것보다 일찍 끊고 교정하는 것이 에이전트 사용의 기본기입니다.

## 9. 즉시 연습(2종 1세트): 첫 세션 시나리오 채우기

배운 것을 직접 손으로 옮겨 보겠습니다. **모범** → **빈 템플릿** → **자가체크** 순서로 진행합니다.

### 연습 1 — 첫 세션 명령 흐름 복기하기

먼저 모범 답안입니다. "프로젝트 폴더에서 점검 스크립트를 하나 만들어 실행"하는, 실무에 가까운 첫 세션의 명령 흐름입니다(5절 TODO 수집과 같은 결의 작업입니다).

```
# 모범: 명령 흐름
cd C:\work\sample-service # 1) 점검할 프로젝트 폴더로 이동
claude                    # 2) 대화형 REPL 시작(첫 실행이면 trust 확인)
# REPL 안에서:
# > 현재 폴더의 .py 파일 줄 수 합계를 출력하는 line_count.py를 만들어 줘.
# > 표준 라이브러리만 쓰고, 다 만들면 python line_count.py로 실행해 결과를 보여 줘.
```

### 작성형 빈 템플릿 — 직접 작성하기

아래 빈칸을 채워, "처음 합류한 저장소를 열어 파악하는" 첫 세션의 명령 흐름을 직접 작성하십시오. 코드를 붙여넣지 않고 에이전트가 파일을 읽게 만드는 것이 핵심입니다(7절과 같은 작업).

```
# 빈칸을 채우세요
cd _____ # 1) 처음 보는 저장소 폴더로 이동
               # 2) 대화형 REPL 시작
# REPL 안에서 던질 지시(코드 붙여넣기 없이):
# > _____
# > (형식 제약 한 가지를 넣어 보세요. 예: "(1)~(4) 항목별로")
```

추가로, 같은 파악 작업을 1회성으로 끝내는 print 모드 명령도 한 줄 작성해 보십시오.

```
claude ____ "_____"
```

**자가체크 — 스스로 점검** - [ ] claude 를 실행하기 전에 점검 대상 폴더로 이동했는가 - [ ] 지시에 코드 붙여넣기 대신 "네가 직접 파일을 읽어"라는 취지가 들어갔는가 - [ ] 출력 형식 제약(항목·줄 수·형식)을 한 가지 이상 넣었는가 - [ ] print 모드 명령에 -p 와 따옴표로 감싼 질의가 들어갔는가

## 연습 2 — 세션 재개·종료 시나리오 채우기

모범 답안입니다. 작업을 잠시 멈췄다가 이어 가는 흐름입니다.

```
# 모범: 중단했다 이어 가기
# (작업 중 방향이 틀렸다고 느껴) Esc 로 현재 작업 중단
/exit # 세션 종료
# 잠시 후 다시:
claude --continue # 가장 최근 세션을 곧바로 이어 열기
```

## 작성형 빈 템플릿 — 직접 작성하기

"여러 작업을 오가다가, 어제 하던 특정 세션으로 돌아가고 싶은" 상황의 명령을 빈칸을 채워 작성하십시오.

```
# 빈칸을 채우세요
claude _____ # 지난 세션 목록을 보고 하나를 골라 재개
# 만약 가장 최근 세션이면 더 짧게:
claude _____ # 최근 세션 즉시 이어 가기
# 세션을 끝낼 때:
_____ # REPL 종료 커맨드
```

**자가체크 — 스스로 점검(체크리스트)** - [ ] "특정 세션 선택"에는 --resume (목록에서 선택)을 골랐는가 - [ ] "최근 세션 즉시 이어 가기"에는 --continue (또는 -c)를 골랐는가 - [ ] 종료 커맨드(/exit)를 정확히 적었는가 - [ ] 중단이 필요하면 Esc로 즉시 멈출 수 있음을 떠올렸는가

# 10. 흔한 오류와 오개념: 자동완성 오해·붙여넣기 습관

**오개념 1 — "Claude Code는 똑똑한 자동완성기다"** 아닙니다. 자동완성은 다음 줄을 제안할 뿐이지만, Claude Code는 파일을 직접 읽고·편집하고·실행하는 **에이전트**입니다. "다음 줄 제안"이 아니라 "완결된 작업 위임"이 본질입니다. 그래서 "이 함수 고쳐 줘"가 아니라 "이 버그를 재현해 원인을 찾아 고치고 테스트로 확인해 줘" 같은 **작업 단위**로 지시하는 것이 맞습니다.

**오개념 2 — "코드를 채팅창에 통째로 붙여넣어야 한다"** 불필요합니다. 에이전트는 @파일 멘션이나 스스로의 탐색(Glob-Grep-Read)으로 실제 파일을 읽습니다. 코드를 복사해 붙여넣으면 오히려 원본과 어긋난 낡은 사본을 보게 될 위험이 있습니다. **실제 파일을 가리키게** 하십시오(자세한 방법은 2장).

**오개념 3 — "세션을 닫으면 맥락이 사라진다"** 아닙니다. `--continue` 로 가장 최근 세션을, `--resume` 으로 고른 세션을 이어 갈 수 있습니다. 자리를 비웠다 돌아와도 직전 대화 맥락이 보존됩니다.

**흔한 오류 — 잘못된 방향을 끝까지 지켜보기** 에이전트가 엉뚱한 파일을 고치기 시작하면 끝까지 기다리지 말고 **Esc**로 즉시 중단 한 뒤 다시 지시하십시오. 일찍 끊고 교정하는 것이 가장 빠른 길입니다.

## 11. 자주 묻는 질문(FAQ)

**Q. 네이티브 설치와 npm 설치 중 무엇을 골라야 하나요?** 기본은 **네이티브 설치(공식 권장)**입니다. Windows에서는 PowerShell에 `irm https://claude.ai/install.ps1 | iex` 한 줄이면 되고, Node.js 의존이 없으며 백그라운드 자동 업데이트를 받습니다. npm 설치(`npm install -g @anthropic-ai/claude-code`)는 Node.js 생태계를 이미 쓰거나, CI에서 버전을 고정하거나, 팀 표준을 npm으로 맞추고 싶을 때의 대안입니다. 둘 다 결과적으로 `claude` 명령을 제공하므로, 특별한 이유가 없으면 네이티브를 고르십시오.

**Q. 인터넷 없이도 동작하나요?** Claude Code는 모델 추론을 위해 네트워크가 필요합니다. 다만 만들어진 코드의 실행·테스트는 로컬에서 이뤄집니다.

**Q. 대화형과 print 모드(-p)는 언제 갈라 씁니까?** 탐색·구현·검토처럼 주고받는 작업은 대화형(`claude`)이 좋습니다. 한 번 묻고 답만 받아 스크립트·CI에 끼워 넣을 때는 print 모드(`claude -p`)가 맞습니다. 자동화의 종착점은 결국 print 모드입니다(13장).

**Q. 첫 실행에서 폴더 신뢰(trust)를 왜 묻습니까?** 에이전트가 그 폴더의 파일을 읽고 도구를 쓰는 것을 사람이 한 번 승인하도록 하는 안전장치입니다. 신뢰는 폴더 단위로 기억됩니다.

**Q. 모델은 어떻게 바꿉니까?** 세션 안에서 `/model` 로 확인·변경하거나, 실행할 때 `--model` 옵션으로 지정합니다. 단순 작업엔 빠른 모델, 복잡한 작업엔 더 강한 모델을 고르는 식으로 비용·품질을 조절합니다.

**Q. 설치·인증이 자주 막히면 무엇부터 봐야 하나요?** `claude doctor` 를 먼저 실행하십시오. 인증 상태·PATH·권한 등 환경을 한 번에 진단해, 무엇이 어긋났는지 짚어 줍니다. 추측으로 설정을 바꾸기 전에 진단 결과를 확인하는 것이 가장 빠릅니다.

## 이 장 요약

- **설치와 첫 세션:** Claude Code는 자동완성이 아니라 파일을 읽고·편집하고·실행하는 **에이전트형 코딩 CLI**입니다. Windows에서는 **네이티브 설치**(`irm https://claude.ai/install.ps1 | iex`, **공식 권장**)가 기본이고 npm 전역 설치도 대안입니다. 구독 로그인 또는 API 키로 인증한 뒤 폴더를 신뢰하면 첫 세션이 열립니다.
- **에이전트 루프:** 관찰 → 계획 → 도구 호출 → 결과 확인을 반복하며, 사람은 목표를 주고 결과를 검토·교정합니다.

- **두 가지 실행 모드:** 대화형 REPL( `claude` )은 개발용, print 모드( `claude -p` )는 스크립트·CI용입니다.
- **세션 관리:** Esc로 중단, `--continue` · `--resume` 으로 재개, `/exit` 로 종료합니다.

다음 장에서는 이 루프의 품질을 좌우하는 **효과적 프롬프팅과 작업 지시** 를 배웁니다. 좋은 지시가 좋은 결과를 만든다는 것을, 모호한 지시와 구체적인 지시를 직접 비교하며 체득하겠습니다. 두 장을 마치면 단원 미니 프로젝트(첫 세션으로 미니 도구 만들기)로 배운 것을 한 산출물에 모읍니다.



# 에이전트 다루기: 효과적 프롬프팅과 작업 지시

**학습 목표** - 목표·맥락·제약·완료 조건을 갖춘 명확한 작업 지시를 작성할 수 있습니다. - 큰 작업을 검증 가능한 단계로 쪼개 지시할 수 있습니다. - 에이전트의 계획·중간 산출을 검토하고 교정 지시를 줄 수 있습니다. - 모호함이 부른 잘못된 결과를 진단하고 프롬프트를 개선할 수 있습니다.

앞 장에서 우리는 에이전트가 **관찰 → 계획 → 도구 호출 → 결과 확인** 루프로 동작한다는 것을 보았습니다. 이 장은 그 루프의 입력, 즉 **사람의 지시** 를 다룹니다. 같은 도구라도 지시의 명확성에 따라 결과가 하늘과 땅 차이로 갈립니다. 이 장의 한 개념을 충실히 익히면, 다음 단원에서 배울 모든 작업(파일 편집·권한·플랜·커밋)이 훨씬 정확하게 굴러갑니다.

이 장의 선수 지식: **1장(설치와 첫 세션)** 입니다. 첫 세션에서 본 "관찰→계획→도구 호출→결과 확인" 루프를 떠올리며 읽으면, 왜 모호한 지시가 잘못된 가정으로 이어지는지 자연스럽게 이해됩니다.

이 장에서 가르치는 개념은 하나입니다.

- **개념 c02 — 효과적 프롬프팅과 작업 지시**: 목표·맥락·제약·완료 조건을 갖춘 지시를 쓰고, 작업을 단계로 쪼개며, @파일로 맥락을 주입하고, 검토→피드백→재시도로 품질을 끌어올리는 법.

## 1. 도입: 결과는 지시의 명확성에 달렸다(선수 상기)

1장에서 우리는 에이전트가 사람의 지시 한 줄을 받아 스스로 여러 단계를 도는 것을 보았습니다. 그런데 에이전트는 사람의 머릿속을 읽지 못합니다. 지시에 빠진 정보는 **가정(assumption)** 으로 메웁니다. 그리고 가정이 어긋나면, 에이전트는 어긋난 가정 위에서 성실하게 잘못된 결과를 만들어 냅니다.

예를 들어 "로그인 좀 고쳐 줘"라는 지시를 생각해 봅시다. 어떤 증상인지, 어떤 동작이 옳은지, 어떤 파일을 건드려도 되는지, 무엇으로 "고쳐졌다"고 판단할지가 전부 빠져 있습니다. 에이전트는 이 빈칸을 추측으로 채웁니다. 운이 좋으면 맞고, 대개는 빗나갑니다. 반대로 같은 작업이라도 증상·기대 동작·건드릴 범위·완료 기준을 함께 주면, 에이전트의 계획이 곧장 정확해집니다.

**한눈에 보는 이 장의 지도** - 좋은 지시의 뼈대 → **목표·맥락·제약·완료 조건(4요소)** - 큰 일을 다루는 법 → **작업 분해(한 번에 하나)** - 실제 코드를 가리키기 → **@파일 멘션** - 한 번에 완벽을 기대하지 않기 → **반복 교정(검토→피드백→재시도)** - 피해야 할 것 → **과대 요청·모호한 형용사·검증 누락**

핵심 명제는 단순합니다. **에이전트의 결과 품질은 작업 지시의 명확성에 비례합니다.** 이 장의 모든 기법은 이 한 문장을 구체적인 습관으로 바꾸는 도구입니다.

## 2. 좋은 지시의 네 요소(목표·맥락·제약·완료 조건)

### 정의

이 장의 핵심 개념인 **효과적 프롬프팅**과 **작업 지시** 를 한 문장으로 정의하겠습니다.

**효과적 프롬프팅(effective prompting)**이란, 에이전트가 잘못된 가정을 하지 않도록 **목표·맥락·제약·완료 조건** 을 명확히 담아 작업을 지시하고, 큰 일을 검증 가능한 단계로 쪼개며, 검토→피드백→재시도의 **반복 교정(iterate)** 으로 결과 품질을 끌어올리는 기법입니다.

좋은 지시는 다음 **네 요소(four elements)** 를 갖춥니다.

| 요소                   | 무엇을 담나               | 빠지면 생기는 문제         |
|----------------------|----------------------|--------------------|
| 목표(goal)             | 무엇을 이루고 싶은가, 왜       | 엉뚱한 결과물을 만든다       |
| 맥락(context)          | 어떤 코드·환경·증상에서        | 존재하지 않는 가정을 한다     |
| 제약(constraints)      | 무엇은 지키고 무엇은 건드리지 말 것 | 의도치 않은 변경이 섞인다     |
| 완료 조건(done criteria) | 무엇으로 "끝났다"를 판단하나     | 검증 없이 "다 됐다"고 보고한다 |

말로만 보면 추상적이니 같은 작업을 두 버전으로 비교해 보겠습니다.

### 나쁜 지시(4요소 결여)

> 로그인 좀 고쳐 줘.

### 좋은 지시(4요소 충족)

> [목표] 로그인 후 빈 화면이 뜨는 버그를 고치고 싶어.  
 > [맥락] auth/login.py 의 handle\_login()에서, 비밀번호가 틀렸을 때  
 >     None을 반환하는데 호출부가 이를 처리하지 않아 빈 화면이 떠.  
 > [제약] 라우팅 구조는 바꾸지 말고, auth/ 폴더 안에서만 수정해 줘.  
 >     새 외부 라이브러리는 추가하지 마.  
 > [완료 조건] 틀린 비밀번호로 로그인하면 "비밀번호가 올바르지 않습니다"  
 >     메시지가 보이고, tests/test\_auth.py 가 모두 통과하면 끝이야.

두 번째 지시는 길어서가 아니라 **빈칸을 남기지 않아서** 좋습니다. 에이전트가 추측할 자리가 없으므로 계획이 곧장 정확해집니다.

**길이보다 빈칸** 좋은 프롬프트의 목표는 "길게 쓰기"가 아니라 "에이전트가 추측해야 할 빈칸을 없애기"입니다. 핵심 제약과 완료 조건 한두 줄이 장황한 배경 설명보다 결과를 더 크게 바꿉니다.

### 3. 동작 원리: 에이전트 루프와 잘못된 가정



왜 4요소가 그렇게 중요한지를 1장의 루프와 연결해 단계로 보겠습니다.

- 지시 입력:** 사람이 작업을 말합니다. 이때 빠진 정보는 그대로 빈칸으로 남습니다.
- 가정으로 빈칸 메우기:** 에이전트는 계획을 세우려면 빈칸을 채워야 하므로, 가장 그럴듯한 값으로 **가정** 합니다. "아마 이 파일이겠지", "아마 전부 다시 쓰라는 뜻이겠지".
- 계획 수립:** 채워진(가정 포함) 정보로 작업 계획을 만듭니다.
- 도구 호출:** 계획대로 파일을 고치고 명령을 실행합니다.
- 결과:** 가정이 맞았으면 좋은 결과, 어긋났으면 성실하게 잘못된 결과가 나옵니다.

여기서 갈림길은 **2번**입니다. 모호한 지시는 잘못된 가정을 부르고, 잘못된 가정은 잘못된 계획과 결과로 이어집니다. 4요소를 갖춘 지시는 2번에서 가정할 거리를 없애므로, 곧장 올바른 계획으로 흐릅니다.

모호한 지시 → (빈칸 많음) → 잘못된 가정 → 잘못된 계획 → 잘못된 결과  
 구체적 지시 → (빈칸 없음) → 가정 불필요 → 올바른 계획 → 올바른 결과

이 그림이 이 장 전체의 뼈대입니다. 우리가 통제할 수 있는 지점은 사실상 **2번을 없애는 것**, 즉 지시 단계에서 빈칸을 메우는 것뿐입니다.

**기억할 한 가지** 에이전트는 거짓말을 하지 않습니다. 다만 빈칸을 가정으로 메울 뿐입니다. 그러므로 "에이전트가 멍청하다"가 아니라 "**내 지시에 빈칸이 있었다**"로 진단하는 습관이 실력을 빠르게 올립니다.

### 4. worked example: 모호한 지시 vs 구체 지시 비교

같은 작업을 모호하게, 그리고 구체적으로 지시했을 때 결과가 어떻게 갈리는지 완전히 풀어 보겠습니다. 작업은 **"외부 결제 게이트웨이 호출에 재시도(retry) 로직 추가"** 입니다. 일시적인 네트워크 오류로 결제 요청이 가끔 실패하니, 안전하게 다시 시도하도록 만들려는 흔한 실무 작업입니다.

## 1차 — 모호한 지시

> 결제 호출에 재시도 좀 넣어 줘.

에이전트는 빈칸(어떤 오류를 재시도할지, 몇 번까지, 영구 실패는 어떻게 다룰지)을 가정으로 메워 다음과 같은 코드를 만들었다고 합니다.

```
# 모호한 지시의 결과 — 모든 예외를 삼키고 영구 실패에서 무한 반복합니다.
def charge(payment):
    while True:
        try:
            return gateway.charge(payment)
        except Exception:
            continue
```

이 코드는 카드 거절처럼 **다시 시도해도 절대 성공하지 않는 영구 오류** 에서도 멈추지 않고 영원히 반복합니다. 게다가 모든 예외를 `except Exception` 으로 삼켜, 무엇이 잘못됐는지조차 드러나지 않습니다.

```
$ python3 -c "from billing import charge; charge(declined_card)"
# (카드가 거절됐지만 종료되지 않고 영원히 재시도 — 프로세스가 멈춘 듯 보입니다)
```

## 2차 — 4요소를 갖춘 구체 지시

```
> [목표] 결제 게이트웨이 호출에 일시적 네트워크 오류를 건디는 재시도 로직을 추가해 줘.
> [맥락] @billing/gateway.py 의 charge()가 가끔 TimeoutError로 실패해.
>       기존 함수들과 같은 스타일(type hint·docstring)로 넣어 줘.
> [제약] TimeoutError·ConnectionError 같은 일시 오류만 재시도하고,
>       카드 거절(CardDeclined)은 재시도 없이 즉시 전파해. 최대 3회,
>       지수 백오프(1초·2초·4초). 외부 라이브러리는 추가하지 마.
> [완료 조건] 3회까지 재시도해도 실패하면 마지막 예외를 다시 던지고,
>       CardDeclined는 호출이 한 번만 일어나야 해. tests/test_gateway.py 가 통과하면 끝이야.
```

그 결과로 만들어진 코드는 다음과 같습니다.

```
# billing/gateway.py — 구체적 지시의 결과
import time

# 다시 시도하면 성공할 수 있는 일시적 오류만 재시도 대상으로 둡니다.
TRANSIENT_ERRORS = (TimeoutError, ConnectionError)

def charge_with_retry(payment: Payment, max_attempts: int = 3) -> ChargeResult:
    """결제를 시도하되 일시적 오류는 지수 백오프로 재시도합니다.

    카드 거절 등 영구 오류는 재시도하지 않고 즉시 전파합니다.
    """
    for attempt in range(1, max_attempts + 1):
        try:
            return gateway.charge(payment)
        except TRANSIENT_ERRORS:
            if attempt == max_attempts:
                raise # 마지막 시도까지 실패하면 마지막 예외를 그대로 전파
            time.sleep(2 ** (attempt - 1)) # 1초 -> 2초 -> 4초 백오프
```

검증 실행 결과는 다음과 같습니다.

```
$ python3 -m pytest tests/test_gateway.py -q
3 passed
```

## 줄별 해설

- **재시도 대상 좁히기(TRANSIENT\_ERRORS):** 제약에 "일시 오류만 재시도"를 못 박자, 1차의 `except Exception` 이 두 일시 오류 튜플로 좁혀졌습니다. 카드 거절은 이 튜플에 없으므로 `except` 에 걸리지 않고 즉시 전파 됩니다. **제약이 곧 안전장치** 입니다.
- **시도 횟수 상한(max\_attempts=3):** "최대 3회"라는 완료 조건이 `for attempt in range(...)` 의 경계가 되어, 1차의 무한 루프가 사라졌습니다.
- **마지막 시도의 raise:** "3회 후에도 실패하면 마지막 예외를 던져"라는 완료 조건이 `if attempt == max_attempts: raise` 로 그대로 옮겨졌습니다. 실패를 조용히 삼키지 않고 호출부에 알립니다.
- **결과 비교:** 같은 도구,같은 작업인데 결과가 갈린 유일한 원인은 **지시의 명확성** 입니다.

## 5. 작업 분해와 단계적 지시(한 번에 하나)

4요소를 갖춰도, 한 번에 너무 많은 것을 요구하면 결과가 흔들립니다. "인증을 새로 만들고, DB를 바꾸고, UI도 갈아엎어 줘"처럼 거대한 지시는 에이전트가 어디서 검증해야 할지 잃게 만듭니다. 해법은 **작업 분해(task decomposition)** 입니다. 큰 일을 **검증 가능한 작은 단계** 로 쪼개, 한 번에 하나씩 지시하고 각 단계의 결과를 확인한 뒤 다음으로 넘어갑니다.

좋은 단계의 조건은 "이 단계가 끝났는지 사람이 곧바로 판단할 수 있는가" 입니다.

| 나쁜 한 방 지시         | 좋은 단계 분해                                                         |
|-------------------|------------------------------------------------------------------|
| "결제 기능 통째로 만들어 줘" | 1) 결제 요청 데이터 모델 정의 → 2) 검증 함수 + 테스트 → 3) 외부 호출 어댑터 → 4) 엔드포인트 연결 |
| "이 코드 리팩터링해 줘"    | 1) 중복 함수 식별 → 2) 공통 함수로 추출 + 테스트 통과 → 3) 호출부 교체 → 4) 죽은 코드 제거    |

각 단계마다 결과를 확인하면, 문제가 생겨도 **어느 단계에서 어긋났는지**가 분명해 교정이 쉽습니다. 한 방에 다 받으면, 무엇이 어디서 틀렸는지 추적하기 어렵습니다.

**플랜 모드와의 연결(미리 보기)** 복잡한 작업을 통째로 맡기고 싶을 때는, 에이전트에게 **먼저 계획만** 세우게 한 뒤 사람이 단계 분해를 검토하는 방법이 있습니다. 이것이 6장에서 배울 **플랜 모드**입니다. 지금은 "큰 일은 사람이 단계로 쪼개 지시한다"는 원칙만 손에 익히면 됩니다.

## 6. @파일 멘션·참조로 맥락 주입하기

1장에서 "코드를 채팅창에 붙여넣을 필요가 없다"고 했습니다. 그 구체적 수단이 **@파일 멘션(@file mention)**입니다. 프롬프트에 **@경로/파일**처럼 적으면, 에이전트가 그 파일을 직접 읽어 맥락으로 삼습니다.

```
> @auth/login.py 와 같은 스타일로 @auth/logout.py 에 로그아웃 함수를 추가해 줘.
> 완료 조건: 세션 토큰을 무효화하고, tests/test_auth.py 가 통과해야 해.
```

붙여넣기 대신 @멘션을 쓰는 이유는 분명합니다.

- **정확성**: 에이전트가 **실제 현재 파일**을 읽습니다. 붙여넣은 사본은 원본과 어긋난 낡은 버전일 수 있습니다.
- **일관성**: "이 파일과 같은 스타일로"라고 가리키면, 기존 코드의 관례(이름·구조·타입 힌트)를 따라갑니다.
- **간결함**: 긴 코드를 프롬프트에 늘어놓지 않아 지시가 읽기 쉽습니다.

폴더를 가리켜 범위를 좁히거나, 참조 문서·URL을 함께 줄 수도 있습니다. 요점은 **"추측하지 말고 실제 자료를 가리켜라"**입니다. 맥락(4요소의 두 번째)을 주입하는 가장 강력한 수단이 바로 @멘션입니다.

## 7. worked example: 3단계(테스트→구현→리팩터) 순차 지시

작업 분해와 4요소, @멘션을 한 흐름에 모아 보겠습니다. 작업은 **"HTTP 쿼리 파라미터에서 페이지네이션 값을 뽑는 함수 추가"**입니다. 목록 API에서 `?page=2&size=50` 같은 파라미터를 안전하게 파싱하는, 웹 백엔드에서 매일 마주치는 작업입니다. **테스트 먼저 → 구현 → 리팩터**의 3단계로 순차 지시합니다.

### 1단계 — 테스트 먼저

- > [목표] `parse_pagination(params)` 함수를 만들 건데, 먼저 테스트부터 작성해 줘.
- > [맥락] `@tests/test_pagination.py` 의 기존 테스트 스타일을 따라 줘.
- > [제약] `page` 기본값 1, `size` 기본값 20, `size` 상한 100이라는 가정으로 테스트를 짜 줘.
- > 아직 구현 코드는 만들지 마.
- > [완료 조건] 빈 `params`는 (1, 20)을 주고, `size=500` 요청은 100으로 깎이는 케이스를 포함해.

그 결과 만들어진 테스트는 다음과 같습니다.

```
# tests/test_pagination.py — 1단계: 구현 전에 먼저 작성한 테스트
from pagination import parse_pagination

def test_defaults_when_params_missing() -> None:
    """파라미터가 비어 있으면 기본값 (page=1, size=20)을 씁니다."""
    assert parse_pagination({}) == (1, 20)

def test_size_is_capped_at_limit() -> None:
    """size가 상한(100)을 넘으면 100으로 깎습니다."""
    assert parse_pagination({"page": "2", "size": "500"}) == (2, 100)
```

## 2단계 — 구현

- > 이제 위 테스트를 통과하는 `parse_pagination(params)`를 `pagination.py` 에 구현해 줘.
- > 완료 조건: `pytest tests/test_pagination.py` 가 모두 통과하면 끝이야.

```
# pagination.py — 2단계: 테스트를 통과하는 최소 구현
def parse_pagination(params: dict[str, str]) -> tuple[int, int]:
    """쿼리 파라미터에서 (page, size)를 뽑되 기본값과 상한을 적용합니다."""
    page = int(params.get("page", 1))
    size = min(int(params.get("size", 20)), 100)
    return page, size
```

```
$ python3 -m pytest tests/test_pagination.py -q
2 passed
```

## 3단계 — 리팩터

- > 동작은 그대로 두고, `page`나 `size`에 0·음수가 들어와도 안전하도록 하한(`page>=1`, `size>=1`)을 적용해 줘.
- > 제약: 기존 테스트는 계속 통과해야 하고, 새 케이스(`page="0" -> 1`, `size="-5" -> 1`) 테스트도 추가해 줘.

## 줄별 해설

- **테스트를 먼저(1단계):** 완료 조건을 코드로 고정해 두면, 구현이 그 기준을 향해 수렴합니다. 각 단계의 "끝났는지"가 `pytest` 통과로 명확해집니다.
- **@tests/test\_pagination.py 멘션:** 기존 테스트 스타일을 실제 파일로 가리켜 일관성을 확보했습니다.
- **단계마다 검증:** 2단계에서 `2 passed` 를 확인한 뒤에야 3단계로 넘어갔습니다. 문제가 생기면 직전 단계에서 멈춰 추적이 쉽습니다.

- **리팩터의 제약("동작은 그대로")**: 3단계에서 "기존 테스트는 계속 통과"라는 제약이 안전망이 되어, 경계 값 처리를 더하다 기존 동작이 깨지는 것을 막습니다.

## 8. 반복 교정(iterate): 검토→피드백→재시도

에이전트 사용의 정석은 **"한 번에 완벽"**이 아니라 **"빠른 반복"**입니다. 첫 결과가 70점이면, 버리고 처음부터 다시 시키는 대신 **무엇이 부족한지 구체적으로 피드백** 해 90점으로 끌어올립니다. 이 검토→피드백→재시도 루프를 **반복 교정(iterate)** 이라고 합니다.

좋은 교정 피드백은 막연한 불만이 아니라 **무엇이·왜·어떻게** 를 담습니다.

| 나쁜 피드백        | 좋은 교정 피드백                                        |
|---------------|--------------------------------------------------|
| "별로야, 다시 해 줘" | "예외 메시지가 영어로 나오는데, 한국어로 바꿔 줘"                    |
| "더 좋게 만들어 줘"  | "함수가 너무 길어. 입력 검증 부분을 별도 함수로 분리해 줘"              |
| "이상해"         | "빈 입력일 때 None을 반환하는데, 완료 조건대로 ValueError를 던져야 해" |

교정할 때도 4요소가 그대로 쓰입니다. 특히 **완료 조건** 을 다시 못 박는 것이 효과적입니다. "이런 경우에 이런 결과가 나오면 통과"라고 기준을 주면, 에이전트가 그 기준을 향해 수렴합니다.

```
> 좋아, 거의 다 됐어. 한 가지만 교정하자.
> [무엇] charge_with_retry가 3회 모두 실패하면 지금 None을 반환하는데
> [왜] 완료 조건은 마지막 시도까지 실패하면 마지막 예외를 다시 던지는 거였어
> [어떻게] 마지막 시도의 except 블록이 raise 하도록 고치고,
> tests/test_gateway.py 에 "3회 모두 실패하면 TimeoutError가 전파된다" 케이스를 추가해 줘
```

**검토 없는 수용**은 금물 에이전트가 "다 됐습니다"라고 보고해도, 사람이 결과를 검토하지 않고 그대로 받아들이면 잘못된 변경이 조용히 섞입니다. 반복 교정의 출발점은 **항상 사람의 검토** 입니다. 검토 없이 통과시키는 습관은 자동완성 시절의 "Tab 연타"와 같은 위험을 에이전트 시대에 그대로 옮겨 옵니다.

## 9. 즉시 연습(2종 1세트): 지시문 4요소 채우기

배운 4요소를 직접 손으로 써 보겠습니다. **모범 → 빈 템플릿 → 자가체크** 순서입니다.

### 연습 1 — 모호한 지시를 4요소로 구체화하기

먼저 모범 답안입니다. "인증 미들웨어에 요청 로깅 추가"를 4요소로 구체화한 예입니다.

```
# 모범: 4요소를 갖춘 지시
> [목표] 인증 미들웨어를 통과하는 모든 요청에 구조화 로그(메서드·경로·사용자 ID)를 남기고 싶어.
> [맥락] @middleware/auth.py 의 authenticate() 끝부분에, 기존 logger 인스턴스와
>     같은 스타일로 로그 한 줄을 추가해 줘.
> [제약] 표준 logging 모듈만 쓰고, 비밀번호·토큰 같은 민감정보는 절대 로그에 넣지 마.
>     인증 로직 자체는 바꾸지 마.
> [완료 조건] 인증 성공 시 'auth ok method=GET path=/orders user=42' 형태의 로그가 남고,
>     tests/test_auth_log.py 가 통과하면 끝이야.
```

## 작성형 빈 템플릿 — 직접 작성하기

아래 모호한 지시를 4요소로 구체화하십시오. 작업은 실무에서 자주 하는 것으로 자유롭게 골라도 됩니다(예: "헬스체크 엔드포인트 추가", "환경 변수 로딩 함수 추가").

원래의 모호한 지시: > 로깅 좀 넣어 줘.

빈칸을 채워 구체화:

```
> [목표] _____
> [맥락] @_____ 와(과) 같은 스타일로 _____
> [제약] _____ 만 쓰고, _____ 는(은) 건드리지 마.
> [완료 조건] _____ 이면 _____ 이고,
>     _____ 테스트가 통과하면 끝.
```

**자가체크 — 스스로 점검(체크리스트)** - [ ] 네 요소(목표·맥락·제약·완료 조건)가 모두 들어갔는가 - [ ] 맥락에 @ 파일 멘션으로 실제 파일을 가리켰는가(붙여넣기 대신) - [ ] 완료 조건이 "구체 값·테스트 통과"처럼 판단 가능한 기준인가 - [ ] "예쁘게·잘"처럼 모호한 형용사 대신 검증 가능한 표현을 썼는가

## 연습 2 — 잘못된 결과를 진단해 교정 지시 쓰기

모범 답안입니다. 에이전트의 결과를 검토하고 구체적으로 교정하는 피드백입니다.

```
# 모범: 무엇·왜·어떻게를 갖춘 교정 피드백
> 거의 됐어. 한 가지 교정하자.
> [무엇] 재시도 함수가 카드 거절(CardDeclined)에도 3회 재시도하고 있어.
> [왜] 완료 조건은 일시 오류(TimeoutError)만 재시도하고, 영구 오류는 즉시 전파하는 거였어.
> [어떻게] except 대상을 TRANSIENT_ERRORS로 좁혀 CardDeclined가 걸리지 않게 하고,
>     tests/test_gateway.py 에 "CardDeclined는 gateway.charge가 1회만 호출된다" 케이스를 추가해 줘.
```

## 작성형 빈 템플릿 — 직접 작성하기

아래 상황을 읽고, **무엇·왜·어떻게** 를 갖춘 교정 지시를 빈칸을 채워 작성하십시오.

상황: "결제 재시도 로직을 추가시켰더니, 카드가 거절돼도 3번 재시도해서 사용자가 응답을 3배 더 기다린다."

```
> [무엇] _____
> [왜] _____
> [어떻게] _____
>     그리고 _____ 테스트도 추가해 줘.
```

**자가체크 — 스스로 점검** - [ ] 막연한 불만("별로다") 대신 "무엇이" 잘못됐는지 짚었는가 - [ ] "왜" 그것이 문제 인지(완료 조건·기대 동작) 근거를 댔는가 - [ ] "어떻게" 교정지 방향을 구체적으로 제시했는가 - [ ] 교정 결과를

확인할 테스트·검증을 요청했는가

## 10. 흔한 오류와 오개념: 과대 요청·모호한 형용사·검증 누락

**오개념 1** — "프롬프트는 길수록 좋다" 아닙니다. 중요한 것은 길이가 아니라 **빈칸을 없앴는가**입니다. 장황한 배경 설명보다 핵심 제약·완료 조건 한두 줄이 결과를 더 크게 바꿉니다. 긴 프롬프트는 오히려 핵심을 묻히게 할 수 있습니다.

**오개념 2** — "코드를 붙여넣어야 정확하다" 반대입니다. 붙여넣은 사본은 원본과 어긋난 낡은 버전일 수 있습니다. @파일 멘션으로 **실제 현재 파일** 을 가리키는 편이 정확하고 일관적입니다.

**오개념 3** — "한 번에 완벽해야 한다" 에이전트 사용의 정석은 한 방이 아니라 **검토→피드백→재시도의 반복** 교정입니다. 70점 결과를 버리지 말고 구체적으로 교정해 90점으로 올리는 것이 더 빠릅니다.

**흔한 오류 A** — **과대 요청(한 방에 다)** "인증·DB·UI를 한꺼번에 바꿔 줘"처럼 거대한 지시는 검증 지점을 잃게 만듭니다. **검증 가능한 단계로 쪼개** 한 번에 하나씩 지시하십시오.

**흔한 오류 B** — **모호한 형용사** "예쁘게", "잘", "깔끔하게"는 판단 기준이 없습니다. "함수 길이 30줄 이하", "테스트 통과", "예외 메시지는 한국어로"처럼 **검증 가능한 표현** 으로 바꾸십시오.

**흔한 오류 C** — **검증 누락** 완료 조건 없이 지시하면 에이전트가 "다 됐다"고 보고해도 그것이 맞는지 알 수 없습니다. "무엇으로 끝났다고 판단할지"를 **반드시 지시에 포함** 하십시오.

## 11. 단원 미니 프로젝트: 첫 세션으로 미니 도구 만들기(2종 1세트)

이제 1장의 **설치와 첫 세션**, 2장의 **효과적 프롬프팅과 작업 지시** 를 한 산출물에 모읍니다. 이 미니 프로젝트는 두 개념만으로 완성하는 작은 도구이며, 뒤에서 만들 종합 프로젝트 repo-audit의 축소판 연습이기도 합니다.

**미니 프로젝트 목표** Claude Code 첫 세션에서, **명확한 단계적 지시만으로** 작은 실행 가능한 CLI 도구와 사용법 README를 만들어 봅니다. 만들 도구는 소스 파일에서 `TODO·FIXME` 주석 수를 세는 `todo-count` 로, 뒤에서 만들 종합 프로젝트 repo-audit의 핵심 점검 항목 하나를 미리 맛보는 셈입니다.

- **산출물:** `todo-count/` 폴더 = `todo_count.py` (파일 경로를 인자로 받아 TODO/FIXME 개수 출력) + `README.md` (사용법)
- **검증:** `python3 todo_count.py app.py` 가 `TODO/FIXME: 3건` 처럼 개수를 출력하고, README만 보고 사용법을 알 수 있으면 성공
- **적용 개념:** c01(첫 세션·REPL·실행 확인) + c02(4요소 지시·단계 분해·반복 교정)

**마일스톤(중분으로 썰기)**

큰 작업을 한 번에 던지지 않고, 2장에서 배운 작업 분해 그대로 세 단계로 썰습니다.

- **M1:** 빈 폴더에서 첫 세션을 열고, 목표·완료 조건을 갖춘 지시로 `todo_count.py` 골격을 생성합니다(c01 첫 세션 + c02 4요소).
- **M2:** 인자 검증(파일이 없을 때)과 출력 형식을 **반복 교정(iterate)** 으로 다듬습니다(c02 검토→피드백→재 시도).
- **M3:** 같은 세션에서 사용법 `README.md` 를 작성하고, 실제 실행으로 확인합니다(c01 결과 확인).

## 모범 — M1 지시(4요소)와 결과

```
# M1 지시(REPL 안)
> [목표] 소스 파일 경로를 인자로 받아 TODO·FIXME 주석 개수를 출력하는 작은 CLI todo_count.py를 만들어 줘.
> [맥락] 빈 폴더야. 표준 라이브러리만 쓰면 돼.
> [제약] 외부 패키지는 쓰지 마. 인자가 없으면 사용법을 안내하고 종료해.
> [완료 조건] python3 todo_count.py app.py → "TODO/FIXME: N건" 형태로 출력,
> 인자가 없으면 "사용법: python3 todo_count.py <파일경로>" 출력.
```

그 결과로 만들어진 스크립트는 다음과 같습니다.

```
# todo_count.py — 파일에서 TODO/FIXME 주석 수를 세는 미니 CLI
import sys

MARKERS = ("TODO", "FIXME")

def count_markers(lines: list[str]) -> int:
    """주어진 줄들에서 TODO·FIXME가 등장하는 줄 수를 셉니다."""
    return sum(1 for line in lines if any(m in line for m in MARKERS))

def main() -> int:
    """파일 경로를 인자로 받아 TODO·FIXME 개수를 출력합니다.

    인자가 없거나 파일이 없으면 안내하고 비정상 종료 코드를 반환합니다.
    """
    if len(sys.argv) < 2:
        print("사용법: python3 todo_count.py <파일경로>")
        return 1
    try:
        with open(sys.argv[1], encoding="utf-8") as f:
            lines = f.readlines()
    except FileNotFoundError:
        print(f"파일을 찾을 수 없습니다: {sys.argv[1]}")
        return 1
    print(f"TODO/FIXME: {count_markers(lines)}건")
    return 0

if __name__ == "__main__":
    raise SystemExit(main())
```

M3에서 같은 세션에 만든 README는 다음과 같습니다.

```
# todo-count

소스 파일에서 TODO·FIXME 주석 수를 세는 작은 CLI입니다.

## 사용법
python3 todo_count.py <파일경로>

## 예시
python3 todo_count.py app.py
-> TODO/FIXME: 3건
```

실행으로 확인한 결과(완료 조건 검증)는 다음과 같습니다.

```
$ python3 todo_count.py app.py
TODO/FIXME: 3건
$ python3 todo_count.py
사용법: python3 todo_count.py <파일경로>
$ echo $?
1
```

- **M1의 완료 조건이 곧 검증 시나리오:** "파일 인자가 있을 때"와 "없을 때" 두 경우를 완료 조건에 박아 두자, 그대로 실행 검증 항목이 되었습니다.
- **반복 교정의 흔적(M2):** 없는 파일 경로를 줬을 때 스택트레이스로 죽지 않고 "파일을 찾을 수 없습니다"를 안내하도록 다듬은 것은, 첫 결과를 검토한 뒤 `FileNotFoundError` 처리를 교정 지시한 결과입니다.
- **종료 코드(`echo $?` → `1`):** 인자가 없거나 파일이 없을 때 비정상 종료(1)를 반환합니다. 이 작은 습관이 13장에서 CI가 성공·실패를 판정하는 기반이 됩니다.

### 작성형 빈 템플릿 — 직접 작성하기

이제 같은 미니 도구를 **여러분의 4요소 지시** 로 직접 만들어 보십시오. 아래 모호한 지시를 4요소로 구체화한 뒤, 첫 세션에서 실행해 완성하십시오.

원래의 모호한 지시: > TODO 세는 거 만들어 줘.

빈칸을 채워 구체화(M1 지시):

```
> [목표] _____
> [맥락] _____
> [제약] _____
> [완료 조건] _____ → "_____" 출력,
> 인자가 없으면 _____ 출력.
```

M2 교정 지시(첫 결과를 검토한 뒤 한 가지 교정):

```
> [무엇] _____ [왜] _____ [어떻게] _____
```

M3 README 작성 지시:

```
> _____
```

**미니 프로젝트 자가체크 — 스스로 점검(체크리스트)** - [ ] M1 지시에 4요소(목표·맥락·제약·완료 조건)가 모두 들어갔는가 - [ ] 큰 작업을 M1→M2→M3 단계로 쪼개 한 번에 하나씩 지시했는가 - [ ] 첫 결과를 검토하고 최소한 번 반복 교정(iterate)했는가 - [ ] `python3 todo_count.py <파일경로>` 가 실제로 실행되어 개수가 나오는가 - [ ] 인자가 없거나 파일이 없을 때 안내하고 종료 코드로 구분되는가 - [ ] README만 보고도 사용법을 알 수 있는가

## 이 장 요약

- **효과적 프롬프팅과 작업 지시:** 에이전트의 결과 품질은 지시의 명확성에 비례합니다. 좋은 지시는 **목표·맥락·제약·완료 조건** 의 4요소로 빈칸을 없앱니다.
- **잘못된 가정 차단:** 모호한 지시는 잘못된 가정 → 잘못된 결과로 이어집니다. 우리가 통제할 지점은 지시 단계에서 빈칸을 메우는 것입니다.
- **작업 분해·@멘션·반복 교정:** 큰 일은 검증 가능한 단계로 쪼개고, @파일 로 실제 코드를 가리키며, 검토→피드백→재시도로 품질을 끌어올립니다.
- **미니 프로젝트:** 두 개념(c01·c02)만으로 실행 가능한 `todo-count` CLI와 README를 첫 세션에서 완성했습니다.

다음 단원에서는 에이전트의 일상 작업인 **파일 탐색과 외과적 편집**(3장), 그리고 그 작업을 안전하게 통제하는 **권한과 허용목록**(4장)으로 나아갑니다. 이 장에서 익힌 "명확한 지시"는 그 모든 작업의 토대가 됩니다.



## 3장 핵심 작업 루프: 파일 탐색과 외과적 편집

이 단원(u2)의 첫 챕터입니다. 이 장에서 배우는 개념은 **파일 탐색과 편집 워크플로우(file navigation and editing workflow, c03)** 한 가지입니다. 앞 단원에서 익힌 효과적 프롬프팅(c02) 위에, Claude Code로 매일 하게 될 가장 기본적이고 빈번한 작업 — 코드베이스를 뒤져 필요한 곳을 찾고, 그 자리만 정확히 고치고, 결과를 실행해 확인하는 일 — 을 본격적으로 다룹니다.

### 이 장을 마치면 할 수 있는 것

- 검색(Glob-Grep)으로 관련 파일을 찾아 읽게 지시할 수 있습니다.
- 외과적 편집(특정 위치만 수정)과 신규 파일 작성을 구분해 요청할 수 있습니다.
- 제시된 diff를 검토하고 수용·수정·되돌리기를 판단할 수 있습니다.
- 큰 변경을 작은 편집 단위로 나눠 안전하게 적용할 수 있습니다.

### 3.1 도입: 탐색 → 편집 → 확인의 일상 루프

앞 단원에서 우리는 목표·맥락·제약·완료 조건을 갖춘 작업 지시를 작성하는 법(c02)을 배웠습니다. 좋은 지시를 줄 수 있게 되었으니, 이제 그 지시를 받은 에이전트가 **실제로 코드를 어떻게 다루는지**를 들여다볼 차례입니다. 한 줄로 요약하면, Claude Code의 하루는 다음 세 박자가 끝없이 반복되는 것입니다.

1. **탐색(navigate)** — "고쳐야 할 코드가 어디 있지?"를 찾는 단계입니다. 파일 이름으로 찾고(Glob), 내용으로 찾고(Grep), 찾은 파일을 읽습니다(Read).
2. **편집(edit)** — 찾은 자리를 바꾸는 단계입니다. 기존 파일의 일부만 외과적으로 고치거나(Edit), 새 파일을 만듭니다(Write).
3. **확인(verify)** — 바꾼 결과가 맞는지 실행해 보는 단계입니다(테스트·실행 명령).

이 루프는 사람이 IDE에서 하는 일과 똑같습니다. 다른 점은, 사람은 머릿속과 손으로 이 박자를 돌리지만 에이전트는 **도구(tool) 호출**로 돌린다는 것입니다. 그래서 우리가 이 도구들의 성격을 이해하면, 에이전트가 왜 그렇게 행동하는지 예측할 수 있고, 더 정확한 지시를 줄 수 있습니다.

#### 왜 이 개념이 먼저인가

탐색·편집은 Claude Code로 하는 거의 모든 작업의 바닥에 깔린 루틴입니다. 버그를 고치든, 기능을 추가하든, 리팩터링을 하든, 결국은 "어디를 찾아 무엇을 어떻게 바꾸느냐"로 환원됩니다. 이 루프를 또렷이 이해하지 못하면, 뒤에 나올 권한(4장)·플랜 모드(6장)·커밋(7장)이 무엇을 통제하고 무엇을 계획하는지가 흐릿해집니다. 그래서 단원 2의 출발점으로 이 개념을 둡니다.

**선수 개념 한 줄 상기(c02).** 좋은 작업 지시는 목표·맥락·제약·완료 조건을 담는다고 배웠습니다. 탐색·편집에서도 이 원칙은 그대로입니다. "로그인 좀 고쳐 줘" 대신 "auth/ 폴더에서 토큰 만료를 처리하는 함수를 찾아, 만료

시 401을 반환하도록 한 군데만 고치고, 기존 테스트가 통과하는지 확인해 줘"처럼 **어디를(맥락) 어떻게(제약) 무엇이 되면 끝인지(완료 조건)**를 주면, 아래에서 볼 탐색·편집 루프가 훨씬 정확하게 돕니다.

## 3.2 코드베이스 탐색: 검색(Glob·Grep)·읽기(Read)

### 정의

**파일 탐색과 편집 워크플로우(c03)**란, 코드베이스를 탐색(Glob·Grep·Read)해 작업 위치를 찾고, 그 위치를 외과적으로 편집(Edit)하거나 새로 작성(Write)한 뒤, 실행해 확인하는 핵심 작업 루프를 말합니다.

탐색은 세 가지 읽기 전용(read-only) 도구로 이뤄집니다. 읽기 전용이라는 말은 **파일을 바꾸지 않고 보기만 한다**는 뜻이고, 그래서 이 도구들은 권한 확인 없이 자유롭게 실행됩니다(권한은 4장에서 다룹니다).

| 도구          | 무엇을 하나                      | 비유               | 언제 쓰나                                    |
|-------------|-----------------------------|------------------|------------------------------------------|
| <b>Glob</b> | 파일 이름·경로 패턴으로 파일 목록을 찾음     | 도서관 서가 번호로 책 찾기  | "*.py" 파일이 어디 있지?", "test_ 로 시작하는 파일 목록" |
| <b>Grep</b> | 파일 <i>내용</i> 에서 문자열·정규식을 검색 | 책 본문에서 단어 찾기(색인) | "calculate_total"이라는 이름이 어디서 쓰이지?"       |
| <b>Read</b> | 특정 파일(또는 일부 줄)을 실제로 읽음      | 찾은 책을 펼쳐 읽기      | 고칠 함수의 본문을 직접 확인                         |

핵심은 순서입니다. 에이전트는 보통 **Glob/Grep으로 후보를 좁히고 → Read로 정확히 읽은 뒤 → 편집**합니다. 사람이 통째로 코드를 채팅창에 붙여넣을 필요가 없는 이유가 여기 있습니다. 에이전트가 직접 저장소를 뒤져 필요한 부분만 읽기 때문입니다.

### Grep은 정규식(ripgrep) 기반입니다

Claude Code의 Grep은 리눅스의 `grep`이 아니라 빠른 검색 엔진 `ripgrep`을 토대로 합니다. 그래서 `calculate.*total`, `def\s+\w+` 같은 정규식이 그대로 통합니다. 지시할 때 "정규식으로 `TODO|FIXME`를 찾아 줘"처럼 패턴을 명확히 주면 탐색이 정확해집니다.

### 탐색을 부르는 지시 예

같은 작업이라도 탐색을 의식한 지시가 결과를 더 좋게 만듭니다. 아래 두 지시를 비교해 봅니다.

- (흐릿함) "결제 코드 어딘가에 버그가 있어. 찾아서 고쳐 줘."
- (탐색을 부름) "src/ 아래에서 `charge_card`를 호출하는 곳을 **Grep으로 찾고**, 그 정의 파일을 **Read로 읽은 뒤**, 금액이 0 이하일 때 예외를 던지지 않는 문제를 한 군데만 고쳐 줘."

두 번째 지시는 에이전트가 어떤 도구로 어디부터 볼지를 안내합니다. 에이전트는 알아서 탐색하지만, 이렇게 탐색의 출발점을 주면 엉뚱한 파일을 헤매는 일이 줄어듭니다.

## 즉시 연습 A — 탐색 지시 설계 (2종 1세트 · 1/2)

다음 상황에서 에이전트가 **탐색부터** 하도록 지시문을 완성해 보세요.

상황: 프로젝트 전체에서 `getUserName` 이라는 함수가 여러 파일에 흩어져 쓰이는데, 반환값이 `None` 일 때 빈 문자열로 바꾸고 싶습니다.

빈 템플릿(채워 보세요):

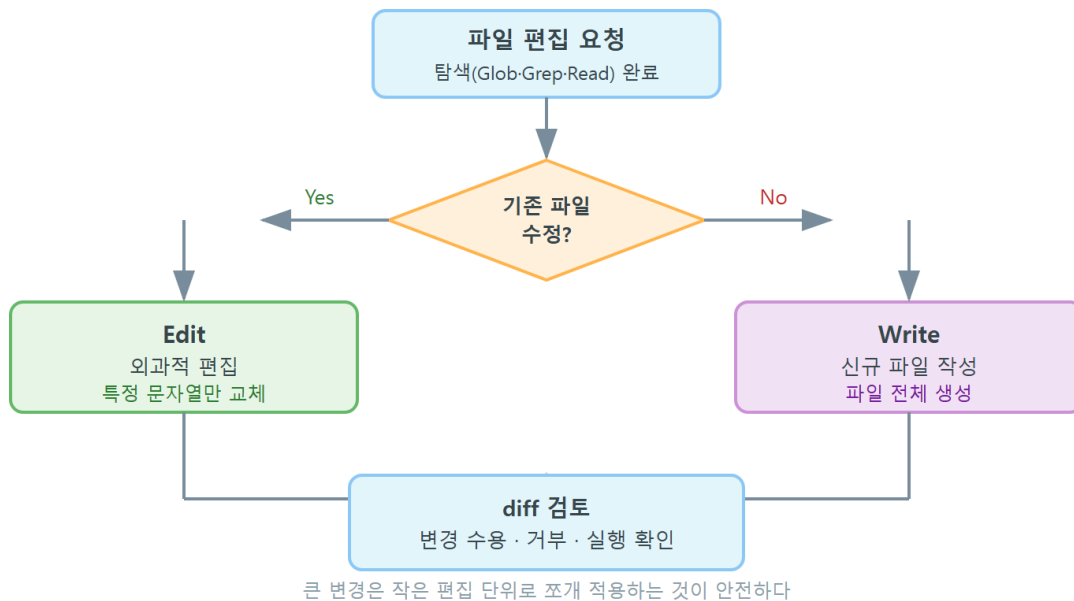
[목표] \_\_\_\_\_  
 [탐색 지시] 먼저 \_\_\_\_\_ 으로(Glob/Grep 중 택1) "getUserName"이 쓰인  
 파일을 모두 찾고, 각 정의를 \_\_\_\_\_ (도구)로 읽어 줘.  
 [제약] 호출부는 건드리지 말고, \_\_\_\_\_ 만 고쳐 줘.  
 [완료 조건] \_\_\_\_\_

자가체크:

- [ ] 탐색 도구(Glob 또는 Grep)를 *내용 검색*에 맞게 골랐는가(이름이 아니라 내용이므로 Grep).
- [ ] "어디를 읽을지"를 Read로 명시했는가.
- [ ] 제약(호출부는 건드리지 않음)과 완료 조건(예: 기존 테스트 통과)을 적었는가.

## 3.3 동작 원리: 외과적 편집(Edit) vs 신규 작성(Write)

### 외과적 편집(Edit) vs 신규 작성(Write) 분기 흐름



탐색으로 자리를 찾았다면, 이제 바꿀 차례입니다. 여기서 가장 중요한 구분이 **외과적 편집(Edit)**과 **신규 작성(Write)**입니다.

### 두 도구의 동작

**Edit**는 기존 파일에서 **정확히 일치하는 문자열(old\_string)**을 **새 문자열(new\_string)**로 바꾸는 도구입니다. 외과의가 환부만 도려내듯, 파일의 나머지는 한 글자도 건드리지 않고 지정한 조각만 교체합니다. 동작 규칙은 다음과 같습니다.

1. 바꾸려는 파일을 **먼저 읽어야** 합니다(Read). 읽지 않은 파일은 편집할 수 없습니다 — 무엇이 들어 있는지 모르고 바꾸는 것을 막기 위함입니다.
2. `old_string`은 파일 안에서 **유일(unique)**해야 합니다. 같은 문자열이 두 군데 있으면 어디를 바꿀지 알 수 없으므로, 앞뒤 맥락을 더 포함해 유일하게 만들거나 "모두 바꾸기(replace\_all)"를 명시합니다.
3. `old_string`은 들여쓰기·공백까지 파일과 **정확히** 같아야 합니다.

**Write**는 **파일 전체를 새로 쓰는**(없으면 생성, 있으면 통째로 덮어쓰기) 도구입니다. 새 파일을 만들 때 자연스럽게, 빈 모듈·설정 파일·README를 만들 때 씁니다.

## 언제 Edit, 언제 Write인가

| 상황               | 권장 도구                    | 이유                        |
|------------------|--------------------------|---------------------------|
| 기존 함수 한 줄 수정     | <b>Edit</b>              | 나머지를 보존, 의도치 않은 변경 차단     |
| 기존 파일에 함수 하나 추가  | <b>Edit</b>              | 추가 위치만 삽입, diff가 작아 검토 쉬움 |
| 변수 이름을 여러 곳에서 통일 | <b>Edit(replace_all)</b> | 같은 문자열을 일괄 교체             |
| 새 모듈·새 설정 파일 생성  | <b>Write</b>             | 애초에 빈 캔버스라 통째로 작성이 자연스러움  |
| 기존 파일을 처음부터 다시 씀 | (가급적 피함)                 | 전체 덮어쓰기는 검토가 어렵고 위험       |

**원칙: 기존 파일은 Edit, 새 파일은 Write.** 이미 존재하는 파일을 Write로 통째로 덮어쓰는 것은, 잘 작동하던 부분까지 다시 쓰는 것이라 오류가 섞이기 쉽고 diff도 거대해져 검토가 어렵습니다. 이 원칙 하나가 에이전트 협업의 안전성을 크게 좌우합니다.

### 외과적 편집이 안전한 이유

100줄짜리 파일에서 한 줄을 고친다고 합시다. Edit는 그 한 줄만 바꾸므로 diff에 한 줄만 빨강고 한 줄만 파랗게 뜹니다. 검토자는 1초면 맞는지 봅니다. 반면 Write로 100줄을 다시 쓰면, 에이전트가 99줄을 똑같이 옮겨 적는 과정에서 공백·주석·순서가 미묘하게 달라질 수 있고, diff는 100줄이 통째로 바뀐 것처럼 보여 무엇이 진짜 변경인지 가려집니다. "작게 바꾸고 작게 검토한다"가 핵심입니다.

## 3.4 worked example: 함수 이름을 전체에서 일관되게 바꾸기

탐색 → Edit(replace\_all) → 확인 루프를 끝까지 따라가 봅니다. 주문 서비스 모듈에서 함수 이름 `amt`가 너무 모호해, 의미가 분명한 `order_subtotal`로 통일하려 합니다. 코드 리뷰에서 "이름이 뭘 계산하는지 모르겠다"는 지적을 받은, 혼한 정리 작업입니다.

## 출발 상태

다음과 같은 `order_service.py` 가 있다고 합시다.

```
# order_service.py — 변경 전
def amt(items):
    # 주문 항목의 (단가 x 수량)을 모두 더해 합계를 반환합니다.
    total = 0
    for line in items:
        total += line["price"] * line["qty"]
    return total

def build_invoice(items):
    # 합계를 포함한 송장 데이터를 만듭니다.
    subtotal = amt(items) # 여기서 amt 호출
    return {"subtotal": subtotal, "currency": "KRW"}

if __name__ == "__main__":
    cart = [{"price": 1200, "qty": 2}, {"price": 3400, "qty": 1}]
    print(build_invoice(cart))
    # amt를 직접 쓰는 또 다른 자리(빠른 점검)
    print("검증:", amt([{"price": 10, "qty": 3}]))
```

`amt` 는 정의 1곳, 호출 2곳, 총 3군데에 등장합니다. 이름을 한 군데만 바꾸면 나머지가 깨지므로 **일관되게** 바뀌어야 합니다. 실제 프로젝트에서는 이런 호출이 여러 파일에 흩어져 있어, 탐색으로 전부 찾는 일이 더 중요해집니다.

## 에이전트에게 준 지시

```
order_service.py에서 함수 amt를 order_subtotal로 이름을 바꿔 줘.
1) 먼저 Grep으로 amt가 쓰인 자리를 모두 찾고,
2) Read로 파일을 읽은 뒤,
3) 정의와 호출을 빠짐없이 일관되게 바꿔 줘(replace_all).
끝나면 python order_service.py 를 실행해 결과가 그대로인지 확인해 줘.
```

## 에이전트가 보여 준 diff

```
# order_service.py — 변경 후
-def amt(items):
+def order_subtotal(items):
    # 주문 항목의 (단가 x 수량)을 모두 더해 합계를 반환합니다.
    total = 0
    for line in items:
        total += line["price"] * line["qty"]
    return total

def build_invoice(items):
    # 합계를 포함한 송장 데이터를 만듭니다.
- subtotal = amt(items) # 여기서 amt 호출
+ subtotal = order_subtotal(items) # 여기서 order_subtotal 호출
    return {"subtotal": subtotal, "currency": "KRW"}

if __name__ == "__main__":
    cart = [{"price": 1200, "qty": 2}, {"price": 3400, "qty": 1}]
    print(build_invoice(cart))
    # amt를 직접 쓰는 또 다른 자리(빠른 점검)
- print("검증:", amt([{"price": 10, "qty": 3}]))
+ print("검증:", order_subtotal([{"price": 10, "qty": 3}]))
```

## 줄별 해설

- `-def amt(items):` / `+def order_subtotal(items):` — 정의를 바꿉니다. 함수 이름이 바뀌는 지점입니다.
- `- subtotal = amt(items)` / `+ subtotal = order_subtotal(items)` — 첫 번째 호출부입니다. 정의만 바꾸고 여기를 빠뜨리면 `NameError` 가 납니다.
- `- print("검증:", amt(...))` / `+ ... order_subtotal(...)` — 세 번째 등장(두 번째 호출)입니다. `replace_all` 로 지시했기에 흩어진 자리까지 한 번에 잡혔습니다.
- 주석 안의 단어 `amt` 는 의미상 설명이라 이름 식별자만 바꾸는 것이 원칙이지만, 여기서는 호출 옆 주석이 자연스럽게 같이 갱신되었습니다. 식별자가 아닌 문서/주석의 단어까지 바꿀지는 **검토 단계에서 사람이 판단**합니다.

## 실행 결과

```
$ python order_service.py
{'subtotal': 5800, 'currency': 'KRW'}
검증: 30
```

이름만 바꾸고 동작은 그대로입니다. 만약 정의만 바꾸고 호출 한 군데를 놓쳤다면 다음처럼 깨졌을 것입니다.

```
$ python order_service.py
Traceback (most recent call last):
...
NameError: name 'amt' is not defined
```

이 대비가 보여 주듯, 이름 바꾸기 같은 작업은 **탐색으로 모든 등장 자리를 먼저 확보**한 뒤 일괄 편집하는 것이 핵심입니다. 한 군데라도 빠지면 코드가 깨지기 때문입니다.

## 3.5 diff 검토와 변경 수용·거부

Edit-Write가 만든 변경은 곧바로 디스크에 반영되는 것이 아니라, 먼저 **diff(차이)** 로 제시됩니다. diff는 무엇이 빠지고(-) 무엇이 더해지는지(+)를 보여 주는 변경 미리보기입니다. 여기서 사람의 역할은 세 갈래입니다.

1. **수용(accept)** — 변경이 맞으면 적용합니다.
2. **수정(refine)** — "여기 변수명은 그대로 두고 위에 한 줄만 추가해 줘"처럼 교정 지시를 줘 다시 받습니다.
3. **거부(reject)** — 방향이 틀렸으면 적용하지 않고 되돌립니다.

diff를 읽는 작은 습관 몇 가지를 정리합니다.

- **변경 줄 수를 먼저 본다.** 한 줄 고쳐 달라 했는데 30줄이 바뀌면, 의도보다 많은 일이 벌어진 신호입니다.
- **- (삭제)된 줄을 의심한다.** 무엇이 사라지는지가 가장 위험합니다. 멀쩡하던 코드가 삭제되고 있지는 않은지 봅니다.
- **범위 밖 파일이 바뀌는지 본다.** "이 함수만"이라 했는데 다른 파일까지 diff에 나오면 멈춰 세웁니다.

### 무검토 일괄 수용의 위험

diff를 보지 않고 모든 변경을 자동 수용하는 습관은, 의도치 않은 변경(주석 삭제, 포맷 변형, 인접 코드 수정)이 조용히 섞여 들어오는 가장 흔한 경로입니다. 특히 변경이 여러 파일에 걸칠수록 위험이 커집니다. "빨리"보다 "diff 한 번 보기"가 결국 더 빠릅니다 — 나중에 원인 모를 버그를 추적하는 시간을 아끼기 때문입니다.

### 즉시 연습 B — diff 판정 (2종 1세트 · 모범 + 작성)

아래 diff를 보고 수용/수정/거부 중 무엇을 할지, 그 이유를 적어 보세요. 지시는 "configure\_logging 에서 기본 로그 레벨을 INFO에서 WARNING으로만 바꿔 줘"였습니다.

```
def configure_logging():
- logging.basicConfig(level=logging.INFO)
+ logging.basicConfig(level=logging.WARNING, format="%asctime)s %(message)s")
-
-def add_file_handler(path):
- logging.getLogger().addHandler(logging.FileHandler(path))
```

판정(채워 보세요):

```
판정: _____ (수용 / 수정 / 거부)
이유: _____
교정 지시(필요하면): _____
```

모범 답안(접어 두었다 비교하세요): 판정은 **거부 또는 수정**입니다. 지시는 로그 레벨만 바꾸는 것이었는데, diff는 (1) 요청하지 않은 `format` 인자를 추가했고 (2) 무관한 `add_file_handler` 함수를 통째로 **삭제**하고 있습니다. 교정 지시 예: "basicConfig 의 level 만 `logging.WARNING` 으로 바꿔 줘. format 추가나 `add_file_handler` 삭제는 하지 마."

자가체크:

- `[]` 삭제(-)되는 줄이 의도한 것인지 확인했는가.

- [ ] 지시 범위 밖 변경(다른 함수 삭제·요청 안 한 인자 추가)을 잡아냈는가.
- [ ] 거부에 그치지 않고 *어떻게 고칠지* 교정 지시를 적었는가.

### 3.6 worked example: 버그 한 줄을 외과적으로 고치고 실행 확인

이번에는 탐색으로 버그 자리를 찾고, **딱 한 줄만** Edit로 고친 뒤, 테스트로 확인하는 전형적인 버그픽스 루프를 따라가 봅시다.

#### 버그가 있는 코드

pagination.py 는 목록 API에서 page (1부터)와 size 로 DB 조회 OFFSET 을 계산합니다. 그런데 첫 페이지의 데이터를 건너뛰는 off-by-one 버그가 있습니다.

```
# pagination.py — 변경 전
def page_offset(page, size):
    # page(1부터)와 size로 DB 조회 OFFSET을 계산합니다.
    return page * size          # 버그: page가 1부터인데 그냥 곱해 첫 페이지를 건너뛴

def test_page_offset():
    # 1페이지는 OFFSET 0이어야 합니다.
    assert page_offset(1, 20) == 0    # 현재는 20이 나와 실패
    # 2페이지는 OFFSET 20.
    assert page_offset(2, 20) == 20
    # 3페이지는 OFFSET 40.
    assert page_offset(3, 20) == 40

if __name__ == "__main__":
    test_page_offset()
    print("모든 테스트 통과")
```

#### 지시와 탐색

pagination.py의 page\_offset에 버그가 있어.  
 page가 1부터인데 page \* size로 계산해서 1페이지가 OFFSET 20이 돼(첫 페이지 데이터를 건너뛴).  
 test\_page\_offset의 첫 단언이 통과하도록,  
 OFFSET 계산식을 한 줄만 외과적으로 고쳐 줘.  
 고친 뒤 python pagination.py 로 테스트를 실행해 확인해 줘.

에이전트는 먼저 파일을 Read로 읽고, OFFSET 계산이 page를 0부터로 환산하지 않는다는 것을 확인한 뒤, return 한 줄만 바꾸는 Edit를 제안합니다.

#### 제시된 diff

```
def page_offset(page, size):
    # page(1부터)와 size로 DB 조회 OFFSET을 계산합니다.
-   return page * size      # 버그: page가 1부터인데 그냥 곱해 첫 페이지를 건너뛴
+   return (page - 1) * size # page를 0부터로 환산해 OFFSET 계산
```

### 줄별 해설

- 함수 시그니처(`def page_offset(page, size):`)와 주석은 **그대로**입니다. 인터페이스는 옳으므로 건드릴 이유가 없습니다. 외과적 편집의 핵심은 "멀쩡한 곳은 손대지 않는 것"입니다.
- `- return page * size` — 1페이지에서 OFFSET 20 을 내던 off-by-one 버그의 자리입니다.
- `+ return (page - 1) * size` — `page - 1` 로 1부터인 페이지를 0부터로 환산합니다. 단 한 줄, 단 한 군데만 바꿉니다. diff가 두 줄뿐이라 검토가 즉시 끝납니다.

### 실행 결과

```
$ python pagination.py
모든 테스트 통과
```

세 단언이 모두 통과합니다. `page_offset(1, 20)` 은 이제  $(1 - 1) * 20$  즉 0 을 반환합니다. **한 줄 버그는 한 줄로 고친다** — 이 절제가 변경을 검토 가능하게 만들고, 회귀(regression)를 줄입니다.

### 고치기 전에 "실패하는 확인"을 먼저 두면 좋습니다

위 예에서 `test_page_offset` 의 첫 단언은 버그를 *드러내*는 장치였습니다. 고치기 전에는 실패하고, 고친 뒤에는 통과합니다. 이렇게 "무엇이 되면 끝인지"를 실행 가능한 형태(테스트·실행 명령)로 먼저 주면, 에이전트는 그 기준을 향해 작업하고 사람은 통과 여부로 검증할 수 있습니다. 2장에서 배운 "완료 조건"을 코드로 표현한 셈입니다.

## 3.7 다중 파일 변경과 일관성·되돌리기

변경이 한 파일을 넘어 여러 파일에 걸치면 난이도가 한 단계 올라갑니다. 예를 들어 새 유틸 모듈을 만들고, 기존 파일이 그것을 import하도록 두 파일을 동시에 고쳐야 할 때가 있습니다. 이때 지켜야 할 원칙은 **일관성(consistency)**과 **되돌릴 수 있음(reversibility)**입니다.

- **일관성**: 한 파일에서 함수 시그니처를 바꿨다면, 그것을 호출하는 *모든* 파일이 같이 바뀌어야 합니다. 한 곳만 바뀌면 코드가 깨집니다(3.4의 이름 바꾸기와 같은 원리).
- **되돌릴 수 있음**: 여러 파일을 한꺼번에 바꿨는데 결과가 틀렸다면, 어디까지 되돌려야 할지 알아야 합니다. 그래서 **작업을 작은 커밋 단위로 끊는 습관**이 중요합니다(7장에서 본격적으로 다룹니다). 변경 전 상태가 git에 저장돼 있으면 언제든지 안전하게 되돌릴 수 있습니다.

### 다중 파일 변경의 권장 순서

1. **탐색으로 영향 범위를 먼저 확정한다.** "이 함수를 바꾸면 어느 파일들이 영향을 받나?"를 Grep으로 모두 찾습니다.

2. **변경을 작은 단위로 쪼갬다.** 모듈 추가 → import 추가 → 호출 교체처럼 단계로 나눕니다.
3. **각 단계마다 실행·확인한다.** 한 번에 다 바꾸고 마지막에 한 번 돌리기보다, 작은 단계마다 확인하면 어느 단계에서 깨졌는지 즉시 압니다.

## 작은 두-파일 예: 유틸 분리

예를 들어 `order_service.py`의 합계 계산 로직을 여러 모듈이 공유할 수 있도록 새 모듈 `pricing.py`로 옮기고, 원래 파일은 그것을 import해 쓰도록 바꾼다고 합시다. 이때 변경은 한 파일이 아니라 두 파일에 걸칩니다.

```
# pricing.py — Write로 새로 생성(신규 파일)
def order_subtotal(items):
    # 주문 항목의 (단가 x 수량) 합계를 반환합니다.
    return sum(line["price"] * line["qty"] for line in items)
```

```
# order_service.py — Edit로 외과적 수정(기존 파일)
+from pricing import order_subtotal
+
def build_invoice(items):
-     subtotal = 0
-     for line in items:
-         subtotal += line["price"] * line["qty"]
+     subtotal = order_subtotal(items)
    return {"subtotal": subtotal, "currency": "KRW"}
```

여기서 두 도구의 역할 분담이 뚜렷합니다. 새 파일 `pricing.py`는 **Write**로 만들고(빈 캔버스), 기존 `order_service.py`는 **Edit**로 import 한 줄을 더하고 중복 로직을 호출로 바꿉니다. 그리고 변경 직후 `python order_service.py`를 실행해 송장 합계가 그대로 나오는지 확인합니다. import를 빠뜨리면 `NameError`가, 호출만 바꾸고 정의를 안 옮기면 동작이 달라지므로, **두 파일의 일관성**이 깨지지 않았는지 실행으로 검증하는 것이 핵심입니다.

### 되돌리기의 전제: 깨끗한 시작점

변경을 시작하기 전에 작업 디렉터리가 깨끗한지(커밋되지 않은 변경이 없는지) 확인하는 습관을 들이세요. 시작점이 커밋돼 있으면 무엇이 잘못돼도 `git`으로 한 번에 되돌릴 수 있습니다. 반대로 시작점이 지저분하면, 에이전트의 변경과 기존의 미커밋 변경이 뒤섞여 무엇을 되돌려야 할지 분간하기 어려워집니다.

## 3.8 작은 단위 편집 원칙과 결과 실행·확인

지금까지 본 모든 예제를 관통하는 한 가지 원칙이 있습니다. **작게 바꾸고, 바로 확인한다.**

- **작게:** 한 번에 한 가지 변경. 이름 바꾸기와 버그 수정과 리팩터링을 한 덩어리로 시키지 않고, 하나씩 끊어 시킵니다. 그래야 diff가 작고, 검토가 쉽고, 문제가 생겼을 때 원인이 분명합니다.
- **확인:** 바꿨으면 실행합니다. 테스트가 있으면 테스트를, 없으면 직접 실행해 출력이 의도대로인지 봅니다. "고쳤습니다"는 실행 결과로 뒷받침될 때만 믿을 수 있습니다.

이 원칙은 에이전트에게 주는 **지시 방식**에도 반영됩니다. "이 모듈을 전부 리팩터링하고 테스트도 새로 짜고 문서도 갱신해 줘"라는 큰 한 방보다, "①먼저 이 함수만 분리해 줘 → (확인) → ②이제 호출부를 바꿔 줘 → (확인) → ③테스트를 추가해 줘"처럼 단계를 끊어 주는 편이 결과가 안정적입니다. 2장에서 배운 **작업 분해**가 여기서 실전으로 이어집니다.

| 안티패턴                    | 무엇이 문제인가                   | 더 나은 방식           |
|-------------------------|----------------------------|-------------------|
| 한 번에 큰 변경을 통째로 요청       | diff가 거대해 검토 불가, 깨지면 원인 불명 | 작은 단계로 쪼개 단계마다 확인 |
| 기존 파일을 Write로 통째로 덮어 쓰기 | 멀쩡한 코드까지 다시 써 회귀 위험        | 필요한 곳만 Edit       |
| 고친 뒤 실행하지 않음            | 동작 보장 없음                   | 테스트·실행으로 확인       |
| diff를 안 보고 전부 수용        | 의도치 않은 변경 유입               | 삭제 줄·범위 밖 변경 점검   |

### 3.9 즉시 연습(2종 1세트): 탐색 → 편집 시나리오 채우기

#### 즉시 연습 C — 탐색 → 편집 → 확인 시나리오 (2종 1세트)

다음 버그 스크립트가 있습니다. 이벤트 로그에서 가장 최근 타임스탬프를 뽑는 함수가 빈 목록에서 에러를 냅니다.

```
# events.py
def latest_event_ts(events):
    # 이벤트 목록에서 가장 최근(최댓값) 타임스탬프를 반환합니다. 빈 목록이면 None이어야 합니다.
    return max(events)          # 빈 목록에서 ValueError 발생

def test_latest_event_ts():
    assert latest_event_ts([1717000000, 1717100000, 1717050000]) == 1717100000
    assert latest_event_ts([]) is None      # 현재 실패
```

#### 모범 흐름(읽고 참고)

- 탐색: `latest_event_ts` 정의 위치를 확인하고 Read로 읽는다.
- 편집: `return max(events)` 한 줄을 빈 목록을 처리하도록 외과적으로 고친다.
- 확인: `python events.py` 로 테스트 통과를 본다.

**빈 템플릿(직접 작성)** — 에이전트에게 줄 지시문을 4요소로 채워 보세요.

```
[목표] _____
[탐색] 먼저 _____ (도구)로 events.py의 latest_event_ts를 읽어 줘.
[편집·제약] return 줄만 외과적으로 고쳐 줘.
              _____(어떻게 고칠지 조건)
[완료 조건] python events.py 실행 시 _____
```

이어서, 당신이 기대하는 diff를 직접 적어 보세요(-/+ 표기).

```
def latest_event_ts(events):
    # 이벤트 목록에서 가장 최근(최댓값) 타임스탬프를 반환합니다. 빈 목록이면 None이어야 합니다.
    -
    +
```

자가체크:

- [ ] 탐색(Read)을 편집보다 먼저 두었는가.
- [ ] 한 줄만 바꾸는 외과적 편집으로 한정했는가(전체 재작성 아님).
- [ ] 완료 조건을 **실행 가능한 형태**(테스트 통과)로 적었는가.
- [ ] 내가 적은 diff에서 삭제되는 줄이 의도한 그 한 줄뿐인가.

참고 모범 diff(스스로 풀어 본 뒤 비교하세요): `- return max(events) → + return max(events) if events else None`. 한 줄, 한 군데만 바꾸고 빈 목록은 `None`을 반환합니다.

### 3.10 흔한 오류와 오개념: 전체 재작성·일괄 변경·무검토 수용

#### 오개념 1 — "에이전트가 파일을 통째로 다시 써 준다"

(틀림) 에이전트가 항상 파일 전체를 새로 작성한다고 생각하면, 거대한 diff를 당연하게 받아들이게 됩니다. (교정) 기존 파일은 **외과적 편집(Edit)**으로 필요한 부분만 바꾸는 것이 기본입니다. 전체를 다시 쓰는 Write는 **새 파일**에 어울립니다. diff가 예상보다 크면 "이거 왜 이렇게 많이 바뀌었지?"라고 멈춰 물어야 합니다.

#### 오개념 2 — "큰 변경 한 방이 효율적이다"

(틀림) 리팩터링·버그픽스·테스트 추가를 한 번에 시키면 빠를 것 같습니다. (교정) 큰 일괄 변경은 검토가 불가능하고, 깨졌을 때 어느 부분이 원인인지 가려집니다. **작은 편집을 검토하며 쌓는** 편이 결국 더 빠르고 안전합니다. 작업을 단계로 쪼개 지시하세요(2장 작업 분해의 실전).

#### 오개념 3 — "diff는 형식적이니 그냥 다 수용하면 된다"

(틀림) diff를 보지 않고 전부 수용하는 습관. (교정) diff는 마지막 안전장치입니다. 특히 **삭제되는 줄과 범위 밖 파일 변경**을 살피세요. 무검토 수용은 주석 삭제·포맷 변형·인접 코드 훼손 같은 조용한 변경이 새어 드는 가장 흔한 경로입니다.

#### 오개념 4 — "고쳤다고 하면 고쳐진 것이다"

(틀림) 에이전트가 "수정 완료"라고 말하면 그대로 믿는 것. (교정) 변경은 **실행으로 확인**될 때만 끝난 것입니다. 테스트를 돌리거나 직접 실행해 출력이 의도대로인지 보세요. 완료 조건을 실행 가능한 형태로 준 뒤, 그 결과로 검증하는 것이 정석입니다.

## 3.11 마무리: 다음 장으로

---

이 장에서 우리는 Claude Code의 일상인 **탐색(Glob·Grep·Read) → 편집(Edit·Write) → 확인** 루프를 배웠습니다. 핵심은 세 가지였습니다. 첫째, 탐색으로 작업 자리를 먼저 확보합니다. 둘째, 기존 파일은 외과적 편집으로 필요한 곳만 바꾸고 새 파일만 Write로 만듭니다. 셋째, diff를 검토하고 실행으로 확인하며, 큰 변경은 작은 단위로 쪼갭니다.

그런데 한 가지 빠진 질문이 있습니다. **편집(Edit)과 실행(Bash)은 파일을 바꾸고 명령을 돌리는 "부수효과"가 있는 작업인데, 에이전트가 이것을 마음대로 해도 될까요?** 다음 장(4장)에서는 바로 이 부수효과 도구를 안전하게 통제하는 **권한과 도구 허용목록(allowlist)**을 배웁니다. 이 장에서 본 Edit·Write·Bash가 *언제 사용자에게 확인을 구하는지*, 그리고 어떻게 신뢰하는 작업만 자동 승인하고 위험한 작업을 막는지를 다룹니다. 그리고 4장 끝에서는 이 두 개념(탐색·편집 + 권한)을 합쳐 **안전한 리팩터링 세션**을 구성하는 단위 미니 프로젝트로 이어집니다.



## 4장 통제: 권한과 도구 허용목록(allowlist)

이 단원(u2)의 두 번째이자 마지막 챕터입니다. 이 장에서 배우는 개념은 **권한과 도구 허용목록 (permissions and tool allowlist, c04)** 한 가지입니다. 앞 장에서 익힌 파일 탐색·편집(c03) 위에, 그 편집·실행이 **언제 사용자에게 확인을 구하고, 어떻게 신뢰하는 작업만 자동 승인하며 위험한 작업을 막는**지를 다룹니다. 장 끝에서는 두 개념(탐색·편집 + 권한)을 합쳐 안전한 리팩터링 세션을 구성하는 단원 미니 프로젝트로 마무리합니다.

### 이 장을 마치면 할 수 있는 것

- 권한 확인이 언제 발생하는지(부수효과 도구) 설명할 수 있습니다.
- `settings.json`의 `allow-deny-ask` 규칙으로 도구를 통제할 수 있습니다.
- 도구·명령 패턴(예: `Bash(git:*)`)으로 세밀한 허용을 작성할 수 있습니다.
- 권한 범위(프로젝트·사용자·로컬)를 구분해 적용할 수 있습니다.

### 4.1 도입: 편집·실행에는 권한이 따른다

앞 장에서 우리는 에이전트가 파일을 읽고(Read), 찾고(Glob·Grep), 고치고(Edit), 만들고(Write), 실행(Bash)하는 루프를 배웠습니다. 그런데 이 도구들은 성격이 둘로 갈립니다.

- **읽기만 하는 도구** — Read·Glob·Grep은 파일을 *보기만* 합니다. 아무리 많이 실행해도 코드베이스가 바뀌지 않습니다.
- **세상을 바꾸는 도구** — Edit·Write는 파일을 바꾸고, Bash는 명령을 실행합니다. 이들은 **부수효과(side effect)**가 있습니다. 파일이 지워질 수도, 패키지가 설치될 수도, 원격 저장소에 푸시될 수도 있습니다.

여기서 자연스러운 질문이 나옵니다. **에이전트가 파일을 지우거나 명령을 실행하는 일을 마음대로 해도 될까요?** 대답은 "아니오, 통제가 필요하다"입니다. Claude Code는 부수효과가 있는 도구를 호출할 때 기본적으로 **사용자에게 확인을 구합니다**(권한 확인). 하지만 매번 모든 것을 일일이 승인하면 마찰이 너무 큼니다. 그래서 신뢰할 수 있는 작업은 **허용목록(allowlist)**으로 자동 승인하고, 위험한 작업은 **차단목록(denylist)**으로 막아, 안전성과 생산성의 균형을 설계합니다.

#### 왜 이 개념이 탐색·편집 바로 다음인가

편집과 실행을 할 줄 알게 된 순간, "이 강력한 도구를 어떻게 안전하게 풀어 줄 것인가"가 곧바로 따라옵니다. 권한을 모르면 둘 중 하나에 빠집니다. 매번 모든 확인을 일일이 누르며 지치거나(생산성 0), 반대로 전부 자동 승인해 버려 사고에 노출됩니다(안전성 0). 권한 설계는 이 둘 사이에서 **꼭 필요한 만큼만 신뢰를 위임**하는 기술입니다.

**선수 개념 한 줄 상기(c03).** 앞 장에서 본 Edit·Write·Bash가 바로 권한 확인의 대상입니다. 이 장은 그 도구들을 어떻게 풀어 주고 막을지를 다루므로, 탐색·편집 루프를 이해하고 있어야 권한 규칙이 무엇을 통제하는지 또렷이 보입니다.

## 4.2 부수효과 도구와 읽기 전용 도구

### 정의

**권한과 도구 허용목록(c04)**이란, 부수효과가 있는 도구(Edit·Write·Bash)의 호출을 `settings.json`의 `allow·deny·ask` 규칙과 명령 패턴으로 통제해, 최소 권한 원칙으로 안전성과 생산성의 균형을 설계하는 체계를 말합니다.

도구를 두 부류로 나누면 권한이 한눈에 들어옵니다.

| 분류    | 도구 예             | 부수효과      | 기본 동작               |
|-------|------------------|-----------|---------------------|
| 읽기 전용 | Read, Glob, Grep | 없음(보기만 함) | 확인 없이 자유롭게 실행       |
| 부수효과  | Edit, Write      | 파일을 바꿈    | 기본은 확인을 구함(설정으로 조정) |
| 부수효과  | Bash             | 임의 명령 실행  | 명령마다 확인을 구함         |

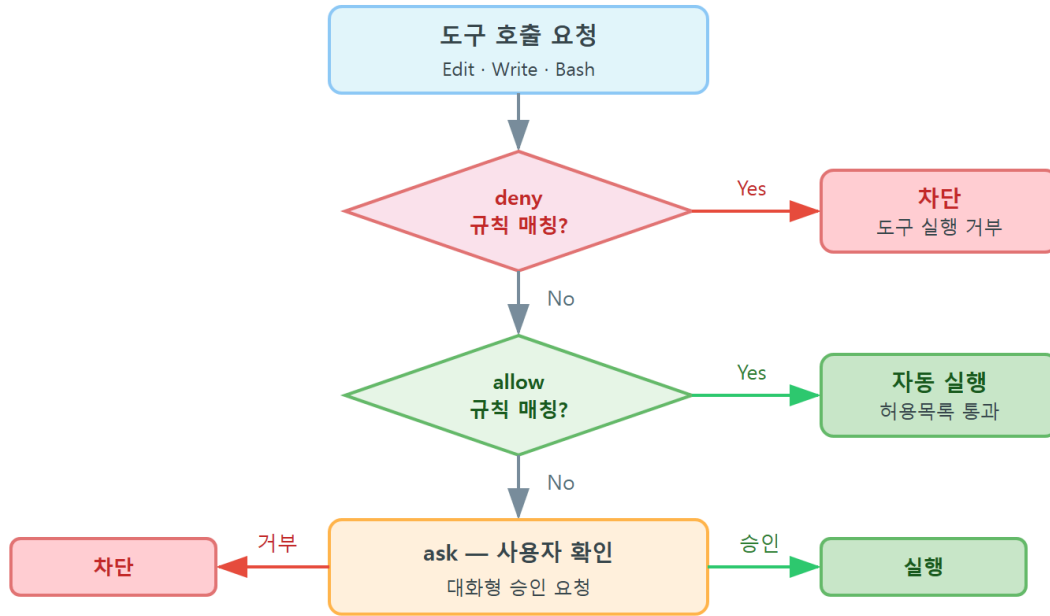
읽기 전용 도구가 확인 없이 실행되는 이유는 단순합니다. 보기만 해서는 무엇도 망가지지 않기 때문입니다. 반면 Bash는 가장 강력하고 가장 위험합니다. `git status` 처럼 무해한 것부터 `rm -rf` 처럼 파괴적인 것까지 무엇이든 될 수 있어서, **명령 단위로** 통제해야 합니다.

### "무엇을 허용하느냐"가 곧 신뢰 경계입니다

권한 설정은 결국 "에이전트에게 어디까지 손대게 둘 것인가"라는 신뢰 경계를 코드로 적는 일입니다. 읽기는 전부 풀어 줘도 안전하지만, 쓰기·실행은 무엇을 풀어 줄지 신중히 골라야 합니다. 이 경계를 잘 그으면 에이전트는 반복 확인 없이 빠르게 일하면서도, 선을 넘는 행동은 막힙니다.

## 4.3 동작 원리: allow·deny·ask 판정 흐름

## 권한 판정 흐름 — allow · deny · ask



판정 순서: deny 먼저 확인 → allow 확인 → 없으면 ask(대화형)

에이전트가 부수효과 도구를 호출하려 할 때, Claude Code는 `settings.json`의 권한 규칙을 정해진 순서로 비취 보고 결과를 정합니다. 규칙은 세 종류입니다.

- **deny** — 이 도구·명령은 **절대 허용하지 않습니다**(차단).
- **ask** — 이 도구·명령은 **매번 사용자에게 묻습니다**(확인).
- **allow** — 이 도구·명령은 **확인 없이 자동 승인합니다**(허용).

## 판정 순서: deny → ask → allow

핵심은 규칙이 비취지는 **순서**입니다. Claude Code는 다음 순서로 평가하며, **먼저 일치하는 규칙이 결과를 정합니다**(규칙의 구체성이 순서를 바꾸지는 않습니다).

1. 먼저 **deny** 목록과 맞춰 봅니다. 일치하면 → **차단**(끝).
2. 일치가 없으면 **ask** 목록과 맞춰 봅니다. 일치하면 → **사용자에게 확인**.
3. 그래도 일치가 없으면 **allow** 목록과 맞춰 봅니다. 일치하면 → **자동 승인**.
4. 어디에도 없으면 → 기본 동작(보통 사용자에게 확인)을 따릅니다.

이 순서가 주는 중요한 성질이 하나 있습니다. **deny가 가장 강합니다**. 어떤 명령을 allow에도 넣고 deny에도 넣었다면, deny가 먼저 평가되므로 차단됩니다. 그래서 "이건 절대 안 돼"라는 규칙은 deny에 두면 다른 허용 규칙이 있어도 안전하게 막힙니다.

| 단계 | 비교 대상    | 일치 시 결과    |
|----|----------|------------|
| 1  | deny 규칙  | 차단(즉시 종료)  |
| 2  | ask 규칙   | 사용자 확인 요청  |
| 3  | allow 규칙 | 자동 승인      |
| 4  | (없음)     | 기본값(대개 확인) |

### 왜 deny를 먼저 보는가

안전 시스템의 황금률은 "막는 규칙을 가장 먼저 본다"입니다. 만약 allow를 먼저 봤다면, 넓은 허용 규칙 (Bash(git:\*)) 때문에 위험한 명령이 새어 나갈 수 있습니다. deny를 맨 앞에 두면, 넓게 허용하더라도 "이것만은 절대"라는 빨간 선을 확실히 지킬 수 있습니다. 즉 **넓게 허용하되 위험한 것은 명시적으로 차단**하는 안전한 기본 구도가 가능해집니다.

## 4.4 worked example: 최소 권한 settings.json 작성

이제 실무에서 그대로 쓸 settings.json 을 작성해 봅시다. 상황은 이렇습니다. **결제 API 서비스 저장소**에서 팀 전체가 Claude Code로 일상 작업을 합니다. 목표는 테스트·린트·타입 검사처럼 하루에도 수십 번 도는 안전한 명령은 자동 승인해 마찰을 없애되, 되돌리기 어렵거나 파괴적인 명령은 차단하는 **팀 공유용 균형 설정**입니다. 권한 규칙은 permissions 객체 아래 allow · ask · deny 배열에 적습니다.

### 작성한 설정

```
{
  "permissions": {
    "allow": [
      "Read",
      "Edit",
      "Bash(git status:*)",
      "Bash(git diff:*)",
      "Bash(pytest:*)",
      "Bash(ruff:*)",
      "Bash(mypy:*)"
    ],
    "ask": [
      "Bash(git push:*)",
      "Bash(alembic upgrade:*)"
    ],
    "deny": [
      "Bash(rm:*)",
      "Bash(git push --force:*)",
      "Edit(./env)",
      "Read(./env)",
      "Read(./secrets/**)"
    ]
  }
}
```

## 줄별 해설

- "permissions" — 모든 권한 규칙이 들어가는 최상위 키입니다.
- "allow": [ "Read", "Edit", ... ] — 확인 없이 자동 승인할 목록입니다. Read 와 Edit 은 일상 작업이라 풀어 주고, Bash는 안전한 명령만 꼭 집어 허용합니다.
- "Bash(pytest:\*)", "Bash(ruff:\*)", "Bash(mypy:\*)" — 테스트·린트·타입 검사는 개발 루프와 CI에서 매일 수십 번 도는 안전한 검증 명령이라 자동 승인합니다. 앞 장에서 강조한 "고친 뒤 바로 검증"을 마찰 없이 하기 위함입니다.
- "Bash(git status:\*)", "Bash(git diff:\*)" — 상태·차이 확인은 읽기에 가까운 안전한 명령입니다. :\* 는 그 뒤에 어떤 인자가 붙어도 된다는 와일드카드라, git status, git status -s 가 모두 매칭됩니다. (패턴 문법은 4.5에서 자세히 봅니다.)
- "ask": [ "Bash(git push:\*)", "Bash(alembic upgrade:\*)" ] — 원격 푸시와 DB 마이그레이션(alembic upgrade)은 영향이 크고 되돌리기 어려워, 자동 승인하지 않고 매번 묻습니다. 프로덕션 데이터에 닿는 명령은 사람이 한 번 더 확인합니다.
- "deny": [ "Bash(rm:\*)", "Bash(git push --force:\*)", ... ] — 파일 삭제(rm)와 강제 푸시는 절대 차단합니다. deny는 가장 먼저 평가되므로 allow에 넣은 규칙이 있어도 안전합니다.
- "Edit(./env)", "Read(./env)", "Read(./secrets/\*\*)" — 자격 증명이 든 .env·secrets 는 읽기뿐 아니라 편집까지 막습니다. 비밀이 모델 컨텍스트로 새거나, 에이전트가 환경 변수 파일을 실수로 덮어서 서비스가 깨지는 일을 함께 차단합니다.

## 설정 적용 후 동작 예

이 설정을 둔 세션에서 에이전트가 여러 도구를 호출하면 다음처럼 갑니다.

|                                         |                                 |
|-----------------------------------------|---------------------------------|
| 에이전트가 Edit로 app/services/pricing.py 수정  | → 자동 승인 (allow)                 |
| 에이전트가 "pytest -q" 실행                    | → 자동 승인 (allow: Bash(pytest:*)) |
| 에이전트가 "ruff check app" 실행               | → 자동 승인 (allow: Bash(ruff:*))   |
| 에이전트가 "alembic upgrade head" 실행         | → 확인 요청 (ask)                   |
| 에이전트가 "git push --force origin main" 실행 | → 차단 (deny)                     |
| 에이전트가 ".env" 편집 시도                      | → 차단 (deny: Edit(./env))        |

같은 세션 안에서도 도구·명령에 따라 자동 승인·확인·차단이 정확히 나뉩니다. 이것이 **최소 권한(least privilege)** 설계입니다 — 일에 꼭 필요한 만큼만 허용하고, 나머지는 묻거나 막습니다.

**allow에 그냥 Bash를 통째로 넣지 마세요**

"allow": ["Bash"] 처럼 도구 이름만 넣으면 **모든 셸 명령이** 자동 승인됩니다 — `rm -rf /` 까지요. 이는 안전 경계를 통째로 없애는 것과 같습니다. Bash는 반드시 `Bash(명령:*)` 형태로 **명령을 좁혀** 허용하세요. 도구 이름만 허용하는 것과 명령 패턴까지 좁히는 것의 차이가 사고를 가릅니다.

## 4.5 도구·명령 패턴 매칭(Bash(git:\*) 등)

권한 규칙의 힘은 **패턴**에서 나옵니다. 규칙은 두 단계로 좁힐 수 있습니다.

1. **도구 단위** — `Read`, `Edit`, `Write` 처럼 도구 이름만 적으면 그 도구의 **모든** 호출에 적용됩니다.
2. **명령·인자 단위** — `Bash(git:*)` 처럼 괄호 안에 명령 패턴을 적으면, 그 도구 중에서도 **특정 명령만** 골라 적용됩니다.

### 패턴 문법 읽기

`Bash(...)` 안의 패턴은 와일드카드 `*`를 씁니다. `*`는 임의의 문자열을 뜻하며, 패턴의 여러 위치에 올 수 있습니다.

| 패턴                                    | 매칭되는 명령 예                                                                         | 매칭 안 되는 예                                    |
|---------------------------------------|-----------------------------------------------------------------------------------|----------------------------------------------|
| <code>Bash(git:*)</code>              | <code>git status</code> , <code>git add .</code> , <code>git commit -m "x"</code> | <code>npm install</code> (git 아님)            |
| <code>Bash(git commit:*)</code>       | <code>git commit -m "feat: x"</code>                                              | <code>git push</code> (commit 아님)            |
| <code>Bash(npm test:*)</code>         | <code>npm test</code> , <code>npm test -- --watch</code>                          | <code>npm install</code>                     |
| <code>Bash(git push --force:*)</code> | <code>git push --force origin main</code>                                         | <code>git push origin main</code> (force 아님) |

핵심 감각은 **"얼마나 좁게 허용할 것인가"**입니다.

- `Bash(git:*)` 는 **모든 git 명령**을 허용합니다 — 편하지만 `git push --force`, `git reset --hard` 처럼 위험한 git 명령까지 들어옵니다.

- `Bash(git commit:*)` + `Bash(git status:*)` + `Bash(git diff:*)` 처럼 **안전한 git 하위 명령만** 골라 허용하면 더 안전합니다.
- 넓게 허용해야 한다면, 위험한 하위 명령은 **deny로 따로 막습니다**(deny가 먼저 평가되므로). 예: allow에 `Bash(git:*)`, deny에 `Bash(git push --force:*)`.

### "넓게 allow + 꼭 집어 deny" 패턴

실무에서 자주 쓰는 균형점은 자주 쓰는 도구는 넓게 풀되(`Bash(git:*)`), 돌이킬 수 없는 소수 명령만 deny로 못 박는 것입니다. deny가 allow보다 먼저 평가되기 때문에, 이 조합은 "편의는 유지하면서 빨간 선만 확실히 지키는" 효과를 냅니다. 다만 무엇이 위험한지(force push-hard reset-rm)를 정확히 알아야 deny 목록을 제대로 채울 수 있습니다.

### 즉시 연습 A — 패턴 좁히기 (2종 1세트 · 1/2)

다음 요구를 만족하도록 패턴을 채워 보세요.

요구: `npm` 명령 중 `test` 와 `run build` 만 자동 허용하고, `npm install` 은 매번 묻고, `npm publish` 는 절대 차단한다.

빈 템플릿:

```
{
  "permissions": {
    "allow": [
      "Bash(_____:*)",
      "Bash(_____:*)"
    ],
    "ask": [
      "Bash(_____:*)"
    ],
    "deny": [
      "Bash(_____:*)"
    ]
  }
}
```

자가체크:

- [ ] allow에는 `npm test` 와 `npm run build` 만 들어갔는가(`npm:*` 처럼 통째로 넣지 않았는가).
- [ ] `npm install` 을 ask에 두어 매번 확인하게 했는가.
- [ ] `npm publish` 를 deny에 두어 차단했는가.
- [ ] 도구 이름만(`Bash`)이 아니라 명령 패턴까지 좁혔는가.

참고 모범 답(스스로 채운 뒤 비교): allow는 `Bash(npm test:*)`, `Bash(npm run build:*)`, ask는 `Bash(npm install:*)`, deny는 `Bash(npm publish:*)`.

## 4.6 권한 범위: 프로젝트·사용자·로컬

권한 규칙을 *어디에 두느냐*에 따라 적용 범위와 공유 여부가 달라집니다. `settings.json` 은 세 곳에 둘 수 있습니다.

| 범위            | 파일 위치                                    | 적용 대상            | 버전관리(공유)          |
|---------------|------------------------------------------|------------------|-------------------|
| 사용자(user)     | <code>~/.claude/settings.json</code>     | 내 모든 프로젝트        | 개인용(공유 안 됨)       |
| 프로젝트(project) | <code>.claude/settings.json</code>       | 그 저장소를 쓰는 모두     | 커밋해 팀과 공유         |
| 로컬(local)     | <code>.claude/settings.local.json</code> | 그 저장소의 <i>나만</i> | gitignore(공유 안 됨) |

이 구분은 실무에서 매우 중요합니다.

- **팀 규칙은 프로젝트 설정에 둡니다.** `.claude/settings.json` 을 저장소에 커밋하면, 그 저장소를 클론한 모든 팀원이 같은 권한 경계를 공유합니다. "우리 프로젝트에서는 force push를 막는다" 같은 정책이 모두에게 적용됩니다.
- **개인 선호는 로컬 설정에 둡니다.** `.claude/settings.local.json` 은 git에 올라가지 않으므로, "나는 이 프로젝트에서 python 실행을 자동 승인하고 싶다" 같은 개인 편의를 다른 사람에게 강요하지 않고 둘 수 있습니다.
- **모든 프로젝트 공통 규칙은 사용자 설정에 둡니다.** `~/.claude/settings.json` 은 내 모든 작업에 적용됩니다. "어디서든 rm 은 항상 deny" 같은 안전 기본값을 두기 좋습니다.

#### 로컬 설정의 오남용

로컬 설정(`settings.local.json`)은 공유되지 않으므로 편리하지만, 여기에 위험한 자동 승인을 몰래 쌓으면 *팀의 안전 경계 밖에서* 사고가 날 수 있습니다. 예컨대 팀 프로젝트 설정은 force push를 막는데 내 로컬에서 풀어 버리면, 나만 위험에 노출됩니다(그리고 그 사실이 코드 리뷰에 드러나지 않습니다). 팀이 함께 지켜야 할 규칙은 반드시 **프로젝트 설정**에 두고, 로컬은 무해한 개인 편의에만 쓰세요.

## 4.7 worked example: 위험 명령 deny로 사고 예방

이번에는 "사고를 예방하는" 관점에서 deny 규칙을 설계해 봅니다. 시나리오는 이렇습니다. 한 팀이 Claude Code로 리팩터링을 자주 하는데, 과거에 에이전트가 정리 작업 중 빌드 산출물 폴더를 지우다 실수로 소스까지 날린 적이 있습니다. 또 누군가 force push로 동료의 커밋을 덮어쓴 사고도 있었습니다. 이 두 사고를 **구조적으로 불가능하게** 만드는 것이 목표입니다.

### 사고 분석과 대응 규칙

| 과거 사고      | 위험 명령                         | 대응 deny 규칙                                                               |
|------------|-------------------------------|--------------------------------------------------------------------------|
| 정리 중 소스 삭제 | <code>rm -rf ...</code>       | <code>Bash(rm:*)</code>                                                  |
| 동료 커밋 덮어쓰기 | <code>git push --force</code> | <code>Bash(git push --force:*)</code> , <code>Bash(git push -f:*)</code> |
| 히스토리 파괴    | <code>git reset --hard</code> | <code>Bash(git reset --hard:*)</code>                                    |
| 비밀 유출      | <code>.env</code> 읽기          | <code>Read(./env)</code>                                                 |
| 환경 파일 손상   | <code>.env</code> 덮어쓰기        | <code>Edit(./env)</code>                                                 |

## 작성한 프로젝트 설정

이 규칙들은 모든 팀원에게 적용되어야 하므로 `.claude/settings.json` (프로젝트 범위)에 둡니다.

```
{
  "permissions": {
    "allow": [
      "Read",
      "Edit",
      "Bash(git status:*)",
      "Bash(git add:*)",
      "Bash(git commit:*)",
      "Bash(git diff:*)",
      "Bash(pytest:*)",
      "Bash(ruff:*)"
    ],
    "deny": [
      "Bash(rm:*)",
      "Bash(git push --force:*)",
      "Bash(git push -f:*)",
      "Bash(git reset --hard:*)",
      "Edit(./env)",
      "Read(./env)",
      "Read(./secrets/**)"
    ]
  }
}
```

## 설계 해설

- allow에는 되돌릴 수 있는 일상 작업만 담았습니다. `git add`·`git commit`은 로컬 작업이라 안전하고 되돌리기 쉽습니다. `push`는 일부러 allow에서 빼서 매번 확인하게 했습니다(되돌리기 어려운 행동).
- deny에는 되돌릴 수 없거나 파괴적인 명령을 담았습니다. `rm`은 통째로 막아 어떤 삭제도 자동으로 일어나지 않게 했습니다. `force push`는 `--force`와 축약형 `-f` 둘 다 막아야 빠져나갈 구멍이 없습니다.
- deny가 allow보다 먼저 평가되므로, 설령 누군가 allow에 `Bash(git:*)`를 추가하더라도 `git push --force`는 여전히 차단됩니다. 이것이 deny를 "안전 그물"로 쓰는 방식입니다.
- 민감 파일 읽기까지 막아, 비밀이 컨텍스트로 새는 것을 차단했습니다.

## 검증: 차단이 실제로 동작하는가

```
$ # 에이전트에게 "build 폴더를 정리해 줘"라고 시켰을 때
에이전트가 "rm -rf build/" 실행 시도 → 차단됨 (deny: Bash(rm:*))
└ 에이전트는 대신 사용자에게 "rm은 차단돼 있어 직접 지우시거나
  허용해 주셔야 한다"고 보고하고 멈춤

$ # 에이전트가 강제 푸시를 제안했을 때
에이전트가 "git push -f origin main" 실행 시도 → 차단됨 (deny: Bash(git push -f:*))
```

규칙이 사고 시나리오를 정확히 막는 것을 확인했습니다. 위험을 사람의 주의력에 맡기지 않고 설정으로 구조화한 것입니다 — 피곤하거나 급할 때도 빨간 선은 항상 지켜집니다.

## 4.8 자동 승인(권한 위임)과 그 위험

Claude Code에는 권한 확인을 광범위하게 건너뛰는 모드도 있습니다. 편집을 일괄 자동 승인하는 모드나, 모든 권한 확인을 통째로 우회하는 옵션(흔히 "전체 bypass"로 불리는 `--dangerously-skip-permissions`)이 그것입니다. 이름에 "dangerously(위험하게)"가 붙은 것은 우연이 아닙니다.

자동 승인을 넓게 켜면 마찰이 사라져 빠르지만, **안전 경계도 함께 사라집니다**. 에이전트의 모든 편집·명령이 확인 없이 실행되므로, 잘못된 판단 하나가 곧바로 파일 삭제·원격 푸시·비밀 노출로 이어질 수 있습니다.

| 모드              | 마찰 | 안전성   | 적합한 상황              |
|-----------------|----|-------|---------------------|
| 매번 확인(기본)       | 높음 | 높음    | 처음 쓰는 저장소, 고위험 작업   |
| allowlist 자동 승인 | 중간 | 중간~높음 | 신뢰 명령을 좁게 허용한 일상 작업 |
| 편집 일괄 자동 승인     | 낮음 | 중간    | 변경이 작고 되돌리기 쉬운 작업   |
| 전체 bypass       | 없음 | 낮음    | 격리된 샌드박스/일회용 컨테이너만  |

**권장 원칙: 전체 bypass는 격리된 환경에서만.** 네트워크가 차단되고 되돌리기 쉬운 일회용 컨테이너·VM처럼 "망가져도 버리면 그만"인 곳이 아니라면, 전체 bypass는 켜지 마세요. 실제 작업 머신에서는 **allowlist로 신뢰 명령만 좁게 자동 승인**하는 편이, 빠르면서도 빨간 선을 지키는 최선입니다.

**자동화일수록 권한을 더 좁혀야 합니다**

사람이 지켜보는 대화형 세션에서는 실수해도 즉시 멈출 수 있습니다. 하지만 뒤 단원에서 배울 무인 자동화(CI·스케줄, 13장)에서는 *아무도 보고 있지 않습니다*. 그래서 자동화 환경일수록 권한을 더 좁히고 deny를 더 촘촘히 깔아야 합니다. 이 장의 최소 권한 원칙은 단지 지금의 편의가 아니라, 나중에 자동화를 안전하게 만드는 토대입니다.

## 4.9 즉시 연습(2종 1세트): allow·deny 규칙 채우기

즉시 연습 B — 균형 잡힌 settings.json 작성 (2종 1세트)

상황: 결제 API 서비스 저장소이고, 팀이 함께 씁니다. 다음을 만족하는 **팀 공유용** 프로젝트 권한 설정을 작성하세요.

요구사항:

1. 파일 읽기·편집은 자동 승인한다.
2. `pytest` 테스트와 `ruff` 린트는 자동 승인한다(CI에서 매일 도는 안전한 검증).
3. `git status`·`git add`·`git commit` 은 자동 승인하되, `git push` 는 매번 묻는다.
4. `rm` 과 `git push --force` 는 절대 차단한다.
5. `.env` 파일은 읽기·편집 모두 차단한다(자격 증명 보호).

**모범(읽고 참고)** — 4.4.4.7의 설정이 이 요구의 모범입니다. 구조를 떠올린 뒤 아래 빈 템플릿을 직접 채워 보세요.

**빈 템플릿(직접 작성)**

```
{
  "permissions": {
    "allow": [
      "_____",
      "_____",
      "Bash(_____:*)",
      "Bash(_____:*)",
      "Bash(git status:*)",
      "Bash(_____:*)",
      "Bash(_____:*)"
    ],
    "ask": [
      "Bash(_____:*)"
    ],
    "deny": [
      "Bash(_____:*)",
      "Bash(_____:*)",
      "Edit(_____) ",
      "Read(_____) "
    ]
  }
}
```

이 설정은 어느 파일(범위)에 두어야 팀 전체에 적용될지도 한 줄로 적어 보세요.

파일 위치: \_\_\_\_\_ (이유: \_\_\_\_\_)

자가체크:

- [ ] Bash는 도구 이름만이 아니라 **명령 패턴**까지 좁혔는가.
- [ ] `git push` 를 ask에 두어 자동 승인하지 않았는가.
- [ ] `rm` ·`git push --force` 를 deny에 넣었는가(되돌릴 수 없는 명령).
- [ ] 민감 파일(`.env`) 읽기·편집을 모두 deny에 넣었는가.
- [ ] 팀 공유가 목적이므로 `.claude/settings.json` (프로젝트 범위)에 두기로 했는가.

## 4.10 흔한 오류와 오개념: 전체 bypass·이름만 허용·로컬 오남용

### 오개념 1 — "전부 자동 승인(bypass)하면 편하고 좋다"

(틀림) 권한 확인이 귀찮으니 전체 bypass를 켜 두는 것. (교정) 전체 bypass는 안전 경계를 통째로 없앱니다. 잘못된 판단 하나가 곧장 파일 삭제·force push·비밀 노출로 이어집니다. 기본은 **최소 권한**이고, 빠르게 가고 싶다면 **신뢰하는 명령만 좁게** allowlist에 넣으세요. 전체 bypass는 망가져도 되는 격리 환경에서만 씁니다.

### 오개념 2 — "도구 이름만 허용하면 충분하다"

(틀림) "allow": ["Bash"] 처럼 도구 이름만 넣는 것. (교정) 그러면 `rm -rf` 를 포함한 모든 셸 명령이 자동 승인됩니다. 허용목록은 도구 이름뿐 아니라 **명령 패턴까지 좁혀야** (`Bash(git commit:*)`) 안전합니다. "무엇을" 허용하는지가 핵심입니다.

### 오개념 3 — "로컬 설정에 팀 규칙을 뒤도 된다"

(틀림) 안전 규칙을 `settings.local.json` 에 두는 것. (교정) 로컬 설정은 git에 올라가지 않아 **공유되지 않습니다**. 팀이 함께 지켜야 할 규칙(force push 차단 등)은 반드시 **프로젝트 설정** (`.claude/settings.json`)에 두어 커밋하세요. 로컬은 무해한 개인 편의에만 씁니다.

### 오개념 4 — "deny와 allow에 같이 넣으면 헛갈린다"

(틀림) 위험 명령을 막으려면 allow에서 빼기만 하면 된다고 생각하는 것. (교정) allow에서 빼면 "문기"가 되지만, 누군가 나중에 넣은 규칙(`Bash(git:*)`)을 allow에 추가하면 다시 새어 나갑니다. **절대 안 되는 것은 deny에 명시하세요**. deny가 가장 먼저 평가되므로, allow에 무엇이 있든 확실히 차단됩니다.

## 4.11 단원 미니 프로젝트: 안전한 리팩터링 세션 구성

이제 단원 2의 두 개념 — 탐색·편집(c03)과 권한·allowlist(c04) — 을 하나로 합쳐 작은 산출물을 만듭니다. 목표는 **운영 중인 서비스의 작은 버그를 탐색·편집으로 고치되, 필요한 검증 명령만 자동 허용하는 settings.json을 함께 두어 안전한 세션을 구성하는 것**입니다. 실무에서 핫픽스를 다룰 때, "고치는 손"과 "막는 가드"를 한 세션에 함께 준비하는 연습입니다.

### 산출물

서비스 저장소에 두 가지를 준비합니다.

1. `app/services/metrics.py` — 신규 매장 대시보드를 깨뜨리던 버그를 외과적으로 고친 수정본.
2. `.claude/settings.json` — 테스트(`pytest`)·린트(`ruff`)만 자동 허용하고 `rm .env` 편집을 차단하는 최소 권한 설정.

완료 기준: 수정 후 단위 테스트가 통과하고, 위험 명령은 차단되며, 필요한 검증 명령만 자동 승인됩니다.

## 마일스톤(증분)

| 마일스톤 | 할 일                                              | 쓰는 개념                     |
|------|--------------------------------------------------|---------------------------|
| M1   | Grep·Read로 버그 위치를 탐색하고, diff를 검토하며 한 줄을 외과적으로 수정 | c03(탐색·외과적 편집·diff 검토)    |
| M2   | pytest·ruff 만 자동 허용하는 .claude/settings.json 작성   | c04(allow 규칙·명령 패턴)       |
| M3   | 위험 명령( rm·.env 편집)을 deny로 막고, 테스트로 수정본을 검증       | c04(deny·범위) + c03(실행 확인) |

## 모범 진행(읽고 참고)

**M1 — 탐색과 외과적 수정.** 다음 app/services/metrics.py 에서 주문 평균 금액(AOV) 계산이 주문이 없는 신규 매장에서 깨집니다. 대시보드가 500 에러를 내고 있어 핫픽스가 필요합니다.

```
# app/services/metrics.py — 변경 전
def average_order_value(orders: list[float]) -> float:
    # 주문 평균 금액(AOV)을 반환합니다. 주문이 없으면 0.0이어야 합니다(대시보드용).
    return sum(orders) / len(orders) # 신규 매장(주문 0건)에서 ZeroDivisionError

def test_average_order_value() -> None:
    assert average_order_value([12000.0, 8000.0, 10000.0]) == 10000.0
    assert average_order_value([]) == 0.0 # 현재 실패: 신규 매장 대시보드가 깨짐
```

지시 예: " app/services/metrics.py 에서 average\_order\_value 를 Read로 읽고, 주문이 없을 때 0.0을 반환하도록 return 한 줄만 외과적으로 고쳐 줘. 끝나면 pytest app/services/test\_metrics.py -q 로 확인해 줘." 에이전트가 제시하는 diff는 다음과 같습니다.

```
def average_order_value(orders: list[float]) -> float:
    # 주문 평균 금액(AOV)을 반환합니다. 주문이 없으면 0.0이어야 합니다(대시보드용).
-   return sum(orders) / len(orders) # 신규 매장(주문 0건)에서 ZeroDivisionError
+   return sum(orders) / len(orders) if orders else 0.0
```

**M2·M3 — 최소 권한 설정.** 이 세션에서 자동 승인할 것은 검증 명령(테스트·린트)뿐이고, 삭제와 환경 파일 손상은 막습니다.

```
{
  "permissions": {
    "allow": [
      "Read",
      "Edit",
      "Bash(pytest:*)",
      "Bash(ruff:*)"
    ],
    "deny": [
      "Bash(rm:*)",
      "Edit(./env)"
    ]
  }
}
```

### 검증(실행 확인)

```
$ pytest app/services/test_metrics.py -q
2 passed in 0.08s
```

수정본이 통과하고, 세션 중 어떤 `rm` 명령이나 `.env` 편집도 자동으로 실행될 수 없습니다.

## 학생 작성형 빈 스타터(직접 완성)

### 미니 프로젝트 — 직접 만들기 (2종 1세트)

아래 빈 스타터를 채워 안전한 리팩터링 세션을 구성하세요.

1단계 — 알림 서비스에 다음 버그가 있습니다. 수신자가 없는 캠페인에서 대표 수신자를 뽑다가 워커가 죽습니다. 에이전트에게 줄 **탐색·편집 지시문**을 작성하세요.

```
# app/services/notify.py — 직접 고칠 대상
def primary_recipient(emails: list[str]) -> str:
    # 알림 수신자 중 대표 1명을 반환합니다. 수신자가 없으면 ""이어야 합니다.
    return emails[0] # 수신자 목록이 빈 캠페인에서 IndexError
```

```
[목표] _____
[탐색] 먼저 _____ (도구)로 primary_recipient를 읽어 줘.
[편집·제약] return 줄만 외과적으로 고쳐 줘. _____
[완료 조건] pytest app/services/test_notify.py 실행 시 _____
```

2단계 — 이 세션의 최소 권한 설정을 채우세요(`pytest` 테스트만 자동 허용, `rm` 차단).

```
{
  "permissions": {
    "allow": [
      "_____",
      "Bash(_____:*)"
    ],
    "deny": [
      "Bash(_____:*)"
    ]
  }
}
```

3단계 — 기대하는 수정 diff를 적어 보세요.

```
def primary_recipient(emails: list[str]) -> str:
    # 알림 수신자 중 대표 1명을 반환합니다. 수신자가 없으면 ""이어야 합니다.
-     _____
+     _____
```

자가체크:

- [ ] 탐색(Read)을 편집보다 먼저 두었는가.
- [ ] 한 줄만 바꾸는 외과적 편집으로 한정했는가.
- [ ] 필요한 명령( `pytest` )만 자동 허용했는가(통째 `Bash` 아님).
- [ ] 위험 명령( `rm` )을 `deny`로 차단했는가.
- [ ] 완료 조건을 실행 가능한 형태로 적고, 실제로 테스트를 돌려 통과를 확인했는가.

참고 모범 diff: `- return emails[0] → + return emails[0] if emails else ""`. 한 줄, 한 군데만 바뀌고 수신자가 없으면 빈 문자열 `""`를 반환합니다. (더 명시적으로는 `if not emails: return ""`를 한 줄 추가하는 가드도 가능합니다 — 둘 다 외과적 범위 안입니다.)

## 4.12 마무리: 단원 2를 닫으며

이 단원에서 우리는 Claude Code의 일상 작업 두 축을 배웠습니다. 3장에서는 **탐색 → 편집 → 확인** 루프와 외과적 편집의 절제를, 4장에서는 그 편집·실행을 안전하게 통제하는 **권한과 허용목록**을 익혔습니다. 두 개념은 따로가 아니라 한 몸입니다 — 편집할 줄 알아야 무엇을 통제할지 알고, 통제할 줄 알아야 편집을 마음 놓고 위임할 수 있습니다. 미니 프로젝트에서 그 둘을 합쳐 "안전한 리팩터링 세션"을 직접 구성해 보았습니다.

다음 단원(u3)에서는 한 걸음 더 나아갑니다. 세션마다 프로젝트를 다시 설명하지 않도록 영속 컨텍스트(**CLAUDE.md**, 5장)를 설계하고, 코드를 바꾸기 전에 계획을 먼저 받아 검토하는 **플랜 모드**(6장)로 복잡하고 위험한 작업을 더 안전하게 다루는 법을 배웁니다. 이 장에서 만든 권한 설정은 그 단원에서도 안전의 토대로 계속 함께합니다.



# 프로젝트 헌법: CLAUDE.md와 컨텍스트 관리

## 이 챕터에서 배우는 것

- CLAUDE.md가 왜 "세션마다 읽히는 프로젝트 헌법"인지 설명하고, 무엇을 담아야 하는지 판단할 수 있습니다.
- 빌드-테스트 명령, 코딩 컨벤션, 아키텍처, 금지사항을 구조화한 CLAUDE.md를 작성할 수 있습니다.
- 컨텍스트 계층(사용자 `~/.claude`·프로젝트·하위 디렉터리)을 구분해 규칙을 적절한 위치에 배치할 수 있습니다.
- `@참조(import)`로 큰 메모리를 여러 파일로 모듈화할 수 있습니다.
- 긴 세션에서 컨텍스트 한계를 이해하고 `/compact`·`/clear`로 품질을 유지할 수 있습니다.

이번 챕터에서 가르치는 개념은 **컨텍스트 관리: CLAUDE.md와 메모리(context management)** 한 가지입니다. 선수 개념은 ch\_02에서 익힌 **효과적 프롬프팅(작업 지시)**입니다. ch\_02에서 우리는 "목표·맥락·제약·완료 조건"을 매 지시마다 명확히 적는 법을 배웠습니다. 그런데 프로젝트의 빌드 명령이나 코딩 컨벤션처럼 **세션이 바뀌어도 변하지 않는 맥락**을 매번 손으로 적는 것은 비효율적입니다. CLAUDE.md는 바로 그 "반복되는 맥락"을 한 번 적어 두고 모든 세션에 자동으로 주입하는 장치입니다. ch\_02의 "맥락 주입"을 영속화한 것이 이번 개념이라고 보면 됩니다.

## 5.1 도입: 매번 설명하지 않으려면 영속 컨텍스트가 필요하다

실무에서 Claude Code를 며칠만 써 보면 같은 설명을 반복하는 자신을 발견하게 됩니다. "이 프로젝트는 `pytest`로 테스트해", "들여쓰기는 스페이스 4칸이야", "`main` 브랜치에 직접 커밋하지 마", "비밀 키가 든 `secrets/` 폴더는 절대 건드리지 마". 세션을 새로 열 때마다 에이전트는 이 맥락을 알지 못한 채 시작하므로, 매번 같은 말을 적어야 합니다.

에이전트는 **세션마다 제한된 컨텍스트 윈도우(context window)** 안에서 동작합니다. 세션을 닫으면 그 대화의 기억은 사라지고, 다음 세션은 백지에서 출발합니다. 그래서 프로젝트 규칙을 한 번 말해도 내일이면 다시 말해야 하는 것입니다. 사람이라면 신입에게 한 번 알려 준 규칙을 기억하겠지만, 에이전트에게는 그 "한 번"을 매 세션 자동으로 다시 들려줄 장치가 필요합니다.

그 장치가 바로 **CLAUDE.md**입니다. 저장소 루트에 이 파일을 두면, Claude Code는 세션을 시작할 때마다 이 파일을 **자동으로 읽어** 프로젝트의 규칙·명령·관례를 컨텍스트에 주입합니다. 한 번 잘 써 두면 팀 전체가, 모든 세션이, 같은 규칙 위에서 일하게 됩니다. 이것이 ch\_02에서 배운 "맥락 주입"을 매번 손으로 하지 않고 **영속화(persist)**하는 방법입니다.

### 한눈에 보는 CLAUDE.md

CLAUDE.md는 저장소의 "헌법"입니다. 세션 시작 시 자동으로 읽어 빌드-테스트 명령, 코딩 컨벤션, 아키텍처 개요, 금지사항 같은 **영속 맥락**을 에이전트에 주입합니다. 사용자(`~/.claude`)·프로젝트(저장소 루트)·하위 디렉터리

수준으로 계층화되며, @ 참조로 여러 파일로 모듈화하고, 긴 세션에서는 `/compact` (압축)·`/clear` (초기화)로 컨텍스트 품질을 유지합니다.

## 5.2 정의: 세션마다 읽히는 헌법

**CLAUDE.md**(클로드 메모리 파일, **project memory file**)는 Claude Code가 세션을 시작할 때 자동으로 읽어 들이는 마크다운 파일로, 프로젝트의 규칙·명령·관례를 에이전트의 컨텍스트에 영속적으로 주입하는 "프로젝트 헌법"입니다.

핵심 성질을 정리하면 다음과 같습니다.

- **자동 로딩**: 별도 지시 없이도 세션 시작 시 읽힙니다. 사용자가 "CLAUDE.md를 읽어"라고 말할 필요가 없습니다.
- **항상 적용**: 그 안에 적힌 규칙은 그 프로젝트의 모든 세션에서 기본 맥락으로 작동합니다.
- **사람도 읽는 문서**: 마크다운이라 사람이 읽고 수정하기 쉽고, Git으로 버전 관리되어 팀이 공유합니다.
- **지침이지 강제 장치가 아님**: CLAUDE.md는 에이전트에게 주는 **강한 권고**입니다. "항상 100% 차단"이 필요하다면 `ch_11`의 **흑(결정론적 게이트)**을 써야 합니다. 이 구분은 5.10에서 다시 짚습니다.

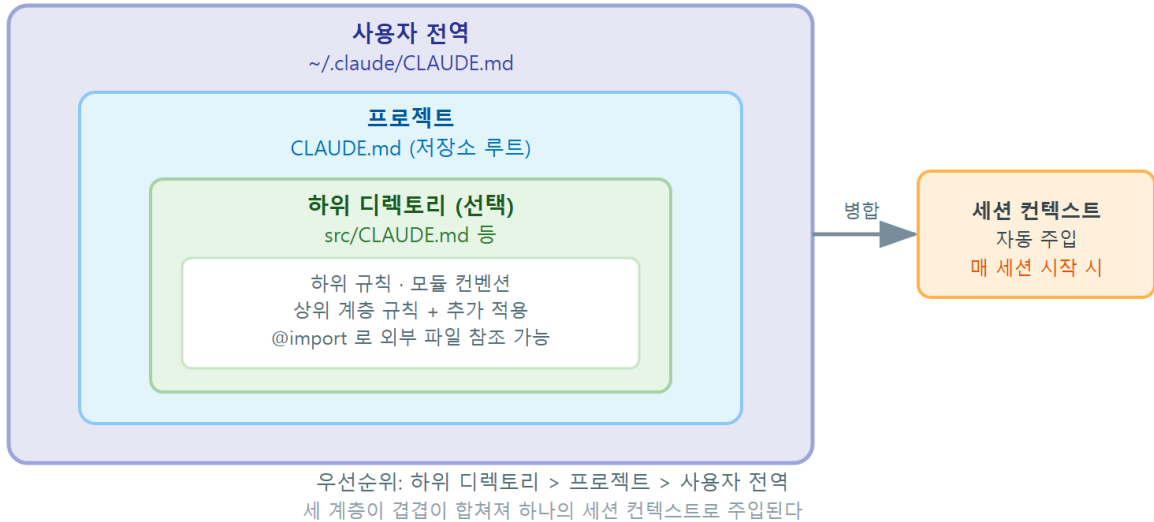
비유하자면, CLAUDE.md는 새 팀원에게 첫날 건네는 **온보딩 문서**입니다. 다만 사람과 달리 에이전트는 매 세션이 "첫날"이므로, 이 문서를 매번 새로 읽혀 주는 것이 자동 로딩의 의미입니다. 잘 쓴 온보딩 문서가 신입의 실수를 줄이듯, 잘 쓴 CLAUDE.md는 에이전트의 헛발질(틀린 빌드 명령, 어긋난 스타일, 금지된 변경)을 크게 줄입니다.

### CLAUDE.md를 빠르게 만드는 법

빈 저장소에서 시작한다면 `/init` 슬래시 커맨드를 실행해 보십시오. Claude Code가 프로젝트 구조를 훑어 CLAUDE.md 초안을 생성합니다. 생성된 초안을 그대로 두지 말고, 5.5에서 배울 항목(명령·컨벤션·아키텍처·금지 사항)에 맞춰 사람이 다듬는 것이 정석입니다. 자동 생성은 출발점일 뿐, 좋은 헌법은 사람이 가꿉니다.

## 5.3 동작 원리: 컨텍스트 계층(사용자·프로젝트·하위 디렉터리)

## 컨텍스트 계층 — 사용자 · 프로젝트 · 하위 디렉토리 병합



CLAUDE.md는 한 곳에만 있는 것이 아닙니다. Claude Code는 여러 위치의 메모리 파일을 계층(layer)으로 겹쳐 하나의 세션 컨텍스트를 구성합니다. 위치마다 적용 범위와 용도가 다릅니다.

| 계층       | 위치                                  | 적용 범위              | 담는 내용 예                                          | Git 공유     |
|----------|-------------------------------------|--------------------|--------------------------------------------------|------------|
| 사용자 (개인) | <code>~/.claude/CLAUDE.md</code>    | 그 사용자의 모든 프로젝트     | 개인 코딩 취향, 자주 쓰는 언어 규칙, 응답 언어                     | 안 됨(개인 머신) |
| 프로젝트 (팀) | 저장소 루트<br><code>./CLAUDE.md</code>  | 그 저장소의 모든 세션·모든 팀원 | 빌드·테스트 명령, 아키텍처, 팀 컨벤션, 금지사항                     | 됨(저장소에 커밋) |
| 하위 디렉터리  | <code>&lt;하위폴더&gt;/CLAUDE.md</code> | 그 폴더 하위 파일을 다룰 때   | 그 모듈에만 해당하는 규칙(예: <code>frontend/</code> 만의 스타일) | 됨          |

동작 흐름은 다음과 같습니다.

- 세션 시작:** 작업 디렉터리(저장소 루트)의 `./CLAUDE.md`와 사용자 `~/.claude/CLAUDE.md`를 함께 읽어 기본 컨텍스트를 만듭니다.
- 상위 탐색:** 작업 디렉터리부터 위로 올라가며 만나는 CLAUDE.md도 포함합니다. 즉 하위 폴더에서 세션을 열어도 상위 프로젝트의 헌법이 빠지지 않습니다.
- 하위 디렉터리 로딩:** 에이전트가 `frontend/` 처럼 하위 폴더의 파일을 실제로 읽거나 다룰 때, 그 폴더의 CLAUDE.md가 그 시점에 컨텍스트로 합쳐집니다. 모든 하위 폴더의 메모리를 처음부터 다 읽지 않고, **필요할 때 끌어오는 것**이 컨텍스트 절약의 핵심입니다.
- 병합:** 이렇게 모인 여러 계층이 하나로 합쳐져 그 세션의 맥락이 됩니다. 더 구체적인(가까운) 규칙이 더 일반적인 규칙을 보충·우선합니다.

이 계층 구조 덕분에 "개인 취향은 내 머신에만, 팀 규칙은 저장소에"라는 분리가 자연스럽게 됩니다. 응답을 한국어로 받고 싶다는 개인 선호를 팀 저장소의 CLAUDE.md에 넣으면 다른 팀원에게 강요가 되지만,

~/claude/CLAUDE.md 에 넣으면 나에게만 적용됩니다. 반대로 "테스트는 pytest 로 돌린다"는 팀 전체가 공유해야 하므로 저장소 CLAUDE.md에 둡니다.

### 왜 계층이 필요한가 — 한 줄 정리

규칙은 적용 범위가 제각각입니다. "나만의 취향"은 사용자 계층에, "이 팀의 약속"은 프로젝트 계층에, "이 폴더만의 사정"은 하위 디렉터리 계층에 둡니다. 범위에 맞는 계층을 고르는 것이 컨텍스트 관리의 첫 번째 기술입니다.

## 5.4 worked example: 빌드·테스트 명령과 컨벤션을 담은 CLAUDE.md

실무에서 흔히 마주치는 백엔드 서비스 `payments-api` 를 예로, 프로젝트 CLAUDE.md를 처음부터 작성해 보겠습니다. 결제 요청을 받아 PG(결제대행)사로 중계하고 거래 내역을 기록하는 FastAPI 서비스입니다. 저장소 루트에 CLAUDE.md 파일을 만들고 다음과 같이 채웁니다.

### # payments-api — 프로젝트 헌법

결제 요청을 받아 PG사로 중계하고 거래 내역을 기록하는 FastAPI 백엔드 서비스입니다.

### ## 빌드·실행·테스트·배포 명령

- 로컬 실행: ``uvicorn app.main:app --reload --port 8000``
- 테스트: ``pytest -q``(커버리지: ``pytest --cov=app``)
- 타입 검사: ``mypy app``
- 린트·포맷: ``ruff check app && ruff format app``
- DB 마이그레이션: ``alembic upgrade head``(로컬·스테이징만)
- 컨테이너 빌드: ``docker build -t payments-api:local .``
- 배포: 프로덕션은 릴리스 PR 머지 시 CI가 수행(수동 배포 금지)

### ## 코딩 컨벤션

- Python 3.11+, 타입 힌트 필수, ruff 규칙 준수(snake\_case)
- 요청·응답 스키마는 Pydantic 모델로 정의(app/schemas)
- 외부 호출(PG·DB)은 services 계층에만 두고 라우터에서 직접 호출 금지
- 커밋 메시지는 Conventional Commits(feat/fix/chore)

### ## 디렉터리 구조

- ``app/main.py``: FastAPI 앱 생성·라우터 등록
- ``app/api/``: 엔드포인트(라우터)
- ``app/services/``: 결제·정산 비즈니스 로직
- ``app/models/``: SQLAlchemy ORM 모델
- ``app/schemas/``: 요청·응답 Pydantic 스키마
- ``migrations/``: Alembic 마이그레이션
- ``tests/``: pytest 단위·통합 테스트

### ## 금지사항

- ``main`` 브랜치 직접 커밋 금지(브랜치 후 PR·리뷰 승인 필수)
- 프로덕션 DB에 마이그레이션 직접 실행 금지(스테이징 검증 후 CI 경유)
- ``.env``·``secrets/``의 자격 증명을 커밋하거나 로그로 출력 금지
- PG 연동 테스트에 실제 카드번호 사용 금지(마스킹된 토큰만)

> 이 프로젝트의 테스트는 어떻게 돌려?

CLAUDE.md에 따르면 ``pytest -q``로 테스트를 실행합니다.  
커버리지까지 보려면 ``pytest --cov=app``입니다. 지금 돌려 볼까요?

- **빌드·실행·테스트·배포 명령** 절을 가장 먼저, 가장 구체적으로 적었습니다. 에이전트가 가장 자주 필요로 하는 정보이고, 틀리면 가장 답답한 정보이기 때문입니다. "테스트 돌려"라는 한마디에 에이전트가 곧장 `pytest -q`를 떠올리는 것이 이 절의 효과입니다. 배포가 CI 전용임을 명시해, 에이전트가 수동 배포 명령을 함부로 제안하지 않게 했습니다.
- **코딩 컨벤션**은 "타입 힌트 필수", "외부 호출은 services 계층에만"처럼 검증 가능하고 아키텍처를 강제하는 형태로 적었습니다. "깔끔하게 짜라" 같은 모호한 형용사(ch\_02의 안티패턴)는 도움이 되지 않습니다.
- **디렉터리 구조**는 폴더별 역할을 한 줄씩 적어, 에이전트가 새 코드를 어느 폴더에 뒀야 할지 스스로 판단하게 합니다. 탐색(ch\_03)에 드는 시간을 줄여 줍니다.
- **금지사항**은 사고를 예방하는 절입니다. "프로덕션 DB 마이그레이션 직접 실행 금지", "자격 증명 커밋 금지"처럼 결제 서비스에서 치명적인 행동을 명확한 부정문으로 못 박았습니다. 두 번째 코드 블록의 세션 기록은, 에이전트가 이 CLAUDE.md를 읽고 곧바로 올바른 테스트 명령을 답하는 모습을 보여 줍니다.

이 헌법이 깔린 세션에서 "당일 정산 금액을 계산하는 서비스 함수를 `app/services/settlement.py`에 추가해 줘"라고 지시하면, 에이전트는 CLAUDE.md의 컨벤션(타입 힌트 필수·services 계층·단일 책임)과 디렉터리 구조(비즈니스 로직은 `app/services/`)를 그대로 따른 코드를 생성합니다.

```
# app/services/settlement.py — CLAUDE.md 컨벤션(타입 힌트·services 계층·단일 책임)을 따른 코드
from datetime import date
from decimal import Decimal
from typing import List, Dict

def daily_settlement(transactions: List[Dict[str, object]], target: date) -> Decimal:
    """target 날짜에 승인(captured)된 거래의 합계를 정산 금액으로 계산한다."""
    total = Decimal("0")
    for tx in transactions:
        # 승인되지 않았거나 다른 날짜의 거래는 정산 대상에서 제외한다
        if tx.get("status") != "approved":
            continue
        if tx.get("captured_at") != target:
            continue
        total += Decimal(str(tx.get("amount", "0")))
    return total
```

> 방금 추가한 daily\_settlement를 간단히 확인해 줘

```
pytest tests/services/test_settlement.py -q
2 passed in 0.11s
2026-06-20 정산 합계: 1,284,500 KRW (승인 거래 37건)
```

- 에이전트가 시키지 않았는데도 `def daily_settlement(transactions: List[Dict[str, object]], target: date) -> Decimal:`처럼 **타입 힌트**를 붙인 것은 CLAUDE.md의 "타입 힌트 필수" 컨벤션을 읽었기 때문입니다. 헌법이 없었다면 타입 힌트가 빠졌을 수 있습니다.
- 금액을 `float`가 아니라 `Decimal`로 다룬 것도 결제 도메인 관행에 맞습니다. 헌법에 "결제·정산 로직"이라는 맥락이 적혀 있어, 에이전트가 돈 계산에 적절한 자료형을 고르도록 유도된 것입니다.

- 함수가 `app/services/`에 놓고 "정산 합계 계산" 한 가지만 하도록 작성된 것은 "services 계층·단일 책임" 컨벤션을 따른 모습입니다. 같은 지시라도 CLAUDE.md의 유무에 따라 산출물의 품질·일관성이 달라진다는 점이 핵심입니다.

이 한 파일을 저장소에 커밋해 두면, 내일의 나도, 옆자리 동료도, CI의 무인 세션도 모두 같은 규칙 위에서 시작합니다. 4개의 절(명령·컨벤션·구조·금지)이 좋은 헌법의 기본 골격입니다.

## 5.5 무엇을 담는가: 명령·컨벤션·아키텍처·금지사항

좋은 CLAUDE.md의 비결은 "많이 담기"가 아니라 **"자주 필요하고, 틀리면 비싼 것만 골라 담기"**입니다. 컨텍스트 원도는 유한하므로(5.8 참고), 모든 것을 담으면 정작 중요한 규칙이 잡음에 묻힙니다. 다음 네 범주를 우선 채우십시오.

| 범주                 | 담을 내용               | 좋은 예                                                  | 피할 예         |
|--------------------|---------------------|-------------------------------------------------------|--------------|
| 명령(commands)       | 빌드·실행·테스트·포맷·린트 명령  | <code>pytest -q</code> , <code>ruff format app</code> | (테스트 방법 누락)  |
| 컨벤션(conventions)   | 언어 버전·네이밍·스타일·커밋 규칙 | "타입 힌트 필수", "Conventional Commits"                    | "코드 예쁘게"     |
| 아키텍처(architecture) | 디렉터리·모듈 역할, 데이터 흐름  | "app/services/ = 비즈니스 로직"                             | 전체 클래스 목록 나열 |
| 금지사항(do-not)       | 건드리면 안 되는 것, 위험 행동  | "secrets/ 수정 금지"                                      | (암묵적 가정)     |

세 가지 원칙을 더 기억하면 좋습니다.

1. **간결·구체**: 한 항목은 한 줄로, 검증 가능하게. 장문의 설계 철학보다 "이 명령으로 테스트한다"가 백 배 유용합니다.
2. **최신 유지**: 관례가 바뀌면 CLAUDE.md도 같이 고칩니다. 낡은 규칙은 도움이 아니라 함정입니다(5.10의 1번 오개념).
3. **금지는 명확히**: "~하지 마라"는 부정문으로 분명히. 에이전트가 추측하지 않게 합니다.

### 즉시 연습: 우선순위 가려내기

아래 다섯 항목 중, CLAUDE.md에 **꼭 넣어야 할 3개**를 고르고 이유를 한 줄씩 적어 보십시오(나머지 2개는 왜 뺐는지도).

1. "테스트는 `npm test`로 실행한다"
2. "이 프로젝트는 2019년에 시작되었다"
3. "환경 변수는 `.env`에 두며 절대 커밋하지 않는다"
4. "컴포넌트 파일은 `PascalCase.tsx`로 명명한다"

## 5. "팀장의 이름은 김OO이다"

## 자가체크

- [ ] 고른 3개가 "자주 필요하고 틀리면 비싼 것"인가
- [ ] 뺀 2개가 작업에 영향이 없는 단순 정보인가
- [ ] 각 항목을 한 줄·검증 가능한 형태로 다듬었는가

## 5.6 메모리 모듈화: @참조(import)

프로젝트가 커지면 규칙도 많아집니다. CLAUDE.md 하나에 수백 줄을 몰아넣으면 읽기 어렵고, 컨텍스트도 무겁게 차지합니다. 이때 **@참조(import)**로 메모리를 여러 파일로 쪼갤 수 있습니다.

CLAUDE.md 본문에 @상대경로 를 적으면, Claude Code가 그 파일의 내용을 그 자리에 끌어와 합칩니다. 마치 한 파일에 적은 것처럼 동작하되, 실제 내용은 주제별 파일로 나뉘어 관리됩니다.

```
# payments-api — 프로젝트 헌법

PG 중계 결제 서비스. 세부 규칙은 주제별 문서로 분리합니다.

## 핵심 명령
- 테스트: `pytest -q`
- 로컬 실행: `uvicorn app.main:app --reload`

## 세부 규칙(분리 관리)
- 코딩·타입 규칙 상세: @docs/style.md
- 커밋·브랜치·릴리스 정책: @docs/git-policy.md
- API 설계 규약(에러 포맷·버저닝): @docs/api-conventions.md

## 개인 메모(있으면)
@~/ .claude/my-python-prefs.md
```

- @docs/style.md 는 저장소 안의 다른 마크다운을 그 자리에 끌어옵니다. 스타일 규칙이 길어도 CLAUDE.md 본문은 짧고 깔끔하게 유지됩니다.
- 여러 파일로 나누면 수정 충돌도 줄어듭니다. 스타일 담당자는 style.md 만, 릴리스 담당자는 git-policy.md 만, API 담당자는 api-conventions.md 만 고치므로 한 파일을 여럿이 동시에 건드릴 일이 줄어듭니다.
- @~/ .claude/my-python-prefs.md 처럼 홈 디렉터리의 개인 파일도 참조할 수 있습니다. 다만 이 줄은 개인 머신에만 그 파일이 있으므로, 팀 저장소 CLAUDE.md보다는 사용자 계층에 두는 편이 깔끔합니다.
- import는 중첩될 수 있습니다(끌어온 파일이 또 @로 다른 파일을 부르는 식). 다만 너무 깊게 얹으면 추적이 어려우니, 한두 단계로 얹게 유지하는 것이 좋습니다.

@참조는 "한 거대한 헌법"을 "주제별 조문"으로 나누는 도구입니다. **모노레포**에서는 이 구도가 특히 유용합니다. 루트 CLAUDE.md 에 전사 공통 규칙을 두고 @docs/... 로 분리하되, 각 서비스·패키지 폴더 (services/payments/ , services/ledger/ 등)에 그 모듈만의 CLAUDE.md 를 두면, 에이전트가 해당 폴더를 다룰 때 모듈 규칙이 그 시점에 합쳐집니다(5.3의 하위 디렉터리 로딩). 작은 단일 서비스라면 단일 CLAUDE.md로 충분하지만, 규칙이 50줄을 넘어가거나 여러 서비스를 한 저장소에서 다루기 시작하면 모듈화를 고려하십시오.

## @ 참조와 @파일 멘션 을 혼동하지 마십시오

대화 중에 입력하는 @파일명 (ch\_02)은 "이번 한 번 이 파일을 봐 줘"라는 일회성 맥락 주입입니다. 반면 CLAUDE.md 안에 적는 @경로는 "이 파일을 헌법의 일부로 항상 포함하라"는 영속 import입니다. 표기는 비슷하지만, 하나는 일시적이고 하나는 영구적이라는 점이 다릅니다.

## 5.7 worked example: 사용자 규칙과 프로젝트 규칙 분리 배치

같은 규칙이라도 "누구에게 적용되어야 하는가"에 따라 둘 위치가 달라집니다. 이번 예제에서는 한 개발자가 자신의 취향과 팀의 약속을 올바른 계층에 나눠 배치합니다.

먼저 개인 머신에만 두는 사용자 계층 파일 ~/.claude/CLAUDE.md 입니다.

### # 개인 공통 규칙 (모든 프로젝트에 적용, 내 머신에만)

#### ## 응답

- 설명은 한국어로, 코드 주석도 한국어로 작성한다.

#### ## 개인 코딩 취향

- 변수명은 줄임말보다 풀어 쓴다(cnt 대신 count).
- 커밋 전 항상 변경 요약 한 줄로 먼저 보여 준다.

다음은 저장소에 커밋해 팀과 공유하는 프로젝트 계층 파일 ./CLAUDE.md 입니다.

### # payments-api — 팀 공유 규칙 (저장소에 커밋)

#### ## 명령

- 테스트: `pytest -q` 린트·포맷: `ruff check app && ruff format app`

#### ## 팀 컨벤션(전원 동일)

- Python 3.11+, 타입 힌트 필수, 외부 호출은 services 계층에만.
- 커밋 메시지는 Conventional Commits(feat/fix/docs).

#### ## 금지

- main 직접 커밋 금지, secrets/.env 수정 금지.

> payments-api에 먹등 키 생성 서비스 함수를 하나 추가해 줘.

(에이전트가 적용하는 규칙)

- 한국어로 설명 ← 사용자 계층(~/.claude)
- 타입 힌트 필수, services 계층에 배치 ← 프로젝트 계층(./CLAUDE.md)
- 커밋 전 변경 요약 한 줄 먼저 ← 사용자 계층
- main 직접 커밋 금지 ← 프로젝트 계층

두 계층이 합쳐진 맥락으로 작업을 시작하겠습니다.

- **응답 언어·개인 취향**은 "나에게만" 해당하므로 사용자 계층(~/.claude/CLAUDE.md)에 두었습니다. 이 규칙을 팀 저장소에 넣으면 한국어를 원치 않는 동료에게도 강요가 됩니다.
- **테스트 명령·타입 힌트·금지사항**은 "팀 전원이 같아야" 하므로 프로젝트 계층(./CLAUDE.md)에 두고 커밋했습니다. 새 팀원이 클론만 해도 이 규칙을 함께 받습니다.

- 세 번째 블록은 두 계층이 **병합**되어 한 세션의 맥락을 이루는 모습입니다. 5.3의 계층 병합이 실제로 어떻게 작동하는지 보여 줍니다. 개인 규칙과 팀 규칙이 충돌하지 않는 한, 둘 다 적용됩니다.

두 계층이 합쳐졌을 때 에이전트가 실제로 내놓는 코드는 양쪽 규칙을 **동시에** 반영합니다. 아래는 같은 "새 서비스 함수 추가" 지시에 대해, 사용자 계층(한국어 주석)과 프로젝트 계층(타입 힌트·services 계층 배치)이 함께 적용된 결과입니다.

```
# app/services/idempotency.py — 프로젝트 규칙(타입 힌트·services 계층)과 개인 규칙(한국어 주석)이 함께 적용됨
import hashlib
from typing import Dict

def idempotency_key(payload: Dict[str, str]) -> str:
    """결제 요청 본문으로 멍등 키를 생성한다(중복 결제 방지)."""
    # 키를 정렬해 같은 요청은 항상 같은 문자열이 되도록 표준화한다(한국어 주석은 개인 규칙)
    canonical = "&".join(f"{k}={payload[k]}" for k in sorted(payload))
    return hashlib.sha256(canonical.encode("utf-8")).hexdigest()
```

- 타입 힌트( `payload: Dict[str, str], -> str` )와 **services 계층 배치**( `app/services/idempotency.py` )는 프로젝트 계층( `./CLAUDE.md` )의 팀 규칙이 적용된 흔적입니다.
- **한국어 주석**("키를 정렬해 같은 요청은...")은 사용자 계층( `~/.claude/CLAUDE.md` )의 개인 규칙이 적용된 흔적입니다. 같은 결과물에 두 계층이 동시에 새겨졌습니다.
- 멍등 키로 중복 결제를 막는 것은 결제 서비스의 흔한 안전장치입니다. 같은 요청이 두 번 들어와도 같은 키가 나오므로, 서버가 중복을 식별해 한 번만 처리할 수 있습니다.
- 이처럼 계층은 "둘 중 하나만"이 아니라 "둘 다 합쳐져" 작동합니다. 적용 범위에 맞게 규칙을 나눠 두면, 충돌 없이 각자의 자리에서 결과에 반영됩니다.

**배치 판단 기준:** "이 규칙이 동료에게도 적용되어야 하는가?" 그렇다면 프로젝트 계층, 아니라면 사용자 계층입니다. 이 한 가지 질문이 대부분의 배치 결정을 해결합니다.

## 5.8 컨텍스트 원도와 압축( `/compact` )·초기화( `/clear` )

CLAUDE.md로 좋은 맥락을 깔아 두어도, **컨텍스트 원도는 무한하지 않습니다**. 대화가 길어지면 그동안 읽은 파일, 실행한 명령, 주고받은 메시지가 컨텍스트에 쌓입니다. 원도가 가득 차면 에이전트는 앞부분을 잊거나(맥락 손실), 응답이 느려지고 산만해집니다. 긴 세션의 품질을 지키려면 컨텍스트를 능동적으로 관리해야 합니다. 두 가지 슬래시 커맨드가 핵심입니다.

| 커맨드                   | 하는 일                              | 언제 쓰나                |
|-----------------------|-----------------------------------|----------------------|
| <code>/compact</code> | 지금까지 대화를 <b>요약으로 압축</b> 하고 핵심만 남김 | 작업을 이어가되 맥락이 무거워졌을 때 |
| <code>/clear</code>   | 컨텍스트를 <b>비우고</b> 새 작업으로 전환        | 다른 작업으로 완전히 넘어갈 때    |

차이가 중요합니다. `/compact` 는 **연속성을 유지**합니다. 진행하던 작업의 핵심(무엇을 하고 있었는지, 어떤 결정을 했는지)을 요약으로 남기고 자잘한 기록을 버리므로, 같은 작업을 계속할 수 있습니다. 반면 `/clear` 는 **단절**입

니다. 직전 맥락을 모두 비우므로, "버그 수정"을 끝내고 전혀 다른 "문서 작성"으로 넘어갈 때처럼 이전 맥락이 오히려 방해가 될 때 씁니다.

### 언제 무엇을 쓸지 — 실전 감각

- 한 기능을 오래 붙들고 있어 대화가 길어졌다 → `/compact` (작업 계속, 맥락만 가볍게)
- 기능 A를 끝내고 무관한 기능 B로 넘어간다 → `/clear` (A의 맥락이 B를 오염시키지 않게)
- 에이전트가 갑자기 앞 내용을 헛갈려한다 → 원도가 찼다는 신호. `/compact`로 정리하거나, 작업이 끝났으면 `/clear`.

CLAUDE.md는 `/clear` 후에도 **다시 자동 로딩**되므로, 초기화해도 프로젝트 헌법은 사라지지 않습니다. 사라지는 것은 그 세션에서 쌓인 대화 기록뿐입니다.

이 관점에서 CLAUDE.md의 가치가 더 분명해집니다. 매 세션 다시 설명해야 할 맥락을 CLAUDE.md에 영속화해 두면, `/clear`로 가볍게 비우고 또 비워도 프로젝트 규칙은 늘 자동으로 깔립니다. **"영속할 것은 CLAUDE.md에, 휘발할 것은 대화에"** — 이것이 컨텍스트 관리의 큰 그림입니다.

## 5.9 즉시 연습(2종 1세트): CLAUDE.md 골격 채우기

여러분이 맡은 작은 백엔드 서비스 `notify-worker` (큐에서 알림 작업을 꺼내 이메일·푸시로 발송하는 백그라운드 워커)를 위한 CLAUDE.md를 작성합니다. 모범 답안, 작성형 빈칸, 자가체크의 2종 1세트입니다.

### 모범 답안

#### # notify-worker — 프로젝트 헌법

큐(Redis)에서 알림 작업을 꺼내 이메일·푸시로 발송하는 백그라운드 워커입니다.

#### ## 명령

- 실행: ``python -m worker.main``
- 테스트: ``pytest -q``
- 린트·포맷: ``ruff check . && ruff format .``
- 컨테이너 빌드: ``docker build -t notify-worker:local .``

#### ## 컨벤션

- Python 3.11+, 타입 힌트 필수, snake\_case
- 외부 발송 클라이언트(SES·FCM)는 `worker/providers`에만 둔다
- 재시도 가능한 예외와 영구 실패를 구분해 처리

#### ## 아키텍처

- ``worker/main.py``: 큐 소비 루프(진입점)
- ``worker/handlers.py``: 알림 종류별 처리
- ``worker/providers/``: 외부 발송 클라이언트 래퍼
- ``tests/``: 핸들러·프로바이더 단위 테스트

#### ## 금지

- 발송 실패를 조용히 삼키지 말 것(반드시 로깅·재큐)
- 자격 증명(SES\_KEY 등)을 코드·로그에 노출 금지
- main 브랜치 직접 커밋 금지(브랜치 후 PR)

- 네 절(명령·컨벤션·아키텍처·금지)을 모두 채웠고, 각 항목을 한 줄·검증 가능한 형태로 적었습니다.
- 명령 절에 실행·테스트·린트·빌드를 함께 두어, 에이전트가 작업 후 스스로 검증·포맷을 돌릴 수 있게 했습니다.
- 금지 절의 "발송 실패를 조용히 삼키지 말 것"은 워커의 가장 위험한 실수(알림 유실)를 막는 부정문으로, 운영 사고를 예방합니다. "자격 증명 노출 금지"는 외부 발송 키를 다루는 서비스에서 필수입니다.

## 직접 작성: CLAUDE.md 골격 (빈칸 채우기)

여러분의(또는 가상의) 작은 프로젝트를 하나 정해, 아래 골격의 빈칸( \_\_\_\_ )을 채워 CLAUDE.md를 완성해 보십시오. 모범 답안을 덮어 두고 도전하는 것이 효과적입니다.

```
# ____ — 프로젝트 헌법

(이 프로젝트가 무엇을 하는지 한 줄): ____

## 명령
- 실행: ____
- 테스트: ____          # 이 프로젝트의 테스트 명령은?

## 컨벤션
- 언어·버전: ____
- 네이밍·스타일: ____

## 아키텍처
- ____: (역할 한 줄)
- ____: (역할 한 줄)

## 금지
- ____          # 건드리면 위험한 것 하나
```

**확장 과제:** 컨벤션 절이 길어졌다고 가정하고, @docs/style.md 로 분리하는 형태로 바꿔 보십시오(5.6 모듈화).

## 자가체크 체크리스트

- [ ] 명령 절에 빌드/실행·테스트 명령이 빠짐없이 들어갔는가
- [ ] 각 규칙이 "코드 예쁘게" 같은 모호한 형용사가 아니라 검증 가능한 형태인가
- [ ] 팀이 공유할 규칙은 프로젝트 계층(./CLAUDE.md)에, 개인 취향은 사용자 계층에 두었는가
- [ ] 금지 절에 사고를 예방하는 부정문이 하나 이상 있는가
- [ ] (확장) 규칙이 길어진 절을 @참조로 분리했는가

## 5.10 흔한 오류와 오개념

CLAUDE.md와 컨텍스트 관리에서 실무자가 자주 부딪히는 함정들입니다.

### 오개념 1: 한 번 쓰면 끝, 방치해도 된다

CLAUDE.md는 한 번 쓰고 방치하면 **낡습니다**. 테스트 명령이 `pytest` 에서 `pytest -n auto` 로 바뀌었는데 CLAUDE.md가 그대로면, 에이전트는 틀린 명령을 자신 있게 실행합니다. 낡은 규칙은 없는 것보다 나쁩니다 —

에이전트가 그것을 "사실"로 믿기 때문입니다.

교정: 관례·명령·구조가 바뀌면 같은 PR에서 CLAUDE.md도 함께 고칩니다. "코드와 헌법은 함께 커밋"을 습관으로 삼으십시오.

### 오개념 2: 많이 답을수록 좋다 (과적)

"혹시 도움 될까" 싶어 회사 연혁, 전체 API 목록, 긴 설계 철학까지 다 담으면 CLAUDE.md가 비대해집니다. 결과는 둘 다 나쁩니다. (1) 정작 중요한 규칙(테스트 명령·금지사항)이 잡음에 묻혀 잘 지켜지지 않고, (2) 컨텍스트 원도를 헛되이 차지해 긴 작업의 여유를 깎아먹습니다.

교정: "자주 필요하고 틀리면 비싼 것만." 분량이 길어지면 @참조로 주제별 파일로 분리하고, 본문은 핵심만 남깁니다.

### 오개념 3: 컨텍스트는 무한하다

"긴 세션이어도 에이전트가 처음 말을 다 기억하겠지"는 틀린 가정입니다. 컨텍스트 원도는 유한하고, 길어지면 앞부분 맥락이 흐려집니다. 한 세션에서 몇 시간씩 끌면 품질이 떨어지는 이유가 이것입니다.

교정: 작업을 이어갈 땐 /compact 로 압축하고, 다른 작업으로 넘어갈 땐 /clear 로 초기화합니다. 영속해야 할 맥락은 대화가 아니라 CLAUDE.md에 둡니다.

### 오개념 4: CLAUDE.md는 강제(차단) 장치다

CLAUDE.md의 규칙은 강한 권고이지, 100% 강제가 아닙니다. 에이전트는 대개 따르지만, "절대 일어나서는 안 되는 일"(예: 프로덕션 DB 삭제 차단, 비밀 파일 수정 차단)을 보장하려면 ch\_04의 **권한(deny)**이나 ch\_11의 **혹(PreToolUse 차단)**을 써야 합니다. 안전 경계는 헌법이 아니라 게이트로 강제하십시오. CLAUDE.md는 "어떻게 일할지"를, 권한·혹은 "무엇을 못 하게 할지"를 담당한다고 기억하면 됩니다.

## 5.11 자주 묻는 질문(FAQ)

### Q. CLAUDE.md와 README.md는 무엇이 다른가요?

README는 사람(특히 처음 온 기여자)을 위한 프로젝트 소개·설치 안내입니다. CLAUDE.md는 에이전트가 작업할 때 따를 규칙·명령·금지를 담은 운영 지침입니다. 둘은 겹치는 내용(빌드 명령 등)이 있지만 독자와 목적이 다릅니다. CLAUDE.md에서 @README 로 README를 끌어와 중복을 줄이는 방법도 있습니다.

### Q. CLAUDE.md를 수정하면 바로 적용되나요?

다음 세션 시작 시 새로 읽혀 적용됩니다. 진행 중인 세션에 즉시 반영하고 싶으면 /memory 로 메모리를 확인·갱신하거나, 세션을 다시 여는 것이 확실합니다.

### Q. 빠르게 규칙 한 줄을 추가하고 싶을 때는요?

대화 입력에서 # 으로 시작하는 줄을 적으면 그 내용을 메모리에 추가할지 물어봅니다. 작업 중 발견한 규칙(예: "이 풀더는 건드리지 말 것")을 즉석에서 헌법에 반영하는 빠른 방법입니다.

**Q. ~/.claude/CLAUDE.md (사용자)와 ./CLAUDE.md (프로젝트)가 충돌하면요?**

충돌하지 않는 규칙은 둘 다 적용됩니다. 같은 주제에서 더 구체적인(프로젝트·하위 디렉터리) 규칙이 더 일반적 인(사용자) 규칙을 보충·우선하는 쪽으로 동작합니다. 애초에 충돌이 나지 않도록, 팀 약속은 프로젝트에·개인 취향은 사용자에게 두는 분리가 중요합니다.

**Q. 하위 디렉터리 CLAUDE.md는 언제 읽히나요?**

그 폴더 하위의 파일을 실제로 다룰 때 그 시점에 합쳐집니다. 모든 하위 메모리를 처음부터 신지 않고 필요할 때 끌어오므로, 큰 모노레포에서도 컨텍스트가 과하게 무거워지지 않습니다.

**Q. CLAUDE.md를 꼭 Git에 커밋해야 하나요?**

팀이 공유할 프로젝트 규칙이라면 커밋하는 것이 맞습니다. 그래야 모든 팀원·CI 세션이 같은 헌법을 받습니다. 반대로 나만의 개인 메모는 사용자 계층(~/.claude)에 두어 저장소를 더럽히지 않습니다.

## 정리

이번 챕터에서는 **컨텍스트 관리: CLAUDE.md와 메모리** 개념을 익혔습니다.

- CLAUDE.md는 세션마다 자동으로 읽히는 **프로젝트 헌법**으로, 매번 반복하던 맥락을 영속화합니다(ch\_02의 맥락 주입을 영구화).
- 명령·컨벤션·아키텍처·금지사항 네 범주를 **간결·구체·최신**으로 담는 것이 좋은 헌법의 골격입니다.
- 사용자(~/.claude)·프로젝트·하위 디렉터리 **계층**으로 규칙을 적용 범위에 맞게 배치합니다.
- 규칙이 많아지면 @참조로 **모듈화**하고, 긴 세션은 /compact·/clear로 **컨텍스트를 관리**합니다.
- CLAUDE.md는 권고이지 강제가 아니라는 점(안전 경계는 권한·흑으로)을 포함해 4대 오개념을 점검했습니다.

이제 에이전트는 "이 프로젝트가 어떤 곳인지"를 매 세션 자동으로 압니다. 그렇다면 다음 단계는 **"바꾸기 전에 무엇을 어떻게 바꿀지 먼저 합의하는 것"**입니다. 다음 챕터(ch\_06)에서 배울 **플랜 모드**는 바로 이 CLAUDE.md의 맥락 위에서 더 정확한 계획을 끌어내, 복잡하고 위험한 변경을 안전하게 다루는 방법입니다.



# 먼저 계획하기: 플랜 모드

## 이 챕터에서 배우는 것

- 플랜 모드가 왜 "변경 전 읽기 전용 계획 단계"인지 설명하고, 켜고 끌 수 있습니다.
- 에이전트가 제시한 계획을 검토하고 단계 추가·삭제·재정렬로 교정할 수 있습니다.
- 계획 승인 후 실행으로 전환하는 흐름을 운용할 수 있습니다.
- 어떤 작업에 계획·실행 분리가 필요한지(복잡·고위험·다중 파일) 판단할 수 있습니다.
- 플랜 모드를 CLAUDE.md·서브에이전트와 연계해 더 정확한 계획을 끌어낼 수 있습니다.

이번 챕터에서 가르치는 개념은 **플랜 모드(plan mode)** 한 가지입니다. 선수 개념은 둘입니다. 하나는 ch\_05의 **CLAUDE.md·컨텍스트 관리**로, 좋은 프로젝트 맥락이 깔려 있어야 에이전트가 정확한 계획을 세웁니다. 다른 하나는 ch\_03의 **파일 탐색·외과적 편집**으로, 플랜 모드는 바로 그 편집을 "곧장 하지 않고 먼저 계획부터 보여 주는" 모드이기 때문입니다. 이 챕터는 u3 단원(컨텍스트와 계획)의 마지막 챕터이므로, 끝에서 두 개념(CLAUDE.md + 플랜 모드)을 묶는 **단원 미니 프로젝트**로 마무리합니다.

## 6.1 도입: 바꾸기 전에 계획을 먼저 본다

ch\_03에서 우리는 에이전트가 파일을 탐색하고 곧바로 외과적으로 편집하는 일상 루프를 배웠습니다. 한 줄 버그를 고치는 작은 작업이라면 이 "바로 편집" 흐름이 빠르고 좋습니다. 그러나 작업이 커지면 이야기가 달라집니다.

"인증 모듈을 토큰 방식에서 세션 방식으로 바꿔 줘" 같은 지시를 떠올려 봅시다. 이건 한 파일이 아니라 라우터·미들웨어·테스트·설정까지 **여러 파일을 동시에** 건드리는 일이고, 방향을 잘못 잡으면 **되돌리기 힘든** 변경이 무더기로 쌓입니다. 에이전트가 절반쯤 고친 뒤에야 "아, 네가 원하는 건 그게 아니었구나"를 알게 되면, 이미 열 군데가 바뀐 상태입니다.

사람도 같은 상황에서 그렇게 일하지 않습니다. 숙련된 개발자는 큰 변경 앞에서 먼저 **계획을 세웁니다**. "어디를 손대야 하지? 순서는? 위험한 부분은?"을 종이에 적고, 동료와 합의한 뒤 키보드에 손을 올립니다. 코드를 한 줄도 바꾸기 전에 말이지요.

**플랜 모드**는 에이전트에게 바로 이 습관을 강제하는 장치입니다. 변경을 시작하기 전에 에이전트가 코드베이스를 **읽기만** 하면서 분석하고, "무엇을 어떤 순서로 바꿀지" 계획을 먼저 제시합니다. 여러분은 그 계획을 검토·교정한 뒤 승인하고, 그제야 실제 편집이 시작됩니다. 잘못된 방향으로 대규모 변경이 일어나는 사고를, 가장 싼 시점(아직 아무것도 안 바꾼 시점)에 막는 것입니다.

### 한눈에 보는 플랜 모드

플랜 모드는 에이전트가 코드를 바꾸기 전에 "무엇을 어떻게 바꿀지" 계획을 먼저 제시하고 승인을 받는 **읽기 전용** 모드입니다. 이 단계에서 에이전트는 파일을 읽고 검색하며 분석하지만 **편집·실행은 하지 않습니다**. 사용자

가 계획을 검토·수정해 방향을 교정하고 승인하면, 그때 실행 단계로 넘어갑니다. 복잡·고위험·다중 파일 변경에서 잘못된 대규모 변경을 예방하고 에이전트의 의도를 투명하게 만듭니다.

## 6.2 정의: 변경 전 읽기 전용 계획 단계

**플랜 모드(plan mode)**는 에이전트가 파일 편집·명령 실행 같은 **부수효과 동작을 멈추고**, 코드베이스를 읽고 분석해 변경 계획을 먼저 제시하는 읽기 전용 작업 모드입니다. 사용자가 그 계획을 승인해야 비로소 실행(편집) 단계로 전환됩니다.

핵심 성질은 다음과 같습니다.

- **읽기 전용:** 플랜 모드에서 에이전트는 탐색(Glob·Grep)·읽기(Read)는 하지만, 편집(Edit·Write)이나 부수효과 명령(Bash)으로 파일·시스템을 **바꾸지 않습니다**. 그래서 이 단계에서는 코드가 안전합니다.
- **계획 산출물:** 분석 결과로 "어디를 어떤 순서로 바꿀지"를 단계 목록으로 제시합니다. 이것이 검토 대상입니다.
- **승인 게이트:** 사용자가 계획을 승인(또는 교정 후 승인)해야 실행으로 넘어갑니다. 승인 전에는 변경이 일어나지 않습니다.
- **계획-실행 분리:** "생각하는 단계"와 "행동하는 단계"를 명확히 나눕니다. 이 분리가 투명성(에이전트의 의도가 먼저 보임)과 안전성(틀린 방향을 미리 차단)을 만듭니다.

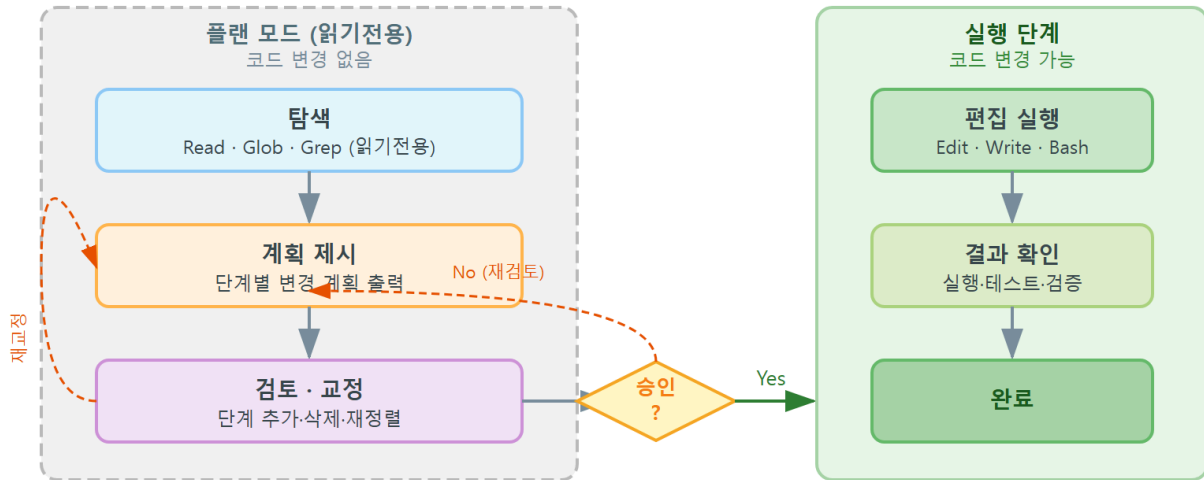
켜고 끄는 법은 두 가지입니다. 대화형 세션에서는 **Shift+Tab** 으로 모드를 순환하면(일반 → 편집 자동 수락 → 플랜 모드) 플랜 모드로 들어갑니다. 또는 처음부터 계획 위주로 시작하고 싶으면 실행 시 `--permission-mode plan` 플래그를 줄 수 있습니다. 계획을 승인하면 모드가 풀리며 실행 단계로 전환됩니다.

**플랜 모드 = "말로 합의하고 손대기"**

플랜 모드의 본질은 단순합니다. "곧장 코드를 바꾸지 말고, 먼저 무엇을 할지 말로 합의하자." 회의에서 화이트보드에 설계를 그려 합의한 뒤 구현에 들어가는 것과 같습니다. 다른 점은, 그 합의가 에이전트의 다음 행동을 그대로 묶는 **실행 가능한 계약**이 된다는 것입니다.

## 6.3 동작 원리: 계획 → 승인 → 실행 분리 흐름

## 플랜 모드 — 계획 · 승인 · 실행 분리 흐름



복잡·고위험 작업에서 잘못된 대규모 변경을 사전에 예방한다  
 플랜 모드: --plan 플래그 또는 "계획을 먼저 보여줘" 지시

플랜 모드는 한 작업을 두 단계로 명확히 가릅니다. 단계마다 에이전트가 할 수 있는 일이 다릅니다.

| 단계           | 에이전트가 하는 일             | 할 수 있는 도구               | 못 하는 일                     |
|--------------|------------------------|-------------------------|----------------------------|
| 1. 계획(읽기 전용) | 코드 탐색·분석, 단계별 계획 작성·제시 | Glob·Grep·Read 등 읽기 도구  | 파일 편집(Edit·Write), 부수효과 명령 |
| (게이트) 검토·승인  | (사용자) 계획 검토·교정·승인      | —                       | —                          |
| 2. 실행        | 승인된 계획대로 편집·실행         | Edit·Write·Bash 등 전체 도구 | (승인 범위를 벗어난 임의 변경)         |

흐름을 순서대로 풀면 다음과 같습니다.

- 진입:** Shift+Tab 으로 플랜 모드로 들어가 작업을 지시합니다("인증을 세션 방식으로 바꿔 줘").
- 탐색·분석:** 에이전트가 관련 파일을 검색·읽기로 파악합니다. 이때 **CLAUDE.md**의 맥락(아키텍처·금지사항)이 분석의 질을 좌우합니다(6.8 연계).
- 계획 제시:** "1) 토큰 검증 미들웨어 제거 2) 세션 스토어 추가 3) 라우터 수정 4) 테스트 갱신" 식의 단계 목록을 보여 줍니다. 아직 아무 파일도 바뀌지 않았습니다.
- 검토·교정(게이트):** 사용자가 계획을 읽고 단계를 더하거나 빼거나 순서를 바꿉니다(6.5). 방향이 틀렸으면 여기서 바로잡습니다 — 코드를 한 줄도 버리지 않고.
- 승인:** 계획이 만족스러우면 승인합니다. 모드가 풀리고 실행 단계로 전환됩니다.
- 실행:** 에이전트가 승인된 계획대로 편집·실행을 진행합니다. 이제 ch\_03의 외과적 편집·diff 검토 루프가 됩니다.

이 분리의 핵심 이득은 "**가장 싼 시점에 틀림을 잡는다**"는 데 있습니다. 잘못된 방향을 계획 단계(아직 변경 0)에서 잡으면 교정 비용이 한 문장이지만, 실행 한참 뒤에 잡으면 열 파일을 되돌려야 합니다. 플랜 모드는 오류를

"되돌리기"에서 "안 만들기"로 옮깁니다.

## 왜 읽기 전용인가 — 한 줄 정리

계획 단계가 읽기 전용이기 때문에, 여러분은 **아무 위험 없이** 에이전트의 의도를 끝까지 들여다볼 수 있습니다. 마음에 안 들면 그냥 계획을 고치면 됩니다. 이미 바뀐 파일을 되돌릴 걱정이 없으므로, 부담 없이 간간하게 검토할 수 있습니다.

## 6.4 worked example: 세션 인증을 JWT로 마이그레이션하는 계획

운영 중인 웹 서비스의 인증 방식을 서버 세션에서 JWT(JSON Web Token)로 전환하는 작업을 플랜 모드로 진행합니다. 로그인 라우터·인증 미들웨어·토큰 유틸·설정·테스트까지 여러 파일을 동시에 건드리고, 잘못되면 전 사용자가 로그인하지 못하는 **되돌리기 비싼 고위험 변경**입니다. "바로 편집"으로 맡길 작업이 아닙니다.

```
> [Shift+Tab으로 플랜 모드 진입]
> 인증을 서버 세션 방식에서 JWT 방식으로 바꿔 줘. 로그인하면 액세스
  토큰을 발급하고, 이후 요청은 Authorization 헤더의 토큰으로 검증해.
```

(에이전트, 읽기 전용으로 인증 관련 파일 탐색 후 계획 제시)

```
## 계획: 세션 → JWT 인증 마이그레이션
1. Grep으로 session[...] 사용처와 로그인/로그아웃 경로 전수 식별
2. auth/tokens.py 신설 — encode_token()/decode_token()
3. routes/login.py: 세션 발급 제거 → 액세스 토큰 발급으로 교체
4. middleware/auth.py: 세션 조회 제거 → Authorization 헤더 검증으로 교체
5. config.py에 JWT_SECRET·만료(exp) 설정 추가
6. pytest -q 로 회귀 확인
```

이대로 진행할까요? (수정할 부분이 있으면 알려 주세요)

```
> 방향은 좋아. 다만 두 가지를 고쳐야 해. (1) 토큰 클레임에 만료(exp)만
  넣지 말고, 나중에 강제 무효화를 할 수 있게 jti(토큰 ID)도 넣어 줘.
  (2) 4번 뒤에 리프레시 토큰 발급·재발급 단계를 추가하고, "기존 세션
  사용자 강제 로그아웃은 이번 범위 밖"임을 계획에 명시해.
```

(에이전트, 계획 갱신)

```
## 계획(수정본): 세션 → JWT 인증 마이그레이션
1. session[...] 사용처·로그인/로그아웃 경로 전수 식별
2. auth/tokens.py 신설 — encode/decode(클레임에 exp·jti 포함)
3. routes/login.py: 세션 발급 제거 → 액세스 토큰 발급
4. middleware/auth.py: Authorization 헤더 토큰 검증
5. (추가) 리프레시 토큰 발급·재발급 엔드포인트
6. config.py에 JWT_SECRET·액세스 만료·리프레시 만료 설정
7. (범위 명시) 기존 세션 사용자 강제 로그아웃은 이번 범위 밖
8. pytest -q 회귀 확인
```

승인하시면 실행하겠습니다.

```
> 승인. 진행해.
```

(모드 해제, 실행 단계 — 1~8을 순서대로 편집·실행)

- 첫 블록에서 에이전트는 **읽기 전용으로 인증 파일만 탐색**한 뒤 6단계 계획을 제시합니다. 이 시점에 `auth/tokens.py` 는 아직 없고 로그인 경로도 그대로입니다 — 순수한 제안일 뿐, 토큰 발급 코드는 한 줄도 작성되지 않았습니다.
- 둘째 블록에서 팀리드가 **보안·운영 관점의 교정**을 넣었습니다. `jti` 추가(추후 토큰 무효화 대비)와 리프레시 토큰 단계 추가는 빠진 요구를, "강제 로그아웃 범위 밖" 명시는 범위를 통제하는 교정입니다. 모두 목록만 고치므로 코드 변경 비용은 0입니다.
- "승인. 진행해"로 게이트를 통과하자 모드가 풀리고 **실행 단계**가 시작됩니다. 이제부터 `ch_03`의 외과적 편집(세션 조회 제거 → 헤더 검증 교체)이 8단계 순서대로 적용됩니다.
- 만약 플랜 모드 없이 곧장 지시했다면, 에이전트가 `jti` 없이 토큰을 설계하고 여러 파일을 바꾼 뒤였을 것이고, 무효화 요구를 뒤늦게 끼워 넣느라 토큰 스키마와 검증 코드를 다시 손봐야 했을 것입니다.

승인 후 실행 단계에서 2번 단계가 만들어 낸 `auth/tokens.py` 와, 4번 단계로 미들웨어가 토큰 검증으로 바뀐 결과는 다음과 같습니다. 계획의 각 단계가 실제 코드로 옮겨진 모습입니다.

```
# auth/tokens.py - 계획 2단계의 산출물(클레임에 exp/jti 포함 발급)
import uuid
from datetime import datetime, timedelta, timezone

import jwt # PyJWT

from config import JWT_SECRET, ACCESS_TTL_MIN

def encode_token(user_id: int) -> str:
    """사용자 ID로 액세스 토큰을 발급한다(만료 exp/무효화용 jti 포함)."""
    now = datetime.now(timezone.utc)
    payload = {
        "sub": str(user_id),
        "exp": now + timedelta(minutes=ACCESS_TTL_MIN), # 계획 6: 만료 설정
        "jti": uuid.uuid4().hex, # 계획 2(교정): 무효화용 토큰 ID
    }
    return jwt.encode(payload, JWT_SECRET, algorithm="HS256")
```

```
# middleware/auth.py - 계획 4단계: 세션 조회 제거 -> 헤더 토큰 검증
import jwt

from config import JWT_SECRET

def authenticate(request) -> int:
    """Authorization 헤더의 JWT를 검증하고 사용자 ID를 돌려준다."""
    header = request.headers.get("Authorization", "")
    token = header.removeprefix("Bearer ").strip()
    payload = jwt.decode(token, JWT_SECRET, algorithms=["HS256"])
    return int(payload["sub"]) # 세션 저장소 조회 대신 토큰 클레임에서 신원 확인
```

- `auth/tokens.py` 는 계획 2번(토큰 유틸 신설)과, 교정으로 들어간 `jti` 요구가 합쳐진 결과입니다. `"jti": uuid.uuid4().hex` 가 들어간 것은 팀리드가 계획 단계에서 **추가한 무효화 요구**가 실행에 그대로 반영됐기 때문입니다.
- `middleware/auth.py` 는 계획 4번(세션 조회 제거 → 헤더 검증)의 모습입니다. 서버 세션 저장소를 뒤흔던 코드가 사라지고, `jwt.decode(...)` 한 번으로 토큰 클레임에서 신원을 확인합니다. 이것이 외과적 편집이 승인된 계획대로 적용된 결과입니다.

- 핵심은 이 코드들이 모두 계획 승인 이후에야 만들어졌다는 점입니다. 계획 단계에서는 토큰 한 개도 발급되지 않았고, 로그인 흐름도 멀쩡히 살아 있었습니다.

이 예제의 교훈은 분명합니다. 인증처럼 전 사용자에게 영향이 가는 다중 파일 변경일수록, 첫 편집 전에 계획을 합의하는 비용이 가장 쌉니다.

## 6.5 계획 교정: 단계 추가·삭제·재정렬

플랜 모드의 진짜 가치는 "계획을 받는 것"이 아니라 "계획을 함께 다듬는 것"에 있습니다. 에이전트가 처음 내놓은 계획을 그대로 승인할 필요는 전혀 없습니다. 오히려 검토 없이 그대로 승인하는 것은 플랜 모드를 형식적으로만 쓰는 것입니다(6.10의 오개념 2). 교정의 세 가지 기본 동작을 익혀 두십시오.

| 교정 동작  | 언제              | 지시 예                         |
|--------|-----------------|------------------------------|
| 단계 추가  | 빠진 작업이 보일 때     | "3번 전에 '기존 테스트 백업' 단계를 넣어 줘" |
| 단계 삭제  | 불필요·과한 단계가 있을 때 | "5번 문서 갱신은 이번 범위 밖이야, 빼 줘"   |
| 단계 재정렬 | 순서가 위험·비효율일 때   | "테스트 갱신을 구현 앞으로 옮겨(테스트 우선)"  |

교정 시 좋은 검토 질문 네 가지를 기억하면 빠짐없이 점검할 수 있습니다.

1. **범위:** 이 계획이 내가 원하는 것보다 넓거나 좁지 않은가? (불필요한 단계는 삭제, 빠진 단계는 추가)
2. **순서:** 위험한 단계가 너무 앞에 있지 않은가? 되돌리기 어려운 일은 검증 뒤로.
3. **가정:** 계획이 틀린 전제를 깔고 있지 않은가?(6.7에서 집중) 예를 들어 없는 함수를 고치려 하지 않는가.
4. **검증:** 끝에 "어떻게 확인할지"(테스트·실행)가 들어 있는가? 검증 없는 계획은 완료를 알 수 없습니다.

### 즉시 연습: 계획에서 문제 찾기

아래는 "orders 테이블에 customer\_phone 컬럼을 추가하고 관련 조회를 맞춰 줘"라는 지시에 에이전트가 내놓은 DB 스키마 변경 계획입니다. 위 네 질문(범위·순서·가정·검증)으로 검토해 **고칠 점 2개 이상**을 찾아 교정 지시 문장으로 적어 보십시오.

```
## 계획: orders 테이블 스키마 변경
1. ALTER TABLE orders DROP COLUMN legacy_memo, ADD COLUMN customer_phone
2. 모든 SELECT orders 쿼리에 customer_phone 포함하도록 일괄 수정
3. 완료
```

#### 자가체크

- [ ] 1번이 요청에 없던 legacy\_memo **DROP**까지 섞은 범위 초과를, 그리고 컬럼 삭제는 되돌리기 어려운 파괴적 변경임을 지적했는가
- [ ] 운영 DB 변경 앞에 백업·마이그레이션 스크립트(롤백 포함) 단계가 빠졌음을 지적했는가
- [ ] 끝에 검증 단계(스키마 반영 확인·관련 쿼리 회귀 테스트)가 없음을 지적했는가
- [ ] 교정을 "추가·삭제·재정렬" 중 어떤 동작인지 분명히 적었는가

## 6.6 언제 쓰는가: 복잡·고위험·다중 파일 변경

플랜 모드는 강력하지만 **모든 작업에 쓰는 것은 아닙니다**. 한 줄 오타 수정에까지 계획을 받으면 오히려 느리고 번거롭습니다. 작업의 성격에 따라 판단해야 합니다.

| 플랜 모드가 어울리는 작업           | 곧장 실행이 나은 작업     |
|--------------------------|------------------|
| 여러 파일을 동시에 바꾸는 리팩터링      | 한 파일·한 줄 버그 수정   |
| 되돌리기 어려운 변경(스키마·마이그레이션)  | 오타·주석 수정         |
| 낮선 코드베이스에서 영향 범위가 불확실할 때 | 이미 방법이 명확한 작은 추가 |
| 고위험(인증·결제·삭제) 로직 변경      | 단순 포맷·이름 바꾸기 한 건 |
| 설계 방향 자체를 합의해야 할 때       | 즉시 확인 가능한 사소한 작업 |

판단 기준을 한 문장으로 줄이면 이렇습니다. **"틀렸을 때 되돌리는 비용이 큰가? 또는 영향 범위가 불확실한가?"** 둘 중 하나라도 '예'라면 플랜 모드가 값을 합니다. 둘 다 '아니오'인 사소한 작업은 곧장 실행하는 편이 빠릅니다.

### 과용도 함정이다

플랜 모드가 안전하다고 모든 사소한 작업까지 계획-검토-승인의 3박자를 거치면, 생산성이 떨어지고 정작 중요한 계획을 검토하는 집중력도 흐려집니다("계획 피로"). 안전장치는 위험이 있는 곳에 집중해 쓸 때 가장 효과적입니다. 무엇이 위험한지 가려내는 판단력 자체가 실무 역량입니다.

## 6.7 worked example: 잘못된 가정을 교정 지시로 바로잡기

플랜 모드의 가장 값진 순간은, 에이전트가 **틀린 전제로 계획을 세웠을 때 그것을 코드 변경 전에 잡아내는 것**입니다. 실무 상황을 봅시다. 상품 상세 화면이 가끔 **이미 품절된 상품을 "재고 있음"으로 표시**한다는 버그 리포트가 들어왔고, 에이전트에게 재고 캐시 무효화 로직을 고치게 했습니다. 그런데 에이전트가 코드베이스를 잘못 이해했습니다.

> [플랜 모드] 상품 화면이 가끔 품절 상품을 '재고 있음'으로 보여줘.  
재고 캐시 무효화 로직을 고쳐 줘.

(에이전트 계획)

## 계획: 재고 캐시 무효화 수정

1. cache.py의 재고 TTL을 60초에서 10초로 줄여 옛 값 노출 시간 단축
2. 테스트로 확인

> 잠깐, 그건 가정이 틀렸어. 문제는 TTL이 아니라, 재고를 차감하는 쓰기 경로에서 invalidate()를 호출하지 않는 곳이 있다는 거야. 주문 확정 시 호출되는 decrement\_stock()를 봐 봐. 거기서 재고 캐시를 안 비우고 있어.

(에이전트, 다시 읽기 전용으로 decrement\_stock 확인 후 계획 갱신)

```
## 계획(수정본): 재고 쓰기 경로의 캐시 무효화 누락 수정
1. 재고를 바꾸는 모든 함수(decrement_stock·restock 등)에서 invalidate() 호출 여부 점검
2. 누락된 경로에 invalidate(key) 추가
3. TTL은 60초 유지(증상 완화가 아니라 원인 해결)
4. 주문 확정 직후 상품 조회 시 재고가 즉시 갱신되는지 테스트로 검증
```

이 방향이 맞을까요?

> 맞아. 승인.

- 에이전트의 첫 계획은 **증상(옛 재고 노출)**을 **완화**하는 방향(TTL 단축)이었습니다. 그럴듯하지만 근본 원인(재고 차감 시 무효화 누락)을 놓친 **틀린 가정**입니다. TTL을 줄여도 60초가 10초가 될 뿐, 품질 상품이 잠깐씩 "재고 있음"으로 보이는 일은 그대로입니다.
- 결정적 장면은, 이 틀린 계획이 **아직 코드를 바꾸기 전**이라는 점입니다. 만약 곧장 실행했다면 TTL만 줄인 채 버그는 그대로 남고, 캐시 부하만 늘리는 변경이 운영에 배포됐을 것입니다.
- 팀리드는 도메인 지식("decrement\_stock가 invalidate를 안 부른다")으로 **가정을 교정**했고, 에이전트는 다시 읽기 전용으로 그 함수를 확인한 뒤 **원인 해결** 계획으로 돌아왔습니다.
- 마지막 계획에 4번 검증 단계("주문 확정 직후 재고 즉시 갱신")가 들어간 것도 중요합니다. 가정을 바로잡았으니, 그 가정이 맞았는지 확인할 방법까지 계획에 넣은 것입니다.

교정된 계획이 승인된 뒤 실행 단계에서 적용된 변경은 다음과 같습니다. 증상(TTL)을 건드리는 대신 원인(쓰기 경로의 무효화 누락)을 고친 모습입니다.

```
# 수정 전: 재고 차감 후 캐시를 비우지 않아 옛 재고가 남는다(원인)
def decrement_stock(product_id: int, qty: int) -> None:
    db.execute("UPDATE products SET stock = stock - ? WHERE id = ?", (qty, product_id))
    # 재고 캐시 무효화 누락 - 다음 조회가 차감 전 재고를 돌려줌

# 수정 후: 재고 차감 직후 해당 상품 캐시를 무효화(원인 해결)
def decrement_stock(product_id: int, qty: int) -> None:
    db.execute("UPDATE products SET stock = stock - ? WHERE id = ?", (qty, product_id))
    cache.invalidate(f"product:{product_id}") # 계획 2단계: 누락 경로에 무효화 추가
```

- 첫 계획대로 TTL만 줄였다면 이 decrement\_stock 은 그대로 남아, 품질 표시가 조금 빨라질 뿐 같은 버그가 재발했을 것입니다.
- 교정된 계획은 cache.invalidate(f"product:{product\_id}") 한 줄을 **누락된 쓰기 경로에 추가**해 원인을 해결합니다. 도메인 지식이 만든 교정이 코드에 정확히 반영됐습니다.
- 이 변경 역시 계획 승인 이후에야 일어났다는 점이 중요합니다. 틀린 첫 계획은 코드에 한 줄도 남기지 않았습니다.

이것이 계획-실행 분리의 정수입니다. 사람의 도메인 지식과 에이전트의 실행력을 **변경이 일어나기 전에** 맞춰, 틀린 방향으로 한 걸음도 가지 않게 합니다.

## 6.8 플랜 모드와 CLAUDE.md·서브에이전트 연계

플랜 모드는 홀로 쓰기보다 **앞서 배운 개념과 결합**할 때 가장 강합니다.

**CLAUDE.md(ch\_05)와의 연계.** 계획의 질은 에이전트가 가진 맥락의 질에 비례합니다. 플랜 모드의 읽기 전용 분석 단계에서 에이전트는 CLAUDE.md의 아키텍처·컨벤션·금지사항을 함께 참고합니다. 예를 들어 CLAUDE.md에 "checks.py = 점검 함수, report.py = 리포트 렌더"라는 아키텍처가 적혀 있으면, 에이전트의 계획은 "새 점검을 checks.py에 추가"처럼 **프로젝트 구조에 맞는** 단계를 내놓습니다. 반대로 헌법이 비어 있으면 계획도 헤맬니다. **좋은 CLAUDE.md가 좋은 계획을 만든다** — 이것이 u3 단원이 두 개념을 한데 묶은 이유입니다.

**서브에이전트(ch\_09 예고)와의 연계.** 큰 작업에서는 계획 단계에서 "조사"를 별도 서브에이전트에 맡겨 메인 맥락을 보호하기도 합니다. 메인 에이전트는 **계획·통합**에 집중하고, 영향 범위 조사 같은 한정 작업은 서브에이전트가 병렬로 처리해 결과만 올려보냅니다. 지금은 "플랜 모드의 계획이 더 큰 오케스트레이션의 출발점이 된다"는 정도만 기억하면 충분합니다. 자세한 위임 설계는 ch\_09에서 배웁니다.

### 조합의 큰 그림

CLAUDE.md(프로젝트가 어떤 곳인지) → 플랜 모드(이번에 무엇을 어떻게 바꿀지) → 실행(외과적 편집) → 커밋(ch\_07)으로 이어지는 흐름은, 실무 개발자가 신뢰할 수 있는 변경을 만드는 표준 파이프라인입니다. 이번 단원은 그 앞쪽 두 칸(맥락·계획)을 완성한 것입니다.

## 6.9 즉시 연습(2종 1세트): 계획 검토·교정 채우기

에이전트가 제시한 계획을 검토해 교정 지시를 작성하는 연습입니다. 모범 답안, 작성형 빈칸, 자가체크의 2종 1세트입니다.

### 모범 답안

상황: "주문 생성 API 핸들러에 요청 검증을 추가해 줘"라고 지시하자 에이전트가 아래 계획을 냈습니다.

```
## 계획: 주문 생성 요청 검증 추가
1. create_order 핸들러에서 모든 요청 필드를 정규식으로 즉시 검증
2. 검증 실패 시 sys.exit(1)
```

검토 결과 교정 지시(모범):

1번은 범위가 너무 넓어. 지금 필요한 건 'product\_id가 존재하는 상품인지'와 'quantity가 1 이상인지' 두 가지뿐이야. 전 필드 정규식 검증은 빼 줘. 그리고 검증 실패 시 프로세스를 종료(sys.exit)하지 말고 HTTP 400과 어떤 필드가 왜 잘못됐는지 담은 응답을 돌려주도록 단계를 바꿔. 마지막에 잘못된 요청(없는 상품·수량 0)으로 호출해 400과 에러 메시지가 제대로 나오는지 확인하는 검증 단계도 넣어 줘.

- 범위를 좁혔습니다(전 필드 정규식 검증 삭제 → product\_id-quantity 두 규칙으로). 원한 것보다 넓은 계획을 잘라 낸 것입니다.

- **단계 교체**(프로세스 종료 → HTTP 400 응답)로, API 핸들러에 맞는 실패 처리를 골랐습니다. `sys.exit(1)`은 요청 하나 때문에 서버를 죽이는 위험한 선택입니다.
- **검증 단계 추가**(잘못된 요청으로 호출해 400 확인)로, "어떻게 완료를 확인할지"를 계획에 넣었습니다. 6.5의 네 검토 질문(범위·순서·가정·검증) 중 셋을 적용한 모범입니다.

## 직접 작성: 계획 교정 (빈칸 채우기)

아래 계획을 6.5의 네 질문으로 검토하고, 빈칸(\_\_\_\_)을 채워 교정 지시를 완성하십시오.

상황: "오래된 임시 업로드 파일을 정리하는 배치 작업을 추가해 줘"

```
## 계획(에이전트 제시)
1. uploads/tmp/ 폴더의 모든 파일을 삭제
2. 완료
```

교정 지시(채우기):

- 범위: 1번은 위험해. "모든 파일 삭제"가 아니라 \_\_\_\_  
(어떤 기준으로 무엇만 지울지 좁혀 적기 — 예: 24시간 지난 파일만)
- 순서/안전: 실제 삭제 전에 \_\_\_\_ 단계를 먼저 넣어 줘  
(운영 파일 삭제는 되돌리기 어렵다 — 무엇을 먼저 할까?)
- 검증: 마지막에 \_\_\_\_ 로 결과를 확인하는 단계를 추가해 줘

**확장 과제:** 이 작업이 애초에 "곧장 실행"이 나은지, "플랜 모드"가 나은지 6.6 기준으로 판단하고 한 줄로 근거를 적어 보십시오.

## 자가체크 체크리스트

- [ ] 범위를 좁혀 위험한 "전부 삭제"를 기준 있는 삭제로 바꿨는가
- [ ] 되돌리기 어려운 삭제 **앞에** 안전 단계(백업·대상 확인 등)를 넣었는가
- [ ] 끝에 **검증** 단계를 추가했는가
- [ ] 교정을 추가·삭제·재정렬 중 무엇인지 분명히 표현했는가
- [ ] (확장) 이 작업에 플랜 모드가 필요한지 근거를 들어 판단했는가

## 6.10 흔한 오류와 오개념

플랜 모드에서 실무자가 자주 부딪히는 함정들입니다.

### 오개념 1: 단순 작업에도 플랜 모드를 써야 안전하다

플랜 모드는 복잡·고위험·다중 파일 작업에서 값을 합니다. 한 줄 오타 수정이나 사소한 추가에까지 계획·검토·승인을 거치면 느려지고, "계획 피로"로 정작 중요한 계획의 검토가 소홀해집니다.

교정: 6.6의 기준("되돌리는 비용이 큰가? 영향 범위가 불확실한가?")로 가려 쓰십시오. 안전장치는 위험한 곳에 집중할 때 효과적입니다.

**오개념 2: 계획을 받았으면 그대로 실행하면 된다**

계획을 검토 없이 곧장 승인하면 플랜 모드를 형식적으로만 쓰는 것입니다. 플랜 모드의 가치는 "계획을 받는 것"이 아니라 "**계획을 검토·교정하는 것**"에 있습니다. 검토 없는 승인은, 틀린 가정(6.7)을 그대로 실행에 통과시킵니다.

교정: 매 계획을 6.5의 네 질문(범위·순서·가정·검증)으로 점검하는 습관을 들이십시오. 최소한 "검증 단계가 있는가"만이라도 늘 확인하십시오.

**오개념 3: 플랜 모드에서도 파일이 바뀐다**

플랜 모드는 **읽기 전용**입니다. 이 단계에서 에이전트는 탐색·읽기·분석만 하고 편집·실행으로 파일이나 시스템을 바꾸지 않습니다. "계획만 보여 줘"라고 했는데 파일이 바뀔까 걱정할 필요가 없습니다 — 그게 이 모드의 보장입니다.

교정: 안심하고 간간하게 검토하십시오. 변경은 오직 승인 후 실행 단계에서만 일어납니다. 만약 계획 단계에서 무언가 바뀌었다면 모드가 제대로 켜지지 않았는지 확인하십시오.

**그 밖의 주의점**

- 계획이 너무 추상적이면("적절히 리팩터링한다") 검토할 수 없습니다. "어느 파일을 어떻게"가 보이도록 구체화를 요청하십시오.
- 승인 후 실행 중 방향이 어긋나면, 멈추고(Esc) 다시 플랜 모드로 돌아가 계획을 갱신할 수 있습니다. 한 번 승인했다고 끝까지 끌려갈 필요는 없습니다.
- 플랜 모드는 권한(ch\_04)·흑(ch\_11)을 대체하지 않습니다. "절대 막아야 할 것"은 여전히 권한·흑으로 강제하고, 플랜 모드는 "방향 합의"를 담당합니다.

---

## 6.11 단원 미니 프로젝트: 프로젝트 헌법과 구현 계획 세우기

---

u3 단원(ch\_05 CLAUDE.md + ch\_06 플랜 모드)을 마무리하는 작은 산출물을 만듭니다. **두 개념만으로** 완성하는 프로젝트입니다. 적용 개념은 c05(CLAUDE.md·컨텍스트)와 c06(플랜 모드)입니다.

**목표:** 샘플 저장소용 CLAUDE.md를 작성하고, 플랜 모드로 한 기능의 단계별 구현 계획을 받아 검토·교정해 PLAN.md로 확정한다. **산출물:** sample\_repo/CLAUDE.md + sample\_repo/PLAN.md

마일스톤(증분)으로 나눠 진행합니다.

| 마일스톤 | 할 일                               | 적용 개념 | 체크포인트                  |
|------|-----------------------------------|-------|------------------------|
| M1   | 빌드·테스트 명령·컨벤션·금지를 담은 CLAUDE.md 작성 | c05   | 명령 절에 테스트 명령이 있는가      |
| M2   | 플랜 모드로 한 기능의 단계별 계획 받기            | c06   | 계획이 검증 가능한 단계로 나뉘는가    |
| M3   | 잘못된 가정·과한 범위를 교정해 PLAN.md로 확정     | c06   | 교정한 흔적(추가·삭제·재정렬)이 있는가 |

## 모범 산출물

먼저 M1의 `sample_repo/CLAUDE.md` 입니다(ch\_05에서 익힌 골격). 사내 운영팀이 쓰는 작은 서비스 헬스체크 CLI를 대상으로 삼았습니다.

### # healthcheck — 프로젝트 헌법

설정에 적합한 서비스 URL들을 호출해 응답 상태를 점검하는 운영용 CLI입니다.

#### ## 명령

- 실행: ``python healthcheck.py --config services.yaml``
- 테스트: ``pytest -q``
- 포맷: ``black .``

#### ## 컨벤션

- Python 3.11+, 타입 힌트 필수, HTTP는 requests만 사용
- 함수 하나는 한 가지 책임만(설정 로딩·점검·리포트 분리)

#### ## 아키텍처

- ``healthcheck.py``: CLI 진입점(인자 파싱·종료 코드)
- ``probe.py``: `check_service(url)` — 상태코드·지연 측정
- ``report.py``: 점검 결과를 표/요약으로 렌더
- ``tests/``: probe·report 단위 테스트

#### ## 금지

- 외부 호출에 timeout 없이 요청하지 말 것(무한 대기 금지)
- 점검 실패해도 종료 코드 0으로 끝내지 말 것(CI가 통과로 오인)
- main 직접 커밋 금지

다음은 M2~M3을 거쳐 확정한 `sample_repo/PLAN.md` 입니다(플랜 모드로 받아 교정한 계획).

**# 구현 계획: --json 출력 옵션 추가**

점검 결과를 CI·모니터링이 파싱하기 쉽도록 JSON으로도 출력하는 옵션을 더한다.

**## 단계**

1. report.py에 to\_json(results) 추가(표준 json 모듈 사용)
2. (교정으로 추가) 실패가 하나라도 있으면 종료 코드 1로 끝내기
3. healthcheck.py 인자 파서에 --json 플래그 연결
4. tests/에 to\_json·종료 코드 단위 테스트(전체 정상/일부 실패 케이스)
5. python healthcheck.py --config services.yaml --json 로 실제 출력 확인
6. (교정으로 추가) black . 포맷

**## 검토 메모**

- 초기 계획엔 2·6번이 없었음 → 종료 코드 처리와 포맷을 추가(범위 보강)
- "출력 직전에 문자열을 직접 조립"을 json.dumps로 단순화(재정렬)

- M1의 CLAUDE.md는 ch\_05의 네 절(명령·컨벤션·아키텍처·금지)을 모두 채워, 이어질 플랜 모드가 **프로젝트 구조에 맞는 계획**을 내도록 맥락을 깔았습니다(6.8 연계). 특히 "timeout 없이 요청 금지"."실패 시 종료 코드 0 금지" 같은 운영 제약이 계획의 2번 단계를 끌어냈습니다.
- PLAN.md의 단계에 "(교정으로 추가)" 표시가 붙은 항목(2·6번)이 핵심입니다. 에이전트의 첫 계획을 **그대로 받지 않고** 종료 코드 처리와 포맷을 추가했음을 보여 줍니다(6.5 교정).
- 맨 아래 "검토 메모"는 무엇을 왜 고쳤는지 기록한 것입니다. 계획·실행 분리의 산물(검토 흔적)을 문서로 남겨, 나중에 결정의 근거를 추적할 수 있게 했습니다.

## 직접 작성: 미니 프로젝트 빈 스타터

아래 빈 스타터를 채워 여러분의 CLAUDE.md 와 PLAN.md 를 완성하십시오. 주제는 자유롭게(예: 간단한 가계부 CLI, 메모 도구) 정하십시오.

CLAUDE.md 골격:

**# \_\_\_\_ — 프로젝트 헌법**

(한 줄 설명): \_\_\_\_

**## 명령**

- 실행: \_\_\_\_
- 테스트: \_\_\_\_

**## 컨벤션**

- \_\_\_\_

**## 아키텍처**

- \_\_\_\_: (역할)

**## 금지**

- \_\_\_\_

PLAN.md 골격(플랜 모드로 받았다고 가정하고 교정까지):

# 구현 계획: \_\_\_\_ 기능 추가

## 단계

1. \_\_\_\_
2. \_\_\_\_
3. (교정으로 추가) \_\_\_\_ # 검토하며 보강한 단계
4. \_\_\_\_ 로 결과 확인 # 검증 단계는 빠뜨리지 말 것

## 검토 메모

- 초기 계획에서 \_\_\_\_ 를 고침(범위/순서/가정/검증 중 무엇?)

## 자가체크 체크리스트

- [ ] CLAUDE.md의 명령 절에 실행·테스트 명령이 들어갔는가(c05)
- [ ] CLAUDE.md에 사고를 예방하는 금지 항목이 하나 이상 있는가(c05)
- [ ] PLAN.md의 단계가 검증 가능하게 나뉘는가(추상적이지 않은가)(c06)
- [ ] 에이전트 초기 계획을 그대로 받지 않고 교정한 흔적(추가·삭제·재정렬)이 있는가(c06)
- [ ] PLAN.md 끝에 검증 단계(테스트·실행 확인)가 있는가(c06)
- [ ] CLAUDE.md가 PLAN.md의 계획 품질을 끌어올렸는가(아키텍처에 맞는 계획인가)(c05+c06 연계)

## 정리

이번 챕터에서는 플랜 모드 개념을 익히고, u3 단원을 미니 프로젝트로 마무리했습니다.

- 플랜 모드는 변경 전 읽기 전용 계획 단계로, 에이전트가 분석 후 계획을 제시하면 검토·교정·승인을 거쳐 실행으로 넘어갑니다.
- 계획 → 승인 → 실행의 분리는 틀린 방향을 "되돌리기"가 아니라 "안 만들기"로 옮겨, 가장 싼 시점에 오류를 잡습니다.
- 계획은 그대로 받지 말고 추가·삭제·재정렬로 교정하며, 범위·순서·가정·검증의 네 질문으로 점검합니다.
- 플랜 모드는 복잡·고위험·다중 파일 작업에 쓰고, 사소한 작업엔 곧장 실행이 낫습니다(과용 주의).
- CLAUDE.md(ch\_05)의 좋은 맥락이 좋은 계획을 만든다 — 두 개념을 묶어 미니 프로젝트로 적용했습니다.

이제 우리는 "프로젝트가 어떤 곳인지(CLAUDE.md)"와 "이번에 무엇을 어떻게 바꿀지(플랜 모드)"를 갖췄습니다. 다음 단원(u4)에서는 그렇게 만든 변경을 작은 논리 단위로 안전하게 기록하는 Git·커밋 통합(ch\_07)과, 반복 작업을 표준화하는 슬래시 커맨드(ch\_08)로 나아갑니다.



## 버전관리: Git과 커밋 통합

**학습 목표** - 변경 사항을 스테이징(staging)하고 관례(Conventional Commits) 메시지로 커밋하게 지시할 수 있습니다. - `git status`·`git diff` 로 변경 맥락을 확인하고 무엇을 한 커밋에 담을지 단위를 통제할 수 있습니다. - 브랜치(branch) 생성·전환과 PR(Pull Request) 초안 작성을 에이전트에 위임할 수 있습니다. - 커밋 관련 권한을 허용목록(allowlist)으로 안전하게 자동화할 수 있습니다.

바로 앞 6장까지 우리는 Claude Code로 파일을 탐색하고(3장), 권한으로 통제하며(4장), CLAUDE.md로 맥락을 주고(5장), 플랜 모드로 먼저 계획을 받는(6장) 일상 작업 루프를 익혔습니다. 그런데 에이전트가 파일을 고치고 새 코드를 만들어 내는 순간, 곧바로 따라오는 질문이 있습니다. "이 변경을 어떻게 기록으로 남길 것인가?" 입니다. 코드를 바꾸기만 하고 그대로 두면, 며칠 뒤에는 무엇을 왜 바꿨는지 아무도 기억하지 못합니다. 잘못된 변경을 되돌릴 수도 없습니다. 바로 이 지점에서 **버전관리(version control)** 가 작업 루프의 마지막 고리로 합류합니다.

이 장에서 배우는 **Git과 커밋 통합** 은, 에이전트가 만든 변경을 작은 논리 단위로 안전하게 기록하고, 그 기록을 의미 있는 메시지로 남기며, 협업을 위한 브랜치·PR까지 이어 가는 방법입니다. 핵심은 단순합니다. 에이전트는 변경의 **맥락** 을 알기 때문에 좋은 커밋 메시지를 제안할 수 있지만, **무엇을 언제 커밋할지** 를 결정하는 통제권은 끝까지 사람에게 있어야 한다는 것입니다. 이 장을 마치면, 종합 프로젝트에서 repo-audit의 각 기능을 "작은 커밋으로 쌓아 가는" 습관의 토대가 완성됩니다.

선수 개념 상기: 이 장은 3장의 **파일 탐색과 편집 워크플로우(c03)** 위에 세워집니다. 탐색→외과적 편집→실행 확인으로 만들어 낸 변경을, 이제 "검토→스테이징→커밋"으로 기록하는 단계가 더해집니다. 또 4장의 **권한과 허용목록(c04)** 도 그대로 쓰입니다. `git commit` 같은 명령은 부수효과가 있으므로 권한 확인이 따르며, 이 장 후반에서 `Bash(git commit:*)` 패턴으로 안전하게 자동화하는 법을 다룹니다.

## 도입: 버전관리를 워크플로우의 일부로

### 변경은 만드는 것으로 끝나지 않는다

3장에서 우리는 버그 한 줄을 외과적으로 고치고 실행으로 확인하는 법을 배웠습니다. 그런데 거기서 멈추면 그 수정은 "지금 이 순간"에만 존재합니다. 동료가 같은 파일을 건드리면 충돌이 나고, 이 수정이 다른 버그를 일으켜도 깔끔하게 되돌릴 방법이 없습니다. 변경을 **커밋(commit)** 으로 봉인해야 비로소 "되돌릴 수 있는 한 점"이 생깁니다.

전통적으로 개발자는 코드를 짜고, 터미널로 전환해 `git add`·`git commit` 을 손으로 치고, 메시지를 고민해 적었습니다. Claude Code는 이 흐름을 작업 루프 안으로 끌어들입니다. "방금 바꾼 걸 커밋해 줘"라고 말하면, 에이

전트는 지금 무엇이 바뀌었는지 스스로 확인하고, 관례에 맞는 메시지를 제안하며, 커밋을 수행합니다. 사람은 기록을 직접 타이핑하는 노동에서 벗어나 무엇을 한 단위로 묶을지 결정하는 판단에 집중하게 됩니다.

## 에이전트에게 버전관리를 맡길 때의 두 얼굴

에이전트가 git을 다루면 두 가지가 동시에 가능해집니다. 하나는 **편의**입니다. 메시지 작성, diff 요약, 브랜치 생성 같은 반복 노동을 위임할 수 있습니다. 다른 하나는 **위험**입니다. 통제 없이 맡기면 에이전트가 관련 없는 변경을 한 커밋에 몰아넣거나, 의도하지 않은 파일까지 스테이징할 수 있습니다. 그래서 이 장은 "편하게 맡기는 법"과 "안전하게 통제하는 법"을 항상 한 쌍으로 다룹니다.

### 왜 '작은 커밋'이 중요한가

커밋은 "되돌릴 수 있는 최소 단위"입니다. 한 커밋에 로그인 버그 수정과 새 결제 기능이 뒤섞여 있으면, 결제 기능만 되돌리고 싶어도 버그 수정까지 함께 사라집니다. 반대로 변경을 논리 단위로 쪼개 커밋하면, 나중에 어느 한 점만 골라 되돌리거나 그 점만 떼어 다른 브랜치로 옮길 수 있습니다. 에이전트와 일할 때 "한 번에 하나의 논리 변경 → 하나의 커밋" 원칙이 특히 중요한 이유입니다.

## git 상태·diff 확인과 변경 맥락 파악

### 정의

**버전관리(version control)**란, 파일의 변경 이력을 시점별로 기록해 언제든지 비교·복원할 수 있게 하는 체계이고, **Git**은 그 가장 널리 쓰이는 분산 버전관리 도구입니다. **커밋(commit)**은 그 이력에 찍는 한 점(스냅샷)이며, **스테이징(staging)**은 그 점에 담을 변경을 미리 골라 두는 대기 단계입니다.

에이전트에게 커밋을 맡기기 전에, 사람과 에이전트 모두 "지금 무엇이 바뀌었는가"를 알아야 합니다. 이때 쓰는 두 명령이 `git status` (어떤 파일이 바뀌었나)와 `git diff` (구체적으로 어떻게 바뀌었나)입니다. Claude Code는 커밋을 요청받으면 거의 항상 이 두 명령을 먼저 실행해 **변경 맥락을 스스로 파악** 합니다. 사람은 그 요약을 보고 커밋 단위를 판단합니다.

### 상태와 diff를 읽는 법

다음은 세션에서 "지금 git 상태를 보여 줘"라고 했을 때 에이전트가 실행하는 명령과 그 의미입니다.

```
git status --short
git diff
```

```
M src/auth.py
M README.md
?? src/utils/token.py
```

`git status --short`의 출력 한 줄 한 줄이 변경의 종류를 알려 줍니다.

| 표시                    | 의미                                   |
|-----------------------|--------------------------------------|
| M src/auth.py         | 추적 중인 파일이 수정됨(Modified), 아직 스테이징 안 됨 |
| ?? src/utils/token.py | git이 아직 모르는 새 파일(Untracked)          |
| A                     | 새로 스테이징된 파일(Added)                   |
| D                     | 삭제된 파일(Deleted)                      |

git diff 는 수정된 내용을 줄 단위로 보여 줍니다. - 로 시작하는 줄은 지워진 줄, + 로 시작하는 줄은 추가된 줄입니다. 에이전트는 이 diff를 읽고 "auth.py에서는 토큰 검증 로직을, README에서는 사용법 설명을 바꿨다"처럼 **변경을 논리별로 분류** 할 수 있습니다. 이 분류가 다음 단계인 "커밋 단위 통제"의 출발점입니다.

## diff 검토: 수용·수정·되돌리기 판단

diff를 읽었다면 이제 세 가지 중 하나를 판단합니다. **수용**(이대로 커밋해도 좋다), **수정**(일부를 더 고쳐야 한다), **되돌리기**(이 변경은 잘못됐으니 버린다)입니다. 이는 3장에서 배운 "제시된 변경을 검토하고 수용·수정·되돌리기를 판단한다"는 편집 루프의 원칙을, 이제 **커밋 직전**에 한 번 더 적용하는 것입니다. 특히 에이전트가 의도보다 많은 줄을 바꿨거나, 디버깅용 출력(print 등)을 남겨 두었는지를 diff에서 잡아내는 것이 사람의 핵심 역할입니다. 검토 없이 곧장 커밋하면, 그 군더더기가 영구 이력에 박제됩니다.

**팁: 커밋 전에 항상 diff를 보게 하라**

"바꾼 거 커밋해 줘"보다 "먼저 git status와 diff를 보여 주고, 무엇을 한 커밋에 담을지 제안해 줘"라고 지시하면 훨씬 안전합니다. 에이전트가 무엇을 커밋하려는지 사람이 먼저 검토할 수 있기 때문입니다. 이는 6장 플랜 모드의 "바꾸기 전에 계획을 본다"는 원칙을 커밋에 적용한 것입니다.

## 동작 원리: 스테이징 → 커밋 단위 통제

### Git 통합 — 작업 디렉토리 · 스테이징 · 커밋 단위 통제



Git의 변경 흐름은 세 개의 공간을 거칩니다. 이 구조를 이해하면 "왜 스테이징이 따로 있는지"가 분명해집니다.

**1단계 — 작업 디렉터리(working directory).** 에이전트가 Edit-Write로 파일을 바꾸면, 변경은 우선 작업 디렉터리에 머물습니다. 아직 git 이력에는 아무것도 기록되지 않은 "초안" 상태입니다. `git status` 에서 **M**(수정)이나 **??**(미추적)으로 보이는 것이 이 단계입니다.

**2단계 — 스테이징 영역(staging area).** `git add <파일>` 을 하면, 그 변경이 "다음 커밋에 담길 후보"로 골라져 스테이징 영역에 올라갑니다. 핵심은 여기서 **무엇을 담고 무엇을 뺄지 선택** 한다는 점입니다. 작업 디렉터리에 여러 변경이 섞여 있어도, 그중 일부만 스테이징해 한 커밋으로 묶을 수 있습니다. 이것이 "커밋 단위 통제"의 기술적 토대입니다.

**3단계 — 저장소(repository).** `git commit` 을 하면 스테이징된 변경이 메시지와 함께 이력에 한 점으로 봉인됩니다. 이 점은 고유한 식별자(해시)를 가지며, 언제든지 이 시점으로 비교·복원할 수 있습니다.

이 3단계 구조 덕분에 **"바꾼 것 전부"와 "이번 커밋에 담을 것"을 분리** 할 수 있습니다. 예를 들어 `auth.py`와 `README`를 둘 다 바꿨더라도, `auth.py`만 스테이징해 `fix:` 커밋으로 봉인하고, `README`는 다음 `docs:` 커밋으로 따로 남길 수 있습니다. 에이전트에게 커밋을 맡길 때 우리가 통제하는 핵심 지점이 바로 이 **스테이징 선택**입니다.

**주의: `git add .` 의 함정**

`git add .` 는 작업 디렉터리의 *모든* 변경을 한꺼번에 스테이징합니다. 빠르지만 위험합니다. 디버깅용 임시 출력, 실험 코드, 비밀 키가 든 설정 파일까지 통째로 커밋에 휩쓸려 들어갈 수 있습니다. 에이전트에게 커밋을 맡길 때는 "관련 파일만 골라 스테이징하라"고 지시하는 편이 안전합니다.

## worked example: 변경을 feat: 메시지로 커밋

이제 실제 세션을 처음부터 끝까지 따라가며, 하나의 변경을 관례 메시지로 커밋하는 흐름을 완전히 풀어 보겠습니다. 상황은 이렇습니다. 외부 결제 API를 호출하는 `api/client.py` 의 `fetch()` 함수에 **요청 타임아웃 옵션**을 새로 추가했습니다. 타임아웃이 없어 외부 서비스가 느릴 때 워커가 무한 대기하던 문제를 막기 위한 변경이고, 이를 커밋하려 합니다.

### 1) 변경 확인을 먼저 지시한다

나: `api/client.py`의 `fetch()`에 요청 타임아웃 인자를 추가했어.  
먼저 `git status`와 `diff`를 보여 주고, 어떤 메시지로 커밋할지 제안해 줘.

에이전트는 다음 명령을 실행해 맥락을 파악합니다.

```
git status --short
git diff api/client.py
```

```
M api/client.py
```

```

@@ def fetch(url):
- def fetch(url):
-     resp = requests.get(url)
+ def fetch(url, timeout=DEFAULT_TIMEOUT):
+     resp = requests.get(url, timeout=timeout)
+     resp.raise_for_status()
+     return resp.json()

```

커밋 단위를 판단하려면 변경 후의 코드가 한 가지 논리만 담는지 보는 것이 좋습니다. 현재 `api/client.py`의 변경은 다음과 같이, 오직 "요청 타임아웃 추가"라는 한 가지 의도만 반영하고 있습니다.

```

import requests

DEFAULT_TIMEOUT = 5.0 # 초

def fetch(url: str, timeout: float = DEFAULT_TIMEOUT) -> dict:
    """외부 API를 호출해 JSON을 돌려준다(timeout으로 무한 대기 방지)."""
    resp = requests.get(url, timeout=timeout) # 새로 추가된 인자 — 이 변경 하나만 커밋
    resp.raise_for_status()
    return resp.json()

```

변경이 한 가지 의도로 깔끔하면, 그만큼 커밋 단위를 나누기도 쉽고 메시지도 명확해집니다.

## 2) 에이전트가 메시지를 제안한다

diff를 읽은 에이전트는 "기능 추가"라는 변경의 성격을 파악하고 다음과 같이 제안합니다.

```

에이전트: api/client.py의 fetch()에 설정 가능한 요청 타임아웃을 추가하셨습니다.
새 기능이므로 feat 타입이 적절합니다. 다음 메시지를 제안합니다.

feat(client): add configurable request timeout to fetch()

api/client.py만 스테이징해서 이 메시지로 커밋할까요?

```

## 3) 승인하면 스테이징과 커밋을 수행한다

```

git add api/client.py
git commit -m "feat(client): add configurable request timeout to fetch()"

```

```

[main 4f2a1c9] feat(client): add configurable request timeout to fetch()
1 file changed, 3 insertions(+), 2 deletions(-)

```

### 줄별 해설

| 단계   | 명령/행동                               | 하는 일                                          |
|------|-------------------------------------|-----------------------------------------------|
| 확인   | <code>git status --short</code>     | 무엇이 바뀌었는지(api/client.py 1개) 파악합니다.            |
| 확인   | <code>git diff api/client.py</code> | 어떻게 바뀌었는지(timeout 인자 추가) 읽습니다.                |
| 판단   | 에이전트가 타입 제안                         | "새 기능"이므로 <code>feat</code> 타입을 골라 메시지를 만듭니다. |
| 승인   | 사람이 확인                              | 메시지와 대상 파일을 검토하고 진행을 허락합니다.                   |
| 스테이징 | <code>git add api/client.py</code>  | api/client.py만 골라 커밋 후보로 올립니다( . 이 아님).       |
| 커밋   | <code>git commit -m "..."</code>    | 스테이징된 변경을 메시지와 함께 한 점으로 봉인합니다.                |
| 결과   | <code>[main 4f2a1c9]</code>         | 커밋 해시·변경 통계로 봉인이 끝났음을 확인합니다.                  |

여기서 핵심은 **순서**입니다. "확인 → 제안 → 승인 → 커밋" 네 단계 중, 사람이 끼어드는 지점은 **승인**입니다. 에이전트가 무엇을 커밋하려는지 보고 나서 진행을 허락하기 때문에, 의도치 않은 파일이나 잘못된 메시지를 사전에 걸러 낼 수 있습니다.

## Conventional Commits 메시지 작성 위임

### 정의

**Conventional Commits(관례 커밋)**란, 커밋 메시지를 타입(범위): 설명 형식으로 통일하는 약속입니다. 메시지 첫머리에 변경의 종류를 붙여, 사람과 도구가 이력을 한눈에 분류·검색할 수 있게 합니다.

에이전트는 변경의 맥락을 알기 때문에 이 형식의 메시지를 잘 만듭니다. 우리가 할 일은 "관례를 따르라"고 한번 지시하고(또는 CLAUDE.md에 적어 두고), 제안된 타입이 맞는지 검토하는 것입니다. 대표 타입은 다음과 같습니다.

| 타입       | 언제 쓰나          | 예시                                      |
|----------|----------------|-----------------------------------------|
| feat     | 새 기능 추가        | feat(auth): add token refresh           |
| fix      | 버그 수정          | fix(parser): handle empty input         |
| docs     | 문서만 변경         | docs(readme): add usage section         |
| refactor | 동작 변화 없는 구조 개선 | refactor(report): extract render helper |
| test     | 테스트 추가·수정      | test(checks): cover broken link case    |
| chore    | 빌드·설정 등 잡무     | chore: bump dependency version          |

## 좋은 메시지는 '왜'를 담는다

타입만 맞춘다고 좋은 메시지가 아닙니다. 설명 부분은 "무엇을 했는가"를 넘어 "왜 했는가"를 담을 때 나중에 도움이 됩니다. 본문(body)을 덧붙이면 맥락을 더 남길 수 있습니다.

```
fix(audit): skip symlinked dirs to avoid infinite scan loop
```

심볼릭 링크가 상위 폴더를 가리키면 점검이 무한 순환에 빠지는 문제가 있었다. 방문한 실제 경로를 집합으로 추적해 재방문을 막는다.

### 팁: CLAUDE.md에 커밋 규칙을 박아 두기

5장에서 배운 CLAUDE.md에 "커밋: Conventional Commits(feat, fix, docs, refactor, test, chore)를 따른다"라고 한 줄 적어 두면, 이후 모든 세션에서 에이전트가 별도 지시 없이도 관례를 지킵니다. 영속 컨텍스트(c05)와 커밋 통합(c07)이 자연스럽게 맞물리는 지점입니다.

## 브랜치 생성·전환과 PR 초안(gh 연계)

혼자가 아니라 팀으로 일할 때, 변경은 보통 **브랜치(branch)** 위에서 이뤄지고 **PR(Pull Request)** 로 합쳐집니다. 에이전트는 이 흐름도 위임받을 수 있습니다.

### 작업 브랜치를 먼저 만든다

기본 브랜치(main)에 직접 커밋하기보다, 작업마다 새 브랜치를 파는 것이 안전합니다. "토론 갱신 기능을 만들 거니까 브랜치부터 만들어 줘"라고 하면 에이전트는 다음을 실행합니다.

```
git switch -c feat/token-refresh
```

```
Switched to a new branch 'feat/token-refresh'
```

이제 이 브랜치 위에서 변경하고 커밋하면, main은 그대로 보존됩니다. 실험이 잘못돼도 브랜치만 버리면 됩니다.

## PR 초안을 gh로 작성한다

GitHub CLI(gh)가 설치돼 있으면, 에이전트는 브랜치를 푸시하고 PR 초안까지 만들 수 있습니다. 에이전트는 그동안의 커밋을 요약해 PR 제목·본문을 제안합니다.

```
git push -u origin feat/token-refresh
gh pr create --title "feat: add token refresh" --body "토큰 만료 시 자동 갱신 로직 추가. 401 응답을 받으면 refres
```

```
https://github.com/acme/api/pull/142
```

에이전트가 PR 본문을 그냥 지어내는 것이 아니라, 그 브랜치의 커밋들을 `git log`로 읽어 **무엇이 왜 바뀌었는지 요약** 한다는 점이 중요합니다. 작은 관련 커밋을 쌓아 두었다면(앞 절의 습관), PR 요약도 그만큼 정확하고 풍부해집니다. 즉 "작게 자주 커밋" 습관은 PR 품질로도 이어집니다.

### 주의: 푸시·PR은 '되돌리기 어려운' 행동이다

브랜치 생성·로컬 커밋은 언제든지 되돌릴 수 있지만, 원격 푸시(`git push`)와 PR 생성은 외부에 공개됩니다. 이런 명령은 권한 확인을 거치게 두는 편이 좋습니다(다음 절에서 다룹니다). 특히 `git push --force`는 동료의 작업을 덮어쓸 수 있으므로, 에이전트가 함부로 쓰지 못하게 막아 두는 것을 권장합니다.

## worked example: 변경을 두 논리 커밋으로 분리

이번에는 더 까다로운 상황입니다. 운영에서 보고된 **청구 금액 반올림 버그**를 고치면서, 그 버그를 **재현·고정하는 회귀 테스트까지** 작성했습니다. 버그 수정(`fix`)과 테스트 추가(`test`)는 성격이 다른 두 변경이므로, 한 커밋에 몰지 않고 논리 단위로 나눠 두 개의 커밋으로 분리하는 과정을 풀어 보겠습니다. 이렇게 나눠 두면 나중에 "수정만" 또는 "테스트만" 떼어 보거나 되돌릴 수 있습니다.

### 1) 섞인 변경을 확인한다

나: 청구 반올림 버그를 고치고 회귀 테스트도 추가했어. 성격이 다른 변경이니 먼저 무엇이 바뀌었는지 분류해서 보여 줘.

```
git status --short
```

```
M src/billing.py
?? tests/test_billing.py
```

에이전트가 diff를 읽고 분류합니다. `billing.py`에는 다음과 같은 버그 수정이 들어 있습니다.

```
def calc_tax(amount: int, rate: float) -> int:
    """청구 세액을 계산한다(원 단위 정수)."""
    # 수정 전: int()로 내림 -> 청구액이 1원씩 적게 잡히던 버그
    # 수정 후: round()로 반올림해 누락분 제거
    return round(amount * rate)
```

반면 `tests/test_billing.py` 는 그 버그를 재현하는 새 회귀 테스트(테스트 추가)일 뿐입니다. 에이전트는 둘을 다음과 같이 분류합니다.

에이전트: 두 종류의 변경이 섞여 있습니다.

- (1) `src/billing.py`: 세액 계산의 내림 -> 반올림 버그 수정 — 버그 수정(fix)
- (2) `tests/test_billing.py`: 그 버그를 고정하는 회귀 테스트 추가 — 테스트(test)

성격이 다르므로 두 개의 커밋으로 나누길 권합니다.

## 2) 첫 번째 논리 단위만 스테이징해 커밋한다

```
git add src/billing.py
git commit -m "fix(billing): round tax amount instead of truncating"
```

```
[main a91f3e2] fix(billing): round tax amount instead of truncating
1 file changed, 2 insertions(+), 1 deletion(-)
```

이때 새 테스트 파일은 일부러 스테이징하지 않았습니다. 그래서 첫 커밋에는 버그 수정만 담깁니다.

## 3) 남은 변경을 두 번째 커밋으로 봉인한다

```
git add tests/test_billing.py
git commit -m "test(billing): add regression test for tax rounding"
```

```
[main c30b7d8] test(billing): add regression test for tax rounding
1 file changed, 9 insertions(+)
```

```
git log --oneline -2
```

```
c30b7d8 test(billing): add regression test for tax rounding
a91f3e2 fix(billing): round tax amount instead of truncating
```

### 줄별 해설

| 단계   | 명령                                              | 하는 일                                                      |
|------|-------------------------------------------------|-----------------------------------------------------------|
| 분류   | <code>git status --short</code> + diff 읽기       | 쉬인 변경을 "버그 수정(billing.py)"과 "테스트(test_billing.py)"로 나눕니다. |
| 선택 1 | <code>git add src/billing.py</code>             | 버그 수정만 스테이징(테스트 파일은 제외).                                  |
| 봉인 1 | <code>git commit -m "fix(billing): ..."</code>  | 수정만 담은 첫 커밋을 만듭니다.                                        |
| 선택 2 | <code>git add tests/test_billing.py</code>      | 남은 테스트 변경을 스테이징.                                          |
| 봉인 2 | <code>git commit -m "test(billing): ..."</code> | 테스트만 담은 둘째 커밋을 만듭니다.                                      |
| 확인   | <code>git log --oneline -2</code>               | 두 커밋이 각각 한 가지 성격만 담았음을 확인합니다.                             |

이렇게 나누면, 핫픽스로 `fix` 커밋만 먼저 릴리스 브랜치에 체리픽하거나, 수정만 되돌릴 때 테스트 변경에 영향을 주지 않습니다. 스테이징 영역에서 무엇을 담을지 고르는 능력 이 커밋 단위 통제의 전부라는 점을 다시 확인할 수 있습니다.

## 커밋 관련 권한(Bash(git commit:\*)) 설계

### 매번 묻는 마찰을 줄이되, 위험은 막는다

4장에서 배운 대로, `git commit`·`git add` 같은 명령은 부수효과가 있어 기본적으로 권한 확인을 거칩니다. 커밋을 자주 한다면 이 확인이 마찰이 됩니다. 그렇다고 모든 Bash를 허용하면 위험합니다. 해법은 **git 관련 명령만 좁게 허용** 하는 것입니다.

`.claude/settings.json` 에 다음과 같이 패턴을 적습니다.

```
{
  "permissions": {
    "allow": [
      "Bash(git add:*)",
      "Bash(git commit:*)",
      "Bash(git status:*)",
      "Bash(git diff:*)",
      "Bash(git log:*)",
      "Bash(git switch:*)"
    ],
    "ask": [
      "Bash(git push:*)"
    ],
    "deny": [
      "Bash(git push --force:*)",
      "Bash(git reset --hard:*)"
    ]
  }
}
```

## 줄별 해설

| 규칙                                 | 의미                                      |
|------------------------------------|-----------------------------------------|
| allow: Bash(git add:*)             | git add 로 시작하는 명령은 묻지 않고 허용합니다.         |
| allow: Bash(git commit:*)          | 커밋은 자동 허용 — 커밋 노동을 마찰 없이 위임합니다.         |
| allow: Bash(git status/diff/log:*) | 읽기 전용 git 명령은 안전하므로 허용합니다.              |
| ask: Bash(git push:*)              | 원격 푸시는 외부에 공개되므로 매번 확인을 받습니다.           |
| deny: Bash(git push --force:*)     | 강제 푸시는 동료 작업을 덮을 수 있어 <b>항상 차단</b> 합니다. |
| deny: Bash(git reset --hard:*)     | 작업 디렉터리를 통째로 날리는 위험 명령을 차단합니다.          |

allow · ask · deny 의 조합이 핵심입니다. 안전한 명령(스테이징·커밋·조회)은 자동 허용해 속도를 얻고, 공개되는 명령(푸시)은 확인을 받으며, 파괴적인 명령(강제 푸시·하드 리셋)은 아예 막습니다. 이것이 4장의 **최소 권한 원칙**을 git에 적용한 모습입니다.

### 위험: Bash(git:\*) 한 줄로 통치지 말 것

"git 관련은 다 허용하자"며 Bash(git:\*) 하나만 적으면, git push --force · git reset --hard · git clean -fd 까지 전부 자동 허용됩니다. 패턴이 넓을수록 위험한 명령이 그 그물에 함께 걸립니다. 허용은 **구체적인 하위 명령 단위**로 좁히고, 위험 명령은 deny 로 명시적으로 막아야 안전합니다.

## 즉시 연습 (2종 1세트)

배운 것을 직접 적용해 봅니다. 각 연습은 ① 모범 흐름을 참고해 ② 빈칸을 채우고 ③ 자가체크로 점검하는 순서입니다.

## 연습 1: 커밋 단위 판단과 메시지 작성

다음 상황에서, 변경을 몇 개의 커밋으로 나눌지 결정하고 각 커밋의 메시지를 관례에 맞게 적어 보십시오.

상황: 결제 서비스 작업 중 한 세션에서 (a) `billing.py` 에 환불 금액 계산 함수를 새로 추가했고, (b) `billing.py` 의 기존 세액 계산 함수에 있던 통화 반올림 버그를 고쳤으며, (c) `tests/test_billing.py` 에 두 변경을 검증하는 테스트를 추가했다.

### 작성형 빈 템플릿 — 커밋 계획을 채우세요

[커밋 단위 계획]  
이 변경은 \_\_\_\_ 개의 커밋으로 나눕니다. (이유: \_\_\_\_\_)

커밋 1

- 스테이징할 파일: \_\_\_\_\_
- 타입: \_\_\_\_\_ 범위: \_\_\_\_\_
- 메시지: \_\_\_\_\_

커밋 2

- 스테이징할 파일: \_\_\_\_\_
- 타입: \_\_\_\_\_ 범위: \_\_\_\_\_
- 메시지: \_\_\_\_\_

(필요하면 커밋 3 추가)

커밋 3

- 스테이징할 파일: \_\_\_\_\_
- 타입: \_\_\_\_\_ 범위: \_\_\_\_\_
- 메시지: \_\_\_\_\_

**자가체크** - [ ] 서로 다른 성격(기능 추가·버그 수정·테스트)을 각각 다른 커밋으로 분리했는가 - [ ] 각 메시지가 타입(범위): 설명 형식을 따르는가 - [ ] 한 커밋에 무관한 파일이 섞이지 않았는가 - [ ] 설명이 "무엇"뿐 아니라 가능하면 "왜"를 암시하는가

## 연습 2: git 권한 규칙 설계

팀 저장소에서 커밋은 자동화되 사고는 막고 싶습니다. `settings.json` 의 권한 규칙을 완성해 보십시오.

### 작성형 빈 템플릿 — `settings.json`을 채우세요

```
{
  "permissions": {
    "allow": [
      "Bash(git ____:*)",
      "Bash(git ____:*)",
      "Bash(git status:*)"
    ],
    "ask": [
      "Bash(git ____:*)"
    ],
    "deny": [
      "Bash(git _____:*)"
    ]
  }
}
```

요구사항: 1. 스테이징과 커밋은 묻지 않고 허용한다. 2. 원격 푸시는 매번 확인을 받는다. 3. 강제 푸시는 항상 차단한다.

**자가체크** - [ ] 스테이징(`git add`)과 커밋(`git commit`)을 `allow`에 넣었는가 - [ ] 푸시(`git push`)를 `ask`에 두어 확인을 받게 했는가 - [ ] 강제 푸시(`git push --force`)를 `deny`로 막았는가 - [ ] `Bash(git:*)`처럼 너무 넓은 패턴으로 통치하지 않았는가

## 흔한 오류와 오개념

### 오개념 1 — "에이전트가 커밋하니 나는 신경 안 써도 된다"

에이전트는 메시지를 *제안* 하고 커밋을 *수행* 할 뿐, **무엇을 한 단위로 묶을지**는 사람이 통제해야 합니다. 통제를 놓으면 무관한 변경이 한 커밋에 뒤섞이거나 임시 파일이 휩쓸려 들어갑니다. "커밋 전에 `status-diff`를 보여주고 제안하라"는 한 마디로 통제권을 유지하십시오.

### 오개념 2 — "한 번에 몰아서 커밋하면 편하다"

세션 끝에 모든 변경을 `git add .` + 한 커밋으로 몰면 당장은 편하지만, 나중에 한 부분만 되돌리거나 떼어 낼 수 없습니다. 커밋은 "되돌릴 수 있는 최소 단위"여야 가치가 있습니다. 변경이 끝날 때마다 작은 논리 단위로 자주 커밋하는 편이 결국 안전하고 빠릅니다.

### 오개념 3 — "메시지는 무엇을 했는지만 적으면 된다"

`fix: bug`처럼 '무엇'만 적은 메시지는 몇 달 뒤 아무 정보도 주지 못합니다. 좋은 메시지는 타입으로 종류를, 설명으로 *왜*를 담습니다. 에이전트는 변경 맥락을 알기에 '왜'가 담긴 메시지를 만들 수 있습니다. 제안된 메시지가 맥락을 담았는지 검토하는 것이 사람의 몫입니다.

### 오개념 4 — "git 권한은 다 열어 두면 빠르다"

`Bash(git:*)` 한 줄로 모두 허용하면 `git reset --hard`·`git push --force` 같은 파괴적 명령까지 자동 실행됩니다. 속도를 위해 안전을 통째로 내주는 거래입니다. 안전한 하위 명령만 좁게 `allow`하고 위험 명령은 `deny`로 막는 것이 진짜로 빠르고 안전한 설계입니다.

## 자주 묻는 질문(FAQ)

### Q. 에이전트가 만든 커밋에 자동으로 표시(서명)가 붙나요?

도구·설정에 따라 커밋 메시지 끝에 협업자 정보(Co-Authored-By)가 붙도록 운용하는 팀도 있습니다. 핵심은 "에이전트가 도왔다"는 사실을 이력에 남길지 여부이며, 이는 팀 컨벤션입니다. 필요하다면 CLAUDE.md에 "커밋 메시지 끝에 협업자 줄을 붙인다"처럼 규칙으로 적어 두면 일관되게 적용됩니다(5장 영속 컨텍스트).

### Q. 커밋을 잘못했습니다. 에이전트에게 되돌리게 해도 됩니까?

아직 푸시하지 않은 로컬 커밋이라면 `git reset` 계열로 되돌릴 수 있습니다. 다만 `git reset --hard`는 작업 디렉터리까지 날리므로 위험합니다(이 장에서 `deny`로 막은 명령). 안전하게는 마지막 커밋만 취소하고 변경은 보존하는 방식을 쓰고, 이미 푸시·공유된 커밋이라면 되돌리기 대신 새 커밋(`revert`)으로 상쇄하는 편이 협업에 안전합니다.

### Q. diff가 너무 길어 검토가 부담스럽습니다. 매번 다 봐야 하나요?

전체를 한 줄씩 다 읽을 필요는 없습니다. 에이전트에게 "변경을 파일별·논리별로 요약해 줘"라고 하면 핵심만 추려 줍니다. 다만 *무엇이 커밋에 담기는지*의 파일 목록과 요약은 반드시 확인하십시오. 검토를 통째로 건너뛰고 자동 수용하는 것이 진짜 위험입니다.

### Q. feat·fix 같은 타입을 꼭 외워야 하나요?

전부 외울 필요는 없습니다. 자주 쓰는 `feat`(기능)·`fix`(수정)·`docs`(문서)·`refactor`(구조)·`test`·`chore` 정도만 알면 충분하고, 에이전트가 변경 성격에 맞는 타입을 제안해 줍니다. 여러분의 일은 제안된 타입이 변경 성격과 맞는지 검토하는 것입니다.

## 정리

이 장에서는 에이전트와 함께 버전관리를 다루는 법을 배웠습니다. 핵심을 다시 짚으면 다음과 같습니다.

- 변경은 **확인(status·diff) → 스테이징 선택 → 커밋 봉인**의 3단계로 기록되며, 통제의 핵심 지점은 **무엇을 스테이징할지 고르는 선택**입니다.
- 에이전트는 변경 맥락을 알기에 **Conventional Commits** 메시지를 잘 제안하지만, 커밋 단위를 결정하는 통제권은 사람에게 있습니다.
- 브랜치 생성·PR 초안(gh 연계)도 위임할 수 있되, 푸시처럼 공개되는 행동은 권한 확인을 거치게 합니다.
- 커밋 권한은 `Bash(git commit:*)`처럼 **좁은 패턴으로 자동화** 하고, 위험 명령은 `deny`로 막아 안전과 속도를 함께 얻습니다.

다음 8장에서는 이렇게 익힌 커밋 워크플로우를 포함해, 자주 반복하는 절차 전체를 /이름 한 번으로 부르는 **슬래시 커맨드**를 배웁니다. 그리고 8장 끝의 단위 미니 프로젝트에서 "커밋 워크플로우 + 커스텀 커맨드"를 한 산출물로 묶어, 이 단위(u4)의 두 개념을 함께 적용하게 됩니다.



## 반복 표준화: 슬래시 커맨드

**학습 목표** - 내장 슬래시 커맨드(`/clear` · `/compact` · `/review` · `/help`)로 세션을 효율적으로 운용할 수 있습니다. - `.claude/commands`에 인자(`$ARGUMENTS`)를 받는 커스텀 커맨드를 정의할 수 있습니다. - 반복 워크플로우를 커맨드로 표준화해 팀과 공유할 수 있습니다. - 커맨드와 스킬(skill)·서브에이전트(subagent)의 관계를 설명할 수 있습니다.

7장에서 우리는 변경을 작은 논리 단위로 커밋하고, 관련 메시지를 위임하며, 브랜치·PR까지 이어 가는 흐름을 배웠습니다. 그런데 이 흐름을 며칠만 반복해 보면 한 가지가 눈에 띄니다. **매번 거의 같은 말을 다시 타이핑하고 있다**는 점입니다. "먼저 `git status`와 `diff`를 보여 주고, Conventional Commits 메시지를 제안한 뒤, 관련 파일만 스테이징해서 커밋해 줘" — 이 긴 지시를 커밋할 때마다 처음부터 칩니다. 코드 리뷰를 요청할 때도, 테스트를 짤 때도 마찬가지입니다. 같은 절차를, 같은 표현으로, 반복해서.

이 장에서 배우는 **슬래시 커맨드(slash command)**는 바로 그 반복을 없앱니다. 자주 쓰는 절차를 한 번 템플릿으로 적어 두면, 이후로는 `/commit` · `/review` 처럼 짧은 이름 하나로 그 절차 전체를 불러올 수 있습니다. 핵심은 슬래시 커맨드가 단순한 글자 단축(매크로)이 아니라, **맥락을 주입하는 재사용 프롬프트 템플릿**이라는 점입니다. 인자를 받고 파일을 참조하기 때문에, 같은 커맨드가 매번 다른 대상에 적용됩니다. 이 장을 마치면, 팀이 "리뷰는 이렇게 한다"를 코드로 공유하는 방법을 갖추게 됩니다.

선수 개념 상기: 이 장은 2장의 **효과적 프롬프팅과 작업 지시(c02)** 위에 세워집니다. 좋은 지시(목표·맥락·제약·완료 조건)를 만드는 법을 배웠다면, 슬래시 커맨드는 **그 좋은 지시를 한 번 잘 써서 재사용**하는 도구입니다. 또 5장의 **CLAUDE.md·컨텍스트(c05)** 와도 맞닿습니다. CLAUDE.md가 "항상 읽히는 규칙"이라면, 커맨드는 "부를 때 주입되는 절차"입니다.

## 도입: 자주 쓰는 절차를 /이름으로 부르기

### 같은 지시를 반복하는 비용

에이전트와 일하다 보면, 잘 다듬은 지시문 몇 개가 손에 익습니다. 코드 리뷰를 요청하는 표현, 테스트를 작성시키는 표현, 릴리스 노트를 뽑는 표현. 문제는 이것들이 *머릿속이나 메모장*에만 있다는 점입니다. 매번 다시 떠올려 타이핑해야 하고, 표현이 조금씩 달라져 결과의 일관성도 흔들립니다. 팀이라면 더 심합니다. 사람마다 리뷰 요청 방식이 달라, 누가 시키느냐에 따라 리뷰 품질이 들쭉날쭉합니다.

슬래시 커맨드는 이 다듬어진 지시를 **저장소 안의 파일**로 박제합니다. `.claude/commands/review.md`에 "이런 관점으로 리뷰하라"는 절차를 한 번 적어 두면, 팀의 누구든 `/review`만 치면 똑같은 고품질 절차를 부릅니다. 좋은 지시가 **개인의 노하우**에서 **팀의 자산**으로 승격되는 것입니다.

### 부르는 즉시 펼쳐지는 프롬프트

슬래시 커맨드의 동작은 직관적입니다. 세션에서 `/review` 라고 입력하면, Claude Code는 `.claude/commands/review.md` 파일의 내용을 읽어 **그 자리에 펼쳐 넣습니다**. 즉 여러분이 그 긴 지시문을 손으로 친 것과 같은 효과가 납니다. 다른 점은 단 하나, 여러분은 다섯 글자만 쳤다는 것입니다.

### 슬래시 커맨드 = 저장된 프롬프트, 그 이상

가장 단순한 슬래시 커맨드는 "자주 쓰는 프롬프트를 파일에 저장해 둔 것"입니다. 하지만 진짜 힘은 그다음에서 나옵니다. 커맨드는 인자(\$ARGUMENTS)를 받아 매번 다른 대상에 적용되고, @파일 로 실제 코드를 참조하며, 프런트매터(frontmatter)로 사용할 도구까지 제한할 수 있습니다. 그래서 "저장된 프롬프트"가 아니라 **"매개변수를 받는 재사용 절차"**라고 부르는 편이 정확합니다.

## 내장 커맨드 개요 (/clear·/compact·/review·/help)

커스텀 커맨드를 만들기 전에, Claude Code가 기본 제공하는 **내장 커맨드(built-in command)** 부터 익혀 둡니다. 이들은 별도 정의 없이 바로 쓸 수 있고, 세션 운용의 기본기입니다.

| 커맨드                   | 하는 일                     | 언제 쓰나                    |
|-----------------------|--------------------------|--------------------------|
| <code>/help</code>    | 사용 가능한 커맨드와 도움말을 표시      | 무엇이 가능한지 막힐 때            |
| <code>/clear</code>   | 대화 컨텍스트를 초기화(새 세션처럼)     | 주제가 완전히 바뀔 때, 맥락이 오염됐을 때 |
| <code>/compact</code> | 대화를 핵심만 남기고 압축           | 긴 세션에서 맥락은 유지하되 토큰을 줄일 때 |
| <code>/review</code>  | 변경/PR을 코드 리뷰             | 커밋/PR 전에 자동 점검을 받을 때     |
| <code>/init</code>    | 저장소를 분석해 CLAUDE.md 초안 생성 | 새 프로젝트에 처음 들어갈 때         |
| <code>/agents</code>  | 서브에이전트 정의를 보고 관리         | 위임 구조를 점검할 때(9장)         |

### /clear와 /compact의 차이

이 둘은 5장에서도 짧게 다뤘지만, 자주 헷갈리므로 분명히 해 둡니다.

- `/clear` 는 대화를 **통째로 비웁니다**. 직전까지의 맥락이 사라지고 새 세션처럼 시작합니다. 앞 작업과 무관한 새 일을 시작할 때 씁니다. 맥락이 길어져 에이전트가 엉뚱한 옛 정보를 끌어다 쓸 때도 효과적입니다.
- `/compact` 는 대화를 **요약해 압축** 합니다. 핵심 결정·맥락은 남기고 장황한 부분만 줄여, 토큰 여유를 확보하면서도 작업의 연속성을 유지합니다. 긴 작업을 이어서 계속할 때 씁니다.

**팁:** 세션이 길어지면 먼저 `/compact`, 주제가 바뀌면 `/clear`

한 작업을 오래 이어 가다 에이전트의 응답이 느려지거나 옛 맥락을 끌어오면 `/compact` 로 정리하십시오. 반대로 "이제 완전히 다른 일을 하자" 싶으면 `/clear` 로 깨끗이 비우는 편이 낫습니다. 무관한 옛 맥락을 끌고 다니면 결과 품질이 떨어집니다(5장 "무한 컨텍스트 가정" 오개념 참고).

## 동작 원리: 커맨드는 맥락을 주입하는 프롬프트 템플릿

### 슬래시 커맨드: 프롬프트 템플릿 주입 흐름



#### 커맨드 = 재사용 가능한 프롬프트 템플릿

반복 워크플로우를 /이름 한 줄로 표준화·공유

프로젝트 커맨드: `.claude/commands/` | 사용자 커맨드: `~/.claude/commands/`

커스텀 슬래시 커맨드가 어떻게 동작하는지 단계로 살펴보겠습니다. `/review-module src/checks.py` 를 입력했다고 가정합니다.

**1단계 — 이름으로 파일을 찾는다.** Claude Code는 `/` 뒤의 이름(`review-module`)을 보고

`.claude/commands/review-module.md` 파일을 찾습니다. 프로젝트에 없으면 사용자 디렉터리(`~/.claude/commands/`)에서 찾습니다(범위는 뒤에서 다룹니다).

**2단계 — 인자를 치환한다.** 커맨드 이름 뒤에 붙은 나머지 문자열(`src/checks.py`)이 템플릿 안의 `$ARGUMENTS` 자리에 그대로 끼워 넣어집니다. 즉 템플릿에 `$ARGUMENTS` 모듈을 리뷰해 줘 라고 적혀 있으면, 실제로는 `src/checks.py` 모듈을 리뷰해 줘 로 펼쳐집니다.

**3단계 — 참조를 해석한다.** 템플릿 안에 `@파일경로`가 있으면 그 파일의 `내용`이 프롬프트에 주입되고, `!명령` 같은 동적 참조가 있으면 그 명령의 `출력`이 주입됩니다(에디션에 따라 지원). 이로써 커맨드는 정적인 글이 아니라 **현재 코드·상태를 끌어다 쓰는 살아 있는 프롬프트**가 됩니다.

**4단계 — 완성된 프롬프트를 세션에 삽입한다.** 치환·참조가 모두 끝난 최종 텍스트가, 마치 여러분이 직접 입력한 것처럼 세션에 들어가 에이전트가 작업을 시작합니다.

정리하면, 슬래시 커맨드는 **이름 → 파일 찾기 → \$ARGUMENTS 치환 → @! 참조 주입 → 완성 프롬프트 삽입**의 파이프라인입니다. 이 구조를 이해하면, 커맨드를 "매크로"가 아니라 **"매개변수를 받는 프롬프트 함수"**로 다룰 수 있습니다.

#### 커맨드 파일의 기본 구조

커스텀 커맨드는 마크다운 파일 하나입니다. 맨 위에 선택적 **프런트매터**(`---`로 감싼 YAML)로 설명·허용 도구·인자 힌트를 적고, 그 아래 본문에 실제 프롬프트(절차)를 씁니다. 파일 이름이 곧 커맨드 이름이 됩니다(`commit.md`

→ `/commit`).

## worked example: `/test` 커스텀 커맨드 만들기

실무 시나리오를 가정하겠습니다. 결제 백엔드 팀이 운영하는 주문 서비스에, 누군가 PR마다 "변경한 모듈의 테스트는 빠뜨리지 말자"고 매번 코드 리뷰에서 지적합니다. 그런데 사람마다 "변경 모듈을 어떻게 찾고, 어떤 경계 조건까지 테스트할지" 기준이 달라 테스트 누락이 반복됩니다. 이 절차를 `/test` 커맨드로 박제 해 팀 전체가 같은 방식으로 변경 모듈의 테스트를 보완하게 만들겠습니다.

### 1) 커맨드 파일을 만든다

`.claude/commands/test.md` 파일을 다음과 같이 작성합니다.

```
---
description: 이번 변경 모듈의 누락된 단위 테스트를 보완하고 실행한다
allowed-tools: Read, Edit, Write, Bash(pytest:*), Bash(git diff:*)
---
```

이번 작업 브랜치에서 변경된 모듈의 단위 테스트를 보완해 주세요.

절차:

1. `git diff --name-only`로 바뀐 파일을 추리고, 비즈니스 로직 모듈을 고른다.
2. 변경된 함수의 정상 경로와 경계 조건(0·음수·상한 초과·None)을 각각 테스트한다.
3. `tests/` 아래에 기존 명명 규칙(`test_<모듈>.py`)에 맞춰 `pytest` 형식으로 작성한다.
4. `pytest`를 실행해 전부 통과하는지 확인하고 통과/실패 개수를 보고한다.

완료 조건: 변경 모듈의 정상·경계 테스트가 추가되고 `pytest`가 전부 통과한다.

### 2) 세션에서 호출한다

나: `/test`

### 3) 에이전트가 절차대로 수행한다

이번 변경은 주문 금액에 쿠폰 할인을 적용하되 할인액 상한을 두는 함수입니다(`src/orders/pricing.py`).

```
def apply_discount(subtotal: int, coupon_rate: float, cap: int) -> int:
    """쿠폰 할인을 적용하되 할인액이 상한(cap)을 넘지 않게 한다."""
    discount = int(subtotal * coupon_rate)
    if discount > cap:
        discount = cap
    return subtotal - discount
```

에이전트: `git diff --name-only`로 변경을 확인했습니다. `src/orders/pricing.py`에 `apply_discount` 함수가 추가됐네요. 정상 적용·상한 절삭·쿠폰율 0의 경계 조건을 테스트하겠습니다. `tests/test_pricing.py` 생성 후 `pytest`를 실행합니다.

에이전트가 생성한 테스트 파일( tests/test\_pricing.py )은 정상 경로와 경계 조건을 모두 다룹니다.

```
from src.orders.pricing import apply_discount

def test_apply_discount_normal_rate() -> None:
    """정상 경로: 10% 쿠폰이 상한 안에서 정상 적용된다."""
    assert apply_discount(50_000, 0.10, 10_000) == 45_000

def test_apply_discount_caps_at_limit() -> None:
    """경계 조건: 할인액이 상한을 넘으면 상한까지만 깎는다."""
    assert apply_discount(300_000, 0.10, 10_000) == 290_000

def test_apply_discount_zero_rate() -> None:
    """경계 조건: 쿠폰을 0이면 원금 그대로다."""
    assert apply_discount(50_000, 0.0, 10_000) == 50_000
```

```
pytest tests/test_pricing.py -q
```

```
... [100%]
3 passed in 0.05s
```

## 줄별 해설

| 부분               | 의미                                                     |
|------------------|--------------------------------------------------------|
| --- 블록           | 프런트매터 — 커맨드의 메타데이터 영역입니다.                              |
| description:     | /help 에 표시될 한 줄 설명입니다.                                 |
| allowed-tools:   | 이 커맨드가 쓸 수 있는 도구를 제한합니다(읽기·편집·pytest·git diff만).       |
| Bash(git diff:*) | 변경 모듈을 스스로 찾도록 diff 조회만 허용 — 다른 git 명령은 막힙니다.          |
| 본문 "절차: 1~4"     | 에이전트가 따를 단계 — 좋은 지시(2장)를 그대로 담았습니다.                    |
| 상한 절삭 테스트        | 결제 로직의 핵심 경계 조건 — apply_discount 가 cap을 넘지 않는지 못 박습니다. |
| 완료 조건:           | 2장에서 배운 완료 조건 — 커맨드도 이를 명시해야 결과가 안정됩니다.                |

여기서 주목할 점은 allowed-tools 로 도구를 좁힌 것입니다. /test 는 변경 모듈을 찾아 테스트를 짜고 실행하는 커맨드이므로, git push 나 배포 명령이 필요 없습니다. 도구를 좁히면, 이 커맨드가 활성화될 때 매번 권한을 묻지 않아도 딱 그 일만 할 수 있어 안전합니다(4장 최소 권한 원칙의 커맨드판). 결제처럼 돈이 오가는 모듈일 수록, "변경하면 경계 조건 테스트부터 보강"하는 절차를 커맨드로 강제하는 것이 사고를 줄입니다.

## 인자(\$ARGUMENTS)와 파일·맥락 참조

## \$ARGUMENTS로 매번 다른 대상에 적용하기

`/test` 는 "지금 변경된 코드"라는 고정 대상에 동작했습니다. 하지만 대상을 **매번 바꾸고 싶을 때** 가 많습니다. "이 모듈을 리뷰해", "저 파일을 리팩터해"처럼 말입니다. 이때 `$ARGUMENTS` 를 씁니다.

`$ARGUMENTS` 는 커맨드 이름 뒤에 붙은 **모든 텍스트** 를 통째로 받는 자리표시자입니다. `/review-module src/checks.py` 라고 하면 `$ARGUMENTS` 에는 `src/checks.py` 가 들어갑니다.

위치별로 나눠 받고 싶으면 `$1`, `$2` 처럼 번호로 참조할 수도 있습니다. 예를 들어 `/fix auth.py 401` 에서 `$1` 은 `auth.py`, `$2` 는 `401` 이 됩니다.

```
---
description: 지정한 파일의 지정한 증상을 진단·수정한다
---

$1 파일에서 "$2" 관련 문제를 진단하고 고쳐 주세요.
먼저 해당 부분을 읽고, 원인을 한 줄로 설명한 뒤, 외과적으로 수정하세요.
```

## @파일로 실제 코드를 주입하기

`$ARGUMENTS` 는 **글자** 를 끼워 넣지만, `@파일경로` 는 그 파일의 **내용 전체** 를 프롬프트에 주입합니다. 커맨드가 특정 기준 문서나 설정을 **항상** 참조해야 한다면 `@` 가 유용합니다.

```
---
description: 코딩 컨벤션에 맞춰 최근 변경을 점검한다
---

@docs/CONVENTIONS.md 의 규칙을 기준으로, 최근 변경(git diff)이
컨벤션을 지키는지 점검하고 위반을 목록으로 보고해 주세요.
```

이 커맨드는 호출될 때마다 `docs/CONVENTIONS.md` 의 최신 내용을 끌어와 기준으로 삼습니다. 컨벤션 문서가 갱신되면 커맨드도 자동으로 새 기준을 따릅니다.

**팁: `$ARGUMENTS`와 `@`의 역할을 구분하라**

`$ARGUMENTS` (또는 `$1`·`$2`)는 "호출할 때마다 달라지는 **대상**"을 받고, `@파일` 은 "늘 참조해야 하는 **기준.맥락**"을 주입합니다. 리뷰 커맨드라면 대상 모듈은 `$ARGUMENTS` 로, 리뷰 기준 체크리스트는 `@` 로 받는 식으로 조합하면 강력합니다.

## 프로젝트 커맨드와 사용자 커맨드의 범위

같은 커맨드 파일이라도 **어디에 두느냐** 에 따라 적용 범위가 달라집니다.

| 위치                               | 범위      | 공유 여부                  | 알맞은 용도             |
|----------------------------------|---------|------------------------|--------------------|
| <code>.claude/commands/</code>   | 프로젝트    | git으로 팀 공유됨            | 팀 표준 절차(리뷰·커밋·릴리스) |
| <code>~/.claude/commands/</code> | 사용자(개인) | 내 모든 프로젝트에서 쓰이나 공유 안 됨 | 개인 습관·유틸 커맨드       |

핵심 판단은 단순합니다. **팀이 함께 써야 하면 프로젝트( `.claude/commands` )**에 두어 저장소에 커밋하고, **나만의 개인 도구면 사용자( `~/.claude/commands` )**에 둡니다. 프로젝트 커맨드는 저장소에 포함되므로, 동료가 `git pull`만 하면 곧바로 같은 커맨드를 쓸 수 있습니다. 바로 이것이 "절차를 팀 자산으로 만든다"는 말의 실체입니다.

**주의:** 팀 표준을 개인 폴더에 두면 공유되지 않는다

"우리 팀 리뷰 커맨드"를 만들어 `~/.claude/commands/`에 두면, 그건 *당신 컴퓨터에서만* 동작합니다. 동료는 그 커맨드의 존재조차 모릅니다. 팀이 함께 쓸 커맨드는 반드시 저장소의 `.claude/commands/`에 두고 커밋해야 합니다. 이는 4장에서 "팀 규칙은 프로젝트 settings에 둔다"고 배운 원칙과 같은 맥락입니다.

## worked example: 인자 받는 /review-module 커맨드

이제 인자와 참조를 결합한 실전 커맨드를 만들어 보겠습니다. "지정한 모듈을 정해진 관점으로 리뷰하는" `/review-module` 커맨드입니다. 팀이 합의한 리뷰 체크리스트를 코드로 박제해, 누가 호출하든 같은 관점으로 같은 형식의 리뷰가 나오게 만드는 것이 목표입니다. 이 커맨드는 단원 미니 프로젝트의 산출물이기도 합니다.

### 1) 커맨드 파일을 작성한다

`.claude/commands/review-module.md`:

```

---
description: 지정한 모듈을 팀 표준 체크리스트로 코드 리뷰한다
argument-hint: <모듈 경로>
allowed-tools: Read, Grep, Bash(git diff:*)
---

$ARGUMENTS 모듈을 코드 리뷰해 주세요.

먼저 해당 파일을 읽고, 아래 관점으로 점검한 뒤 발견 사항을
[심각도] 경로:줄 - 설명 형식의 목록으로 보고하세요. 코드는 수정하지 마세요.

리뷰 관점:
1. 정확성: 경계 조건(None·음수 금액·중복 요청) 처리가 빠졌는가
2. 안전: 외부 호출 실패를 삼키지 않는가, 멍등성이 보장되는가
3. 가독성: 함수가 한 가지 일만 하는가, 이름이 의도를 드러내는가
4. 관측성: 실패 시 추적할 로그·반환값이 남는가

마지막에 가장 중요한 개선 1가지를 'TOP 권장'으로 짚어 주세요.
```

## 2) 모듈을 지정해 호출한다

나: /review-module src/payments/refund.py

\$ARGUMENTS 에 src/payments/refund.py 가 치환되어, 커맨드 본문이 "src/payments/refund.py 모듈을 코드 리뷰해주세요..."로 펼쳐집니다. 리뷰 대상이 된 환불 처리 코드는 다음과 같습니다.

```
def process_refund(order_id: str, amount: int) -> dict:
    order = db.get_order(order_id)          # order가 None이면 다음 줄에서 터진다
    if amount > order.paid_amount:
        amount = order.paid_amount          # 과다 환불은 막지만 음수 금액은 무방비
    try:
        gateway.refund(order.pg_key, amount)
    except Exception:
        pass                                # 게이트웨이 실패를 삼켜 환불 누락이 조용히 발생
    return {"order_id": order_id, "refunded": amount}
```

## 3) 에이전트가 표준 형식으로 리뷰한다

에이전트: src/payments/refund.py를 읽고 팀 표준 체크리스트로 리뷰했습니다.

[error] src/payments/refund.py:9 - gateway.refund 실패를 except: pass로 삼킵니다.  
결제 게이트웨이 오류 시 환불이 안 됐는데도 성공으로 응답합니다.  
[error] src/payments/refund.py:2 - db.get\_order가 None이면 다음 줄에서  
AttributeError. 주문 미존재를 명시적으로 처리해야 합니다.  
[warn] src/payments/refund.py:4 - amount 음수 검증이 없습니다. 마이너스 환불은  
오히려 청구가 되므로 0 이상 가드가 필요합니다.  
[warn] src/payments/refund.py:1 - 멍등성 키가 없어 재시도 시 이중 환불 위험.

TOP 권장: 9번 줄의 예외 삼킴 — 결제 실패를 숨기면 정산 불일치로  
이어져 가장 시급합니다. 실패를 올려보내거나 finding으로 남기세요.

### 줄별 해설

| 부분                                          | 의미                                       |
|---------------------------------------------|------------------------------------------|
| argument-hint: <모듈 경로>                      | /review-module 입력 시 어떤 인자를 줄지 힌트를 표시합니다. |
| allowed-tools: Read, Grep, Bash(git diff:*) | 읽기·검색·diff만 허용 — 수정 도구가 없어 코드를 못 바꿉니다.   |
| \$ARGUMENTS 모듈을 코드 리뷰                       | 호출 시 준 경로가 여기에 치환됩니다.                    |
| 코드는 수정하지 마세요                                | 리뷰는 진단만 — 본문 지시와 도구 제한이 이를 이중으로 보장합니다.   |
| "안전·멍등성" 관점                                 | 결제·환불처럼 사고 비용이 큰 모듈을 겨냥한 팀 체크리스트입니다.     |
| [심각도] 경로:줄 - 설명 형식                          | 출력 형식을 고정해 누가 호출해도 일관된 결과가 납니다.          |

이 커맨드의 묘미는 **도구 제한과 지시가 함께 안전을 만든다**는 점입니다. 본문에서 "수정하지 마라"고 지시하고, `allowed-tools` 에서 `Edit·Write`를 아예 빼 두었습니다. 설령 에이전트가 수정하려 해도 도구가 없어 불가능합니다. 지시(확률적)와 권한(결정론적)을 겹쳐, 리뷰 커맨드가 절대 코드를 건드리지 않도록 보장한 것입니다. 팀 리드 입장에서는, 환불처럼 위험한 모듈일수록 "리뷰는 진단만" 이라는 경계를 도구 수준에서 못 박아 두는 편이 안전합니다.

## 커맨드·스킬·서브에이전트의 관계 (2026 통합)

슬래시 커맨드를 배우면 자연스럽게 떠오르는 질문이 있습니다. "그럼 스킬(skill)이나 서브에이전트(subagent)와는 뭐가 다른가?" 2026 기준으로 세 가지를 정리하면 다음과 같습니다.

| 구분        | 누가 호출하나           | 무엇인가                        | 컨텍스트            |
|-----------|-------------------|-----------------------------|-----------------|
| 슬래시 커맨드   | 사람이 /이름 으로 명시 호출  | 재사용 프롬프트 템플릿                | 현재 세션 안에서 실행    |
| 스킬(skill) | 모델이 필요를 판단해 자동 호출 | 절차 + 보조 파일(스크립트·예시)을 묶은 패키지 | 현재 세션 안에서 실행    |
| 서브에이전트    | 메인 에이전트가 작업을 위임   | 독립 컨텍스트·도구·프롬프트를 가진 별도 세션   | 격리된 별도 컨텍스트(9장) |

**커맨드와 스킬** 은 2026 들어 사실상 통합되었습니다. `.claude/commands/*.md` (커맨드)와 `.claude/skills/*/SKILL.md` (스킬)는 같은 /이름 인터페이스를 만들며, 기존 커맨드 파일은 그대로 계속 동작합니다. 가장 큰 차이는 **호출 주체** 입니다. 슬래시 커맨드는 *사람이 명시적으로* 부르고, 스킬은 *모델이 프롬프트와 설명을 보고 알아서* 불러옵니다. 또 스킬은 프롬프트 한 장을 넘어 템플릿·스크립트 같은 보조 파일을 함께 묶고, 프런트매터로 모델 자동 호출 여부(`disable-model-invocation`) 같은 동작을 더 세밀히 제어할 수 있습니다.

**서브에이전트** 는 결이 다릅니다. 커맨드·스킬이 *현재 세션 안에서* 프롬프트를 펼치는 것이라면, 서브에이전트는 *맥락이 격리된 별도 세션* 에 작업을 통째로 떠넘깁니다(9장에서 자세히). 그래서 위의 `/review-module` 을 더 키우면, 리뷰 전담 *서브에이전트* 를 호출하는 커맨드로 발전시킬 수도 있습니다. 즉 셋은 경쟁이 아니라 **층위가 다른 재사용 단위** 입니다. 짧은 절차는 커맨드, 자동 적용이 필요한 능력은 스킬, 격리·병렬이 필요한 큰 작업은 서브에이전트로 갑니다.

### 언제 무엇을 쓰나 — 한 줄 기준

- **커맨드**: 내가 *그때그때 명시적으로* 부르고 싶은 짧은 절차(커밋·리뷰·테스트).
- **스킬**: 특정 상황이면 모델이 *알아서* 적용했으면 하는 능력(특정 포맷 변환 등).
- **서브에이전트**: 맥락을 *격리* 하거나 *병렬* 로 돌려야 하는 큰 작업(전담 리뷰·조사).

## 즉시 연습 (2종 1세트)

## 연습 1: 커스텀 커맨드 본문 채우기

팀이 릴리스 직전에 매번 손으로 돌리던 점검(테스트·린트·변경 로그 확인)을 표준화하는 `/release-check` 커맨드를 만들어 합니다. 빈칸을 채워 커맨드 파일을 완성하십시오. 인자로 "릴리스 대상 태그(예: v1.4.0)"를 받아 `$ARGUMENTS` 로 쓰도록 설계하세요.

작성형 빈 템플릿 — `release-check.md`를 채우세요

```
---
description: _____
allowed-tools: _____, _____, _____
---
```

`$ARGUMENTS` 릴리스 전 점검을 수행해 주세요. 코드는 수정하지 말고  
통과/실패를 체크리스트로 보고하세요.

절차:

1. 테스트를 실행해 \_\_\_\_\_ 확인한다.
2. 직전 태그 이후 커밋을 모아 \_\_\_\_\_ 점검한다.
3. 버전 번호와 변경 로그(CHANGELOG)가 \_\_\_\_\_ 확인한다.

완료 조건: \_\_\_\_\_

**자가체크** - [ ] `description` 이 "릴리스 전 점검"이라는 목적을 한 줄로 설명하는가 - [ ] `allowed-tools` 에 점검용 도구만(예: `Bash(pytest:*)` · `Bash(git log:*)`) 넣고 배포·push 도구는 뺐는가 - [ ] `$ARGUMENTS` 로 릴리스 대상 태그를 매번 다르게 받을 수 있는가 - [ ] 본문에 절차와 **완료 조건**이 명시돼 있는가(2장 4요소) - [ ] "코드는 수정하지 말고" 처럼 점검 커맨드의 경계를 본문에 못 박았는가

## 연습 2: 커맨드 범위와 호출 판단

다음은 한 결제 서비스 팀에서 후보로 나온 커맨드들입니다. 각 커맨드를 어디에 둘지(프로젝트/사용자)와 그 이유를 적고, 호출 예시를 작성하십시오.

작성형 빈 템플릿 — 범위와 호출을 채우세요

| 커맨드                                    | 둘 곳(프로젝트/사용자) | 이유   | 호출 예시                             |
|----------------------------------------|---------------|------|-----------------------------------|
| 팀 표준 코드 리뷰 <code>/review-module</code> | ____          | ____ | <code>/review-module _____</code> |
| 팀 릴리스 전 점검 <code>/release-check</code> | ____          | ____ | <code>/release-check _____</code> |
| 내 개인 단축 메모 <code>/scratch</code>       | ____          | ____ | <code>/scratch _____</code>       |

**자가체크** - [ ] 팀이 공유해야 하는 커맨드(`/review-module` · `/release-check`)를 `.claude/commands/` (프로젝트)에 두었는가 - [ ] 개인 전용 커맨드를 `~/.claude/commands/` (사용자)에 두었는가 - [ ] 인자가 필요한 커맨드의 호출 예시에 실제 모듈 경로·태그를 적었는가 - [ ] 각 배치의 이유가 "팀 공유 여부"로 설명되는가

## 흔한 오류와 오개념

### 오개념 1 — "슬래시 커맨드는 단순 매크로(글자 단축)다"

커맨드는 글자를 펼치는 데서 끝나지 않습니다. `$ARGUMENTS` 로 매번 다른 대상을 받고, `@` 로 실제 코드를 주입하며, `allowed-tools` 로 도구를 제한합니다. "매크로"가 아니라 **매개변수를 받는 프롬프트 함수** 로 이해해야 진짜 힘을 끌어낼 수 있습니다.

### 오개념 2 — "커맨드는 그냥 텍스트라 한계가 분명하다"

본문이 텍스트인 것은 맞지만, 그 텍스트가 좋은 지시(목표·절차·완료 조건)를 담고 인자·참조를 결합하면 매우 강력해집니다. 게다가 도구를 제한하면 안전성까지 얻습니다( `/review-module` 이 수정 도구 없이 리뷰만 하던 예). 텍스트의 한계가 아니라, 텍스트를 *어떻게 설계하느냐* 의 문제입니다.

### 오개념 3 — "커맨드를 만들면 팀이 자동으로 같이 쓴다"

공유 여부는 **위치** 가 결정합니다. `~/ .claude/commands/` (사용자)에 둔 커맨드는 내 컴퓨터에서만 동작합니다. 팀이 함께 쓰려면 반드시 저장소의 `.claude/commands/` (프로젝트)에 두고 커밋해야 합니다. 범위를 혼동하면 "왜 동료에게는 그 커맨드가 없지?"가 됩니다.

### 오개념 4 — "커맨드·스킬·서브에이전트는 같은 것의 다른 이름이다"

셋은 층위가 다릅니다. 커맨드는 *사람이 명시 호출* 하는 현재 세션 프롬프트, 스킬은 *모델이 자동 호출* 하는 패키지, 서브에이전트는 *격리된 별도 세션* 입니다. 큰 작업을 커맨드 하나에 욕여넣으려 하기보다, 격리·병렬이 필요하면 서브에이전트로(9장) 넘기는 판단이 중요합니다.

## 단원 미니 프로젝트: 커밋 워크플로우와 /커맨드 표준화 (2종 1세트)

이 미니 프로젝트는 단원 u4의 두 개념을 함께 적용합니다. **Git·커밋 통합(c07, 7장)** 으로 변경을 관례 메시지로 작게 커밋하고, **슬래시 커맨드(c08, 이 장)** 로 반복 리뷰 절차를 표준화해 저장소에 둡니다.

### 미니 프로젝트 목표

변경을 작은 논리 단위로 스테이징해 Conventional Commits 메시지로 커밋하는 흐름을 운용하고, 인자 (`$ARGUMENTS`)로 받은 모듈을 표준 관점으로 리뷰하는 커스텀 슬래시 커맨드를 정의해 호출까지 확인합니다. 산출물은 **두 개의 관례 커밋 기록** 과 `.claude/commands/review-module.md` 입니다.

**적용 개념:** c07(커밋 단위 통제·관례 메시지·git 권한) + c08(커스텀 커맨드·`$ARGUMENTS`·프로젝트 범위)

### 마일스톤 (증분으로 진행)

| 마일스톤 | 할 일                                                                         | 적용 개념 | 체크포인트(검증)                                                             |
|------|-----------------------------------------------------------------------------|-------|-----------------------------------------------------------------------|
| M1   | 쉬인 변경을 논리 단위로 나눠 관례 메시지로 두 커밋 으로 분리                                         | c07   | <code>git log --oneline -2</code> 에 성격이 다른 두 커밋이 보인다                  |
| M2   | 인자를 받는 <code>/review-module</code> 커맨드를 <code>.claude/commands/</code> 에 정의 | c08   | 파일이 저장소에 있고 <code>allowed-tools</code> 에 수정 도구가 없다                    |
| M3   | 커맨드를 호출해 특정 모듈 리뷰 결과를 표준 형식으로 확인                                            | c08   | <code>/review-module &lt;경로&gt;</code> 호출 시 [심각도] 경로:줄 - 설명 형식으로 출력된다 |

## 모범 산출물 (참고용)

**M1 — 두 논리 커밋의 결과:** 환불 모듈에 음수 금액 가드를 추가하면서 관련 없는 문서 수정이 섞여 들어온 상황을, 두 개의 성격이 다른 커밋으로 분리합니다.

```
git log --oneline -2
```

```
8d1c4a0 docs(payments): document refund idempotency key usage
3b9e07f fix(refund): reject negative amount and missing order
```

**M2 — 커맨드 정의( `.claude/commands/review-module.md` ):** 앞 worked example의 `/review-module` 과 동일한 구조 (프런트매터에 `argument-hint` · `allowed-tools` , 본문에 `$ARGUMENTS` ·표준 출력 형식)를 사용합니다.

**M3 — 호출 결과:** `/review-module src/payments/refund.py` 호출 시 `[error]/[warn]/[info]` 등급과 `경로:줄 - 설명` 형식, 마지막 TOP 권장 이 출력됩니다.

## 학생 작성형 빈 스타터

빈 템플릿 — 직접 채워 완성하세요

### M1 커밋 계획

변경 분류:

- 논리 1: \_\_\_\_\_ → 타입 \_\_\_\_\_, 파일 \_\_\_\_\_
- 논리 2: \_\_\_\_\_ → 타입 \_\_\_\_\_, 파일 \_\_\_\_\_

커밋 1 메시지: \_\_\_\_\_

커밋 2 메시지: \_\_\_\_\_

### M2 커맨드 파일 ( `.claude/commands/review-module.md` )

```

---
description: _____
argument-hint: _____
allowed-tools: _____, _____, _____
---

$ARGUMENTS 모듈을 코드 리뷰해 주세요. (코드는 수정하지 마세요.)

리뷰 관점:
1. _____
2. _____
3. _____

출력 형식: [심각도] 경로:줄 - 설명

```

### M3 호출

```
/review-module _____
```

**자가체크** - [ ] (M1) 두 커밋이 각각 한 가지 성격만 담고, 메시지가 타입(범위): 설명 을 따르는가 - [ ] (M1) 한 커밋에 무관한 파일이 섞이지 않았는가 - [ ] (M2) 커맨드를 `~/.claude` 가 아니라 저장소의 `.claude/commands/` 에 두어 팀이 공유 가능한가 - [ ] (M2) `allowed-tools` 에 `Edit-Write`를 빼서 리뷰가 코드를 못 바꾸게 했는가 - [ ] (M3) `$ARGUMENTS` 로 받은 모듈이 본문에 치환되어 표준 형식으로 리뷰됐는가

## 정리

이 장에서는 반복 절차를 /이름 으로 표준화하는 슬래시 커맨드를 배웠습니다.

- 내장 커맨드(`/clear` · `/compact` · `/review` · `/help`)는 세션 운용의 기본기이며, `/clear` (초기화)와 `/compact` (압축)의 차이를 구분해 씁니다.
- 커스텀 커맨드는 `.claude/commands/*.md` 로 정의하는 **매개변수 받는 프롬프트 템플릿** 입니다. `$ARGUMENTS` 로 대상을, `@` 로 기준 맥락을 주입하고, `allowed-tools` 로 도구를 제한해 안전을 더합니다.
- 위치가 공유 범위를 결정 합니다 — 팀 표준은 프로젝트(`.claude/commands`), 개인 도구는 사용자(`~/.claude/commands`)에 둡니다.
- 2026 기준 커맨드와 스킬은 통합되었고, 둘은 **현재 세션** 안에서, 서브에이전트는 **격리된 별도 세션** 에서 동작합니다.

이로써 단원 u4(버전관리와 커스텀 커맨드)를 마칩니다. 다음 단원 u5에서는 격리된 별도 세션에 작업을 통째로 위임하는 **서브에이전트(9장)** 와 외부 도구-데이터를 잇는 **MCP(10장)** 로 나아가, 에이전트의 능력을 확장 하는 법을 배웁니다.



# 위임: 서브에이전트로 일 나누기

## 이 챕터에서 배우는 것

- 서브에이전트(subagent)가 컨텍스트 격리·병렬화에 왜 유리한지 설명할 수 있습니다.
- `.claude/agents/이름.md`에 역할·도구(tools)·모델(model)을 갖춘 서브에이전트를 정의할 수 있습니다.
- 경계가 분명한 작업(리뷰·테스트·조사·QA)을 서브에이전트에 위임할 수 있습니다.
- 메인 에이전트의 계획·통합 책임과 서브에이전트의 한정 작업을 분업으로 설계할 수 있습니다.

## 도입: 한정된 일을 전문가에게 맡기기

앞 단원(u4)에서 우리는 자주 쓰는 절차를 /이름 슬래시 커맨드로 표준화하는 법을 배웠습니다. 커맨드는 "같은 프롬프트를 반복해 부르는 수고"를 없애 주었습니다. 그런데 커맨드만으로는 풀리지 않는 문제가 하나 있습니다. 바로 **하나의 세션이 너무 많은 일을 떠안을 때 생기는 혼잡**입니다.

실무를 떠올려 봅시다. 새 기능을 구현하는 세션을 길게 돌리다 보면, 메인 대화에는 요구사항·설계 토론·수십 개의 편집·실행 로그가 뒤섞입니다. 이 상태에서 "방금 만든 코드를 꼼꼼히 리뷰해 줘"라고 하면 에이전트는 이미 자기가 짰 코드에 익숙해진 채, 산만한 맥락 속에서 리뷰합니다. 사람으로 치면 **밤새 코드를 짰 본인이 그 코드를 셀프 리뷰**하는 상황입니다. 객관성도 떨어지고, 긴 맥락 때문에 중요한 디테일이 묻힙니다.

이럴 때 사람 팀은 어떻게 할까요? 구현자와 별개로 **리뷰 담당자**에게 "이 PR만 보고 의견 줘"라고 맡깁니다. 리뷰어는 구현 과정의 잡음을 모른 채, 결과물만 깨끗한 눈으로 봅니다. Claude Code의 **서브에이전트(subagent)**가 바로 이 역할을 합니다. 메인 에이전트가 "이 일만 해서 결과를 가져와"라고 별도의 전문가에게 위임하면, 그 전문가는 자기만의 깨끗한 작업 공간에서 일하고 결과만 돌려줍니다.

핵심 비유: 서브에이전트는 **외부 전문가에게 맡기는 하청**입니다. 의뢰서(역할·범위)를 주면, 전문가는 자기 사무실(독립 컨텍스트)에서 일하고 보고서(결과)만 보내 옵니다. 내 책상(메인 세션)은 그 사람의 작업 중간물로 어질러지지 않습니다.

이번 챕터는 그 위임의 메커니즘과 설계를 다룹니다. 이어지는 ch\_10에서는 외부 도구를 잇는 MCP를, 그리고 단원 끝의 미니 프로젝트에서 둘을 합쳐 "전문 에이전트 + 외부 연동" 세션을 구성합니다.

## 선수 개념 상기

이 챕터는 **효과적 프롬프팅(c02)**과 **CLAUDE.md-컨텍스트 관리(c05)**를 선수로 합니다. 서브에이전트에게 일을 맡길 때 결국 우리가 하는 일은 "범위·제약·완료 조건을 갖춘 작업 지시"(c02)를 그 에이전트의 시스템 프롬프트로 박아 두는 것입니다. 또 서브에이전트가 메인의 맥락을 자동으로 물려받지 않는다는 점은 "컨텍스트는 무한

하지 않고 명시적으로 관리해야 한다"(c05)는 감각의 연장선입니다. 두 개념이 흐릿하다면 ch\_02의 4요소(목표·맥락·제약·완료 조건)와 ch\_05의 컨텍스트 계층을 한 번 훑고 오면 좋습니다.

## 서브에이전트란: 독립 컨텍스트·프롬프트·도구

### 한 문장 정의

**서브에이전트(subagent)**는 자체 컨텍스트 윈도우·시스템 프롬프트·도구 권한을 가진 별도의 Claude 세션으로, 메인 에이전트가 경계가 분명한 작업(리뷰·테스트·조사 등)을 위임하면 자신만의 깨끗한 작업 공간에서 그 일을 처리해 **결과만** 돌려주는 전문 일꾼입니다.

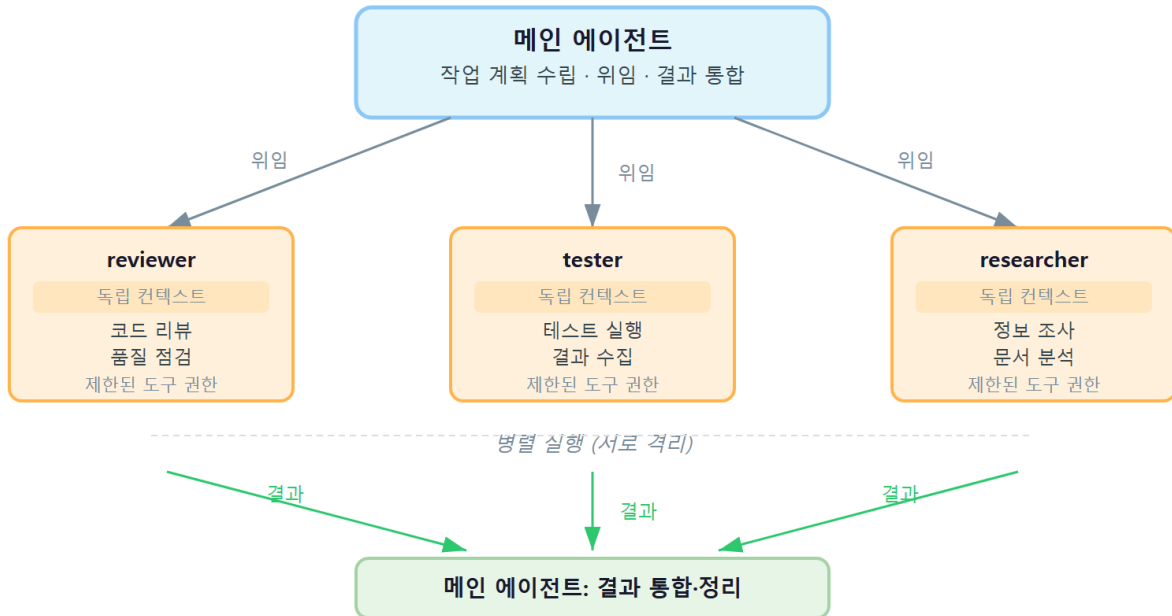
정의에 담긴 세 가지 "독립"을 풀어 봅니다. 이 셋이 서브에이전트를 그냥 "프롬프트 한 번 더 보내기"와 구별 짓는 핵심입니다.

- **독립 컨텍스트(context):** 서브에이전트는 메인 세션의 대화 기록을 통째로 물려받지 않습니다. 메인이 건넨 "작업 지시"와 자신의 시스템 프롬프트, 그리고 작업 디렉터리 같은 기본 환경 정보만 가지고 시작합니다. 그래서 메인 세션이 아무리 길어도 서브에이전트는 늘 가벼운 상태에서 출발합니다.
- **독립 프롬프트(prompt):** 서브에이전트는 자신의 역할을 규정하는 별도의 시스템 프롬프트를 가집니다. "너는 시니어 코드 리뷰어다"처럼 페르소나·기준·산출 형식을 못 박아 두면, 그 에이전트는 늘 그 기준으로 일합니다.
- **독립 도구 권한(tools):** 서브에이전트가 쓸 수 있는 도구를 따로 줍힐 수 있습니다. 예컨대 리뷰어에게는 `Read·Grep·Glob` 만 주고 `Edit·Write` 는 주지 않으면, 그 에이전트는 **읽고 분석할 수는 있어도 파일을 고칠 수는 없습니다**. 위임의 안전성이 권한 설계에서 나옵니다(c04 권한 개념의 연장).

정리하면 서브에이전트는 "메인이 부르는 또 하나의 나"가 아니라, **역할·기억·권한이 분리된 별도의 작업자**입니다. 이 분리가 주는 두 가지 실익이 다음 절의 주제, 곧 컨텍스트 격리와 병렬 실행입니다.

## 동작 원리: 컨텍스트 격리와 병렬 실행

## 서브에이전트: 위임·병렬 실행·결과 통합



메인 에이전트가 서브에이전트에게 일을 맡길 때 내부에서 벌어지는 일을 단계로 풀어 봅니다.

- 위임 결정:** 메인 에이전트가 "이 작업은 경계가 분명하고 결과만 받으면 된다"고 판단하면(또는 사용자가 명시적으로 요청하면) 적절한 서브에이전트를 고릅니다. 어떤 서브에이전트가 있는지는 각 정의 파일의 `description` 을 보고 판단합니다.
- 작업 패키징:** 메인 에이전트는 서브에이전트에게 건넬 **작업 지시**를 만듭니다. 서브에이전트는 메인의 대화를 못 보므로, 필요한 맥락(어떤 파일을 볼지, 무엇을 판단할지, 결과를 어떤 형식으로 줄지)을 이 지시에 명시적으로 담아야 합니다.
- 격리 실행:** 서브에이전트는 자신의 시스템 프롬프트 + 전달받은 작업 지시 + 허용된 도구만으로 새 컨텍스트에서 일합니다. 이때 만들어지는 중간 산출(수십 번의 파일 읽기, 분석 메모 등)은 **서브에이전트의 컨텍스트 안에만** 쌓이고 메인 세션을 어지럽히지 않습니다. 이것이 **컨텍스트 격리(context isolation)**입니다.
- 결과 반환:** 서브에이전트는 작업을 마치면 **요약된 결과 하나**를 메인에게 돌려줍니다. 메인 에이전트는 그 결과만 받아 자기 맥락에 통합합니다. 수백 줄을 읽어 본 과정은 버려지고, 결론(예: 리뷰 지적 5건)만 남습니다.

이 구조에서 두 가지 이점이 자연스럽게 나옵니다.

**(1) 컨텍스트 격리 — 메인 맥락 보호.** 서브에이전트가 코드 전체를 훑느라 토큰을 많이 써도, 메인 세션의 컨텍스트 윈도우에는 최종 요약만 들어옵니다. 길고 복잡한 조사·분석을 메인 대화 밖으로 밀어내므로, 메인 에이전트는 핵심 흐름에 집중한 채 오래 일할 수 있습니다.

**(2) 병렬 실행 — 동시 처리.** 서브에이전트들은 서로 독립이므로 **동시에** 돌 수 있습니다. "보안 관점 점검", "성능 관점 점검", "테스트 커버리지 점검"을 각각 다른 서브에이전트에 맡기면 세 작업이 병렬로 진행되고, 메인 에이전트는 세 결과를 한꺼번에 모아 통합합니다. 순차로 하면 세 배 걸릴 일을 한 번에 끝낼 수 있습니다.

아래 표로 메인 에이전트와 서브에이전트의 역할을 대비하면 분업의 그림이 분명해집니다.

| 구분 | 메인 에이전트           | 서브에이전트             |
|----|-------------------|--------------------|
| 맥락 | 전체 작업 흐름·대화 기록 보유 | 전달받은 지시 + 자기 프롬프트만 |
| 책임 | 계획·분배·결과 통합       | 위임받은 한정 작업 수행      |
| 도구 | 폭넓게 사용            | 역할에 필요한 것만(좁게)     |
| 산출 | 최종 결정·완성물         | 메인에 돌려줄 요약 결과      |
| 수명 | 세션 내내 지속          | 작업 1건 처리 후 종료      |

## 격리의 양면

컨텍스트 격리는 이점이자 제약입니다. 메인 맥락을 보호한다는 건, 거꾸로 **서브에이전트가 메인이 아는 것을 모른다**는 뜻이기도 합니다. "방금 우리가 정한 명명 규칙대로 리뷰해 줘"라고만 하면 서브에이전트는 그 규칙을 모릅니다. 그 규칙을 작업 지시에 적어 주거나, CLAUDE.md(c05)에 박아 두어야 합니다. 격리를 이해한다는 건 곧 "무엇을 명시적으로 건네야 하는가"를 아는 것입니다.

## worked example: code-reviewer 서브에이전트 정의·호출

가장 흔한 위임 대상인 **코드 리뷰어**를 직접 만들어 보겠습니다. 실무에서는 일반 가독성 리뷰보다 **보안·성능 결함**을 놓치지 않는 전담 리뷰어의 가치가 큼니다. 목표는 "구현과 분리된 깨끗한 눈으로, 읽기 전용 권한만 가지고 보안·성능 관점에서 코드를 점검하는 전문가"를 정의하는 것입니다.

먼저 `.claude/agents/code-reviewer.md` 파일을 만듭니다. 서브에이전트는 **YAML 프런트매터(머리말) + 시스템 프롬프트 본문** 구조의 마크다운 파일입니다.

```
---
name: code-reviewer
description: 작성·수정된 코드를 보안(인젝션·인증·비밀 노출)과 성능(N+1·불필요한 I/O) 관점에서 점검하는 시니어 리
tools: Read, Grep, Glob, Bash(git diff:*)
model: sonnet
---
```

당신은 10년 차 보안·성능 전문 코드 리뷰어입니다. 전달받은 파일만 검토하고 다음 기준으로 지적합니다.

검토 기준

- 보안: SQL/명령 인젝션, 인증·인가 누락, 비밀키·토큰 노출
- 성능: 반복문 안 쿼리(N+1), 불필요한 전체 조회, 동기 블로킹 I/O
- 정확성: 경계 조건·예외 처리 누락, 명백한 버그
- 안전: 입력 검증, 위험한 호출(eval·shell=True 등)

산출 형식

- 심각도(높음/중간/낮음)·파일:줄·문제·수정 제안을 한 건씩 표로 정리
- 코드를 직접 수정하지 말 것. 지적과 제안까지만 한다.

- `name: code-reviewer`: 이 서브에이전트의 식별자입니다. 메인 에이전트가 위임할 때 부르는 이름이자, 파일명과 맞추는 것이 관례입니다.

- `description: ...`: 메인 에이전트가 "언제 이 서브에이전트를 부를지" 판단하는 근거입니다. "보안·성능 관점", "코드를 짰 직후"처럼 시점과 전문성을 분명히 적을수록 메인에 적절히 위임합니다.
- `tools: Read, Grep, Glob, Bash(git diff:*)`: 읽기·검색·변경 조회만 줍니다. `Edit·Write`를 일부러 뺐습니다. 리뷰어는 읽고 분석만 하면 되고, 코드를 고치는 권한까지 줄 이유가 없습니다(최소 권한, c04).
- `model: sonnet`: 이 작업에 쓸 모델입니다. 리뷰는 빠르고 정확하면 충분하므로 `sonnet`을 택했습니다. 더 깊은 추론이 필요하면 `opus`, 단순 분류는 `haiku`를 고를 수 있습니다.
- 프론트matter 아래 **본문 전체가 서브에이전트의 시스템 프롬프트**가 됩니다. 페르소나·검토 기준·산출 형식을 못 박아, 매번 같은 품질로 일하게 만듭니다.

리뷰 대상은 사용자 검색 API의 다음 함수라고 합시다. 보안 결함과 성능 결함이 함께 숨어 있습니다.

```
# src/api/users.py - 리뷰 대상 코드
def find_users(db, keyword):
    q = f"SELECT * FROM users WHERE name LIKE '%{keyword}%" # SQL 인젝션
    rows = db.execute(q).fetchall()
    result = []
    for r in rows:
        orders = db.execute(
            f"SELECT * FROM orders WHERE user_id={r['id']}" # 반복문 안 쿼리 = N+1
        ).fetchall()
        result.append({"user": r, "order_count": len(orders)})
    return result
```

이 함수는 정상 입력에서는 잘 도는 것처럼 보이지만, 악의적 입력이나 사용자가 많을 때 문제가 드러납니다.

```
실행 결과:
>>> find_users(db, "kim") # 사용자 1,000명 → 쿼리 1,001회 발생
[... 정상처럼 보이지만 응답이 느림 ...]
>>> find_users(db, "' OR '1'='1") # 전체 행이 노출되는 인젝션
[모든 사용자 레코드 반환]
```

이제 메인 세션에서 이 서브에이전트를 호출합니다. 방법은 두 가지입니다. (1) 그냥 자연어로 리뷰를 요청하면 메인에 `description`을 보고 알아서 `code-reviewer`에게 위임하거나, (2) 명시적으로 지목합니다.

> 방금 만든 `src/api/users.py`를 `code-reviewer` 서브에이전트로 보안·성능 리뷰해 줘.

이렇게 지시하면 메인 에이전트는 `code-reviewer`를 별도 컨텍스트에서 띄우고, "`src/api/users.py`를 검토하라"는 작업 지시를 건넵니다. 서브에이전트가 일을 마치면 메인 세션에는 다음과 같은 **요약 결과만** 돌아옵니다.

```
code-reviewer 서브에이전트 결과 (src/api/users.py)
| 심각도 | 위치 | 문제 | 제안 |
| 높음 | users.py:3 | keyword를 문자열 보간해 SQL 인젝션 | 파라미터 바인딩(?) 사용 |
| 높음 | users.py:7 | 반복문마다 orders 조회로 N+1 | JOIN + GROUP BY 단일 쿼리 |
| 중간 | users.py:3 | SELECT * 로 불필요한 컬럼 전송 | 필요한 컬럼만 명시 |
지적 3건. 코드는 수정하지 않았습니다.
```

핵심은, 서브에이전트가 파일을 열어 보고 분석하며 쌓은 긴 중간 과정은 **메인 세션에 들어오지 않고**, 표로 압축된 결론만 메인에 통합된다는 점입니다. 메인 에이전트는 이 표를 받아 "높음 두 건을 고쳐 줘"라고 다음 작업을 이어갈 수 있습니다. 메인에 지적을 반영해 고치면 코드는 이렇게 바뀝니다.

```
# src/api/users.py - 리뷰 지적을 반영한 수정본
def find_users(db, keyword: str) -> list[dict]:
    rows = db.execute(
        "SELECT u.id, u.name, COUNT(o.id) AS order_count "
        "FROM users u LEFT JOIN orders o ON o.user_id = u.id "
        "WHERE u.name LIKE ? GROUP BY u.id, u.name",
        (f"%{keyword}%",),          # 파라미터 바인딩으로 인젝션 차단
    ).fetchall()                  # 단일 쿼리로 N+1 제거
    return [dict(r) for r in rows]
```

```
실행 결과:
>>> find_users(db, "kim")          # 사용자 수와 무관하게 쿼리 1회
[{'id': 1, 'name': 'kim', 'order_count': 3}, ...]
>>> find_users(db, "' OR '1'='1")  # 바인딩되어 리터럴로 처리됨
[]
```

리뷰(서브에이전트)와 수정(메인)이 분리된 덕분에, 짠 사람이 셀프 리뷰하며 자기 코드의 인젝션·N+1을 놓치는 사각지대를 줄였습니다. 이처럼 **서브에이전트 위임**은 "구현"과 "보안·성능 검증"을 다른 작업자에게 나누어 품질을 끌어올립니다.

## 호출 위치 주의

서브에이전트 정의 파일을 `.claude/agents/` (프로젝트 범위)에 두면 그 저장소에서만 동작하고 팀과 공유됩니다. `~/.claude/agents/` (사용자 범위)에 두면 내 모든 프로젝트에서 쓸 수 있지만 팀원에게는 보이지 않습니다. CLAUDE.md의 컨텍스트 계층(c05)과 같은 범위 구분이 서브에이전트에도 적용됩니다.

## 정의 파일: `.claude/agents/이름.md(역할·tools·model)`

worked example에서 본 정의 파일의 구조를 항목별로 정리합니다. 서브에이전트 파일은 **프런트매터(설정) + 본문(시스템 프롬프트)** 두 부분으로 이루어집니다.

| 프런트매터 키                  | 의미                     | 설계 팁                         |
|--------------------------|------------------------|------------------------------|
| <code>name</code>        | 서브에이전트 식별자             | 파일명과 일치, 역할이 드러나는 이름         |
| <code>description</code> | 메인이 위임 여부를 판단하는 한 줄 설명 | "무엇을·언제" 부를지 명시              |
| <code>tools</code>       | 허용 도구 목록               | 역할에 필요한 최소만(읽기 전용이면 Edit 제외) |
| <code>model</code>       | 사용할 모델                 | 난도에 맞춰 haiku/sonnet/opus 선택  |

그리고 프런트매터 아래 **본문 전체가 시스템 프롬프트**로 그대로 들어갑니다. 좋은 본문은 다음을 담습니다.

- **역할·페르소나**: "당신은 ...입니다" — 일관된 기준을 갖게 합니다.

- **작업 범위·제약:** 무엇을 하고 무엇을 하지 않는지(예: "코드를 수정하지 말 것").
- **산출 형식:** 결과를 어떤 모양으로 돌려줄지(표·목록·JSON 등). 메인이 통합하기 좋게.

## tools를 생략하면

tools를 적지 않으면 서브에이전트는 메인이 가진 도구를 상속받습니다. 편하지만 위임의 안전 이점을 일부 잃습니다. 위임의 핵심은 "필요한 권한만 줘게 주는 것"이므로, **읽기 전용 작업에는 tools를 명시해 쓰기 도구를 빼는 습관**을 권합니다. 권한을 좁히는 것은 c04에서 배운 최소 권한 원칙의 실천입니다.

## 위임에 적합한 작업(리뷰·테스트·조사·QA)

모든 일을 서브에이전트에 맡기는 것이 좋은 건 아닙니다. 위임이 빛나는 작업에는 공통점이 있습니다. **경계가 분명하고, 결과를 요약해 돌려줄 수 있으며, 메인 맥락과 어느 정도 독립적인 작업**입니다. 대표적인 위임 대상을 봅니다.

| 위임 대상     | 왜 적합한가              | 도구 예시            |
|-----------|---------------------|------------------|
| 코드 리뷰     | 구현과 분리된 객관성·읽기 전용   | Read, Grep, Glob |
| 테스트 실행·요약 | 긴 테스트 로그를 결론으로 압축   | Bash, Read       |
| 코드베이스 조사  | 많은 파일 탐색을 격리(맥락 보호) | Glob, Grep, Read |
| QA·보안 점검  | 별도 기준의 전문 점검        | Read, Grep, Bash |

반대로 **위임이 부적합한 경우도** 분명합니다. 메인의 진행 맥락이 매 순간 필요한 대화형 설계 토론, 한두 줄 고치면 끝나는 사소한 편집, 결과를 요약할 수 없고 계속 주고받아야 하는 작업은 서브에이전트로 보내면 오히려 맥락 전달 비용만 늘고 비효율적입니다. **"이 일만 떼어 맡기고 결과만 받으면 되는가?"**가 위임 판단의 기준입니다.

## 위임 판단 한 줄 기준

작업을 한 문장으로 의뢰서에 적을 수 있고(범위 명확), 그 결과를 표·목록으로 받아 보면 충분하다면(요약 가능) 위임에 적합합니다. "계속 옆에서 같이 봐야" 하는 일은 메인이 직접 하는 편이 낫습니다.

## worked example: 병렬 조사 서브에이전트로 동시 수행

서브에이전트의 두 번째 이점인 **병렬 실행**을 활용하는 예제입니다. 실무 상황을 가정하겠습니다. 결제 게이트웨이(PG)를 새 업체로 교체해야 하는데, 코드베이스가 수년간 자란 레거시 모놀리스라 **결제 호출이 어디에 어떻게 흩어져 있는지** 아무도 전모를 모릅니다. 마이그레이션 계획을 세우려면 먼저 현황을 빠르게 파악해야 합니다. 한 세션이 모든 디렉터리를 순차로 뒤지면 느리고 메인 맥락이 탐색 로그로 가득 찹니다. 대신 **조사 전담 서브에이전트(researcher)**를 정의하고, 도메인 영역을 나눠 동시에 조사시킵니다.

먼저 조사용 서브에이전트를 정의합니다.

```
---
name: researcher
description: 코드베이스의 특정 주제(결제·인증·로그 등)가 어디에 어떻게 구현됐는지 조사해 위치·요약을 보고한다. 큰
tools: Glob, Grep, Read
model: sonnet
---
```

당신은 코드베이스 조사 전문가입니다. 지정된 주제에 한해 관련 파일을 찾아 읽고, 다음 형식으로만 보고합니다. 코드를 수정하지 않습니다.

보고 형식

- 관련 파일 경로 목록
- 각 파일에서 그 주제가 어떻게 다루지는지 1~2줄 요약
- 주의할 점(중복 구현·불일치·외부 의존)을 한 줄

이제 메인 세션에서 조사 영역을 나눠 동시에 위임합니다.

```
> researcher 서브에이전트 둘을 병렬로 띄워 줘.
> 하나는 services/checkout 아래의 PG 결제 호출,
> 다른 하나는 services/billing 아래의 정기결제·환불 호출을 조사해 보고하게 해 줘.
```

- researcher 서브에이전트 둘을 병렬로: 같은 정의를 가진 서브에이전트를 두 인스턴스 띄웁니다. 각자 독립 컨텍스트라 서로 간섭하지 않습니다.
- 영역을 services/checkout (주문 결제)과 services/billing (정기결제·환불)으로 **겹치지 않게 분담**시킨 점이 중요합니다. 분담이 명확해야 결과를 합칠 때 충돌이 없습니다.
- 두 조사는 **동시에** 진행됩니다. 순차로 하면 두 배 걸릴 탐색이 한 번에 끝납니다.

두 서브에이전트가 각각 조사를 마치면 메인 세션에는 두 개의 요약이 한꺼번에 돌아옵니다.

```
researcher #1 (services/checkout) 결과
- services/checkout/pay.py: PG SDK로 결제 승인 요청(레거시 v1 SDK 직접 호출)
- services/checkout/webhook.py: PG 결제 결과 콜백 수신·검증
- 주의: PG SDK 키가 코드에 하드코딩되어 마이그레이션 시 분리 필요
```

```
researcher #2 (services/billing) 결과
- services/billing/subscribe.py: 정기결제 등록·갱신
- services/billing/refund.py: 부분·전액 환불 처리
- 주의: 환불이 PG SDK를 직접 부르는 곳과 내부 래퍼를 거치는 곳이 혼재
```

메인 통합 메모: PG 호출이 checkout(승인·콜백)과 billing(정기·환불)에 흩어져 있고, SDK 직접 호출과 래퍼가 섞임. 마이그레이션 1순위는 하드코딩 키 분리와 호출부 래퍼 단일화.

메인 에이전트는 두 서브에이전트가 각자 읽은 수십 개 파일의 세부를 떠안지 않고, **두 요약과 한 줄짜리 통합 메모만** 자기 맥락에 가집니다. 조사의 무거운 과정은 서브에이전트 안에 격리되고, 메인은 결론으로 마이그레이션 다음 결정을 이어갑니다. 이것이 컨텍스트 격리와 병렬 실행이 함께 작동하는 모습입니다.

## 메인 에이전트의 계획·통합 책임

위임이 잘 굴러가려면 메인 에이전트가 분명한 역할을 맡아야 합니다. 서브에이전트는 "시키는 일"을 하지만, **무엇을 누구에게 어떻게 나눌지, 그리고 받은 결과를 어떻게 합칠지**는 메인의 책임입니다. 메인의 책임은 크게 세 가지입니다.

1. **계획(분해)**: 큰 작업을 위임 가능한 단위로 쪼갭니다. "인증 코드 조사"를 `api / web` 으로 나눈 것이 분해의 예입니다. 단위는 서로 겹치지 않고 합치면 전체가 되도록 나뉘어야 합니다.
2. **분배**: 각 단위에 맞는 서브에이전트를 고르고, 필요한 맥락을 담은 작업 지시를 건넵니다. 서브에이전트는 메인 맥락을 모르므로, 이 지시의 품질이 결과 품질을 좌우합니다(c02 프롬프팅의 연장).
3. **통합**: 돌아온 결과들을 검토하고 모순을 조정하며 하나의 결론으로 엮습니다. 병렬 조사 예에서 "인증이 이원화됨"이라는 통합 메모를 만든 것이 이 단계입니다.

이 분업은 `ch_12`에서 배운 **멀티에이전트 오케스트레이션**의 씨앗입니다. 지금은 "메인은 지휘자, 서브는 연주자"라는 그림만 분명히 잡아 두면 됩니다. 메인이 직접 모든 일을 연주하려 들면 위임의 의미가 없고, 반대로 통합을 게을리하면 서브들의 결과가 따로 놀게 됩니다.

## 분업 설계 원칙

- 메인은 **직접 다 하지 않습니다**. 분해·분배·통합에 집중합니다.
- 위임 단위는 **겹치지 않게(MECE)** 나눕니다. 합치면 전체가 되도록.
- 서브에이전트에는 **필요한 맥락을 명시적으로** 건넵니다(자동 공유 없음).
- 받은 결과는 **반드시 통합·검증**합니다. 모아만 두면 가치가 없습니다.

## 즉시 연습 (2종 1세트)

이제 직접 서브에이전트를 설계해 봅시다. 먼저 모범을 보고, 빈 템플릿을 채운 뒤, 자가체크로 점검하는 순서입니다.

### 연습 1: test-writer 서브에이전트 정의하기

#### 모범 예시

변경 모듈의 누락된 단위 테스트를 작성·실행하고 결과를 요약하는 `test-writer` 서브에이전트의 모범입니다. 실무에서 "구현은 했는데 테스트를 안 짰다"는 PR 지연을 줄이려고 자주 위임하는 역할입니다.

```

---
name: test-writer
description: 변경된 모듈의 누락된 단위 테스트를 정상·경계 조건까지 작성하고 실행해 결과를 보고한다. 구현 직후 테
tools: Read, Edit, Write, Bash(pytest:*), Bash(git diff:*)
model: sonnet
---

```

당신은 테스트 작성 담당자입니다. 변경된 모듈을 읽고 누락된 단위 테스트를 보완한 뒤, 다음 형식으로만 보고합니다.

#### 작업

- git diff로 변경 모듈을 찾고, 변경 함수의 정상 경로와 경계 조건(0·음수·상한 초과·None)을 각각 테스트로 작성한다.
- pytest를 실행해 통과를 확인한다.

#### 보고 형식

- 추가한 테스트 함수 목록
- 통과/실패 개수
- 커버하지 못한 위험 경로 한 줄

## 학생 작성형 빈 템플릿

아래 빈칸을 채워 **보안 점검(security-auditor)** 서브에이전트를 직접 정의해 보세요. "변경된 코드에서 비밀키 하드코딩·인젝션·인증 누락을 찾아내는" 역할입니다. 릴리스 전 보안 게이트로 위임하기 좋습니다.

```

---
name: _____
description: _____
tools: _____
model: _____
---

당신은 _____ 입니다. 다음 일을 합니다.

작업
- _____ (예: 비밀키·토큰 하드코딩 탐지)
- _____ (예: SQL/명령 인젝션·인증 누락 점검)

제약
- _____ (예: 코드를 수정하지 말 것)

보고 형식
- _____ (예: 심각도·파일:줄·문제·제안 표)

```

채울 때 생각할 점: - tools 에 Edit·Write 가 들어가야 할까요? (점검만 한다면 필요 없습니다) - description 이 "언제 이 에이전트를 부를지"를 메인에게 알려 주나요? - 보고 형식이 메인에 통합하기 좋은 모양(표·목록)인가요?

## 자가체크

- [ ] name 이 역할을 드러내고 파일명과 맞는가
- [ ] description 에 "무엇을·언제" 위임할지가 적혀 있는가
- [ ] tools 가 역할에 필요한 **최소**로 좁혀졌는가(읽기 전용이면 쓰기 도구 제외)
- [ ] 본문에 역할·제약·산출 형식이 모두 담겼는가
- [ ] 코드를 수정하면 안 되는 역할인데 수정 권한을 주지 않았는가

## 연습 2: 위임 vs 직접 처리 판단하기

### 모범 예시

아래는 실무 작업별로 "위임 적합 여부"를 판단한 모범입니다.

| 작업                    | 판단    | 이유                         |
|-----------------------|-------|----------------------------|
| 새 결제 모듈 전체를 보안·성능 리뷰  | 위임 적합 | 경계 명확·결과 요약 가능·구현과 분리된 객관성 |
| 로그 메시지 오타 한 글자 수정     | 직접 처리 | 너무 사소, 맥락 전달 비용이 더 큼       |
| 레거시 모놀리스의 결제 호출 전수 조사 | 위임 적합 | 대규모 탐색 격리로 메인 맥락 보호        |
| PO와 다음 스프린트 우선순위 토론   | 직접 처리 | 매 순간 메인 맥락·상호작용 필요         |

### 학생 작성형 빈 템플릿

아래 실무 작업들을 "위임 적합 / 직접 처리"로 판단하고 이유를 적어 보세요.

| 작업                      | 판단(위임/직접) | 이유    |
|-------------------------|-----------|-------|
| CI 테스트 스위트를 돌려 실패 원인 요약 | _____     | _____ |
| import 순서 한 줄 정렬        | _____     | _____ |
| 인증 미들웨어를 보안 관점으로 점검     | _____     | _____ |
| 새 PG로 교체할지 아키텍처 방향 결정   | _____     | _____ |

### 자가체크

- [ ] "결과만 받으면 되는 일"을 위임으로 분류했는가
- [ ] "계속 같이 봐야 하는 일"을 직접 처리로 분류했는가
- [ ] 사소한 편집을 굳이 위임으로 보내지 않았는가
- [ ] 각 판단에 한 줄 이유를 달 수 있는가

## 흔한 오류와 오개념

### 1. "서브에이전트가 메인의 맥락을 안다"는 오해

가장 흔한 실수입니다. "방금 우리가 정한 규칙대로 리뷰해 줘"라고만 하면 서브에이전트는 그 규칙을 모릅니다. 서브에이전트는 메인 대화를 물려받지 않고, 전달받은 작업 지시 + 자기 시스템 프롬프트 + 기본 환경만 가집니다. 필요한 맥락은 작업 지시에 명시하거나 CLAUDE.md(c05)에 박아 두어야 합니다.

**교정:** 위임할 때 "어떤 파일을 어떤 기준으로 어떤 형식으로"를 작업 지시에 빠짐없이 적습니다. 격리는 이점이자 "명시 책임"입니다.

## 2. 과잉 위임 — 모든 걸 서브에이전트로

서브에이전트가 멋져 보여 사소한 편집까지 전부 위임하면, 맥락을 패키징하고 결과를 받아 통합하는 **부대 비용**이 작업 자체보다 커집니다. 위임은 "경계가 분명하고 결과만 받으면 되는" 일에 씁니다. 한두 줄 편집이나 메인 맥락이 계속 필요한 대화형 작업은 직접 처리가 빠릅니다.

**교정:** "이 일만 떼어 결과만 받으면 되는가?"를 물어 보고, 아니면 메인이 직접 합니다.

## 3. 넓은 도구 권한 — tools를 안 좁히기

`tools`를 명시하지 않으면 서브에이전트가 메인의 도구를 그대로 상속합니다. 리뷰어·조사자처럼 읽기만 하면 되는 에이전트에 `Edit·Write·Bash`를 다 주면, 의도치 않게 파일을 고치거나 위험한 명령을 실행할 여지가 생깁니다. 위임의 안전 이점은 권한을 좁히는 데서 나옵니다.

**교정:** 역할에 필요한 도구만 `tools`에 명시합니다. 읽기 전용 역할이면 `Read`, `Grep`, `Glob`만 주고 쓰기·실행 도구는 뺍니다(c04 최소 권한).

## 자주 묻는 질문(FAQ)

**Q. 서브에이전트와 슬래시 커맨드(c08)는 무엇이 다른가요?** 슬래시 커맨드는 "정해진 프롬프트를 메인 세션에 끼워 넣는 템플릿"이라 같은 컨텍스트에서 실행됩니다. 서브에이전트는 "별도 컨텍스트·권한을 가진 다른 작업자"입니다. 반복 절차를 부르고 싶으면 커맨드, 일을 떼어 격리해 맡기고 싶으면 서브에이전트가 맞습니다. ch\_12에서 둘을 엮습니다.

**Q. 서브에이전트가 또 다른 서브에이전트를 부를 수 있나요?** 설계상 위임은 메인이 지휘하는 것이 기본입니다. 깊게 중첩하면 맥락 전달과 비용이 빠르게 늘어 통제가 어려워지므로, "메인이 분배·통합한다"는 단순한 구조를 권합니다.

**Q. 모델(model)은 어떻게 고르나요?** 작업 난도에 맞춥니다. 단순 분류·요약은 `haiku`, 일반 리뷰·조사는 `sonnet`, 깊은 추론이 필요한 설계 검토는 `opus`가 무난합니다. 병렬로 여러 개를 돌릴 때는 비용도 고려합니다.

## 마무리

이 챕터에서는 일을 나누는 도구인 **서브에이전트**를 배웠습니다. 핵심은 세 가지였습니다. 서브에이전트는 독립 컨텍스트·프롬프트·도구를 가진 별도 작업자이고(정의), 컨텍스트 격리로 메인 맥락을 보호하며 병렬 실행으로 동시에 일하고(동작 원리), 메인은 계획·분배·통합을 맡고 서브는 경계가 분명한 한정 작업을 맡습니다(분업). `.claude/agents/이름.md`에 역할·tools·model을 적어 정의하고, 읽기 전용 역할에는 권한을 좁히는 것이 안전한 위임의 기본입니다.

다음 챕터(ch\_10)에서는 에이전트를 **바깥 세상**과 잇는 MCP를 배웁니다. 서브에이전트가 "일을 나누는" 확장이었다면, MCP는 "도구를 늘리는" 확장입니다. 단원 끝의 미니 프로젝트에서 이 둘을 합쳐, 외부 자료를 참조하는

전문 서브에이전트 세션을 구성하게 됩니다.



## 연결: MCP 서버로 외부 도구·데이터 연동

### 이 챕터에서 배우는 것

- MCP(Model Context Protocol)가 외부 도구·데이터를 어떻게 표준화해 연결하는지 설명할 수 있습니다.
- `.mcp.json`으로 MCP 서버를 등록·구성하고 범위(로컬·프로젝트·사용자)를 구분할 수 있습니다.
- MCP 서버가 노출한 도구(tool)·리소스(resource)·프롬프트(prompt)를 작업에 활용할 수 있습니다.
- MCP 도구에 권한·범위를 적용하고 비밀(토큰)을 안전하게 주입할 수 있습니다.

### 도입: 에이전트를 바깥 세상과 잇기

앞 챕터(ch\_09)에서 우리는 일을 나누는 확장, 곧 서브에이전트를 배웠습니다. 메인이 경계가 분명한 작업을 별도 전문가에게 위임해 컨텍스트를 보호하고 병렬로 처리했습니다. 이번 챕터는 다른 방향의 확장입니다. 일을 나누는 것이 아니라 에이전트가 **닿을 수 있는 세계를 넓히는** 것입니다.

지금까지 Claude Code는 주로 "현재 작업 디렉터리의 파일을 읽고·편집하고·명령을 실행하는" 일을 했습니다. 그런데 실무 작업은 로컬 폴더 안에서 끝나지 않습니다. GitHub의 이슈와 PR을 확인해야 하고, 데이터베이스의 스키마를 봐야 하고, 사내 위키나 디자인 문서를 참조해야 합니다. 이런 외부 시스템마다 에이전트가 직접 붙는 코드를 일일이 짜 넣는다면, 통합은 제각각이고 깨지기 쉬우며 보안 구멍이 생깁니다.

비유하자면, 새 가전을 살 때마다 벽을 뜯어 전선을 직접 잇는다고 생각해 보세요. 끔찍합니다. 그래서 우리는 **표준 콘센트**를 씁니다. 가전은 표준 플러그를 달고, 벽은 표준 소켓을 제공하면, 어떤 가전이든 꽂기만 하면 됩니다. **MCP(Model Context Protocol)**가 바로 에이전트 세계의 표준 콘센트입니다. 외부 시스템이 "MCP 서버"라는 표준 플러그를 제공하면, Claude Code는 그것을 표준 방식으로 꽂아 그 시스템의 기능을 도구처럼 씁니다.

핵심 비유: MCP는 에이전트와 외부 도구·데이터를 잇는 **표준 콘센트(USB 같은 표준 규격)**입니다. 통합마다 전용 배선을 까는 대신, 표준에 맞춘 서버를 꽂으면 에이전트가 그 도구·데이터를 일관된 방식으로 쓸 수 있습니다.

### 선수 개념 상기

이 챕터는 **권한과 도구 허용목록(c04)**을 선수로 합니다. MCP가 추가하는 도구도 결국 에이전트가 호출하는 "도구"이므로, c04에서 배운 `allow-deny-ask` 권한 체계가 그대로 적용됩니다. 외부 시스템을 건드린다는 점에서 오히려 권한 통제가 더 중요합니다. 권한 판정 흐름이 흐릿하다면 ch\_04를 한 번 훑고 오면 좋습니다. 또 ch\_09의 서브에이전트와 결합하면, 외부 자료를 참조하는 전문 에이전트를 만들 수 있습니다(이 단원의 미니 프로젝트).

### MCP란: 외부 통합을 위한 표준 프로토콜

## 한 문장 정의

**MCP(Model Context Protocol, 모델 컨텍스트 프로토콜)**는 Claude Code 같은 AI 에이전트를 GitHub·데이터베이스·파일시스템·내부 API 등 외부 서비스에 연결하는 **표준 프로토콜**로, 외부 시스템이 "MCP 서버"로 통합을 캡슐화해 도구·리소스를 노출하면 에이전트가 이를 표준 방식으로 호출해 활용합니다.

여기서 두 단어를 정확히 잡아 둡니다.

- **프로토콜(protocol)**: 서로 다른 프로그램이 대화하기 위한 **약속(규격)**입니다. HTTP가 브라우저와 웹 서버의 약속이듯, MCP는 에이전트와 외부 통합 서버의 약속입니다. 약속이 표준이라, 한 번 MCP를 지원하는 에이전트는 MCP를 따르는 **어떤 서버든** 꽂아 쓸 수 있습니다.
- **MCP 서버(server)**: 특정 외부 시스템(GitHub·DB 등)과의 연결을 캡슐화한 작은 프로그램입니다. 서버가 "내가 제공하는 도구는 이런 것들"이라고 노출하면, 에이전트는 그 목록을 받아 필요할 때 호출합니다. 서버는 외부 시스템과의 지저분한 통신을 안에 숨기고, 에이전트에는 깔끔한 도구만 보여 줍니다.

왜 이것이 중요할까요? **표준이 없으면 통합이 N x M 문제로 폭발**하기 때문입니다. 에이전트가 5종, 외부 시스템이 10종이면 전용 연결이 50개 필요합니다. MCP라는 공통 규격이 있으면, 각 외부 시스템은 MCP 서버 하나만 만들면 되고 모든 에이전트가 그것을 씁니다. N + M으로 줄어듭니다. 이것이 "플러그인 난립"과 "표준 프로토콜"의 결정적 차이입니다.

## 동작 원리: 서버·도구·리소스·프롬프트 구조

### MCP 구조: 클라이언트 → 서버 → 외부 시스템



MCP = 외부 통합 표준 프로토콜 — 서버 하나로 여러 도구·리소스를 에이전트에 연결

MCP가 작동하는 구조를 단계로 풀어 봅니다. Claude Code(클라이언트)와 MCP 서버가 어떻게 대화하는지가 핵심입니다.

1. **연결**: Claude Code는 설정( .mcp.json )에 등록된 MCP 서버를 시작·연결합니다. 서버는 로컬에서 실행되는 프로그램일 수도( `stdio` 방식), 원격 HTTP 엔드포인트일 수도( `http` 방식) 있습니다.

2. **능력 교환(handshake)**: 연결되면 서버가 "내가 제공하는 것들"의 목록을 에이전트에 알립니다. 이 목록이 **도구·리소스·프롬프트** 세 종류입니다(아래 표).
3. **호출**: 에이전트가 작업 중 필요를 느끼면(예: "이 저장소의 열린 이슈를 보여 줘") 해당 서버의 도구를 호출합니다. 이때 **권한 체계(c04)**가 개입해 허용 여부를 판정합니다.
4. **결과 반영**: 서버가 외부 시스템과 통신해 결과를 돌려주면, 에이전트는 그 결과를 추론에 반영해 다음 행동을 결정합니다.

MCP 서버가 노출하는 세 가지 구성요소를 구분하는 것이 이 절의 핵심입니다.

| 구성요소          | 무엇인가                         | 누가 주도하나         | 예시                   |
|---------------|------------------------------|-----------------|----------------------|
| 도구(tool)      | 에이전트가 호출해 <b>행동</b> 하는 함수    | 에이전트(모델)가 호출    | 이슈 생성, 쿼리 실행, 파일 읽기  |
| 리소스(resource) | 에이전트가 참조하는 <b>데이터</b> (읽기)   | 애플리케이션/에이전트가 선택 | 파일 내용, DB 스키마, 문서    |
| 프롬프트(prompt)  | 서버가 제공하는 <b>재사용 프롬프트 템플릿</b> | 사용자가 선택해 사용     | "이 PR을 리뷰하라" 정형 프롬프트 |

쉽게 구분하면 이렇습니다. **도구**는 "**할 수 있는 동작**"(능동), **리소스**는 "**참조할 데이터**"(수동), **프롬프트**는 "**정해진 작업 절차 템플릿**"입니다. 실무에서 가장 자주 쓰는 것은 도구이며, 이 chapter의 worked example도 도구 사용을 중심으로 진행합니다.

## MCP 도구의 이름 규칙

MCP 서버가 추가한 도구는 보통 `mcp__서버이름__도구이름` 형태의 이름을 가집니다. 예컨대 GitHub 서버의 이슈 생성 도구는 `mcp__github__create_issue` 같은 이름입니다. 이 이름 규칙은 c04의 권한 규칙을 쓸 때 그대로 씁니다. "어느 서버의 어느 도구를 허용/차단할지"를 이 이름으로 지정합니다.

## worked example: 파일시스템/문서 MCP 등록·참조

실무 상황을 하나 설정하겠습니다. 여러분은 결제 서비스(payment-service) 레포에서 새 환불 API 핸들러를 작성하는 중입니다. 그런데 회사에는 **API 설계 가이드**(에러 응답 포맷·페이지네이션 규약·로깅 표준)가 별도의 사내 문서 레포(/srv/eng-docs/api-guide)로 관리되고, 결제 레포 안에는 없습니다. 가이드는 자주 갱신되어 복붙해 두면 금세 낡습니다. 에이전트가 작업 중 그 **외부 문서를 직접 참조**해 새 핸들러가 사내 표준을 지키는지 점검하게 만들고 싶습니다.

여기서 **파일시스템 MCP 서버**가 들어갑니다. 이 서버는 지정한 디렉터리의 파일을 에이전트가 안전하게 읽게 해 줍니다. "Claude Code는 이미 파일을 읽는데 왜?"라고 생각할 수 있습니다. 핵심은 **현재 작업 디렉터리(payment-service) 밖에 있는 문서 레포**를 작업 맥락으로 끌어온다는 점입니다.

결제 레포 루트에 `.mcp.json`을 만들고, 사내 문서 폴더를 노출하는 서버를 등록합니다.

```
{
  "mcpServers": {
    "eng-docs": {
      "type": "stdio",
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-filesystem",
        "/srv/eng-docs/api-guide"
      ]
    }
  }
}
```

- `mcpServers`: 등록할 MCP 서버들을 담는 최상위 키입니다. 각 서버를 이름 아래에 적습니다.
- `"eng-docs"`: 이 서버에 붙인 이름입니다. 도구 이름이 `mcp__eng-docs__...` 형태가 되고, 권한 규칙에서도 이 이름을 씁니다.
- `"type": "stdio"`: 서버를 로컬 프로세스로 띄워 표준 입출력으로 통신하는 방식입니다. 원격 HTTP 서버라면 `"type": "http"` 에 `"url"` 을 적습니다.
- `"command": "npx"`, `"args": [...]`: 서버 프로그램을 실행하는 명령과 인자입니다. 여기서는 npx로 파일 시스템 서버 패키지를 띄우고, 마지막 인자로 **노출할 디렉터리**( `/srv/eng-docs/api-guide` )를 지정했습니다. 이 폴더 밖(예: 다른 서비스의 소스)은 건드리지 못합니다(안전 경계).

서버를 등록한 뒤 세션을 시작하면, Claude Code가 이 서버를 연결합니다. `/mcp` 명령으로 연결 상태와 노출된 도구를 확인할 수 있습니다.

```
> /mcp

Connected MCP servers:
  eng-docs (stdio) ... connected
    tools: read_file, list_directory, search_files
    resources: file:// (api-guide 트리)
```

이제 메인 세션에서, 작성 중인 환불 핸들러가 사내 표준을 지키는지 외부 가이드를 기준으로 점검시킬 수 있습니다.

```
> eng-docs 서버로 api-guide에서 '에러 응답 포맷'과 '로깅 표준' 문서를 찾아 읽고,
> 지금 작성 중인 services/payment/handlers/refund.py 가 그 표준을 지키는지 점검해 줘.
```

- `eng-docs` 서버로: 에이전트가 `eng-docs` MCP 서버의 `search_files`·`read_file` 도구를 호출해 사내 문서 레포에서 에러 응답·로깅 표준 문서를 찾고 읽습니다.
- 그 문서 폴더는 **현재 결제 레포 밖**입니다. MCP가 없었다면 에이전트는 닿을 수 없었습니다. 표준 서버 등록만으로 최신 사내 가이드가 작업 맥락에 들어옵니다.
- 동시에, 서버에 노출한 `/srv/eng-docs/api-guide` 밖은 읽지 못하므로 **참조 범위가 안전하게 제한**됩니다. 다른 팀의 소스나 비밀이 든 폴더는 애초에 노출하지 않습니다.

핵심은, 가이드를 결제 레포에 복붙하거나 별도 통합 코드를 한 줄도 짜지 않고 **설정 파일 한 개**로 항상 최신인 외부 표준 문서 접근을 추가했다는 점입니다. 이것이 표준 프로토콜의 힘입니다.

## 서버 등록: .mcp.json과 범위(로컬·프로젝트·사용자)

MCP 서버는 어디에 등록하느냐에 따라 누가 쓸 수 있는지가 달라집니다. CLAUDE.md의 컨텍스트 계층(c05)·권한 범위(c04)와 같은 발상입니다. 세 가지 범위를 구분합니다.

| 범위                | 저장 위치                                   | 누가 쓰나              | 언제 쓰나                 |
|-------------------|-----------------------------------------|--------------------|-----------------------|
| 프로젝트<br>(project) | 저장소의 <code>.mcp.json</code>             | 그 저장소를 쓰는 모든<br>팀원 | 팀 공용 서버(공유·버전관리)      |
| 사용자(user)         | <code>~/.claude.json</code>             | 내 모든 프로젝트          | 개인용 서버(여러 프로젝트<br>공통) |
| 로컬(local)         | <code>~/.claude.json</code> (해당 머<br>신) | 이 머신의 나만           | 실험·개인 비밀이 든 서버        |

가장 중요한 구분은 **프로젝트 범위**입니다. `.mcp.json`은 저장소에 커밋되어 팀과 공유되므로, 팀 전체가 같은 외부 통합을 쓰게 만드는 표준 경로입니다. 다만 공유되는 만큼, 신뢰가 중요합니다.

### 프로젝트 서버는 승인 후 사용

보안을 위해 Claude Code는 `.mcp.json`에 등록된 **프로젝트 범위 서버**를 처음 쓸 때 **사용자 승인을** 요구합니다. 저장소를 클론했더니 누군가 심어 둔 수상한 MCP 서버가 자동으로 도는 일을 막기 위해서입니다. 모르는 저장소의 `.mcp.json`은 내용을 확인한 뒤 승인하세요.

명령행으로도 서버를 추가할 수 있고, 이때 `--scope`로 범위를 지정합니다.

```
> claude mcp add eng-docs --scope project -- npx -y @modelcontextprotocol/server-filesystem /srv/eng-docs/api-gui
```

이 명령은 `eng-docs` 서버를 **프로젝트 범위**(`.mcp.json`)에 추가합니다. `--scope user`로 바꾸면 내 모든 프로젝트에서 쓰는 사용자 범위로, `--scope local`이면 이 머신에만 두는 로컬 범위로 등록됩니다. `--` 뒤는 서버를 실행하는 명령입니다.

## 대표 MCP(파일시스템·GitHub·DB·브라우저)

실무에서 자주 쓰는 MCP 서버들을 용도별로 정리합니다. 각각이 어떤 외부 세계를 열어 주는지 감을 잡아 두면, 새 통합이 필요할 때 "이건 MCP 서버가 있겠다"고 떠올릴 수 있습니다.

| 서버     | 무엇을 잇나      | 대표 도구 예                          | 쓰임                |
|--------|-------------|----------------------------------|-------------------|
| 파일시스템  | 지정 디렉터리의 파일 | read_file, search_files          | 프로젝트 밖 문서·자료 참조   |
| GitHub | 이슈·PR·저장소   | list_issues, create_pr, get_file | 이슈 분류·PR 작성·코드 조회 |
| 데이터베이스 | DB 스키마·쿼리   | list_tables, run_query           | 스키마 읽고 쿼리 작성 보조   |
| 브라우저   | 웹 페이지 자동화   | navigate, screenshot, click      | E2E 점검·웹 자료 수집    |

각 서버는 그 시스템 고유의 도구를 노출하지만, **에이전트가 호출하는 방식은 모두 동일합니다**. 이것이 표준 프로토콜의 본질입니다. 새 서버를 배워도 "도구를 호출한다"는 사용법은 그대로이고, 도구 이름과 인자만 바뀝니다.

## 우리 프로젝트와의 연결

이 단원의 종합 프로젝트인 repo-audit는 표준 라이브러리만으로 동작하도록 설계됩니다(NFR1, 외부 의존 최소화). 그래서 MCP는 **필수가 아니라 선택적 확장**으로 들어갑니다. 예컨대 점검 정책 문서를 외부 폴더에 두고 문서 MCP로 참조하거나(M5), GitHub MCP로 점검 결과를 이슈로 남기는 식입니다. MCP는 "있으면 외부와 잇는 힘"이지, 도구의 핵심 로직까지 외부에 의존시키라는 뜻이 아닙니다.

## worked example: GitHub MCP로 이슈·PR 다루기

이번에는 팀리드로서 자주 마주치는 일을 자동화해 봅니다. 매주 월요일 결제 서비스 레포( acme/payment-service )에는 'bug' 라벨이 붙은 신규 이슈가 쌓이고, 그중 다수는 재현 정보가 부족해 분류가 필요합니다. 이 **이슈 트리아지(triage)**를 에이전트에 위임하려면 GitHub에 닿아야 합니다. 대표 통합인 **GitHub MCP 서버**를 등록하고, 인증 토큰을 안전하게 주입해 봅니다. 비밀(토큰) 관리가 함께 등장하므로 주의 깊게 봅니다.

`.mcp.json` 에 GitHub 서버를 추가합니다.

```
{
  "mcpServers": {
    "github": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {
        "GITHUB_TOKEN": "${GITHUB_TOKEN}"
      }
    }
  }
}
```

- `"github"`: 서버 이름입니다. 도구는 `mcp__github__...` 형태가 됩니다.
- `"env": { ... }`: 서버 프로세스에 전달할 환경 변수입니다. GitHub 서버는 API 인증을 위해 토큰이 필요합니다.

- "GITHUB\_TOKEN": "\${GITHUB\_TOKEN}": **여기가 핵심입니다.** 토큰 값을 파일에 직접 적지 않고, `${GITHUB_TOKEN}` 문법으로 **셸 환경 변수에서 읽어 옵니다.** 연결 시점에 실제 환경 변수 값이 채워집니다. 그래서 `.mcp.json` 을 저장소에 커밋해도 진짜 토큰은 새어 나가지 않습니다.

토큰은 파일이 아니라 셸 환경에 둡니다. 예컨대 셸에서 다음처럼 내보내 둡니다.

```
> export GITHUB_TOKEN=ghp_여러분의_실제_토큰
```

이제 세션에서 트리아지 작업을 위임할 수 있습니다.

```
> github 서버로 acme/payment-service 의 열린 이슈 중 'bug' 라벨이 붙고
> 재현 단계가 없는 것만 골라, 가장 오래된 이슈에
> '재현 단계·결제 수단·환경(앱 버전)을 알려 달라'는 분류 코멘트를 달아 줘.
```

작업이 진행되면 권한 확인이 개입합니다. 이슈 목록을 읽는 것(`list_issues`)은 읽기성이라 비교적 가볍지만, 코멘트를 다는 것(`create_comment`)은 외부 협업 레포의 상태를 바꾸는 **부수효과**이므로 권한 판정을 거칩니다.

```
github 서버 도구 호출 요청: mcp__github__create_comment
대상: acme/payment-service issue #1382
허용하시겠습니까? (y/N)
```

- 읽기 도구(`list_issues`)와 쓰기 도구(`create_comment`)가 **권한 면에서 다르게** 다뤄집니다. 외부 상태를 바꾸는 호출은 c04의 권한 체계가 그대로 막아섭니다.
- 매번 묻기 번거롭다면 `settings.json`에 `mcp__github__list_issues` 를 allow로 추가해 읽기만 자동 허용하고, 쓰기 도구는 ask로 남겨 둘 수 있습니다(다음 절).

여기서 두 가지를 확인했습니다. 첫째, 외부 시스템(GitHub) 통합을 설정 한 곳으로 추가했고, 둘째, 비밀은 환경 변수로 주입해 파일에 노출하지 않았습니다. 이 두 습관이 안전한 MCP 운용의 토대입니다.

## MCP 도구의 권한 통제와 비밀 관리

MCP는 에이전트의 손이 닿는 범위를 넓히는 만큼, **통제 없이 쓰면 위험**합니다. 외부 시스템에 잘못된 호출이 가면 로컬 파일 실수보다 파장이 큼니다. 두 축으로 안전을 설계합니다.

### 1) 권한 통제 — MCP 도구도 같은 권한 체계

MCP가 추가한 도구는 **예외가 아닙니다.** c04에서 배운 allow·deny·ask 규칙이 그대로 적용됩니다. `settings.json` 에서 서버·도구 단위로 통제합니다.

```
{
  "permissions": {
    "allow": [
      "mcp__eng-docs__read_file",
      "mcp__github__list_issues"
    ],
    "ask": [
      "mcp__github__create_comment"
    ],
    "deny": [
      "mcp__github__delete_repo"
    ]
  }
}
```

- **allow**: 읽기성 도구(`read_file`·`list_issues`)는 자동 허용해 마찰을 줄입니다.
- **ask**: 외부 상태를 바꾸는 코멘트 작성은 매번 확인을 거치게 둡니다.
- **deny**: 저장소 삭제처럼 돌이킬 수 없는 도구는 **아예 차단**합니다. **deny**는 **allow**보다 우선하므로 사고를 원천 봉쇄합니다.
- 도구 이름이 `mcp__서버__도구` 규칙을 따르므로, "어느 서버의 어느 동작"을 정밀하게 통제할 수 있습니다.

## 2) 비밀 관리 — 토큰은 코드가 아니라 환경으로

API 토큰·DB 비밀번호 같은 **비밀(secret)**은 절대 `.mcp.json` 이나 코드에 평문으로 적지 않습니다. 이유는 분명합니다. `.mcp.json` 은 저장소에 커밋되어 공유되므로, 토큰을 적으면 **전체 팀과 git 이력에 비밀이 노출**됩니다. 앞 예제처럼 `${GITHUB_TOKEN}` 환경 변수 참조로 두고, 실제 값은 각자의 셸 환경·시크릿 매니저에 둡니다.

| 하지 말 것                            | 대신 할 것                                                            |
|-----------------------------------|-------------------------------------------------------------------|
| <code>.mcp.json</code> 에 토큰 평문 기입 | <code>\${VAR}</code> 로 환경 변수 참조                                   |
| 토큰을 코드·커밋에 포함                     | 셸 환경·시크릿 매니저에 보관                                                  |
| 모든 MCP 도구 자동 허용                   | 읽기만 <code>allow</code> , 쓰기는 <code>ask</code> · <code>deny</code> |
| 알 수 없는 프로젝트 서버 무심코 승인             | 내용 확인 후 신뢰 시 승인                                                   |

## MCP 안전 운용 체크

- 비밀은 `${VAR}` 환경 변수로 주입한다(파일·커밋에 평문 금지).
- 외부 상태를 바꾸는 도구는 `ask`, 위험 도구는 `deny`.
- 서버 노출 범위를 최소로(파일시스템 서버는 꼭 필요한 폴더만).
- 프로젝트 범위 서버는 신뢰를 확인하고 승인한다.

## 즉시 연습 (2종 1세트)

### 연습 1: `.mcp.json` 서버 등록 채우기

## 모범 예시

사내 API 설계 가이드 문서 폴더를 프로젝트 범위로 등록한 모범입니다(앞 worked example과 같은 상황).

```
{
  "mcpServers": {
    "eng-docs": {
      "type": "stdio",
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-filesystem",
        "/srv/eng-docs/api-guide"
      ]
    }
  }
}
```

## 학생 작성형 빈 템플릿

아래 빈칸을 채워, **주문 서비스 운영 DB의 읽기 전용 복제본 스키마를 조회하는 PostgreSQL MCP 서버**를 등록해 보세요. 쿼리 작성 보조를 위해 스키마(테이블·컬럼)를 읽되, 접속 비밀번호는 절대 평문으로 적지 말고 환경 변수로 주입합니다.

```
{
  "mcpServers": {
    "_____": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "_____", "_____"],
      "env": {
        "POSTGRES_PASSWORD": "_____"
      }
    }
  }
}
```

채울 때 생각할 점: - 마지막 인자에 운영 쓰기 DB가 아니라 **읽기 전용 복제본**의 접속 정보를 적었나요? - `POSTGRES_PASSWORD` 값에 진짜 비밀번호를 적었나요, `${...}` 참조를 적었나요? - 서버 이름이 도구 이름(`mcp__이름__...`)으로 쓰일 것을 고려했나요? - 이 서버를 팀과 공유하려면 어느 범위(프로젝트/사용자/로컬)여야 할까요?

## 자가체크

- [ ] `mcpServers` 아래에 서버 이름을 두었는가
- [ ] `command` · `args` 로 서버 실행 방법을 적었는가
- [ ] 비밀을 평문이 아니라 `${VAR}` 환경 변수로 주입했는가
- [ ] 공유가 필요하면 프로젝트 범위(`mcp.json`)에 둘 것을 이해했는가

## 연습 2: MCP 도구 권한 규칙 채우기

## 모범 예시

결제 서비스 트리아지에 쓰는 GitHub 서버 도구를 읽기·쓰기·위험으로 나눈 권한 규칙의 모범입니다.

```
{
  "permissions": {
    "allow": ["mcp_github__list_issues", "mcp_github__get_file"],
    "ask": ["mcp_github__create_comment"],
    "deny": ["mcp_github__delete_repo"]
  }
}
```

## 학생 작성형 빈 템플릿

사내 문서 서버 `eng-docs` (파일시스템)와 주문 DB 서버 `orders-db` (PostgreSQL) 도구를 위험도에 따라 분류해 채워 보세요. 운영 DB를 다루므로 분류가 곧 사고 예방입니다.

```
{
  "permissions": {
    "allow": [ "_____", "_____" ],
    "ask": [ "_____" ],
    "deny": [ "_____" ]
  }
}
```

분류 힌트: - 읽기만 하는 도구(예: `mcp_eng-docs__read_file`, `mcp_orders-db__list_tables`)는 어디에? - 운영 데이터베이스를 바꾸는 쓰기 쿼리 실행(예: `mcp_orders-db__run_query`)은 어디에? - 테이블을 통째로 지우는 도구(예: `mcp_orders-db__drop_table`)가 있다면 어디에?

## 자가체크

- [ ] 읽기성 도구를 allow에 두어 마찰을 줄였는가
- [ ] 외부 상태를 바꾸는 도구를 ask로 두어 확인을 거치게 했는가
- [ ] 돌이킬 수 없는 위험 도구를 deny로 차단했는가
- [ ] 도구 이름을 `mcp_서버__도구` 규칙으로 적었는가

## 흔한 오류와 오개념

### 1. "MCP는 플러그인 난립"이라는 오해

MCP를 "이것저것 붙이는 잡다한 플러그인"으로 오해하기 쉽습니다. 정반대입니다. MCP는 통합을 **하나의 표준 규격으로 캡슐화**해, 제각각인 전용 연결의 난립을 막는 프로토콜입니다. 표준이 있어  $N \times M$  통합 폭발이  $N + M$ 으로 줄고, 서버가 외부 시스템의 복잡함을 안에 숨겨 에이전트에는 깔끔한 도구만 보입니다.

**교정:** MCP를 "표준 콘센트"로 이해합니다. 난립을 늘리는 게 아니라 줄이는 규격입니다.

### 2. "MCP 도구는 권한 밖"이라는 오해

MCP가 추가한 도구가 일반 도구와 다른 특별 대우를 받아 권한 체계를 우회한다고 생각하면 위험합니다. **MCP 도구도 동일한 allow·deny·ask 권한 체계로 통제**됩니다. 오히려 외부 시스템을 건드리므로 더 엄격해야 합니다. 외부 상태를 바꾸는 도구는 ask, 위험한 도구는 deny로 둡니다.

**교정:** mcp\_\_서버\_\_도구 이름으로 권한 규칙을 명시해, 읽기는 allow·쓰기는 ask·위험은 deny로 분리합니다(c04).

### 3. 토큰을 .mcp.json에 평문으로 적기

가장 위험한 실수입니다. .mcp.json 은 저장소에 커밋되어 팀과 git 이력에 공유됩니다. 토큰·비밀번호를 평문으로 적으면 **비밀이 통째로 노출**되고, 한 번 커밋되면 이력에서 지우기도 어렵습니다.

**교정:** 비밀은 "GITHUB\_TOKEN": "\${GITHUB\_TOKEN}" 처럼 환경 변수 참조로 두고, 실제 값은 각자의 셸 환경·시크릿 매니저에 둡니다. 연결 시점에 값이 채워지므로 파일에는 비밀이 남지 않습니다.

## 단원 미니 프로젝트: 전문 에이전트와 외부 연동 구성 (2종 1세트)

이 단원(u5)에서 배운 두 개념, **서브에이전트 위임(c09)**과 **MCP 연동(c10)**을 합쳐 하나의 작은 산출물을 만듭니다. 목표는 "외부 자료를 참조해 코드를 리뷰하는 전문 세션"을 구성하는 것입니다. 종합 프로젝트 repo-audit의 M5(안전 게이트와 외부 연동) 단계를 미리 연습하는 셈입니다.

### 미니 프로젝트 목표

- 산출물: .claude/agents/reviewer.md (코드 리뷰 서브에이전트) + .mcp.json (문서 MCP 서버 등록)
- 완료 조건: reviewer 서브에이전트가 문서 MCP로 **외부 점검 정책 문서**를 참조해, 우리 코드가 그 정책을 지키는지 리뷰 결과를 낸다.
- 적용 개념: c09(서브에이전트 정의·위임) + c10(MCP 등록·권한·비밀)

### 마일스톤(증분)

| 마일스톤 | 할 일                                            | 적용 개념     |
|------|------------------------------------------------|-----------|
| M1   | 역할·tools·프롬프트를 갖춘 reviewer 서브에이전트 정의           | c09       |
| M2   | 점검 정책 문서를 문서 MCP 서버로 .mcp.json 에 등록(비밀은 환경 변수) | c10       |
| M3   | reviewer가 외부 정책 문서를 참조해 코드를 리뷰하는지 확인           | c09 + c10 |

### 모범 예시

**M1 — reviewer 서브에이전트( .claude/agents/reviewer.md ):**

```

---
name: reviewer
description: 외부 점검 정책 문서를 기준으로 코드 품질·관례 위반을 점검하는 리뷰어. 코드 작성 직후 정책 준수 리뷰.
tools: Read, Grep, Glob
model: sonnet
---

```

당신은 시니어 코드 리뷰어입니다. docs MCP 서버가 제공하는  
점검 정책 문서를 먼저 읽고, 그 기준으로만 전달받은 코드를 점검합니다.  
코드는 수정하지 않으며, 다음 형식으로 보고합니다.

보고 형식

- 정책 항목별 준수/위반 표(항목·파일:줄·판정·근거)
- 위반 우선 수정 순서 한 줄

## M2 — 문서 MCP 등록( .mcp.json ):

```

{
  "mcpServers": {
    "docs": {
      "type": "stdio",
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-filesystem",
        "/home/team/review-policies"
      ]
    }
  }
}

```

**M3 — 호출과 결과:** 리뷰 대상은 다음 파이썬 코드의 발췌라고 합시다. 정책이 요구하는 타입 힌트가 빠져 있습니다.

```

# src/repo_audit.py (발췌) - 리뷰 대상
def main(argv):                                # 반환 타입 힌트 누락(정책 위반)
    args = parse_args(argv)
    findings = run_checks(args.path)
    print(render_text(findings))

```

실행 결과:

```

$ python src/repo_audit.py ./sample
점검 3건 (error 1, warn 2)

```

이 코드를 reviewer 서브에이전트에 위임해 외부 정책 문서를 기준으로 점검시킵니다.

- > reviewer 서브에이전트로 src/repo\_audit.py를 리뷰해 줘.
- > docs 서버의 정책 문서를 기준으로 점검하고 표로 보고해.

```
reviewer 결과 (정책 기준: review-policies/python.md)
| 정책 항목 | 위치 | 판정 | 근거 |
| 공개 함수 타입 힌트 | repo_audit.py:30 | 위반 | main 반환 타입 누락 |
| 입력 검증 | repo_audit.py:18 | 준수 | 경로 존재 확인 있음 |
| 함수 길이 50줄 이하 | checks.py:12 | 위반 | check_links 72줄 |
우선 수정: 타입 힌트 보강 -> 함수 분리.
```

메인은 이 지적을 반영해 타입 힌트를 보강합니다.

```
# src/repo_audit.py (발췌) - 정책 위반 수정본
def main(argv: list[str]) -> int:    # 타입 힌트 보강(정책 준수)
    args = parse_args(argv)
    findings = run_checks(args.path)
    print(render_text(findings))
    return 1 if has_error(findings) else 0
```

```
실행 결과:
$ python src/repo_audit.py ./sample
점검 3건 (error 1, warn 2)
$ echo $?
1
```

서브에이전트(c09)가 격리된 컨텍스트에서, **MCP 서버 연동**(c10)으로 들어온 외부 정책 문서를 기준으로 리뷰하는 모습입니다. 두 개념이 한 산출물로 맞물립니다.

## 학생 작성형 빈 스타터

아래 골격을 채워 여러분의 "전문 에이전트 + 외부 연동" 세션을 구성하세요.

(1) 서브에이전트 `.claude/agents/_____`.md :

```
---
name: _____
description: _____
tools: _____
model: _____
---
```

당신은 \_\_\_\_\_ 입니다. \_\_\_\_\_ MCP 서버가 제공하는  
\_\_\_\_\_ 를 참조해 다음 일을 합니다.

작업

- \_\_\_\_\_

제약

- \_\_\_\_\_

보고 형식

- \_\_\_\_\_

(2) MCP 등록 `.mcp.json` :

```
{
  "mcpServers": {
    "_____": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "_____", "_____", "_____"],
      "env": { "_____": "_____"}
    }
  }
}
```

### (3) 호출 지시(메인 세션):

```
> _____ 서브에이전트로 _____ 를 점검해 줘.
> _____ 서버의 _____ 를 기준으로 _____ 해.
```

## 미니 프로젝트 자가체크

- [ ] 서브에이전트 권한(tools)을 역할에 필요한 **최소**로 줄였는가(리뷰면 쓰기 도구 제외)
- [ ] MCP 비밀을 코드·파일 평문이 아니라 환경 변수(`${VAR}`)로 주입했는가
- [ ] MCP 서버가 노출하는 폴더 범위를 꼭 필요한 곳으로 제한했는가
- [ ] 서브에이전트가 외부 자료를 **실제로 참조해** 결과를 내는지 확인했는가
- [ ] 외부 상태를 바꾸는 도구가 있다면 ask/deny로 통제했는가

## 자주 묻는 질문(FAQ)

**Q. MCP 서버는 직접 만들어야 하나요?** 대부분은 아닙니다. 파일시스템·GitHub·DB 같은 흔한 통합은 공식·커뮤니티 서버를 npx 한 줄로 띄워 씁니다. 사내 고유 시스템을 잇고 싶을 때만 직접 서버를 만들면 됩니다. MCP가 표준이라 한 번 만든 서버는 모든 MCP 지원 에이전트가 씁니다.

**Q. MCP 도구와 서브에이전트(c09)는 어떻게 함께 쓰나요?** 서브에이전트는 "일을 나누는" 확장, MCP는 "도구를 늘리는" 확장입니다. 둘은 직교합니다. 미니 프로젝트처럼, MCP로 외부 자료에 닿을 수 있는 전문 서브에이전트를 만들면 두 확장이 곱해집니다.

**Q. `${GITHUB_TOKEN}` 이 안 채워지면요?** 연결 시점에 셸 환경에 그 변수가 없으면 빈 값이 들어가 인증이 실패합니다. 세션을 시작하기 전 환경 변수를 내보냈는지(`export GITHUB_TOKEN=...`) 확인하세요. 시크릿 매니저를 쓰면 더 안전합니다.

## 마무리

이 챕터에서는 에이전트를 바깥 세상과 잇는 **MCP(Model Context Protocol)**를 배웠습니다. MCP는 외부 통합을 표준화하는 프로토콜이고(정의), 서버가 도구·리소스·프롬프트를 노출하면 에이전트가 표준 방식으로 호출하며(동작 원리), `.mcp.json`으로 범위를 나눠 등록하고(로컬·프로젝트·사용자), MCP 도구도 동일한 권한 체계로 통제하며 비밀은 환경 변수로 주입합니다(안전). 그리고 단원 미니 프로젝트에서 서브에이전트(c09)와 MCP(c10)를 합쳐, 외부 정책을 참조하는 전문 리뷰 세션을 구성했습니다.

이로써 단위 u5(확장 — 위임과 외부 연동)를 마칩니다. 다음 단위 u6에서는 자동화를 결정론적으로 강제하는 **혹**, 여러 에이전트를 엮는 **오케스트레이션**, 사람 없이 도는 **CI-스케줄 자동화**로 나아갑니다. 위임과 연동으로 넓힌 에이전트의 능력을, 이제 "항상 안전하게 자동으로" 돌리는 단계입니다.



## 11장 결정론적 게이트: 흑으로 자동화 강제하기

이 장은 단원 6 "자동화 게이트와 오케스트레이션"의 첫 챕터입니다. 여기서는 개념 **흑(자동화 게이트, hooks)** 하나를 깊이 있게 배웁니다. 흑은 에이전트가 "알아서 잘하기를 바라는" 확률적 기대를, 코드로 "항상 일어나는" 결정론적 규칙으로 바꾸는 장치입니다.

### 이 장의 학습 목표

- 흑이 에이전트 생애주기의 어느 시점에 실행되는지 설명할 수 있습니다.
- PreToolUse-PostToolUse 등 흑으로 게이트를 구성할 수 있습니다.
- 결정론적 자동화(포맷·린트·정책)를 흑으로 강제할 수 있습니다.
- 흑의 차단·허용(exit code) 동작과 안전 설계를 할 수 있습니다.

### 11.1 도입: 확률이 아니라 '항상' 강제하기

지난 단원까지 우리는 에이전트에게 "편집한 파일은 Black으로 포맷해 줘", "secrets 폴더는 건드리지 마"처럼 **부탁**을 해 왔습니다. 좋은 프롬프트(2장)와 CLAUDE.md(5장)는 이 부탁이 잘 지켜지도록 돕습니다. 하지만 부탁은 본질적으로 확률적입니다. 같은 규칙을 CLAUDE.md에 적어 두어도, 긴 세션에서 컨텍스트가 길어지거나 모델이 다른 일에 집중하면 한두 번은 빠뜨릴 수 있습니다. 실무에서 "거의 항상 지켜지는" 규칙과 "반드시 지켜지는" 규칙의 차이는 큼니다. 비밀 파일을 99퍼센트 보호하는 것은 보호하지 못하는 것과 같습니다.

4장에서 배운 **권한과 도구 허용목록(allowlist)**은 이 문제의 절반을 풀어 줍니다. 권한은 "이 도구 호출을 허용 할지/물어볼지/막을지"를 판정합니다. 그러나 권한이 할 수 있는 일은 허용·거부·확인이라는 세 가지 판정으로 제한됩니다. "편집이 끝나면 자동으로 포매터를 돌려라", "이 경로를 수정하려 하면 막되 그 사실을 로그 파일에 남겨라"처럼 **임의의 로직을 실행**하는 것은 권한의 영역이 아닙니다. 바로 이 지점에서 흑이 등장합니다.

선수 개념 한 줄 상기: 4장의 권한(allow·deny·ask)은 부수효과 도구 호출을 **판정**했습니다. 이번 장의 흑은 그 생애주기의 여러 시점에서 **임의의 스크립트**를 실행해, 판정만으로는 할 수 없는 자동화와 강제를 더합니다.

이 장을 마치면 여러분은 "이 규칙은 사람이나 LLM의 선의에 맡길 수 없다"고 판단되는 작업을, 코드로 못 박는 방법을 갖게 됩니다. 그리고 그 게이트가 정상 작업까지 막지 않도록 안전하게 설계하는 법까지 배웁니다.

### 11.2 정의: 흑이란 생애주기 이벤트에 실행되는 스크립트

**정의.** 훅(hook)이란 Claude Code가 에이전트 생애주기의 특정 이벤트(도구 호출 직전·직후, 세션 시작, 정지, 서브에이전트 완료 등)에 도달할 때 **결정론적으로 실행되는 사용자 정의 셸 명령·스크립트**입니다. 훅은 LLM의 확률적 판단이 아니라 우리가 작성한 코드이므로, 등록된 이벤트가 발생하면 **항상** 실행됩니다.

핵심 용어를 한글(영문)으로 정리합니다.

- **생애주기 이벤트(lifecycle event):** 에이전트 작업의 단계 구분점. 도구를 쓰기 직전(PreToolUse), 직후(PostToolUse), 세션이 시작될 때(SessionStart), 에이전트가 응답을 끝내려 할 때(Stop) 등이 있습니다.
- **매처(matcher):** 어떤 도구·패턴에 훅을 적용할지 거르는 필터입니다. 예를 들어 `Write|Edit` 는 파일을 만들거나 고치는 도구에만 훅을 붙입니다.
- **종료 코드(exit code):** 훅 스크립트가 끝나며 운영체제에 돌려주는 숫자입니다. `0` 은 정상, `2` 는 차단(blocking)을 의미하며, 이 숫자로 Claude Code는 작업을 계속할지 막을지 결정합니다.

훅과 권한을 혼동하기 쉬우므로 한 표로 못 박아 둡니다.

| 구분      | 권한(allowlist)             | 훅(hooks)                        |
|---------|---------------------------|---------------------------------|
| 무엇을 하는가 | 도구 호출을 허용·거부·확인 <i>판정</i> | 이벤트 시점에 <i>임의</i> 로직 실행         |
| 표현력     | allow / deny / ask 세 가지   | 셸로 할 수 있는 모든 일(포맷·린트·테스트·로깅·차단) |
| 동작 시점   | 도구 호출 직전 1회               | 도구 전후·세션 시작·정지 등 여러 시점          |
| 결정론성    | 규칙 매칭(결정론적)               | 스크립트 실행(결정론적)                   |
| 대표 용도   | "git commit은 허용, rm은 거부"  | "편집 후 자동 포맷", "위험 경로 수정 차단·로깅"  |

즉 권한은 "문을 열어 줄지 말지"를 정하는 경비원이고, 훅은 "문을 지날 때마다 신발을 닦고 방명록을 쓰게 하"는" 정해진 절차입니다. 둘은 경쟁 관계가 아니라 함께 쓰여 안전한 자동화를 만듭니다.

## 11.3 동작 원리: 이벤트와 차단·허용(exit code) 제어

## 혹: 생애주기 이벤트 → 스크립트 → exit code 제어



### 혹 이벤트 종류



혹의 동작은 하나의 타임라인으로 이해하면 명확합니다. 에이전트가 도구를 하나 쓰려 할 때 무슨 일이 벌어지는지 단계로 따라가 봅니다.

- 이벤트 발생.** 에이전트가 도구(예: Edit) 호출 파라미터를 만들어 냅니다. 이때 Claude Code는 그 도구 이름이 등록된 혹의 매치에 맞는지 봅니다.
- PreToolUse 혹 실행.** 매치에 맞으면, 도구를 실제로 실행하기 *전에* 등록된 혹 스크립트를 실행합니다. 이때 Claude Code는 표준 입력(stdin)으로 JSON을 넘겨 주는데, 여기에는 `hook_event_name`, `tool_name`, `tool_input` (편집 대상 경로·내용 등), `cwd`, `session_id` 같은 맥락이 담깁니다.
- 종료 코드로 판정.** 혹이 끝나며 돌려준 종료 코드를 Claude Code가 읽습니다.
- 종료 코드 0: 정상. 도구 호출을 그대로 진행합니다.
- 종료 코드 2: 차단(blocking). PreToolUse에서는 그 도구 호출이 막히고, 혹이 표준 오류(stderr)에 쓴 메시지가 에이전트에게 피드백으로 전달됩니다. 에이전트는 그 이유를 보고 다른 방법을 시도합니다.
- 그 밖의 0이 아닌 코드: 비차단 오류. 메시지는 기록되지만 작업은 막지 않습니다.
- 도구 실행.** 차단되지 않았다면 도구가 실제로 실행됩니다(파일이 편집됩니다).
- PostToolUse 혹 실행.** 도구 호출이 성공한 직후 등록된 혹이 실행됩니다. 여기서 포맷터·린터·테스트를 돌리는 것이 전형적입니다. 이 시점의 차단은 이미 일어난 편집을 되돌리진 못하지만, 문제를 에이전트에게 알려 후속 수정을 유도합니다.

종료 코드 외에 더 정교한 제어가 필요하면 혹은 표준 출력(stdout)에 JSON을 써서 의사를 전달할 수도 있습니다. PreToolUse의 경우 `hookSpecificOutput.permissionDecision`에 `allow` (권한 프롬프트를 건너뛰고 자동 승인), `deny` (차단), `ask` (사용자에게 확인 요청)를 담아 더 세밀하게 결정할 수 있습니다. 입문 단계에서는 "종료 코드 2면 차단, 0이면 통과"라는 단순 규약만으로도 강력한 게이트를 만들 수 있으니, 우선 이 규약에 익숙해지는 것을 권합니다.

**기억하세요.** 혹의 힘은 두 가지에서 나옵니다. 첫째, **결정론** — 코드라서 등록된 이벤트마다 빠짐없이 실행됩니다. 둘째, **차단력** — PreToolUse에서 종료 코드 2를 내면 위험한 도구 호출을 *실행되기 전에* 막을 수 있습니다.

## 11.4 worked example: 편집 후 자동 포맷 PostToolUse 훅

실무 상황을 하나 깔겠습니다. 여러분의 백엔드 팀은 모든 파이썬 코드를 Black으로 포맷팅하기로 합의했고, CI에 `black --check` 게이트도 걸어 두었습니다. 그런데 에이전트가 결제 모듈을 편집할 때 포맷을 빠뜨리면, 기능은 멀쩡한데 CI가 포맷 위반으로 빨갛게 떨어져 리뷰가 지연됩니다. "포맷도 해 줘"를 매번 부탁하는 것은 확률적이라 미덥지 않습니다.

가장 흔하고 안전한 훅으로 이 문제를 못 박습니다. "에이전트가 파이썬 파일을 편집하면, 그 직후 자동으로 Black 포맷터를 돌린다"는 규칙입니다. 부탁이 아니라 항상 실행되는 절차로 만듭니다.

먼저 훅 스크립트를 작성합니다. 표준 입력으로 들어오는 JSON에서 편집된 파일 경로를 꺼내 포맷터를 돌립니다.

```
#!/usr/bin/env python3
"""편집된 파이썬 파일을 Black으로 자동 포맷하는 PostToolUse 훅."""
import json
import subprocess
import sys

def main() -> int:
    # 혹은 표준 입력(stdin)으로 이벤트 맥락 JSON을 받습니다.
    raw = sys.stdin.read()
    try:
        event = json.loads(raw)
    except json.JSONDecodeError:
        # 입력이 비정상이어도 작업을 막지 않습니다(fail-open).
        return 0

    # tool_input.file_path 에 편집 대상 경로가 들어 있습니다.
    tool_input = event.get("tool_input", {})
    file_path = tool_input.get("file_path", "")

    # 파이썬 파일이 아니면 포맷할 필요가 없습니다.
    if not file_path.endswith(".py"):
        return 0

    # 포맷터를 실행합니다. 실패해도 편집은 이미 끝났으므로 차단하지 않습니다.
    result = subprocess.run(
        ["black", file_path],
        capture_output=True,
        text=True,
    )
    if result.returncode != 0:
        # 포맷 실패는 표준 오류로 알려되 비차단(0)으로 둡니다.
        sys.stderr.write(f"black 실행 경고: {result.stderr}\n")
    return 0

if __name__ == "__main__":
    sys.exit(main())
```

### 줄별 해설

- `raw = sys.stdin.read()` : 혹은 인자가 아니라 표준 입력으로 이벤트 맥락을 받습니다. 이 한 줄로 Claude Code가 넘긴 JSON 전체를 읽습니다.

- `event = json.loads(raw)`: 받은 문자열을 딕셔너리로 바꿉니다. `except` 에서 파싱 실패 시 `return 0` 으로 둔 것은, 혹 자체의 사소한 오류가 정상 작업을 막지 않게 하려는 **fail-open** 설계입니다(11.8에서 자세히 다룹니다).
- `tool_input.get("file_path", "")`: `PostToolUse` 이벤트의 `tool_input` 에는 방금 편집한 도구의 입력이 그대로 담깁니다. `Edit-Write` 도구라면 `file_path` 가 있습니다.
- `if not file_path.endswith(".py")`: 파이썬 파일이 아니면 즉시 통과시킵니다. 매처로 1차로 거르지만, 스크립트 안에서 한 번 더 확인해 불필요한 포맷 실행을 막습니다.
- `subprocess.run(["black", file_path], ...)`: 실제 포매터를 돌립니다. 리스트 형태로 인자를 넘겨 셸 인젝션을 피합니다.
- `return 0`: 포맷이 성공하든 경고가 나든 `PostToolUse`에서는 0을 반환합니다. 포맷 실패로 에이전트 작업을 막을 이유는 없기 때문입니다.

이제 이 스크립트를 `.claude/settings.json` 의 `hook` 설정에 연결합니다. 매처 `Write|Edit` 로 파일을 만들거나 고치는 도구에만 붙입니다.

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Write|Edit",
        "hooks": [
          {
            "type": "command",
            "command": "python3 .claude/hooks/format_py.py"
          }
        ]
      }
    ]
  }
}
```

위 설정에서 `matcher` 는 어떤 도구에 반응할지, `command` 는 무엇을 실행할지를 정합니다. 이제 에이전트가 어떤 파이썬 파일이든 편집하면, 그 직후 `hook` 이 자동으로 실행됩니다.

```
실행 결과 (세션 로그 발췌)
> services/billing/invoice.py 의 부가세 계산 로직을 고쳐 줘
Edit services/billing/invoice.py (1 edit)
[PostToolUse hook] python3 .claude/hooks/format_py.py
reformatted services/billing/invoice.py
All done! 1 file reformatted.
```

에이전트가 편집을 마치자마자 포매터가 돌아 들여쓰기·따옴표·줄 길이가 팀 표준으로 정리됩니다. 우리가 매번 "포맷도 해 줘"라고 부탁할 필요가 없어졌고, 포맷 위반으로 CI가 떨어지는 일도 사라집니다. 이 규칙은 이제 부탁이 아니라 **항상 일어나는 절차**입니다.

## 11.5 이벤트 종류: 언제 `hook` 이 불리는가

`hook` 을 잘 쓰려면 어떤 이벤트가 있는지 알아야 합니다. 자주 쓰는 다섯 가지를 정리합니다.

| 이벤트                 | 실행 시점             | 대표 용도               | 차단 가능?           |
|---------------------|-------------------|---------------------|------------------|
| <b>PreToolUse</b>   | 도구 호출 직전          | 위험 편집·명령 차단, 정책 검사  | 예(exit 2로 도구 차단) |
| <b>PostToolUse</b>  | 도구 호출 성공 직후       | 자동 포맷·린트·테스트·로깅     | 사실상 사후 알림        |
| <b>Stop</b>         | 에이전트가 응답을 끝내려 할 때 | 완료 조건 검증(테스트 통과 확인) | 예(작업을 더 시킴)      |
| <b>SessionStart</b> | 세션이 시작될 때         | 컨텍스트 주입, 환경 점검      | 아니오              |
| <b>SubagentStop</b> | 서브에이전트가 작업을 끝낼 때  | 위임 결과 검증·집계         | 예                |

각 이벤트의 쓰임을 한 줄씩 더 풀어 봅니다.

- **PreToolUse**는 게이트의 핵심입니다. 도구가 *실행되기 전에* 끼어들어 위험을 막습니다. 비밀 파일 수정·`rm -rf`·운영 DB 접근 같은 것을 여기서 차단합니다.
- **PostToolUse**는 "한 행동이 끝난 직후의 반응"입니다. 편집 후 포맷(11.4)이 가장 대표적입니다. 테스트를 돌려 회귀를 즉시 잡을 수도 있습니다.
- **Stop**은 "이 정도면 끝"이라고 에이전트가 판단했을 때 한 번 더 검증합니다. "테스트가 다 통과해야만 끝낼 수 있다"는 완료 게이트를 여기에 둡니다.
- **SessionStart**는 세션마다 필요한 맥락(예: 현재 브랜치·최근 이슈)을 자동 주입하거나, 필수 도구가 설치돼 있는지 점검합니다.
- **SubagentStop**은 9장에서 배운 서브에이전트가 일을 마칠 때 그 결과를 검증·집계합니다. 12장의 오케스트레이션에서 통합 지점으로 유용합니다.

**선택 기준.** "행동을 막아야 한다" → PreToolUse, "행동 직후 반응해야 한다" → PostToolUse, "완료를 검증해야 한다" → Stop, "세션 준비가 필요하다" → SessionStart, "위임 결과를 검증해야 한다" → SubagentStop. 무엇을 강제하려는지부터 정하면 이벤트는 자연히 정해집니다.

Stop 이벤트가 특히 유용한 장면을 짧은 예제로 보겠습니다. 결제 환불 로직처럼 회귀가 곧 매출 사고로 이어지는 코드를 에이전트가 수정한 세션을 생각해 보세요. "에이전트가 작업을 끝내려 할 때 테스트가 모두 통과하는지 확인하고, 실패하면 끝내지 못하게 막는" 완료 게이트가 필요합니다. 이 훅이 있으면 "환불 테스트가 깨진 채로 작업이 끝났다고 보고"하는 일을 코드로 차단할 수 있습니다.

```
#!/usr/bin/env python3
"""완료 전 테스트 통과를 강제하는 Stop 훅."""
import subprocess
import sys

def main() -> int:
    # 에이전트가 응답을 끝내려 할 때 테스트를 한 번 돌립니다.
    result = subprocess.run(
        ["python3", "-m", "pytest", "-q"],
        capture_output=True,
        text=True,
    )
    if result.returncode != 0:
        # 테스트 실패면 완료를 막고 사유를 에이전트에 피드백합니다.
        sys.stderr.write(
            "테스트가 실패해 작업을 끝낼 수 없습니다. "
            "실패한 테스트를 먼저 고치세요.\n"
        )
        return 2 # Stop 차단: 에이전트가 작업을 더 진행하게 됨

    return 0 # 테스트 통과: 정상 완료 허용

if __name__ == "__main__":
    sys.exit(main())
```

## 해설

- Stop 혹은 에이전트가 "끝났다"고 판단한 그 시점에 실행됩니다. 여기서 테스트를 돌려 진짜로 끝나도 되는지 검증합니다.
- `return 2`: Stop에서 차단하면 에이전트는 완료하지 못하고, 표준 오류의 메시지를 보고 작업을 더 진행합니다(실패 테스트 수정).
- `return 0`: 테스트가 통과해야만 정상 완료를 허용합니다. "완료 조건"을 사람의 확인이 아니라 코드로 못 박은 것입니다.
- 설정은 다른 훅과 같습니다. "Stop": [ { "hooks": [ { "type": "command", "command": "python3 .claude/hooks/verify\_tests.py" } ] } ] 형태로 등록하며, Stop은 도구가 아니라 시점 이벤트라 매치가 필요 없습니다.

## 11.6 훅 설정과 매치: 도구·패턴 매칭

훅은 `settings.json`의 `hooks` 키 아래에 **이벤트별로** 등록합니다. 구조는 항상 같습니다. 이벤트 이름 아래에 `matcher`와 실행할 `command`들의 묶음을 둡니다.

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          { "type": "command", "command": "python3 .claude/hooks/guard_bash.py" }
        ]
      },
      {
        "matcher": "Write|Edit",
        "hooks": [
          { "type": "command", "command": "python3 .claude/hooks/guard_path.py" }
        ]
      }
    ],
    "PostToolUse": [
      {
        "matcher": "Write|Edit",
        "hooks": [
          { "type": "command", "command": "python3 .claude/hooks/format_py.py" }
        ]
      }
    ]
  }
}
```

매처 작성의 핵심 규칙입니다.

- 매처 값은 **도구 이름**입니다. Bash, Write, Edit, Read 등이 들어갑니다.
- `|` 로 여러 도구를 묶을 수 있습니다(Write|Edit 는 둘 다 매칭).
- 한 이벤트에 매처가 다른 여러 항목을 둘 수 있습니다(위 예처럼 Bash용 가드와 파일용 가드를 분리).
- 매처를 비우거나 `*` 로 두면 그 이벤트의 모든 도구에 적용됩니다(넓게 거니 주의해서 씁니다).

설정 파일의 **범위(scope)** 는 4장에서 배운 권한과 같습니다. 프로젝트 공용 규칙은 `.claude/settings.json` (팀과 공유, git에 커밋), 개인용은 `.claude/settings.local.json` (공유 안 함)에 둡니다. 팀이 함께 강제해야 할 게이트 (비밀 보호.포맷)는 반드시 프로젝트 범위에 두어야 모두에게 적용됩니다.

**주의.** 매처는 도구 이름만 거릅니다. "git commit 명령만"처럼 명령 패턴까지 좁히려면, 매처는 Bash 로 두고 후 스크립트 안에서 `tool_input.command` 문자열을 검사해 판단해야 합니다. 매처에 정규식 같은 명령 패턴을 직접 쓰는 것이 아니라는 점을 기억하세요.

## 11.7 worked example: 위험 경로 수정 막는 PreToolUse 차단 훅

이번에는 훅의 차단력을 직접 씁니다. 실무에서 가장 비싼 사고 중 하나는 에이전트가 **운영 비밀이나 인프라 설정**을 잘못 수정하는 것입니다. 운영 DB 접속 문자열이 든 `.env.production` 이나 운영 테라폼 디렉터리 (`infra/prod/`)를 에이전트가 리팩터링하다 건드리면, 한 번의 편집이 전체 서비스 장애로 번질 수 있습니다. "에이전트가 `secrets/`, `.env.production`, `infra/prod/` 아래 파일을 수정하려 하면 **실행 전에** 막고, 그 시도를 감사 로그에 남긴다"는 안전 게이트를 만듭니다. 종합 프로젝트(repo-audit)의 NFR3 "위험 경로 편집을 PreToolUse 훅으로 차단"이 바로 이것입니다.

```
#!/usr/bin/env python3
"""운영 비밀·인프라 경로 편집을 막는 PreToolUse 차단 후크."""
import json
import sys
from datetime import datetime
from pathlib import Path

# 수정을 금지할 경로 조각(부분 문자열로 매칭).
PROTECTED = ("secrets/", ".env.production", "infra/prod/")
LOG_PATH = Path(".claude/guard.log")

def log_attempt(message: str) -> None:
    # 차단 시도를 추적 가능하도록 로그에 남깁니다.
    stamp = datetime.now().isoformat(timespec="seconds")
    with LOG_PATH.open("a", encoding="utf-8") as fp:
        fp.write(f"{stamp} {message}\n")

def main() -> int:
    try:
        event = json.loads(sys.stdin.read())
    except json.JSONDecodeError:
        # 가드 자신의 오류로 작업을 막지 않습니다(fail-open).
        return 0

    file_path = event.get("tool_input", {}).get("file_path", "")

    # 보호 경로에 해당하면 차단(exit 2)합니다.
    if any(token in file_path for token in PROTECTED):
        log_attempt(f"BLOCKED edit: {file_path}")
        sys.stderr.write(
            f"보호 경로는 수정할 수 없습니다: {file_path}. "
            f"이 파일은 사람이 직접 다뤄야 합니다.\n"
        )
        return 2

    return 0

if __name__ == "__main__":
    sys.exit(main())
```

## 줄별 해설

- `PROTECTED = ("secrets/", ".env.production", "infra/prod/")`: 보호할 경로 조각을 상수로 모아 둡니다. 부분 문자열로 매칭하므로 `secrets/db.key·infra/prod/main.tf` 처럼 하위 파일도 함께 걸립니다.
- `log_attempt(...)`: 차단된 시도를 타임스탬프와 함께 `.claude/guard.log`에 남깁니다. **막는 것만큼 막은 사실을 기록하는 것이 중요합니다**(11.8). 나중에 "왜 이 작업이 막혔지?"를 데이터로 확인할 수 있습니다.
- `event.get("tool_input", {}).get("file_path", "")`: `PreToolUse` 시점이라 *아직 편집되지 않은* 대상 경로를 미리 꺼냅니다. 그래서 실행 전에 막을 수 있습니다.
- `if any(token in file_path for token in PROTECTED)`: 대상 경로가 보호 조각 중 하나라도 포함하면 위험으로 판정합니다.
- `sys.stderr.write(...)`: 차단 사유를 표준 오류로 씁니다. 종료 코드 2와 함께 이 메시지가 *에이전트에게 피드백*되어, 에이전트는 이유를 알고 다른 방법(예: 사람에게 요청)을 택합니다.
- `return 2`: 핵심입니다. `PreToolUse`에서 2를 반환하면 그 편집 도구가 호출이 **실행되기 전에 차단**됩니다.

- 보호 경로가 아니면 `return 0` 으로 정상 통과시킵니다. 이 가드는 보호 경로에만 반응하고 나머지는 방해하지 않습니다.

`.claude/settings.json` 에 `PreToolUse`로 등록합니다.

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Write|Edit",
        "hooks": [
          { "type": "command", "command": "python3 .claude/hooks/guard_path.py" }
        ]
      }
    ]
  }
}
```

이제 에이전트가 보호 경로를 수정하려는 순간을 봅니다.

실행 결과 (세션 로그 발췌)

```
> .env.production 의 DATABASE_URL 을 새 운영 DB 주소로 바꿔 줘
[PreToolUse 훅] python3 .claude/hooks/guard_path.py
Edit blocked: 보호 경로는 수정할 수 없습니다: .env.production.
이 파일은 사람이 직접 다뤄야 합니다.
> (에이전트) .env.production 은 보호되어 직접 수정할 수 없습니다.
대신 변경할 키 이름과 값을 알려 주시면 적용 절차(시크릿 매니저 경유)를 안내하겠습니다.
```

그리고 감사 로그 파일에는 다음 한 줄이 남습니다.

```
.claude/guard.log
2026-06-21T14:03:11 BLOCKED edit: .env.production
```

CLAUDE.md에 "운영 설정을 건드리지 마"라고 적어 두는 것과 달리, 이 훅은 **선의를 기대지 않습니다**. 모델이 규칙을 잊거나 다르게 해석할 여지 없이, 코드가 항상 같은 자리에서 막습니다. 그리고 누가 무엇을 막았는지 감사 로그로 남아 사후 추적이 됩니다.

## 11.8 안전 설계: fail-open 대 fail-closed와 로깅

혹은 강력해서 잘못 설계하면 오히려 작업을 마비시킵니다. 두 가지 설계 철학을 구분해야 합니다.

- **fail-open(열린 실패):** 훅 자체에 오류가 나면(입력 파싱 실패, 의존 도구 없음 등) **작업을 막지 않고 통과**시킵니다. 가드의 버그가 정상 작업까지 멈추게 하지 않으려는 설계입니다. 앞의 두 예제에서 `json.JSONDecodeError` 를 만나면 `return 0` 으로 둔 것이 fail-open입니다.
- **fail-closed(닫힌 실패):** 훅이 확신을 못 하면 **안전하게 차단**합니다. 금융·의료처럼 한 번의 누락이 치명적인 영역에서 씁니다. 대신 가드의 사소한 오류로 전체 작업이 멈출 수 있으니, 가드 자체의 신뢰성을 매우 높게 유지해야 합니다.

대부분의 개발 워크플로우에서는 **fail-open**을 기본으로 하되, 명확히 위험한 조건(보호 경로 매칭 등)만 단호히 차단하는 혼합이 실무적입니다. 즉 "가드가 고장 나서 막는" 일은 없게 하고, "위험이 분명할 때만 막는" 것입니다.

또 하나의 축은 **로깅**입니다. 차단이든 통과든, 게이트가 무슨 판단을 했는지 흔적을 남겨야 합니다. 로그가 없으면 "왜 이 작업이 막혔는지", "이 가드가 과잉 차단을 하는지"를 알 수 없습니다. 이 프로젝트의 셀 위생 가드도 같은 원칙으로, 차단 시 사유와 명령을 별도 로그에 적어 데이터로 튜닝합니다. 게이트는 "막기만" 하면 안 되고 "관측 가능"해야 합니다.

**설계 점검 3문.** 새 혹을 만들 때 스스로 물어보세요. (1) 이 혹이 고장 나면 fail-open인가 fail-closed인가, 그 선택이 의도된 것인가? (2) 차단·통과 판단을 로그로 남기는가? (3) 차단 메시지가 에이전트(또는 사람)에게 "다음에 무엇을 하라"를 알려 주는가? 세 질문에 답할 수 있으면 안전한 게이트입니다.

## 11.9 즉시 연습 (2종 1세트)

지금까지 배운 PreToolUse 차단 혹과 fail-open 설계를 직접 손으로 완성해 봅니다. 각 연습은 ① 모범 골격을 참고해 ② 빈칸을 채우고 ③ 자가체크로 검증하는 2종 1세트입니다.

### 연습 1 — 대용량 추가 차단 혹

**과제.** 에이전트가 한 번에 너무 큰 내용을 새 파일로 쓰려 할 때(예: 50KB 초과) PreToolUse에서 차단하고 로그를 남기는 혹을 완성하세요. 실무에서는 에이전트가 마이그레이션 SQL 덤프, 벤더 번들, 대형 픽스처 같은 **자동 생성 대용량 파일**을 레포에 무심코 써넣어 리뷰와 git 이력을 오염시키는 일이 잦습니다. 이런 파일은 레포가 아니라 별도 저장소·아티팩트에 뒤야 하므로, 임계 크기를 넘는 신규 작성을 게이트로 막습니다.

빈칸 \_\_①\_\_ ~ \_\_④\_\_ 를 채워 넣으세요.

```
#!/usr/bin/env python3
"""새 파일 내용이 임계 크기를 넘으면 막는 PreToolUse 훅."""
import json
import sys
from datetime import datetime
from pathlib import Path

MAX_BYTES = 50 * 1024 # 50KB
LOG_PATH = Path(".claude/guard.log")

def main() -> int:
    try:
        event = json.loads(sys.stdin.read())
    except json.JSONDecodeError:
        # 가드 오류로 작업을 막지 않습니다.
        return __①__ # fail-open 종료 코드

    tool_input = event.get("tool_input", {})
    content = tool_input.get("content", "")
    size = len(content.encode("utf-8"))

    if size > __②__: # 임계 비교 대상
        stamp = datetime.now().isoformat(timespec="seconds")
        with LOG_PATH.open("a", encoding="utf-8") as fp:
            fp.write(f"{stamp} BLOCKED big write: {size} bytes\n")
        sys.stderr.write(f"{size} 바이트는 한 번에 쓰기엔 큼니다.\n")
        return __③__ # 차단 종료 코드

    return __④__ # 정상 통과 종료 코드

if __name__ == "__main__":
    sys.exit(main())
```

그리고 이 훅을 Write 도구에만 붙이는 settings.json 조각도 작성하세요.

### 자가체크

- [ ] \_\_①\_\_ 과 \_\_④\_\_ 는 둘 다 정상/통과를 뜻하는 같은 코드인가요?
- [ ] \_\_③\_\_ 은 PreToolUse에서 차단을 일으키는 코드인가요?
- [ ] \_\_②\_\_ 에는 MAX\_BYTES 가 들어가 임계와 비교하나요?
- [ ] settings.json의 매처가 Write 로 되어 새 파일 작성에만 반응하나요?
- [ ] 차단 시 로그가 남고, 가드 자체 오류는 fail-open으로 통과하나요?

## 연습 2 — 위험 명령 차단 훅(Bash)

**과제.** 운영 사고로 직결되는 파괴적 셸 명령(운영 데이터 삭제·강제 푸시·운영 네임스페이스 삭제)을 에이전트가 Bash로 실행하려 할 때 막는 PreToolUse 훅을 완성하세요. 매처는 Bash 이고, 실제 명령 문자열은 tool\_input.command 에 들어 있습니다(11.6의 주의 참고).

빈칸 \_\_㉠\_\_ ~ \_\_㉢\_\_ 를 채우세요.

```
#!/usr/bin/env python3
"""파괴적 셸 명령을 막는 PreToolUse 훅."""
import json
import sys

# 운영 사고로 직결되는 명령 패턴(부분 문자열 매칭).
DANGEROUS = ("rm -rf", "git push --force", "DROP TABLE", "kubectl delete namespace prod")

def main() -> int:
    try:
        event = json.loads(sys.stdin.read())
    except json.JSONDecodeError:
        return 0

    # Bash 도구의 명령 문자열을 꺼냅니다.
    command = event.get("tool_input", {}).get("__@__", "")

    if any(bad in command for bad in _@_):
        sys.stderr.write(f"위험 명령은 차단됩니다: {command}\n")
        return _@_

    return 0

if __name__ == "__main__":
    sys.exit(main())
```

## 자가체크

- [ ] \_\_@\_\_ 는 Bash 도구 입력에서 명령 문자열을 가리키는 키(command)인가요?
- [ ] \_@\_ 는 위험 패턴 목록(DANGEROUS)인가요?
- [ ] \_@\_ 는 차단 코드(2)인가요?
- [ ] 위험 패턴이 아니면 정상 통과(0)하나요?
- [ ] 매처를 Bash 로 등록해 파일 편집 훅과 분리했나요?

## 11.10 흔한 오류와 오개념

**오개념 1: "혹도 결국 AI가 판단하는 것 아닌가?"** 아닙니다. 혹은 우리가 작성한 코드입니다. LLM의 확률적 판단과 달리, 등록된 이벤트가 발생하면 항상 같은 로직으로 실행됩니다. "거의 항상"이 아니라 "항상"이 핵심 차이입니다. 보호 경로 차단을 CLAUDE.md 규칙이 아니라 혹은으로 두는 이유가 바로 이 결정론입니다.

**오개념 2: "게이트는 뽀뽀할수록 안전하다."** 과잉 차단은 또 다른 사고입니다. 너무 넓은 매처나 과도한 차단 조건은 정상 작업까지 막아, 사용자가 결국 가드를 통째로 꺼 버리게 만듭니다(가드가 없는 것보다 나쁩니다). 차단 조건은 명확히 위험한 것에 한정하고, fail-open을 기본으로 두며, 로깅으로 과잉 차단 여부를 관측해 데이터로 좁히거나 넓히세요.

**오개념 3: "혹과 권한은 같은 것이다."** 다릅니다. 권한(4장)은 도구 호출을 allow·deny·ask로 판정만 합니다. 혹은 그 시점에 임의의 로직(포맷·테스트·로깅·조건부 차단)을 실행합니다. "이 명령을 허용할까?"는 권한, "편집 후 포맷하고 위험 경로는 막고 로그를 남겨라"는 혹은입니다. 둘은 함께 써서 권한으로 큰 틀을 정하고, 혹은으로 세밀한 절차와 강제를 더합니다.

**오개념 4: "PostToolUse 훅으로 위험한 편집을 막을 수 있다."** PostToolUse는 도구가 *이미* 실행된 뒤입니다. 그 시점의 차단은 편집을 되돌리지 못하고 후속 알림에 그칩니다. *실행 전에 막아야* 한다면 반드시 **PreToolUse**를 써야 합니다. "막기"는 Pre, "사후 반응"은 Post로 구분하세요.

---

## 마무리

---

이 장에서 우리는 단원 6의 첫 개념인 **훅(자동화 게이트)** 을 배웠습니다. 훅이 무엇이고(생애주기 이벤트에 실행되는 결정론적 스크립트), 어떻게 동작하며(이벤트→종료 코드→차단/통과), 자동 포맷 PostToolUse와 보호 경로 차단 PreToolUse를 직접 만들고, fail-open·로깅으로 안전하게 설계하는 법까지 익혔습니다.

다음 장에서는 여러 서브에이전트(9장)와 커맨드(8장)를 하나의 워크플로우로 엮는 **오케스트레이션**을 배웁니다. 훅이 "각 행동을 항상 안전하게" 만든다면, 오케스트레이션은 "여러 전문가의 작업을 신뢰성 있게 통합"합니다. 둘은 마지막 장(CI·자동화)에서 무인 자동화로 합쳐집니다.



## 12장 역기: 멀티에이전트 워크플로우 오케스트레이션

이 장은 단원 6의 두 번째 챕터입니다. 여기서는 개념 **워크플로우**와 **멀티에이전트 오케스트레이션(multi-agent orchestration)** 하나를 깊이 배웁니다. 한 명의 에이전트로 버거운 큰 작업을, 여러 전문 에이전트의 협업으로 나눠 정복하는 설계입니다.

### 이 장의 학습 목표

- 여러 서브에이전트-커맨드를 하나의 워크플로우로 설계할 수 있습니다.
- 오케스트레이터-전문가 분업(계획·분배·통합)을 구성할 수 있습니다.
- 병렬·순차 단계를 조합한 파이프라인을 만들 수 있습니다.
- 워크플로우의 실패·통합 지점을 통제할 수 있습니다.

### 12.1 도입: 단일 에이전트로 어려운 일 나눠 정복하기

9장에서 우리는 **서브에이전트**로 경계가 분명한 한 가지 일(리뷰·테스트·조사)을 위임하는 법을 배웠습니다. 8장에서는 반복 절차를 **슬래시 커맨드**로 표준화했습니다. 이 둘을 하나씩 쓰는 것만으로도 생산성은 옵니다. 하지만 실무의 큰 작업은 "조사하고, 설계하고, 구현하고, 리뷰하고, 통합한다"처럼 여러 단계가 사슬로 이어집니다. 이런 작업을 한 세션이 처음부터 끝까지 혼자 하면 두 가지 문제가 생깁니다.

첫째, **컨텍스트 오염**입니다. 조사하며 읽은 수십 개 파일, 시도하다 버린 코드, 테스트 로그가 한 세션의 컨텍스트에 쌓이면 정작 중요한 결정에 쓸 자리가 줄고 품질이 떨어집니다. 둘째, **순차 병목**입니다. 서로 독립적인 세 모듈을 조사하는 일을 한 세션이 하나씩 차례로 하면 느립니다. 동시에 할 수 있는 일을 직렬로 처리하는 셈입니다.

선수 개념 한 줄 상기: 9장의 서브에이전트는 **독립 컨텍스트**로 한 가지 일을 격리해 처리했고, 8장의 슬래시 커맨드는 **반복 절차**를 /이름으로 표준화했습니다. 이번 장은 이 둘을 **여러 단계로 엮어** 하나의 신뢰할 수 있는 워크플로우로 만듭니다.

오케스트레이션은 이 두 문제를 "분업"으로 푼다. 메인 에이전트가 전체를 지휘하며 일을 쪼개 전문가들에게 나눠 주고, 각 전문가는 자기 컨텍스트에서 자기 일만 깔끔히 하고 결과만 돌려줍니다. 메인은 그 결과들을 모아 통합합니다. 이 장을 마치면 여러분은 "이건 한 세션이 다 하기엔 크다"고 느껴지는 작업을, 신뢰성 있는 파이프라인으로 설계할 수 있습니다.

## 12.2 정의: 오케스트레이션이란 다중 에이전트·도구 워크플로우

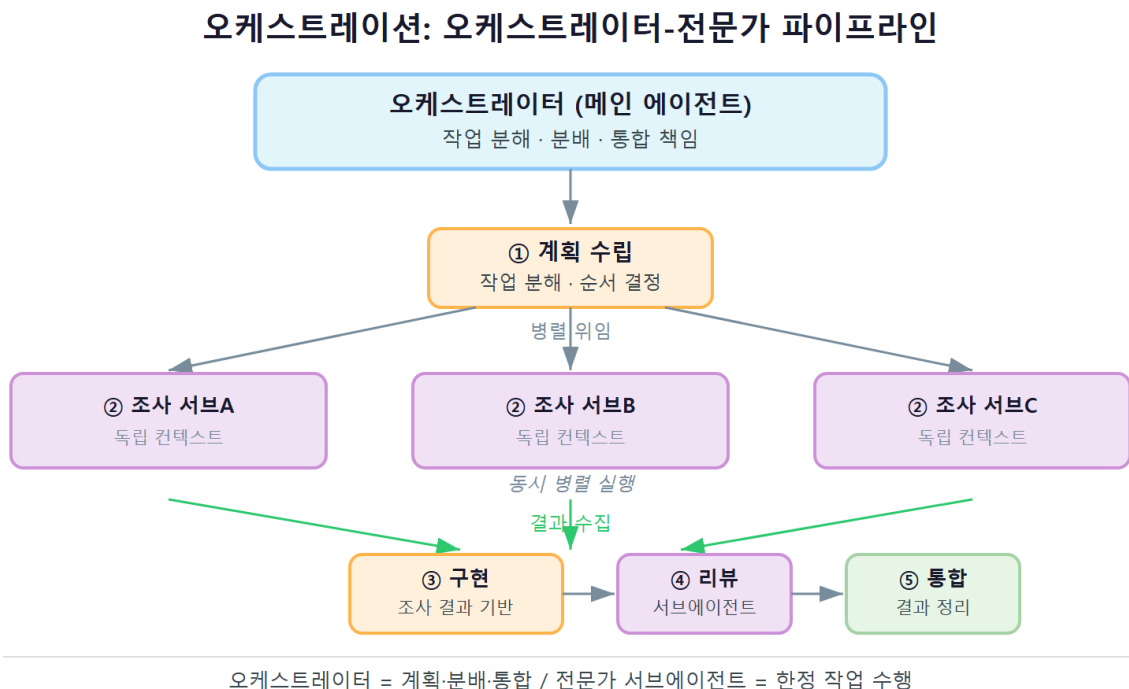
**정의.** 오케스트레이션(orchestration)이란 여러 서브에이전트·커맨드·도구를 하나의 워크플로우(workflow)로 엮어, 복잡한 작업을 단계적으로 수행하는 것입니다. 메인 에이전트(오케스트레이터, orchestrator)가 작업을 분해해 전문 서브에이전트에 분배하고, 각자의 결과를 모아 통합하며, 그 절차를 슬래시 커맨드로 표준화합니다.

핵심 용어를 한글(영문)으로 정리합니다.

- **오케스트레이터(orchestrator):** 전체 워크플로우를 지휘하는 메인 에이전트. 직접 코드를 다 짜기보다 **계획·분배·통합**에 집중합니다. 오케스트라의 지휘자가 직접 악기를 연주하지 않는 것과 같습니다.
- **전문가(specialist) 서브에이전트:** 조사·구현·리뷰처럼 한정된 역할을 맡는 서브에이전트. 각자 독립 컨텍스트를 가집니다.
- **파이프라인(pipeline):** 단계들이 순서대로(순차) 또는 동시에(병렬) 연결된 작업 흐름. 예: 계획 → 병렬 조사 → 구현 → 리뷰 → 통합.
- **통합(integration):** 여러 전문가가 따로 만든 결과를 일관성 있게 합치는 단계. 오케스트레이션에서 가장 중요하고 누락하기 쉬운 책임입니다.

비유하면, 단일 에이전트는 "혼자 다 하는 1인 개발자"이고, 오케스트레이션은 "기획·구현·리뷰를 나눠 맡고 리드가 통합하는 팀"입니다. 작은 일은 1인이 빠르지만, 큰 일은 잘 조직된 팀이 더 신뢰성 있게 해냅니다.

## 12.3 동작 원리: 오케스트레이터-전문가 파이프라인



오케스트레이션이 어떻게 도는지 한 사이클을 단계로 따라가 봅니다. 전형적인 "계획 → 병렬 조사 → 구현 → 리뷰 → 통합" 파이프라인을 예로 듭니다.

1. **계획(분해)**. 오케스트레이터가 큰 작업을 *서로 독립적인 하위 작업*으로 쪼갭니다. 좋은 분해의 기준은 "각 조각이 다른 조각의 중간 결과를 몰라도 시작할 수 있는가"입니다. 독립적일수록 병렬화가 쉽습니다.
2. **분배(병렬)**. 독립적인 조사 작업들을 여러 전문가 서브에이전트에 *동시에* 맡깁니다. 각 서브에이전트는 자기 컨텍스트에서 자기 부분만 조사하고 *요약된 결과*만 돌려줍니다. 메인 컨텍스트는 수십 개 파일 원문으로 더럽혀지지 않습니다.
3. **수집·통합(병렬→순차 경계)**. 오케스트레이터가 각 전문가의 요약 را 받아 하나의 일관된 그림으로 통합합니다. 충돌(서로 다른 권고)이 있으면 여기서 조정합니다. 이 통합 결과가 다음 단계의 입력이 됩니다.
4. **구현(순차)**. 통합된 계획을 바탕으로 실제 변경을 만듭니다. 구현은 보통 순차입니다. 같은 코드베이스를 동시에 여러 에이전트가 고치면 충돌하기 때문입니다.
5. **리뷰(전문가)**. 구현 결과를 리뷰 전문 서브에이전트(9장의 reviewer)에 넘겨 독립적으로 점검합니다. 리뷰어는 구현 맥락에 몰들지 않은 새 눈으로 봅니다.
6. **최종 통합·정리**. 오케스트레이터가 리뷰 지적을 반영하고, 전체가 일관되게 맞물리는지 확인한 뒤 마무리합니다.

여기서 두 가지 흐름 패턴을 구분해야 합니다.

| 패턴             | 언제 쓰는가                 | 예                     |
|----------------|------------------------|-----------------------|
| 병렬(parallel)   | 하위 작업이 서로 독립적일 때       | 모듈 A·B·C를 동시에 조사      |
| 순차(sequential) | 한 단계의 출력이 다음 단계의 입력일 때 | 조사 결과로 구현 → 구현 결과로 리뷰 |

실전 파이프라인은 둘을 섞습니다. "병렬 조사 → (통합) → 순차 구현 → 리뷰"처럼, 독립적인 구간은 병렬로 빠르게, 의존하는 구간은 순차로 안전하게 잇습니다.

**기억하세요.** 오케스트레이션의 두 축은 (1) **분업** — 오케스트레이터는 분배·통합, 전문가는 한정 작업, (2) **흐름 설계** — 독립이면 병렬, 의존이면 순차. 이 둘만 잡으면 어떤 워크플로우든 설계의 출발점이 됩니다.

## 12.4 worked example: 조사→구현→리뷰 파이프라인 구성

말로만 설명하면 추상적이니, 실제 팀 작업을 파이프라인 절차로 적어 봅니다. 목표는 "결제 서비스(payment-service)에 외부 PG사 결제 호출이 실패하면 지수 백오프로 재시도하는 기능을 추가"하는 작업을, 조사·구현·리뷰 세 단계로 오케스트레이션하는 것입니다. 결제는 한 번 잘못 호출하면 이중 청구로 이어질 수 있어, 기존 호출 패턴과 멍등성(idempotency) 규약을 먼저 조사하지 않으면 위험한 변경입니다.

먼저 전문가 서브에이전트가 9장에서 정의돼 있다고 가정합니다(.claude/agents/ 에 `researcher.md`, `reviewer.md`). 오케스트레이터(메인 세션)에게 줄 지시를 단계로 구성합니다.

```
# 오케스트레이터에게 주는 지시 (메인 세션)
다음 작업을 조사 -> 구현 -> 리뷰 파이프라인으로 진행해 줘.
작업: payment-service 의 PG사 결제 호출에 실패 시 지수 백오프 재시도를 추가한다.

1) [조사·병렬] researcher 서브에이전트에게 두 가지를 동시에 조사시켜 줘.
- (a) 기존 payment_client.py 의 HTTP 호출·예외 처리·타임아웃 패턴
- (b) 사내 표준 재시도 유틸(common/retry.py)의 사용 규약과 멍등성 키 정책
각 결과는 요약만 받아 와.

2) [통합] 두 조사 요약을 합쳐, 재시도를 어디에 어떤 정책(횟수·백오프·
멍등성 키)으로 끼워 넣을지 한 문단 명세로 정리해 줘.

3) [구현·순차] 정리된 명세대로 payment_client.py 에 재시도를 적용하고,
tests/ 에 실패-재시도 시나리오 단위 테스트를 추가해 줘. 직접 구현은 메인이 한다.

4) [리뷰] reviewer 서브에이전트에게 구현 diff 를 넘겨 독립 리뷰를 받아 줘.
특히 "재시도가 이중 청구를 유발하지 않는가"를 집중 점검하게 하고,
지적이 있으면 반영한 뒤 마무리해 줘.
```

## 단계별 해설

- **1) 조사·병렬:** 두 조사 (a)·(b)는 서로의 결과를 몰라도 시작할 수 있는 독립 작업이므로 병렬로 묶었습니다. 각 결과를 요약만 받는 것이 핵심입니다. 호출부 코드 원문을 다 받으면 메인 컨텍스트가 오염됩니다(9장의 컨텍스트 격리 이점).
- **2) 통합:** 오케스트레이터의 핵심 책임입니다. 호출 패턴 조사와 재시도 규약 조사를 하나의 명세(어디에·몇번·어떤 멍등성 키로 재시도할지)로 합칩니다. 이 단계가 없으면 구현이 일관성을 잃습니다.
- **3) 구현·순차:** 통합 명세에 의존하므로 조사·통합 뒤에 옵니다. 결제 경로를 동시에 여러 에이전트가 고치면 충돌하므로 구현은 메인이 순차로 합니다.
- **4) 리뷰:** 구현 결과에 의존하므로 마지막입니다. 리뷰는 구현 맥락에 몰들지 않은 reviewer에게 맡겨 "재시도가 이중 청구를 부르지 않는가" 같은 위험을 객관적으로 잡습니다. 지적 반영까지가 한 사이클입니다.

이 지시를 받은 메인 세션의 동작을 로그로 보면 다음과 같습니다.

```
실행 결과 (세션 로그 발췌)
> [위 파이프라인 지시]
Task(researcher): 조사 (a) payment_client.py 호출 패턴 ... 동시 실행
Task(researcher): 조사 (b) common/retry.py 규약·멍등성 ... 동시 실행
- (a) 요약: charge() 가 requests.post 직접 호출, 타임아웃 5s,
  예외는 상위로 그대로 전파
- (b) 요약: retry_with_backoff(max=3, base=0.5s) 데코레이터 제공,
  멍등성 키는 X-Idempotency-Key 헤더로 전달이 규약
[통합] charge() 호출을 retry_with_backoff 로 감싸되, 첫 시도에서
  생성한 멍등성 키를 모든 재시도에서 재사용(이중 청구 방지)
Edit payment_client.py (+22 lines) Write tests/test_charge_retry.py
Task(reviewer): diff 독립 리뷰 ...
- 리뷰 지적: 4xx(클라이언트 오류)는 재시도 대상에서 제외할 것 -> 반영 완료
All done. 지수 백오프 재시도 추가, 테스트 4건 통과.
```

한 세션이 모든 것을 혼자 했다면, 조사 원문이 컨텍스트를 채우고 리뷰도 "자기가 짠 코드를 자기가 보는" 한계에 빠졌을 것입니다. 오케스트레이션은 각 단계를 적임자에게 맡겨 품질과 속도를 함께 끌어올립니다.

## 파이프라인 산출물을 검증하는 파이썬 예제

위 파이프라인에서 통합(2단계) 직전에 오케스트레이터가 반드시 해야 할 일이 하나 있습니다. 바로 **각 조사 결과가 통합에 쓸 수 있는 형태인지 검증**하는 것입니다. researcher가 돌려준 요약이 필수 항목(어떤 모듈을 봤는지, 결론이 무엇인지, 근거가 있는지)을 빠뜨리면, 그 위에서 만든 명세도 부실해집니다. 이 검증을 표준 라이브러리만으로 작은 파이썬 함수로 만들어 보겠습니다. 각 조사 결과를 딕셔너리로 표현하고, 통합 가능 여부를 판정합니다.

```
# pipeline_validate.py — 조사 단계 산출물을 통합 전에 검증한다
from typing import TypedDict

class Finding(TypedDict):
    # researcher 가 돌려주는 조사 요약 한 건의 구조
    source: str      # 조사 대상(예: "payment_client.py")
    conclusion: str   # 한 줄 결론
    evidence: str     # 결론의 근거

# 통합에 쓰려면 반드시 채워져 있어야 하는 항목
REQUIRED_FIELDS = ("source", "conclusion", "evidence")

def validate_finding(finding: dict) -> list[str]:
    """조사 결과 한 건을 검사해 누락·빈 항목의 사유 목록을 돌려준다."""
    problems: list[str] = []
    for field in REQUIRED_FIELDS:
        value = finding.get(field, "")
        if not value or not str(value).strip():
            problems.append(f"'{field}' 누락 또는 비어 있음")
    return problems

def ready_for_integration(findings: list[dict]) -> bool:
    """모든 조사 결과가 통합 가능한 형태인지 판정하고 사유를 출력한다."""
    all_ok = True
    for finding in findings:
        problems = validate_finding(finding)
        source = finding.get("source", "(미상)")
        if problems:
            all_ok = False
            print(f"[보류] {source}: {' '.join(problems)}")
        else:
            print(f"[통과] {source}: {finding['conclusion']}")
    return all_ok

if __name__ == "__main__":
    # 병렬 조사 3건이 돌아왔다고 가정한다(한 건은 근거가 비어 있음)
    findings = [
        {"source": "payment_client.py", "conclusion": "charge()는 requests.post 직접 호출, 타임아웃 5s",
         "evidence": "결제 호출부 3곳이 모두 동일 패턴"},
        {"source": "common/retry.py", "conclusion": "retry_with_backoff(max,base) 데코레이터 제공",
         "evidence": "사내 표준 유틸 문서·기존 5개 서비스 사용"},
        {"source": "명등성 정책", "conclusion": "재시도 시 동일 명등성 키 재사용 필요",
         "evidence": ""}, # 근거 누락 — 통합 보류 대상
    ]
    if ready_for_integration(findings):
        print("> 모든 조사 통과: 통합 단계로 진행")
    else:
        print("> 일부 보류: 보완 후 통합")
```

## 줄별 해설

- `class Finding(TypedDict)`: researcher가 돌려주는 조사 요약 한 건의 구조를 타입으로 못 박습니다. `source` (조사 대상)·`conclusion` (결론)·`evidence` (근거) 세 항목이 통합의 최소 단위입니다.
- `REQUIRED_FIELDS`: 통합에 들어가려면 반드시 채워져 있어야 하는 항목입니다. 이 목록이 곧 "통합 게이트"의 입력 조건입니다.
- `validate_finding(...)`: 한 건을 검사해 *누락·빈 항목의 사유*를 목록으로 돌려줍니다. `str(value).strip()`으로 공백만 든 값도 빈 것으로 처리합니다.
- `ready_for_integration(...)`: 세 조사 결과를 모두 검사해 하나라도 부실하면 `False`를 돌려줍니다. 오케스트레이터는 이 판정을 통합 진입의 *게이트*로 씁니다.
- `__main__` 블록: 병렬 조사 3건이 돌아온 상황을 흉내 냅니다. 멍등성 정책 결과는 근거(`evidence`)가 비어 있어 보류됩니다 — 부실한 조사 위에 결재 변경 명세를 쌓지 않으려는 것입니다.

### 실행 결과

[통과] `payment_client.py`: `charge()`는 `requests.post` 직접 호출, 타임아웃 5s  
 [통과] `common/retry.py`: `retry_with_backoff(max,base)` 데코레이터 제공  
 [보류] 멍등성 정책: 'evidence' 누락 또는 비어 있음  
 => 일부 보류: 보완 후 통합

이처럼 통합 직전에 산출물을 *기계적으로 검증*하면, 오케스트레이터의 통합이 부실한 입력 위에서 흔들리지 않습니다. 사람이 눈으로 일일이 확인하던 "이 조사 결과 쓸 만한가?"를 작은 파이썬 함수가 일관되게 판정해 주는 것입니다. 이 검증은 뒤(12.8)에서 다룰 *통합 게이트*의 코드 버전이기도 합니다.

## 12.5 에이전트 팀: 계획·분배·통합 분업

오케스트레이션을 "여러 에이전트를 부르는 것"으로만 이해하면 절반만 본 것입니다. 진짜 핵심은 **역할 분담**입니다. 잘 조직된 에이전트 팀의 역할을 표로 정리합니다.

| 역할      | 누가                | 책임             | 하지 말아야 할 것     |
|---------|-------------------|----------------|----------------|
| 오케스트레이터 | 메인 세션             | 분해·분배·통합·최종 판단 | 모든 구현을 직접 다 하기 |
| 조사 전문가  | researcher 서브에이전트 | 한정된 범위 조사·요약   | 코드 수정(읽기만)     |
| 구현 전문가  | 메인 또는 구현 서브에이전트   | 통합 명세대로 변경     | 명세 없이 즉흥 구현    |
| 리뷰 전문가  | reviewer 서브에이전트   | 독립적 품질·안전 점검   | 직접 고치기(지적만)    |

이 분업에서 가장 중요한 통찰은 **오케스트레이터는 직접 다 하지 않는다**는 것입니다. 지휘자가 모든 악기를 연주하려 들면 컨텍스트가 폭발하고 병렬화의 이점이 사라집니다. 오케스트레이터의 가치는 "무엇을 누구에게 맡기고, 그 결과를 어떻게 합칠지"를 정하는 데 있습니다.

또 하나, **서브에이전트 간 맥락은 자동으로 공유되지 않습니다**(9장). researcher가 알아낸 것을 reviewer가 자동으로 알지 못합니다. 그래서 통합은 *오케스트레이터를 거쳐야만* 일어납니다. 이 사실을 잊으면 "왜 리뷰어가 조

사 결과를 모르지?" 같은 혼란에 빠집니다. 필요한 맥락은 오케스트레이터가 명시적으로 다음 단계에 전달해야 합니다.

이 "맥락 전달"을 어떻게 설계하느냐가 통합 품질을 좌우합니다. 핵심은 **다음 단계에 필요한 최소 맥락만 추려 넘기는 것**입니다. 조사 원문을 통째로 다음 단계에 던지면 컨텍스트가 다시 오염돼 서브에이전트를 쓴 의미가 사라집니다. 반대로 너무 적게 넘기면 다음 단계가 잘못된 가정으로 진행합니다. 단계별로 무엇을 넘길지 표로 정리해 두면 설계가 명확해집니다.

| 단계 전이   | 넘겨야 할 최소 맥락                    | 넘기지 말 것           |
|---------|--------------------------------|-------------------|
| 조사 → 통합 | 각 조사의 <i>요약 결론</i> 과 근거 한두 줄   | 읽은 파일 전문, 시행착오 로그 |
| 통합 → 구현 | 확정된 함수 시그니처·반환 형식 <i>명세</i>    | 통합에 이르기까지의 논의 전체  |
| 구현 → 리뷰 | 변경 <i>diff</i> 와 리뷰 관점(버그·가독성) | 구현 중 버린 대안들       |

이렇게 단계마다 "무엇을 넘기고 무엇을 버릴지"를 정하면, 각 전문가가 깨끗한 컨텍스트에서 자기 일에 집중하면서도 워크플로우 전체의 일관성이 유지됩니다. 오케스트레이터의 통합 책임은 결국 "결과를 합치는 일"이자 "다음 단계로 *적절한 맥락만* 전달하는 일"입니다.

## 12.6 커맨드로 워크플로우 절차 표준화

같은 파이프라인을 매번 길게 지시하는 것은 비효율적이고 사람마다 달라집니다. 8장에서 배운 **슬래시 커맨드**로 이 절차를 못 박으면, 팀 누구나 `/feature` 나 `/audit` 한 줄로 같은 워크플로우를 돌릴 수 있습니다. 오케스트레이션 절차를 표준화하는 것입니다.

`.claude/commands/feature.md` 로 "조사→구현→리뷰" 파이프라인을 표준 커맨드로 만듭니다.

```
---
description: 새 기능을 조사->구현->리뷰 파이프라인으로 추가한다
---

다음 기능을 조사 -> 구현 -> 리뷰 순서로 진행하세요: $ARGUMENTS

1. [조사·병렬] researcher 서브에이전트에게 관련 기존 코드의 패턴과
   필요 한 표준 라이브러리 방법을 각각 조사시키고, 요약만 받으세요.
2. [통합] 조사 요약을 합쳐 함수 시그니처와 반환 형식을 명세로 정리하세요.
3. [구현] 명세대로 구현하고 대응되는 단위 테스트를 추가하세요.
4. [리뷰] reviewer 서브에이전트에게 diff 독립 리뷰를 받아 지적을 반영하세요.
5. 테스트가 모두 통과하면 마무리하고, 변경 요약을 보고하세요.
```

### 해설

- `description`: `/feature` 를 호출할 때 표시되는 한 줄 설명입니다. 팀원이 무엇을 하는 커맨드인지 바로 압니다.
- `$ARGUMENTS`: 8장에서 배운 인자 자리입니다. `/feature` 주문 취소 시 재고 원복 이벤트 발행 추가 처럼 부르면 그 기능 설명이 이 자리에 주입됩니다.

- 본문 1~5단계: 12.4에서 손으로 적던 파이프라인을 **템플릿**으로 고정했습니다. 병렬 조사·통합·순차 구현·독립 리뷰라는 구조가 매번 동일하게 재현됩니다.
- 이 커맨드 하나로 팀의 "기능 추가 방식"이 표준화됩니다. 사람마다 빠뜨리던 리뷰 단계가 절차에 박혀 항상 수행됩니다.

이제 사용은 한 줄입니다.

```
실행 결과 (세션 로그 발췌)
> /feature 주문 취소 시 재고 원복 이벤트 발행 추가
[조사·병렬] researcher x2 ...
[통합] 명세 정리 ...
[구현] order_service + 이벤트 발행 + tests ...
[리뷰] reviewer ...
테스트 5건 통과. 재고 원복 이벤트 발행 추가 완료.
```

커맨드(절차 표준화)·서브에이전트(전문 위임)·오케스트레이터(통합)가 하나로 맞물립니다. 이 셋의 결합이 멀티 에이전트 오케스트레이션의 실전 형태입니다. 팀에서는 이 `/feature` 커맨드를 프로젝트 범위(`.claude/commands`)에 두어 모두가 같은 절차를 공유하고, 절차가 개선되면 커맨드 한 곳만 고치면 전원에게 반영됩니다.

## 12.7 worked example: 병렬 결과를 메인이 통합·정리

오케스트레이션에서 가장 자주 틀리는 지점이 **통합**입니다. 병렬로 잘 나눠 놓고도 결과를 제대로 합치지 못하면 일관성이 깨집니다. 이번 예제는 통합 단계에 초점을 둡니다. 분기 안정화 주간을 앞두고 프로덕션 핵심 3개 서비스를 병렬로 점검한 뒤, 메인이 충돌을 조정해 하나의 안정화 계획으로 합치는 과정입니다.

```
# 오케스트레이터 지시 (통합 강조)
프로덕션 핵심 3개 서비스(payment-service, order-service, inventory-service)를
각각 researcher 에 병렬로 맡겨 "지금 가장 시급한 안정성 위험 1순위"를 받아 줘.
세 요약은 받은 뒤, 다음 규칙으로 통합해 줘.
- 서로 충돌하는 권고가 있으면 "데이터 정합성·장애 전파 위험"이 큰 쪽을 택한다.
- 세 권고를 우선순위로 정렬해 단일 안정화 계획으로 제시한다.
- 계획만 내고, 실제 변경은 승인 후에 한다.
```

병렬 조사가 돌아오면 메인은 다음처럼 통합합니다.

**실행 결과 (세션 로그 발췌)**

```
Task(researcher): payment-service 조사 ... 동시
Task(researcher): order-service 조사 ... 동시
Task(researcher): inventory-service 조사 ... 동시
- payment 요약: 결제 콜백에 먹등성 키가 없어 중복 결제 가능(정합성)
- order 요약: 주문 생성이 결제·재고를 동기 호출해 한 곳 장애가 주문 전체로 전파(장애 전파)
- inventory 요약: 헬스체크 엔드포인트가 없어 롤링 배포 시 다운 감지 지연(가용성)
[통합·충돌 조정]
* payment(중복 결제)는 금전·정합성을 직접 해치므로 1순위
* order(장애 전파)는 가용성 연쇄 위험으로 2순위
* inventory(헬스체크)는 단일 가용성 문제로 3순위
[단일 안정화 계획]
1. payment 결제 콜백에 먹등성 키 검증 도입
2. order 의 결제·재고 호출을 비동기 이벤트로 분리(장애 격리)
3. inventory 헬스체크 엔드포인트 추가
계획만 제시했습니다. 진행을 승인해 주세요.
```

**해설**

- **병렬 조사**: 세 서비스는 서로 독립적이라 동시에 맡깁니다. 직렬이었다면 세 배 느렸을 것입니다.
- **충돌 조정 규칙을 미리 줌**: "충돌 시 데이터 정합성·장애 전파 위험을 우선한다"는 *판단 기준*을 오케스트레이터에게 미리 주었습니다. 이 기준이 없으면 통합이 자의적이 됩니다. 좋은 통합은 *명시적 기준*으로 충돌을 해소합니다.
- **단일 결론으로 수렴**: 세 갈래 요약을 우선순위가 매겨진 *하나의 안정화 계획*으로 합쳤습니다. "세 의견을 나열"하는 것은 통합이 아닙니다. *결정*까지 내려야 통합입니다.
- **계획만 제시(플랜 모드 정신)**: 6장의 계획·실행 분리를 오케스트레이션에도 적용해, 큰 변경 전에 사람의 승인을 받습니다.

**병렬 결과를 모아 통합하는 파이썬 예제**

위 통합은 메인 세션이 *대화 안에서* 머리로 합친 모습입니다. 하지만 서브에이전트가 많아지면, 각자 결과를 **JSON 파일로 떨어뜨리게 하고 메인이 그 파일들을 읽어 집계**하는 방식이 더 안정적입니다. 파일로 주고받으면 결과가 사라지지 않고, 통합 규칙(정렬·집계)을 코드로 못 박을 수 있기 때문입니다. 세 researcher가 각각 `findings/<서비스>.json` 을 남겼다고 가정하고, 메인이 그 파일들을 읽어 우선순위로 정렬된 단일 안정화 계획으로 통합하는 작은 스크립트를 만들어 보겠습니다. 표준 라이브러리 `json` · `pathlib` 만 씁니다.

```
# integrate_findings.py — 병렬 점검 결과(JSON 파일)들을 단일 안정화 계획으로 통합한다
import json
from pathlib import Path

# 충돌 조정 기준: 데이터 정합성 > 장애 전파 > 가용성 순으로 우선순위를 높인다
PRIORITY_WEIGHT = {"정합성": 3, "장애전파": 2, "가용성": 1}

def load_findings(findings_dir: Path) -> list[dict]:
    """디렉터리의 모든 *.json 조사 결과를 읽어 리스트로 모은다."""
    findings: list[dict] = []
    for path in sorted(findings_dir.glob("*.json")):
        # 파일이 깨졌으면 통합에서 제외하되 사실을 알린다
        try:
            findings.append(json.loads(path.read_text(encoding="utf-8")))
        except json.JSONDecodeError:
            print(f"[건너뛴] {path.name}: JSON 파싱 실패")
    return findings

def integrate(findings: list[dict]) -> list[dict]:
    """조정 기준(가중치)으로 정렬해 단일 안정화 계획을 만든다."""
    ranked = sorted(
        findings,
        key=lambda f: PRIORITY_WEIGHT.get(f["category"], 0),
        reverse=True, # 가중치가 큰 문제를 앞으로
    )
    # 1부터 매긴 순위를 붙여 단일 계획으로 정리한다
    return [
        {"rank": i, "source": f["source"], "action": f["action"]}
        for i, f in enumerate(ranked, start=1)
    ]

if __name__ == "__main__":
    findings = load_findings(Path("findings"))
    plan = integrate(findings)
    print("[단일 안정화 계획]")
    for step in plan:
        print(f" {step['rank']}. {step['source']}: {step['action']}")
```

## 줄별 해설

- `PRIORITY_WEIGHT`: 12.7에서 말로 주던 "충돌 시 데이터 정합성을 우선한다"는 조정 기준을 가중치 표로 코드화했습니다. 통합 규칙이 본문이 아니라 데이터에 박혀 매번 동일하게 적용됩니다.
- `load_findings(...)`: `pathlib`의 `glob("*.json")`으로 결과 파일을 정렬된 순서로 모읍니다. 깨진 파일은 `JSONDecodeError`로 잡아 건너뛰되 사실을 알립니다 — 한 파일이 전체 통합을 무너뜨리지 않습니다(부분 실패 격리).
- `integrate(...)`: `sorted`의 `key`로 가중치가 큰 문제를 앞에 둡니다. "세 의견 나열"이 아니라 우선순위가 매겨진 단일 계획으로 수렴시키는 것이 통합의 핵심입니다.
- `enumerate(ranked, start=1)`: 1부터 순위를 매겨 사람이 바로 실행할 수 있는 번호 매긴 계획으로 만듭니다.
- `__main__` 블록: `findings/` 디렉터리의 결과를 읽어 통합 계획을 출력합니다. 서브에이전트가 늘어도 메인 코드는 그대로입니다.

`findings/` 디렉터리에 세 결과 파일이 있다고 할 때 실행 결과는 다음과 같습니다.

**실행 결과****[단일 안정화 계획]**

1. payment-service: 결제 콜백에 역등성 키 검증 도입
2. order-service: 결제·재고 호출을 비동기 이벤트로 분리(장애 격리)
3. inventory-service: 헬스체크 엔드포인트 추가

대화 안에서 머리로 합치던 12.7의 통합이, 여기서는 *재현 가능한 코드*가 되었습니다. 조정 기준(가중치)을 바꾸면 우선순위가 일관되게 따라 바뀌고, 결과가 파일로 남아 추적과 검증이 쉬워집니다. 서브에이전트가 셋이든 열이든 통합 로직은 그대로라는 점이, 파일 기반 집계가 규모에 강한 이유입니다.

## 12.8 실패·통합 지점 통제

병렬·순차로 단계를 잘 엮었더라도, 어느 한 단계가 실패하면 전체 워크플로우가 흔들립니다. 신뢰할 수 있는 오케스트레이션은 *실패를 전제로* 설계합니다. 통제해야 할 두 지점을 봅니다.

첫째, **단계 실패의 격리**입니다. 병렬 조사 3건 중 1건이 실패했을 때, 오케스트레이터는 "전체를 중단"할지 "성공한 2건으로 진행하되 실패를 기록"할지 *미리 정해 두어야* 합니다. 기준이 없으면 매번 다르게 처리돼 결과가 들쭉날쭉해집니다.

둘째, **통합 게이트**입니다. 통합 단계는 단순히 합치는 곳이 아니라 *품질을 검증하는 관문*이기도 합니다. 합친 결과가 일관된지, 충돌이 해소됐는지, 다음 단계로 넘겨도 되는지를 여기서 판정합니다. 통합 게이트에서 막히면 구현으로 넘어가지 않습니다.

```
# 실패 처리 규칙을 명시한 오케스트레이터 지시
세 서비스를 researcher 에 병렬로 점검시켜 줘. 실패 처리는 다음 규칙을 따른다.
- 점검 1건이 실패하면: 중단하지 말고 성공한 결과로 진행하되,
  실패한 서비스는 "점검 미완"으로 통합 리포트에 표시한다.
- 통합 게이트: 성공 점검이 2건 미만이면 안정화 계획으로 넘어가지 말고
  사람에게 보고한다(신뢰할 만한 근거 부족).
- 충돌하는 권고는 "데이터 정합성 우선" 기준으로 해소한 뒤에만 다음 단계로 넘긴다.
```

**해설**

- **실패 격리 규칙:** 1건 실패가 전체를 무너뜨리지 않게 "성공분으로 진행 + 실패 표시"를 미리 정했습니다. 부분 실패에 강한 워크플로우가 됩니다.
- **통합 게이트의 임계:** "성공 2건 미만이면 사람에게 보고"라는 *판정 기준*을 두었습니다. 근거가 부족하면 다음 단계로 자동 진행하지 않습니다. 게이트가 품질을 지킵니다.
- **충돌 해소 기준 명시:** 통합이 자의적이 되지 않도록 "데이터 정합성 우선"이라는 기준을 줍니다. 이 기준이 다음 단계 진입 조건의 일부가 됩니다.

이렇게 *실패 처리·통합 게이트*를 규칙으로 미리 못 박으면, 워크플로우가 예외 상황에서도 예측 가능하게 동작합니다. 11장의 `SubagentStop`으로 위임 결과 검증)과 결합하면, 통합 게이트를 코드로 강제할 수도 있습니다. 이것이 다음 장의 무인 자동화에서 특히 중요해집니다. 사람이 지켜보지 않는 환경에서는 *실패를 자동으로 안전하게 처리*하는 규칙이 없으면 사고가 조용히 번지기 때문입니다.

## 12.9 한계: 비용·복잡도와 단일 에이전트가 나은 경우

오케스트레이션이 항상 정답은 아닙니다. 오히려 작은 일에 멀티에이전트를 쓰면 손해입니다. 한계를 분명히 알아야 올바르게 씁니다.

- **비용:** 서브에이전트는 각자 자기 컨텍스트로 도는 별도 세션입니다. 여럿을 병렬로 돌리면 토큰 사용량과 비용이 급절로 늘어납니다. 한 줄짜리 오타 수정에 조사·리뷰 파이프라인을 거는 것은 낭비입니다.
- **복잡도:** 단계가 늘수록 설계·디버깅이 어려워집니다. 어느 전문가가 어떤 맥락을 받았는지, 통합이 어디서 틀어졌는지 추적해야 합니다. 단순 작업이라면 이 복잡도가 이득을 넘어섭니다.
- **통합 비용:** 병렬화로 아낀 시간을, 결과를 합치는 통합이 다시 잡아먹을 수 있습니다. 통합이 어려운 작업(조각들이 강하게 얽힌 작업)은 애초에 잘 나뉘지 않습니다.

언제 단일 에이전트가 낫고 언제 오케스트레이션이 나은지 판단 기준을 표로 둡니다.

| 상황                          | 권장      |
|-----------------------------|---------|
| 한두 파일의 작은 변경·버그 수정          | 단일 에이전트 |
| 절차가 단순하고 단계가 1~2개           | 단일 에이전트 |
| 독립적 하위 작업이 여럿(병렬 이득)        | 오케스트레이션 |
| 단계가 길고 각 단계에 전문성이 필요        | 오케스트레이션 |
| 리뷰의 객관성(자기 코드 자기 검토 회피)이 중요 | 오케스트레이션 |

**판단 한 줄.** "이 작업을 독립적인 조각으로 나눌 수 있고, 각 조각에 다른 전문성이 필요하며, 통합 비용보다 분업 이득이 크다"면 오케스트레이션입니다. 그렇지 않으면 단일 에이전트가 빠르고 싸고 단순합니다.

## 12.10 즉시 연습 (2종 1세트)

오케스트레이션 설계를 직접 해 봅니다. 각 연습은 모범 골격을 참고해 빈칸을 채우고 자가체크로 검증하는 2종 1세트입니다.

### 연습 1 — 파이프라인 단계 채우기

**과제.** "여러 서비스(api-gateway, user-service, order-service)에 중복 구현된 JWT 토큰 검증 로직을 공통 미들웨어(common/auth.py)로 추출"하는 대규모 리팩터링을 오케스트레이션 파이프라인으로 설계하세요. 아래 지시문의 빈칸 `__①__` ~ `__④__` 를 채우세요. 각 단계가 병렬인지 순차인지, 누구(역할)에게 맡길지를 판단하는 것이 핵심입니다.

# 오케스트레이터 지시

작업: 세 서비스에 중복된 JWT 토큰 검증 로직을 common/auth.py 로 추출한다.

- 1) [\_\_①\_\_] researcher 에게 (a) 각 서비스의 토큰 검증 코드가 실제로 동일한지·차이가 있는지, (b) 공통 미들웨어의 후보 인터페이스를 조사시키고 요약만 받는다.  
-> 이 두 조사는 서로 독립적인가? 그에 맞는 흐름을 적으세요.
- 2) [통합] 두 조사 요약을 합쳐 \_\_②\_\_ 를 한 문단 명세로 정리한다.  
-> 무엇을 통합해야 하는가?
- 3) [\_\_③\_\_] 명세대로 공통 미들웨어를 만들고 각 서비스가 그것을 쓰도록 교체한다. 공유되는 인증 경로를 손대므로 이 단계의 흐름은?
- 4) [리뷰] \_\_④\_\_ 서브에이전트에게 diff 독립 리뷰를 받아 반영한다.  
-> "검증을 우회할 구멍이 생기지 않았는가"를 누구에게 맡겨 점검하는가?

### 자가체크

- [ ] \_\_①\_\_ 은 독립 조사이므로 "조사·병렬"로 두었나요?
- [ ] \_\_②\_\_ 는 "공통 미들웨어의 인터페이스(입력·검증 규칙·반환 형식)" 같은 통합 산출물인가요?
- [ ] \_\_③\_\_ 은 공유되는 인증 경로 수정이므로 "구현·순차"인가요?
- [ ] \_\_④\_\_ 는 reviewer (독립 리뷰 전문가)인가요?
- [ ] 통합 단계를 거쳐야만 서브에이전트 결과가 합쳐진다는 점을 반영했나요?

## 연습 2 — 단일 vs 오케스트레이션 판단

**과제.** 아래 다섯 작업 각각에 대해 **단일 에이전트**가 나온지 **오케스트레이션**이 나온지 고르고, 한 줄 근거를 쓰세요. 12.8의 판단 기준을 사용하세요.

| # | 작업                         | 선택(단일/오케스트레이션) | 근거 한 줄 |
|---|----------------------------|----------------|--------|
| 1 | README 오타 한 글자를 고치는 핫픽스    | __a__          |        |
| 2 | 8개 마이크로서비스를 동시에 의존성 취약점 점검 | __b__          |        |
| 3 | 환경변수 이름 하나를 한 서비스에서 변경     | __c__          |        |
| 4 | 결제 모듈 신규 기능(조사→설계→구현→리뷰)   | __d__          |        |
| 5 | 자신이 작성한 인증 코드의 객관적 보안 리뷰   | __e__          |        |

### 자가체크

- [ ] 작은 단일 파일 변경(1:3)은 단일 에이전트로 골랐나요?
- [ ] 독립 작업이 여럿인 2(8개 서비스 동시 점검)는 병렬 이득이 있어 오케스트레이션인가요?
- [ ] 단계가 길고 전문성이 필요한 4(결제 신규 기능)는 오케스트레이션인가요?
- [ ] 객관적 보안 리뷰가 필요한 5는 별도 reviewer를 두는 오케스트레이션인가요?
- [ ] 각 선택의 근거가 "비용·복잡도 대 분업 이득"으로 설명되나요?

## 12.11 혼한 오류와 오개념

**오개념 1: "멀티에이전트가 항상 더 낫다."** 아닙니다. 단순 작업에 오케스트레이션을 쓰면 비용·복잡도만 늘고 느려집니다. 한두 파일 수정에는 단일 에이전트가 빠르고 쉽습니다. "독립적으로 나눌 수 있고, 다른 전문성이 필요하며, 분업 이득이 통합 비용보다 클 때"만 오케스트레이션을 쓰세요.

**오개념 2: "오케스트레이터가 직접 다 하면 된다."** 오케스트레이터가 모든 구현을 직접 하면 컨텍스트가 폭발하고 병렬화 이점이 사라집니다. 오케스트레이터의 일은 *분배와 통합*입니다. 직접 하는 것은 통합이 어려운 핵심 부분(주로 순차 구현)에 한정하고, 나눌 수 있는 일은 전문가에게 맡기세요.

**오개념 3: "서브에이전트끼리 맥락이 공유된다."** 공유되지 않습니다. 각 서브에이전트는 독립 컨텍스트입니다(9장). researcher가 알아낸 것을 reviewer가 자동으로 알지 못합니다. 필요한 맥락은 *오케스트레이터가 명시적*으로 다음 단계에 전달해야 합니다. 통합은 항상 오케스트레이터를 거칩니다.

**오개념 4: "병렬로 나누기만 하면 통합은 알아서 된다."** 가장 흔한 실패입니다. 병렬 결과를 나열하는 것은 통합이 아닙니다. 충돌하는 권고를 *명시적 기준*으로 조정하고 *단일 결론*으로 수렴시켜야 통합입니다. 통합 기준(예: "결정론을 우선한다")을 오케스트레이터에게 미리 주어, 통합이 자의적이 되지 않게 하세요.

## 마무리

이 장에서 우리는 단원 6의 두 번째 개념인 **워크플로우와 멀티에이전트 오케스트레이션**을 배웠습니다. 오케스트레이션이 무엇이고(여러 전문가를 엮은 워크플로우), 어떻게 도는지(계획→병렬 조사→통합→순차 구현→리뷰), 커맨드로 절차를 표준화하고, 단계 간 맥락 전달과 실패·통합 게이트를 어떻게 통제하며, 언제 단일 에이전트가 더 나은지 익혔습니다. 핵심은 "오케스트레이터는 분배·통합에 집중하고, 독립이면 병렬, 의존이면 순차, 통합은 명시적 기준으로"입니다.

실무에서 이 개념을 쓸 때 스스로 던질 세 질문으로 정리합니다. (1) 이 작업은 *독립적인 조각으로 나뉘는가* — 나뉘면 병렬 이득이 있습니다. (2) 각 단계에 *다른 전문성*이 필요한가 — 필요하면 전문 서브에이전트에 위임할 가치가 있습니다. (3) 단계가 실패하면 *어떻게 처리할지 미리 정했는가* — 정하지 않았다면 아직 무인 환경에 내보낼 준비가 안 된 것입니다. 이 세 질문에 답할 수 있으면, 다음 장의 무인 자동화로 이 워크플로우를 안전하게 확장할 수 있습니다.

다음 장에서는 이 워크플로우를 *사람이 지켜보지 않는* 환경으로 내보냅니다. 헤드리스(-p) 모드·GitHub Actions·스케줄로 **무인 자동화**를 만들고, 11장의 흑과 이 장의 오케스트레이션을 결합해 신뢰할 수 있는 CI 파이프라인을 완성합니다. 그리고 그것이 단원 6 미니 프로젝트로 모입니다.



## 13장 무인 자동화: 헤드리스·CI·스케줄

이 장은 단원 6의 마지막 챕터입니다. 여기서는 개념 **CI·스케줄·자동화(CI, scheduling, and automation)**를 깊이 배우고, 단원 전체를 묶는 **미니 프로젝트**로 마무리합니다. 지금까지의 모든 것이 *사람이 옆에서 지켜보는 대화형 세션*이었다면, 이번 장은 *사람 없이* 도는 자동화로 나아갑니다.

### 이 장의 학습 목표

- 헤드리스(-p) 모드로 Claude Code를 스크립트·CI에 통합할 수 있습니다.
- GitHub Actions 등에서 무인 워크플로우를 구성할 수 있습니다.
- 스케줄 작업으로 주기적 자동화를 운용할 수 있습니다.
- 무인 환경의 권한·안전(최소 권한·로깅)을 설계할 수 있습니다.

### 13.1 도입: 대화형을 넘어 사람 없이 도는 자동화로

1장부터 우리는 대화형 REPL에서 에이전트와 주고받으며 일했습니다. 사람이 지시하고, 결과를 보고, 교정했습니다. 이 방식은 강력하지만 한계가 분명합니다. 사람이 그 자리에 있어야만 동작합니다. 하지만 실무의 많은 작업은 사람이 없을 때 자동으로 일어나야 합니다. PR이 올라올 때마다 코드 리뷰가 달리고, 매주 월요일 아침마다 모든 저장소가 점검되고, 빌드가 깨지면 알림이 가는 일은 사람이 매번 손으로 할 수 없습니다.

11장의 **혹은** 한 세션 안에서 각 행동을 결정론적으로 강제했고, 12장의 **오케스트레이션**은 여러 전문가의 작업을 신뢰성 있게 통합했습니다. 이제 이 둘을 *사람이 지켜보지 않는 환경*으로 내보낼 차례입니다. 그러려면 두 가지가 필요합니다. 첫째, 에이전트가 대화창 없이 *한 번의 명령으로 실행되고 결과를 기계가 읽을 수 있게* 내야 합니다(헤드리스 모드). 둘째, 그 실행을 *언제 무엇이 트리거할지* 정해야 합니다(CI·스케줄).

선수 개념 한 줄 상기: 11장의 **혹은** 결정론적 **게이트**를, 12장의 오케스트레이션은 **전문가 분업**을 주었습니다. 이번 장은 이 둘을 헤드리스 실행 위에 올려, 사람이 없어도 안전하고 신뢰할 수 있게 도는 무인 자동화를 완성합니다.

무인 자동화의 보상은 큼니다. 한 번 잘 설계하면 같은 품질의 점검·리뷰가 영원히 반복됩니다. 그러나 위험도 큼니다. 사람이 막아 줄 마지막 안전망이 없으므로, *권한을 더 좁히고 결과를 반드시 검증*해야 합니다. 이 장은 자동화의 힘과 안전을 함께 다룹니다.

### 13.2 정의: 헤드리스(-p) 모드와 종료 코드·구조화 출력

**정의.** 헤드리스(headless) 모드는 Claude Code를 대화형 인터페이스 없이 **한 번의 명령으로 실행해 결과를 표준 출력으로 받고, 종료 코드로 성공·실패를 알리는** 비대화 실행 방식입니다. `claude -p "지시"` (print 모드)로 진입하며, 스크립트·CI·스케줄 같은 자동화에 끼워 넣을 수 있습니다.

핵심 용어를 한글(영문)으로 정리합니다.

- **헤드리스/프린트 모드(-p, print mode):** 대화창을 열지 않고 단일 프롬프트를 보내 응답만 받는 모드.  
`claude -p "이 저장소의 README가 있는지 확인해 줘"` 처럼 씁니다.
- **종료 코드(exit code):** 실행이 끝나며 돌려주는 숫자. 유닉스 관례를 따라 `0`은 성공, `0`이 아닌 값은 오류입니다. `&&`·`||`로 다음 단계를 조건부로 잇는 데 씁니다.
- **구조화 출력(structured output):** `--output-format json`을 주면 결과를 사람이 읽는 글이 아니라 `result·usage·cost_usd` 같은 필드를 가진 JSON으로 받습니다. 기계가 파싱해 후속 처리를 하기 좋습니다.

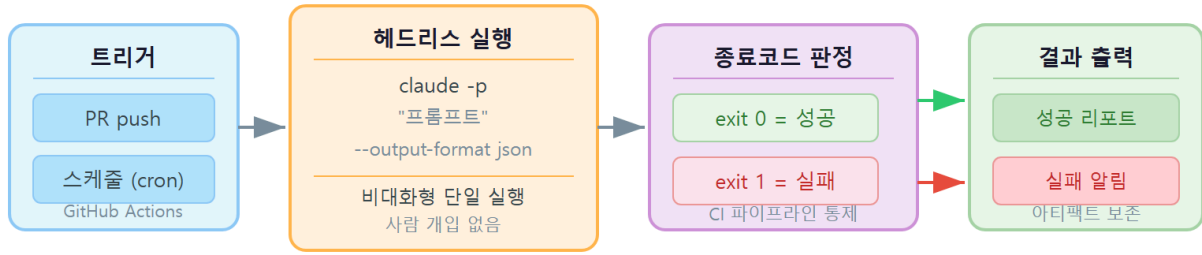
대화형과 헤드리스를 한 표로 대비합니다.

| 구분       | 대화형 REPL | 헤드리스(-p)         |
|----------|----------|------------------|
| 사람 개입    | 매 단계 가능  | 없음(한 번 실행)       |
| 출력       | 화면에 대화로  | stdout(텍스트/JSON) |
| 성공·실패 판정 | 사람이 봄    | 종료 코드            |
| 쓰임       | 탐색·설계·교정 | CI·스케줄·스크립트      |
| 안전망      | 사람의 검토   | 혹·권한·로깅·검증       |

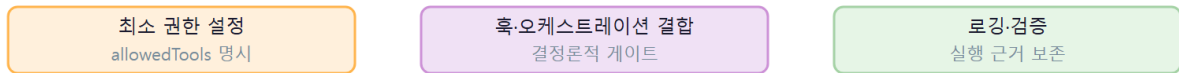
핵심은 헤드리스가 *자동화 친화적인 인터페이스*라는 점입니다. 종료 코드로 통과/실패를 알리니 CI가 판정할 수 있고, JSON 출력을 내니 다른 도구가 결과를 이어받을 수 있습니다.

## 13.3 동작 원리: 트리거→헤드리스 실행→판정 파이프라인

## 무인 자동화: 트리거 → 헤드리스 실행 → 판정 파이프라인



### 무인 자동화의 안전 설계 원칙



-p(헤드리스) 모드 = 사람 없이 실행, 종료코드로 CI 파이프라인 통제

무인 자동화는 언제나 같은 모양의 파이프라인을 가집니다. 단계로 따라가 봅니다.

- 트리거(trigger).** 무엇이 자동화를 시작시키는가입니다. 대표적으로 *이벤트 트리거*(PR이 열림·푸시됨)와 *스케줄 트리거*(매주 월요일 09시)가 있습니다. 트리거가 없으면 자동화는 돌지 않습니다.
- 환경 준비(setup).** CI 러너(예: GitHub Actions의 가상 머신)가 저장소를 체크아웃하고, Claude Code를 설치하며, 비밀(API 키)을 환경에 안전하게 주입합니다.
- 헤드리스 실행.** `claude -p "지시"` 로 에이전트를 한 번 실행합니다. 이때 *최소 권한*만 부여하고, 11장의 *혹*을 함께 적용해 위험한 행동을 막습니다. 결과는 stdout(텍스트 또는 JSON)으로 나옵니다.
- 종료 코드 판정.** 실행이 끝나며 낸 종료 코드를 CI가 읽습니다. `0` 이면 통과(녹색), `0` 이 아니면 실패(빨강)로 처리합니다. 우리 repo-audit는 "심각도 임계 초과 시 비정상 종료"로 이 판정을 만듭니다.
- 결과 보존(artifact-log).** 리포트(JSON·텍스트)를 아티팩트로 저장하거나 PR에 코멘트로 답니다. *결과를 남기지 않으면 무인 자동화는 신뢰할 수 없습니다.* 무슨 판단을 왜 내렸는지 추적 가능해야 합니다.

이 파이프라인의 두 가지 트리거 유형을 구분합니다.

| 트리거 | 언제             | 대표 자동화               |
|-----|----------------|----------------------|
| 이벤트 | PR 열림·푸시·이슈 생성 | PR 코드 리뷰, 누락 문서 점검   |
| 스케줄 | 주기적(cron)      | 주간 의존성 감사, 릴리스 노트 생성 |

**기억하세요.** 무인 자동화 = 트리거 + 헤드리스 실행 + 종료 코드 판정 + 결과 보존. 이 네 박자에 11장의 *혹*·권한과 12장의 오케스트레이션을 끼워 넣으면 "안전하고 신뢰할 수 있는" 자동화가 됩니다.

## 13.4 worked example: -p 결과를 셸 스크립트에 통합

헤드리스의 가장 단순한 형태부터 봅시다. 셸 스크립트 안에서 `claude -p` 를 호출하고, 그 종료 코드로 다음 동작을 가르는 예제입니다. "배포 전, 이번 변경에 DB 마이그레이션이 들어왔는데 롤백(down) 스크립트가 빠졌으면 배포를 막는" 작은 게이트 자동화입니다. 사람이 매 PR을 눈으로 확인하지 않아도 위험한 마이그레이션을 걸러냅니다.

```
#!/usr/bin/env bash
# 배포 전 게이트: 마이그레이션 변경에 롤백(down)이 동반됐는지 헤드리스로 점검.
set -euo pipefail

BASE="${1:-origin/main}"

# 변경된 파일 목록을 -p(print) 모드로 한 번 판정. 출력은 stdout 으로 받습니다.
result=$(git diff --name-only "${BASE}...HEAD" \
  | claude -p "표준입력의 변경 파일 목록에서 migrations/ 아래 새 마이그레이션이
  있는데 대응하는 롤백(down) 정의가 없으면 RISK, 모두 롤백을 갖췄거나
  마이그레이션 변경이 없으면 OK 만 한 단어로 답해 줘." --output-format text)

echo "배포 전 점검: ${result}"

# 결과 문자열로 종료 코드를 가릅니다.
if echo "${result}" | grep -q "RISK"; then
  echo "롤백 없는 마이그레이션 -> 배포 차단"
  exit 1
fi

echo "마이그레이션 안전 -> 배포 허용"
exit 0
```

## 줄별 해설

- `set -euo pipefail`: 오류 발생 시 즉시 중단하고, 정의되지 않은 변수를 막습니다. 자동화 스크립트의 안전 기본기입니다.
- `BASE="${1:-origin/main}"`: 첫 인자로 비교 기준 브랜치를 받되, 없으면 `origin/main` 을 씁니다.
- `git diff --name-only ... | claude -p "..."`: 변경된 파일 목록을 표준입력으로 넘겨 헤드리스로 한 번 판정합니다. 에이전트에게 *필요한 맥락(변경 파일)만* 주는 것이 핵심입니다. 답 형식을 한 단어로 못 박아 기계가 파싱하기 쉽게 했습니다.
- `grep -q "RISK"`: 결과에 위험 신호가 있는지 봅니다.
- `exit 1 / exit 0`: 종료 코드로 성공/실패를 알립니다. 이 스크립트를 배포 파이프라인이 `&&` 로 이어 받아, 위험하면 다음 단계(배포)로 넘어가지 않게 합니다.

```
실행 결과
$ ./predeploy_check.sh origin/main
배포 전 점검: OK
마이그레이션 안전 -> 배포 허용
$ echo $?
0
```

종료 코드 `0` 이 셸 변수 `$?` 로 확인됩니다. 이 한 줄이 자동화의 토대입니다. 사람이 결과를 읽지 않아도 기계가 통과/실패를 판정할 수 있기 때문입니다. 더 정교하게는 `--output-format json` 으로 받아 `result·cost_usd` 필드를 파싱할 수 있지만, 핵심 원리는 "헤드리스 실행 + 종료 코드"입니다.

## 13.5 CI 통합: GitHub Actions 무인 실행

스크립트를 *언제* 돌릴지 정하는 것이 CI(지속적 통합, Continuous Integration)입니다. GitHub Actions는 저장소 이벤트에 반응해 워크플로우를 자동 실행합니다. "PR이 열리면 보안 게이트(시크릿 스캔·의존성 취약점 점검)를 돌려 종료 코드로 통과/실패를 판정"하는 워크플로우를 만들어 봅니다. 두 도구 모두 문제를 발견하면 비정상 종료 코드를 내므로, 별도 판정 코드 없이 종료 코드만으로 PR을 막을 수 있습니다.

```
name: security-gate
on:
  pull_request:      # PR 이벤트가 트리거
    branches: [main]

permissions:
  contents: read      # 최소 권한: 저장소 읽기만 허용

jobs:
  scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4      # 1) 저장소 체크아웃
        with:
          fetch-depth: 0                # 히스토리에 유출된 비밀까지 탐지

      - name: 시크릿 스캔(gitleaks)      # 2) 하드코딩 비밀 발견 시 비정상 종료
        uses: gitleaks/gitleaks-action@v2

      - uses: actions/setup-python@v5    # 3) 파이썬 준비
        with:
          python-version: "3.12"

      - name: 의존성 취약점 점검          # 4) 취약점 발견 시 비정상 종료
        run: |
          pip install pip-audit
          pip-audit --strict --format json --output security-report.json

      - name: 리포트 업로드                # 5) 결과 아티팩트 보존
        if: always()                     # 실패해도 리포트는 남긴다
        uses: actions/upload-artifact@v4
        with:
          name: security-report
          path: security-report.json
```

### 줄별 해설

- `on: pull_request`: 트리거입니다. main을 향한 PR이 열리거나 갱신될 때마다 이 워크플로우가 자동 실행됩니다.
- `permissions: contents: read`: 무인 환경의 핵심 안전장치입니다. 이 작업에는 *읽기만* 허용하고 쓰기·배포 권한을 주지 않습니다. 사람이 없으니 권한을 더 좁힙니다(NFR4 최소 권한).
- `gitleaks/gitleaks-action@v2`: 변경·히스토리에서 하드코딩된 API 키·토큰·비밀번호를 찾습니다. *하나라도 발견하면 비정상 종료* 코드를 내, GitHub Actions가 PR을 빨강으로 판정합니다.
- `pip-audit --strict ...`: 설치된 의존성에 알려진 취약점(CVE)이 있는지 점검합니다. `--strict`는 *취약점이 있으면 비정상 종료*하게 합니다. 자체 점검 CLI를 따로 만들지 않아도, 실무에서 바로 쓰는 표준 도구가 종료 코드 판정을 그대로 제공합니다.

- `if: always()` + `upload-artifact`: 점검이 실패해도 리포트는 보존합니다. 무인 자동화에서 결과 보존은 신뢰의 조건입니다. 나중에 "어떤 취약점이 왜 걸렸는지" JSON으로 확인할 수 있습니다.

이 예제는 표준 도구의 종료 코드로 판정하지만, "이 변경이 우리 코드 컨벤션을 따르는가"처럼 *에이전트의 판단*이 필요한 점검은 같은 자리에서 `claude -p`를 호출하거나 공식 액션 `anthropics/claude-code-action`을 써서 헤드리스로 실행합니다. 이때 비밀(API 키)은 코드가 아니라 GitHub Secrets로 주입합니다(13.8). 바로 다음 13.7에서 그 형태를 봅니다.

```
실행 결과 (Actions 로그 발췌)
Run gitleaks/gitleaks-action@v2
  leaks found: 0
Run pip-audit --strict ...
  Found 1 known vulnerability in 1 package
  Name      Version ID          Fix Versions
  requests  2.19.1  PYSEC-2023-74  2.31.0
  exit code: 1
Error: Process completed with exit code 1.  <- PR 빨강(실패)
Upload security-report ... done
```

취약한 `requests 2.19.1`이 잡혀 `pip-audit`가 종료 코드 1을 냈고, GitHub Actions가 이를 실패로 판정해 PR에 빨간 표시가 달렸습니다. 사람이 아무도 보지 않았지만, 보안 점검은 빠짐없이 수행됐습니다.

## 13.6 스케줄·주기 작업

이벤트가 아니라 *시간*으로 트리거하는 것이 스케줄입니다. "매주 월요일 아침, 의존성 취약점을 감사하고 지난 한 주의 변경을 릴리스 노트 초안으로 정리한다" 같은 주기 작업에 씁니다. GitHub Actions는 cron 표현으로 스케줄을 정합니다.

```

name: weekly-maintenance
on:
  schedule:
    - cron: "0 0 * * 1" # 매주 월요일 00:00 UTC 에 실행

permissions:
  contents: read

jobs:
  maintenance:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0 # 지난 한 주 커밋을 모으기 위해 히스토리 확보
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: 주간 의존성 감사(리포트만, 실패시키지 않음)
        run: |
          pip install pip-audit
          pip-audit --format json --output dep-audit.json || true

      - name: 릴리스 노트 초안 생성
        env:
          ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
        run: |
          git log --since="7 days ago" --pretty="- %s (%an)" > week.log
          claude -p "week.log 의 커밋 목록을 사용자 관점 릴리스 노트로
          정리해 줘. feat/fix/기타로 분류하고 내부 리팩터링은 제외해." \
            --output-format text > release-notes.md

      - name: 주간 산출물 보존
        uses: actions/upload-artifact@v4
        with:
          name: weekly-maintenance
          path: |
            dep-audit.json
            release-notes.md

```

## 줄별 해설

- `on: schedule / cron: "0 0 * * 1"`: cron 표현입니다. 다섯 자리는 차례로 분·시·일·월·요일이며, `0 0 * * 1`은 "매주 월요일 00시 00분"을 뜻합니다. 시간 트리거이므로 사람·PR과 무관하게 주기적으로 돕니다.
- `pip-audit ... || true`: 주간 감사는 *보고가 목적*이라 취약점이 있어도 `|| true`로 잡 전체를 실패시키지 않습니다. PR 게이트(차단)와 달리 추세를 쌓아 관찰하는 모니터링이기 때문입니다.
- `git log ... | claude -p "..."`: 지난 한 주 커밋을 모아 헤드리스 에이전트가 *사용자 관점 릴리스 노트*로 정리합니다. 사람이 매주 손으로 쓰던 작업을 무인으로 자동화한 것입니다. API 키는 GitHub Secrets로만 주입합니다(13.8).
- `upload-artifact`: 매주 감사 리포트와 릴리스 노트 초안을 아티팩트로 쌓습니다. 시간에 따른 의존성 건강 추이와 변경 이력을 함께 볼 수 있습니다.

이벤트 트리거(13.5)와 스케줄 트리거(13.6)는 목적이 다릅니다. *이벤트*는 "변경을 게이트"(나쁜 변경을 막음)하고, *스케줄*은 "상태를 모니터링"(추세를 관찰)합니다. 같은 의존성 점검이라도 PR에서는 취약점을 *차단*하는 게이트로, 매주 스케줄에서는 추세를 *관찰*하는 모니터로 다르게 씁니다.

## 13.7 worked example: PR 자동 코드 리뷰 Actions

이번에는 에이전트의 *판단*이 필요한 자동화입니다. 점검 CLI로는 잡기 어려운 "코드 품질·가독성"을 PR마다 에이전트가 헤드리스로 리뷰해 코멘트로 다는 워크플로우입니다. 12장의 reviewer 서브에이전트 개념을 무인 환경으로 확장합니다.

```
name: pr-review
on:
  pull_request:
    types: [opened, synchronize]

permissions:
  contents: read
  pull-requests: write # 리뷰 코멘트를 달기 위한 최소 추가 권한

jobs:
  review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0 # diff 비교를 위해 전체 히스토리

      - name: Claude Code 설치
        run: npm install -g @anthropic-ai/claude-code

      - name: 변경 diff 리뷰
        env:
          ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
        run: |
          git diff origin/main...HEAD > changes.diff
          claude -p "changes.diff 의 변경을 리뷰해 줘.
          보안 취약점·동시성 버그·하위 호환성 위험만 5줄 이내로 짚고,
          심각한 문제가 없으면 'LGTM' 한 단어로 답해 줘." \
            --output-format text > review.txt
          cat review.txt
```

### 줄별 해설

- `types: [opened, synchronize]`: PR이 열릴 때와 새 커밋이 푸시될 때마다 리뷰가 돕니다.
- `permissions`: 읽기에 더해 `pull-requests: write` 만 추가했습니다. 코멘트를 달려면 필요하지만, 그 이상(코드 푸시·배포)은 주지 않습니다. *필요한 최소만* 더하는 것이 원칙입니다.
- `npm install -g @anthropic-ai/claude-code`: CI 러너에 Claude Code를 설치합니다(1장의 npm 전역 설치를 무인 환경에서 재현).
- `env: ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }`: 비밀 관리의 핵심입니다. API 키를 *코드에 적지 않고* GitHub Secrets에서 환경 변수로 주입합니다(13.8).
- `git diff ... > changes.diff`: 변경만 추려 파일로 만듭니다. 에이전트에게 *필요한 맥락만* 줍니다.
- `claude -p "..." --output-format text > review.txt`: 헤드리스로 리뷰를 실행하고 결과를 파일로 받습니다. 지시에서 *분량과 형식을 못 박아* 출력이 일관되게 합니다.

**실행 결과 (Actions 로그 발췌)**

Run git diff origin/main...HEAD &gt; changes.diff

Run claude -p "." &gt; review.txt

review.txt:

- payment\_client.py: 재시도 루프가 먹등성 키를 매번 새로 생성 -> 이중 청구 위험
- user\_repository.py: f-string 으로 SQL 조립 -> SQL 인젝션 위험
- 나머지 변경은 양호

사람 리뷰어가 없어도 PR마다 같은 기준으로 1차 리뷰가 답입니다. 다만 이것이 *사람 리뷰를 대체*하지는 않습니다. 무인 리뷰는 1차 거름망이고, 최종 판단은 사람이 합니다(13.10의 오개념).

## 13.8 무인 환경의 권한·비밀·안전과 혹·오케스트레이션 결합

무인 자동화는 사람이라는 마지막 안전망이 없습니다. 그래서 *대화형보다 더 엄격하게* 안전을 설계해야 합니다. 네 가지 축으로 정리합니다.

- **최소 권한(least privilege):** CI 작업에 꼭 필요한 권한만 줍니다. 점검만 하면 `contents: read`, 코멘트를 달면 `pull-requests: write` 까지만. *기본을 좁게* 두고 필요할 때만 넓힙니다(4장의 권한 원칙을 무인 환경에 적용).
- **비밀 관리(secrets):** API 키·토큰을 *절대 코드·로그에 남기지 않습니다*. GitHub Secrets 같은 안전한 저장소에 두고 환경 변수로만 주입합니다. 비밀이 로그에 찍히지 않게 출력도 점검합니다(10장 MCP의 비밀 관리 원칙과 동일).
- **결정론·재현성:** 같은 입력에 같은 출력을 내야 무인 판정을 신뢰할 수 있습니다. repo-audit가 외부 URL 검사를 제외하고 JSON 키를 정렬하는 것이 이 때문입니다(NFR5). 비결정적 결과는 무인 게이트에서 거짓 통과·거짓 실패를 만듭니다.
- **로깅·검증:** 무인 실행은 *반드시* 결과를 남깁니다. 종료 코드, 리포트 아티팩트, 차단 로그(11장)를 보존해 "무엇을 왜 했는지" 추적합니다. 로깅 없는 자동화는 신뢰할 수 없습니다.

여기에 11장과 12장이 결합합니다. 헤드리스 실행 안에서도 **혹**은 그대로 작동해 위험 편집을 결정론적으로 차단하고(사람이 못 막으니 더 중요), **오케스트레이션**은 조사→구현→리뷰 파이프라인을 무인으로 재현합니다. 즉 무인 자동화는 단원 6의 세 개념(**혹·오케스트레이션·CI**)이 하나로 모이는 지점입니다.

**무인 환경 안전 3원칙.** (1) 권한은 대화형보다 *더 좁게*(사람 안전망 없음). (2) 비밀은 *코드·로그가 아니라 안전한 설정*으로. (3) 결과는 *반드시 검증·로깅*(무검증 자동화는 사고를 자동으로 반복함). 이 셋을 어기면 무인 자동화는 자산이 아니라 위험이 됩니다.

## 13.9 즉시 연습 (2종 1세트)

헤드리스·CI 자동화를 직접 구성해 봅니다. 각 연습은 모범 골격을 참고해 빈칸을 채우고 자가체크로 검증하는 2종 1세트입니다.

### 연습 1 — CI 워크플로우 채우기

**과제.** PR마다 의존성 취약점을 점검해 종료 코드로 판정하고, 결과를 아티팩트로 보존하는 GitHub Actions 보안 게이트를 완성하세요. 빈칸 `__①__` ~ `__⑤__` 를 채우세요.

```
name: security-gate
on:
  __①__:                # PR 이벤트 트리거 키
    branches: [main]

permissions:
  contents: __②__      # 최소 권한(읽기만)

jobs:
  scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - name: 의존성 취약점 점검
        run: |
          pip install pip-audit
          pip-audit __③__ --format json --output audit.json # 취약점 발견 시 실패시키는 플래그
      - name: 리포트 보존
        if: __④__      # 실패해도 리포트는 남긴다
        uses: actions/upload-artifact@v4
        with:
          name: security-report
          path: __⑤__   # 위에서 생성한 리포트 파일명
```

### 자가체크

- `[] __①__` 은 `pull_request` 로 PR을 트리거하나요?
- `[] __②__` 는 `read` 로 최소 권한(읽기만)인가요?
- `[] __③__` 은 `--strict` 로 취약점 발견 시 비정상 종료하나요?
- `[] __④__` 는 `always()` 로 실패 시에도 리포트를 보존하나요?
- `[] __⑤__` 는 위에서 생성한 리포트 파일명(`audit.json`)과 일치하나요?

## 연습 2 — 헤드리스 스크립트 완성

**과제.** 셸 스크립트에서 `claude -p` 로 변경분의 "하드코딩된 시크릿 의심 줄 수"를 세고, 한 건이라도 있으면 실패시키는 배포 전 자동화를 완성하세요. 빈칸 `__㉑__` ~ `__㉔__` 를 채우세요.

```
#!/usr/bin/env bash
set -euo pipefail
THRESHOLD=0 # 하드코딩 시크릿은 한 건도 허용하지 않는다

# 변경분을 헤더리스로 점검해 시크릿 의심 줄 수만 숫자로 받습니다.
count=$(git diff origin/main...HEAD \
  | claudes --__ "표준입력 diff 에서 하드코딩된 비밀번호·API 키·토큰으로
의심되는 줄의 개수만 숫자로 답해 줘." --output-format text)

echo "시크릿 의심: ${count}건"

if [ "${count}" -gt "${THRESHOLD}" ]; then
  echo "하드코딩 시크릿 의심 -> 실패"
  exit __
fi

echo "시크릿 없음 -> 통과"
exit __
```

### 자가체크

- [ ] \_\_\_\_ 은 헤더리스 진입 플래그(-p)인가요?
- [ ] \_\_\_\_ 은 실패를 알리는 0이 아닌 종료 코드인가요?
- [ ] \_\_\_\_ 은 성공을 알리는 0인가요?
- [ ] 지시에서 출력 형식(숫자만)을 못 박아 파싱이 쉬운가요?
- [ ] 종료 코드로 배포 파이프라인이 이 스크립트를 이어 받아 위험 시 배포를 막나요?

## 13.10 흔한 오류와 오개념

**오개념 1: "무인 자동화는 권한을 더 넓게 줘야 편하다."** 정반대입니다. 사람이 막아 줄 안전망이 없으므로 *대화* 형보다 더 좁게 줘야 합니다. CI 작업에는 꼭 필요한 권한만( `contents: read` 같은), 비밀은 안전한 설정으로만 주입하세요. 넓은 권한 + 무인 = 사고의 자동 반복입니다.

**오개념 2: "헤더리스 모드는 안전장치가 필요 없다."** 헤더리스도 같은(오히려 더 강한) 안전장치가 필요합니다. 11장의 혹은 헤더리스 실행 안에서도 작동해 위험 편집을 차단합니다. 사람이 못 보는 만큼 *혹·권한·로깅*이 더 중요합니다.

**오개념 3: "자동화 결과는 그대로 믿어도 된다."** 무검증 신뢰는 위험합니다. 자동화 결과는 *반드시 로깅·검증*해야 합니다. 종료 코드·리포트 아티팩트·로그로 "무엇을 왜 했는지" 추적할 수 있어야 합니다. 무인 PR 리뷰도 1차 거름망일 뿐, 최종 판단은 사람이 합니다.

**오개념 4: "비결정적이어도 자동화는 돌아간다."** 무인 게이트에서 비결정성은 거짓 통과·거짓 실패를 만듭니다. 같은 입력에 같은 출력을 내도록(외부 의존 제거·출력 정렬) 설계해야 무인 판정을 신뢰할 수 있습니다(NFR5).

## 13.11 단원 미니 프로젝트: 자동화 게이트와 CI 파이프라인

이제 단원 6의 세 개념(혹.오케스트레이션.CI)을 *하나의 작은 산출물*로 묶습니다. 종합 프로젝트(repo-audit)로 가기 전, 단원 6 개념만으로 완성하는 적용 연습입니다. 목표는 `project_spec.json`의 단원 6 미니 프로젝트와 같습니다.

**미니 프로젝트 목표.** 위험 편집을 막는 PreToolUse 혹과 서브에이전트 파이프라인을 만들고, 헤드리스(-p) 모드를 GitHub Actions CI로 무인 실행하는 최소 자동화를 구성합니다. **산출물.** `.claude/hooks/guard.py` (위험 경로 차단·로깅 혹) + `.github/workflows/claude.yml` (PR 시 헤드리스 점검 무인 실행). **완료 기준.** 혹이 보호 경로 편집을 `exit 2`로 차단·로깅하고, CI가 종료 코드로 통과/실패를 판정합니다.

## 마일스톤(증분)과 사용 개념

작업을 세 증분으로 나눕니다. 각 마일스톤이 *어느 개념*을 쓰는지 명시합니다.

| 마일스톤 | 내용                                          | 사용 개념       | 체크포인트                                       |
|------|---------------------------------------------|-------------|---------------------------------------------|
| M1   | 위험 경로 수정을 막는 PreToolUse 혹 (guard.py) 작성·로깅  | c11 혹       | 보호 경로 편집이 <code>exit 2</code> 로 차단되고 로그가 남음 |
| M2   | 조사→구현→리뷰 서브에이전트를 순차 호출하는 워크플로우 정리           | c12 오케스트레이션 | 파이프라인이 단계별로 전문가에 분배·통합됨                     |
| M3   | 헤드리스(-p) 실행을 GitHub Actions로 무인화하고 종료 코드 판정 | c13 CI·자동화  | CI가 종료 코드로 통과/실패를 판정함                       |

## 모범 산출물 (참고용 골격)

**M1 — guard.py(c11 혹).** 11.7에서 만든 보호 경로 차단 혹을 미니 프로젝트의 게이트로 씁니다.

```
#!/usr/bin/env python3
"""미니 프로젝트 안전 게이트: 보호 경로 편집 차단·로그."""
import json
import sys
from datetime import datetime
from pathlib import Path

PROTECTED = ("secrets/", ".env", "prod_config/")
LOG_PATH = Path(".claude/guard.log")

def main() -> int:
    try:
        event = json.loads(sys.stdin.read())
    except json.JSONDecodeError:
        return 0 # fail-open: 가드 오류로 작업을 막지 않음

    file_path = event.get("tool_input", {}).get("file_path", "")
    if any(token in file_path for token in PROTECTED):
        stamp = datetime.now().isoformat(timespec="seconds")
        with LOG_PATH.open("a", encoding="utf-8") as fp:
            fp.write(f"{stamp} BLOCKED edit: {file_path}\n")
        sys.stderr.write(f"보호 경로는 수정할 수 없습니다: {file_path}\n")
        return 2 # PreToolUse 차단

    return 0

if __name__ == "__main__":
    sys.exit(main())
```

**M2 — 오케스트레이션 절차(c12).** 조사→구현→리뷰 파이프라인을 `/feature` 커맨드로 표준화합니다(12.6 참고). 핵심은 독립 조사는 병렬, 구현은 순차, 리뷰는 reviewer에 위임, 통합은 메인에 담당하는 것입니다.

**M3 — claude.yml(c13 CI).** 헤드리스 점검을 PR 트리거로 무인 실행합니다.

```
name: claude
on:
  pull_request:
    branches: [main]

permissions:
  contents: read

jobs:
  audit:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - name: 헤드리스 점검
        run: python3 repo_audit.py . --fail-on error
      - name: 리포트 보존
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: audit-report
          path: audit_report.json
```

통합 실행 결과 (기대 동작)

- 에이전트가 .env 편집 시도 -> guard.py 가 exit 2 로 차단, guard.log 기록
- /feature 로 새 점검 추가 -> 조사·구현·리뷰 파이프라인 수행
- PR 생성 -> Actions 가 repo\_audit 헤드리스 실행 -> 종료 코드로 판정

## 학생 작성형 빈 템플릿 (2종 1세트)

여러분의 차례. 위 모범을 참고해, 차단 조건과 종료 코드를 직접 채운 게이트와 CI를 완성하세요.

(가) guard.py 빈 템플릿 — 보호 경로 목록과 종료 코드를 채우세요.

```
#!/usr/bin/env python3
import json
import sys
from pathlib import Path

PROTECTED = ( __①__ ) # 보호할 경로 조각들(튜플)
LOG_PATH = Path(".claude/guard.log")

def main() -> int:
    try:
        event = json.loads(sys.stdin.read())
    except json.JSONDecodeError:
        return __②__ # fail-open 종료 코드

    file_path = event.get("tool_input", {}).get("file_path", "")
    if any(token in file_path for token in PROTECTED):
        with LOG_PATH.open("a", encoding="utf-8") as fp:
            fp.write(f"BLOCKED edit: {file_path}\n")
            sys.stderr.write(f"보호 경로 차단: {file_path}\n")
        return __③__ # PreToolUse 차단 종료 코드

    return __④__ # 정상 통과 종료 코드

if __name__ == "__main__":
    sys.exit(main())
```

(나) claude.yml 빈 템플릿 — 트리거·권한·판정 부분을 채우세요.

```
name: claude
on:
    __⑤__: # 트리거(PR)
        branches: [main]
permissions:
    contents: __⑥__ # 최소 권한
jobs:
    audit:
        runs-on: ubuntu-latest
        steps:
            - uses: actions/checkout@v4
            - uses: actions/setup-python@v5
              with:
                python-version: "3.12"
            - run: python3 repo_audit.py . --fail-on __⑦__
```

## 자가체크

- [ ] guard.py: \_\_①\_\_ 에 secrets/ · .env 등 보호 조각을 튜플로 넣었나요?
- [ ] guard.py: \_\_②\_\_ · \_\_④\_\_ 는 통과(0), \_\_③\_\_ 은 차단(2)인가요?
- [ ] 혹은 보호 경로 편집을 실제로 막고 로그를 남기나요?
- [ ] M2의 오케스트레이션이 조사(병렬)→구현(순차)→리뷰(위임)→통합(메인)으로 분업되나요?
- [ ] claude.yml: \_\_⑤\_\_ = pull\_request , \_\_⑥\_\_ = read , \_\_⑦\_\_ = error 로 최소 권한·종료 코드 판정을 갖췄나요?
- [ ] CI가 종료 코드로 통과/실패를 판정하고, fail-open 설계로 정상 작업을 막지 않나요?

## 마무리

이 장에서 우리는 단원 6의 마지막 개념인 **CI·스케줄·자동화**를 배우고, 단원 전체를 미니 프로젝트로 묶었습니다. 헤드리스(-p) 모드로 에이전트를 기계가 읽을 수 있게 실행하고, GitHub Actions의 이벤트·스케줄 트리거로 무인 실행하며, 종료 코드로 판정하고 결과를 보존하는 법을 익혔습니다. 무엇보다 무인 환경에서는 *권한을 더 좁히고, 비밀을 안전하게 다루며, 결과를 반드시 검증해야* 한다는 안전 원칙을 배웠습니다.

이로써 단원 6(혹·오케스트레이션·CI)이 완성됐고, 1장부터 13장까지 13개 개념을 모두 배웠습니다. 다음 장(14장)부터는 이 모든 개념을 *하나의 완성품*으로 통합하는 종합 프로젝트 **repo-audit**를 분석→설계→구현→발표의 PBL 수명주기로 만듭니다. 지금까지 따로 배운 기능들이, 신뢰할 수 있는 하나의 자동화로 어떻게 모이는지 직접 체득하게 됩니다.



## 종합 프로젝트: repo-audit — 분석(요구사항·범위)

지금까지 13개 챕터에 걸쳐 Claude Code의 핵심 기능을 하나씩 배웠습니다. 설치와 첫 세션부터 효과적 프롬프팅, 파일 탐색·편집, 권한, CLAUDE.md, 플랜 모드, Git 통합, 슬래시 커맨드, 서브에이전트, MCP, 혹은, 오케스트레이션, CI 자동화까지 — 각 기능을 단위 미니 프로젝트로 따로 써 봤습니다. 그러나 기능을 하나씩 배우는 것과 그 기능들을 **하나의 실제 산출물로 엮어 쓰는 것**은 전혀 다른 경험입니다. 이제부터 네 챕터(ch\_14~ch\_17)에 걸쳐 그 통합 경험을 합니다. 우리가 만들 것은 **저장소 건강검진 CLI, repo-audit**입니다.

이번 챕터는 종합 프로젝트의 첫 단계인 **분석(analysis)**입니다. 공장 코드를 짜는 대신, "무엇을 만들 것인가"를 먼저 뚜렷하게 정의합니다. 막연한 아이디어("저장소를 점검하는 도구")를 검증 가능한 **요구사항(requirement)**으로 바꾸고, 누가 어떻게 쓰는지를 **사용자 스토리·유스케이스**로 그리며, 무엇을 만들고 무엇을 만들지 않을지 **범위(scope)**로 통제합니다. 마지막으로 각 요구사항이 앞 단원에서 배운 **어느 개념으로 구현되는지**를 매핑해, 커리큘럼과 프로젝트가 정확히 맞물리는지 확인합니다.

분석을 건너뛰고 바로 구현에 들어가면 "다 만들었는데 정작 필요한 게 빠졌다"거나 "범위가 끝없이 늘어나 완성하지 못하는" 일이 흔합니다. 분석은 작은 프로젝트라도 결과를 좌우하는 단계입니다. 특히 이 도구는 Claude Code로 만들 것이므로, 잘 정의된 요구사항은 곧 ch\_02에서 배운 **명확한 작업 지시**의 재료가 됩니다 — 분석 결과가 좋을수록 에이전트에게 줄 지시도 정확해집니다.

### 이 챕터의 학습 목표

- 드라이빙 퀘스천을 검증 가능한 기능·비기능 요구사항으로 구체화할 수 있습니다.
- 사용자 스토리와 유스케이스로 도구의 사용 흐름을 표현할 수 있습니다.
- 범위(In/Out)를 정해 프로젝트가 끝없이 늘어나지 않도록 통제할 수 있습니다.
- 각 요구사항이 어느 단위 개념으로 구현되는지 추적성 표로 매핑할 수 있습니다.

이 챕터는 종합 프로젝트(capstone) 챕터입니다. 새로운 Claude Code 기능을 가르치지 않고, ch\_01~ch\_13에서 배운 개념(c01~c13)을 통합 적용하기 위한 설계의 출발점을 다집니다.

## ch\_14\_s01. 드라이빙 퀘스천과 POV 다시 보기

본격적인 분석에 앞서, 이 프로젝트가 답하려는 큰 질문과 관점을 다시 확인합니다. 교재 도입부에서 종합 프로젝트의 방향을 잡을 때 정했던 **드라이빙 퀘스천(driving question)**과 **POV(관점 진술)**입니다. 이 둘은 분석 내내 "우리가 왜 이걸 만드는가"를 잃지 않게 하는 나침반 역할을 합니다.

### 드라이빙 퀘스천

실무 개발자가 Claude Code를 페어 프로그래머이자 자동화 엔진으로 삼아, 신뢰할 수 있는 '저장소 건강검진 CLI'를 분석부터 CI 자동화까지 하나의 에이전트 워크플로우로 어떻게 만들어 낼 수 있을까?

## POV(관점 진술) — 사용자·필요·통찰

- **사용자:** 여러 저장소를 관리하며 코드 품질·문서·링크 상태를 일관되게 점검하려는 실무 개발자·팀리드
- **필요:** 저장소의 건강 상태(깨진 링크·비대 파일·누락 문서·미해결 TODO)를 한 번에 점검·보고하고, 그 점검을 CI로 무인 자동화한다
- **통찰:** 점검 도구를 Claude Code로 직접 만들어 보면, 프롬프팅·플랜 모드·서브에이전트·혹·CI가 어떻게 하나의 신뢰할 수 있는 자동화로 모이는지를 비로소 체득한다 — 기능을 따로 배우는 것과 한 산출물로 엮어 쓰는 것은 다르다

이 진술에서 분석의 방향이 그대로 읽힙니다. 사용자는 "팀리드·실무 개발자"이므로 도구는 **명령줄에서 빠르게 돌고**, 결과는 **사람이 읽는 요약과 기계(CI)가 읽는 형식**을 모두 내야 합니다. 필요가 "한 번에 점검·보고 + CI 무인 자동화"이므로 요구사항은 점검 항목들과 자동화 항목으로 갈라집니다. 드라이빙 퀘스천이 "분석부터 CI 자동화까지 하나의 에이전트 워크플로우"라고 못 박았으니, 우리는 도구의 기능뿐 아니라 **그것을 만드는 Claude Code 워크플로우**까지 설계 대상으로 삼습니다.

도구의 산출물은 `repo_audit.py (+ checks.py · report.py + tests/)`라는 실행 가능한 CLI이고, 함께 Claude Code 스캐폴딩(`CLAUDE.md` · `/audit` 커스텀 커맨드·reviewer 서브에이전트·PreToolUse 린트 혹은 `.mcp.json` · GitHub Actions CI 워크플로우)도 만듭니다. 즉 **만드는 결과물과 만드는 방법**이 모두 학습 대상입니다.

## ch\_14\_s02. 기능 요구사항(FR) 정의

**기능 요구사항(Functional Requirement, FR)**은 "도구가 무엇을 해야 하는가"를 행위 단위로 적은 것입니다. 드라이빙 퀘스천의 "점검·보고·자동화"를 더 작은 검증 가능한 항목으로 쪼갭니다. 좋은 FR은 모호하지 않아 "됐다/안 됐다"를 분명히 판정할 수 있어야 합니다. repo-audit의 기능 요구사항은 모두 아홉 개입니다.

| ID  | 기능 요구사항                           | 검증(완료 조건)                                                                                       |
|-----|-----------------------------------|-------------------------------------------------------------------------------------------------|
| FR1 | 대상 저장소 경로·임계치·출력 형식을 CLI 인자로 받기   | <code>repo_audit.py ./repo --max-file-kb 500 --format json</code> 이 인자대로 동작                     |
| FR2 | 마크다운 파일에서 로컬 링크·앵커를 추출해 깨진 링크 검출  | 없는 파일 <code>[x](./none.md)</code> 는 error, 파일은 있으나 깨진 앵커 <code>(./a.md#없는섹션)</code> 는 warn으로 보고 |
| FR3 | 지정 임계치를 초과하는 파일 크기 점검             | 임계 초과 파일을 warn 이상으로 보고, 임계 변경이 결과에 반영                                                           |
| FR4 | 필수 문서(README·LICENSE) 존재 확인       | README·LICENSE가 없으면 누락을 보고                                                                      |
| FR5 | TODO/FIXME 주석 수집·집계               | 소스의 <code># TODO</code> (info)· <code># FIXME</code> (error)를 경로·결과 함께 수집하고 등급별 개수 집계           |
| FR6 | 사람용 텍스트 요약과 기계용 JSON 리포트 생성       | <code>--format text</code> · <code>--format json</code> 이 같은 발견을 다른 형식으로 출력                     |
| FR7 | 심각도 임계 초과 시 비정상 종료 코드 반환(헤드리스 판정) | error 발견 시 <code>exit 1</code> , 문제없으면 <code>exit 0</code>                                      |
| FR8 | GitHub Actions에서 PR마다 자동 실행       | PR 이벤트가 워크플로우를 트리거하고 종료 코드로 체크 판정                                                               |
| FR9 | 주간 스케줄로 정기 점검 실행                  | <code>schedule</code> 트리거가 주 1회 점검을 무인 실행하고 리포트 보존                                              |

각 FR을 조금 더 풀어 보면 다음과 같습니다.

- **FR1 인자 파싱:** 도구는 대상 경로를 받아야 하고, 파일 크기 임계(`--max-file-kb`)와 출력 형식(`--format text|json`)을 옵션으로 받습니다. 이는 ch\_03에서 배운 파일 탐색의 출발점이자, ch\_02의 "명확한 입력·완료 조건"을 코드 인터페이스로 옮긴 것입니다.
- **FR2 링크 점검:** 마크다운의 [텍스트](경로) 중 외부 URL(`http://`, `https://`)이 아닌 로컬 경로만 대상으로, 그 파일이 실제로 있는지 확인합니다. 한 걸음 더 나아가 `파일.md#섹션` 처럼 헤딩 앵커가 붙은 링크는 대상 문서에 그 헤딩이 실제로 있는지까지 검사합니다 — 파일은 있으나 앵커가 깨진 경우(문서 리팩터링 후 흔한 실수)는 깨진 링크(error)보다 한 단계 낮은 경고(warn)로 보고합니다. 외부 URL을 제외하는 이유는 뒤(ADR3)에서 설계 결정으로 다룹니다.
- **FR3 파일 크기:** 저장소에 실수로 들어간 큰 바이너리·로그를 잡습니다. 임계는 인자로 바꿀 수 있어야 합니다(FR1과 연결).
- **FR4 필수 문서:** README와 LICENSE가 없는 저장소를 표시합니다. 팀 표준 점검의 단골 항목입니다.
- **FR5 TODO 수집:** 코드 전반의 미해결 작업 흔적을 모아 한눈에 보여 줍니다. 이때 `TODO` (언젠가 할 일)와 `FIXME` (지금 잘못된 것)를 같은 등급으로 뭉뚱그리지 않고, 관례대로 TODO는 정보(info), FIXME는 결함(error)으로 경중을 갈라 보고합니다. 이 구분이 있어야 CI에서 `--fail-on error` 로 FIXME만 빌드를 막고 평범한 TODO는 통과시키는 정교한 게이트(FR7)를 세울 수 있습니다.

- **FR6 두 형식 리포트:** 사람은 텍스트 요약물, CI·아티팩트는 JSON을 소비합니다. 둘이 같은 발견을 담아야 합니다.
- **FR7 종료 코드:** CI가 통과/실패를 판정하려면 도구가 종료 코드로 결과를 알려야 합니다. 이것이 헤드리스 자동화(ch\_13)의 핵심 인터페이스입니다.
- **FR8·FR9 자동화:** PR마다(FR8), 그리고 주간 스케줄로(FR9) 사람 없이 점검이 돌아야 합니다.

좋은 FR은 **명사가 아니라 동사**로 시작합니다("링크 검사" 대신 "깨진 링크를 검출한다"). 또 각 FR에 **검증 가능한 완료 조건**을 붙이면, 나중에 ch\_02의 작업 지시로 옮길 때 "이렇게 되면 끝"이라는 기준이 그대로 따라옵니다. 위 표의 오른쪽 열이 바로 그 완료 조건입니다.

## ch\_14\_s03. 비기능 요구사항(NFR) 정의

**비기능 요구사항(Non-Functional Requirement, NFR)**은 "도구가 어떤 품질을 갖춰야 하는가"를 적은 것입니다. 기능이 "무엇을 하는가"라면, 비기능은 "얼마나 안전·견고·재현 가능한가"입니다. repo-audit는 무인 CI에서 돌 도구이므로 비기능 요구사항이 특히 중요합니다 — 사람이 지켜보지 않는 자동화는 안전과 재현성이 생명이기 때문입니다.

| ID   | 비기능 요구사항                                | 이유                           |
|------|-----------------------------------------|------------------------------|
| NFR1 | 표준 라이브러리만으로 동작(외부 의존 최소화)               | 설치가 간단하고 CI가 가벼우며 공급망 위험이 적음 |
| NFR2 | 각 점검 함수를 단위 테스트로 검증(전문 서브에이전트가 작성·실행)   | 점검 로직의 정확성을 자동으로 보장          |
| NFR3 | 위험 경로 편집을 PreToolUse 혹은 차단(안전 게이트)      | 에이전트가 실수로 중요 파일을 건드리지 못하게 막음 |
| NFR4 | 무인 실행을 최소 권한 allowlist로 통제              | 자동화일수록 권한을 좁혀 사고를 예방         |
| NFR5 | 리포트가 결정론적·재현 가능(같은 입력=같은 출력)            | CI 판정이 흔들리지 않아야 신뢰할 수 있음     |
| NFR6 | 프로젝트 규칙·명령은 CLAUDE.md로 영속화해 세션 간 일관성 유지 | 매 세션 같은 컨벤션·명령으로 작업          |

비기능 요구사항을 풀어 보면, 이들은 대부분 **앞 단원에서 배운 안전·자동화 기능과 직결**됩니다.

- **NFR1 무의존:** 외부 패키지 없이 파이썬 표준 라이브러리(`argparse`·`pathlib`·`json`·`re`)만 씁니다. CI에서 설치 단계가 거의 없어 빠르고, 의존성 취약점 위험도 줄어듭니다. 이는 MCP로 외부 연결할지(ch\_10) 판단하는 기준이기도 합니다 — 점검 자체엔 외부 연결이 불필요합니다.
- **NFR2 테스트:** 각 점검 함수가 픽스처 저장소에서 정확히 동작하는지 단위 테스트로 검증하고, 그 테스트 작성·실행은 reviewer/tester 서브에이전트(ch\_09)에 위임합니다.

- **NFR3 안전 게이트:** 도구를 만드는 과정에서 에이전트가 위험 경로(예: `.git/`, `secrets`)를 건드리지 못하도록 `PreToolUse` 혹은 `ch_11`)으로 막습니다.
- **NFR4 최소 권한:** 무인 CI에서 도는 만큼, 필요한 명령만 `allowlist(ch_04)`로 허용합니다.
- **NFR5 결정론:** 같은 저장소를 두 번 점검하면 같은 리포트가 나와야 합니다. 발견을 정렬하고 시각·난수 같은 비결정 요소를 배제합니다. 이는 `혹(ch_11)`·`CI(ch_13)`의 신뢰 기반입니다.
- **NFR6 연속 컨텍스트:** 빌드·테스트 명령과 컨벤션을 `CLAUDE.md(ch_05)`에 적어, 어느 세션에서 작업해도 같은 규칙이 적용되게 합니다.

비기능 요구사항은 "있으면 좋은 것"이 아니라 **무인 자동화의 전제 조건**입니다. NFR5(결정론)가 깨지면 CI가 같은 코드에서 한 번은 통과·한 번은 실패해 신뢰를 잃습니다. NFR3·NFR4(안전·권한)가 없으면 무인 환경에서 사고가 나도 막을 장치가 없습니다. 기능보다 먼저 흔들리면 안 되는 토대입니다.

## ch\_14\_s04. 사용자 스토리와 유스케이스

요구사항을 정의했으니, 이제 **누가 이 도구를 어떻게 쓰는가**를 사용자 관점에서 그립니다. **사용자 스토리(user story)**는 "역할로서, 목적을 위해, 무엇을 원한다" 형식의 짧은 진술이고, **유스케이스 (use case)**는 한 시나리오의 입력·흐름·결과를 더 구체적으로 적은 것입니다.

### 사용자 스토리

| #   | 사용자 스토리                                                   |
|-----|-----------------------------------------------------------|
| US1 | 팀리드로서, 새 저장소를 받으면 repo-audit 한 번으로 문서·링크·품질 상태를 한눈에 보고 싶다 |
| US2 | 개발자로서, PR을 올릴 때 깨진 링크나 누락 문서가 자동으로 걸러지길 원한다               |
| US3 | 팀리드로서, 매주 모든 저장소가 자동 점검돼 리포트가 쌓이길 원한다                     |
| US4 | 개발자로서, 점검 도구를 만들면서 Claude Code의 플랜·서브에이전트·혹을 실제로 써 보고 싶다  |

각 스토리는 앞의 요구사항과 맞물립니다. US1은 로컬 점검(FR1~FR6), US2는 PR 자동 점검(FR7·FR8), US3은 주간 스케줄(FR9), US4는 이 프로젝트 자체의 메타 목표(드라이빙 퀘스천)입니다.

### 유스케이스

사용자 스토리를 실행 시나리오로 구체화하면 다음 세 유스케이스가 됩니다. 각 유스케이스는 트리거 → 흐름 → 결과의 구조를 갖습니다.

| ID  | 유스케이스 | 흐름과 결과                                                                                 |
|-----|-------|----------------------------------------------------------------------------------------|
| UC1 | 로컬 점검 | <code>repo_audit.py ./myrepo</code> 실행 → 텍스트 요약과 발견 목록 출력 → 문제 있으면 <code>exit 1</code> |
| UC2 | CI 점검 | PR 이벤트 → GitHub Actions가 헤드리스 실행 → 종료 코드로 통과/실패 판정                                     |
| UC3 | 정기 점검 | 주간 스케줄 → JSON 리포트 산출 → 아티팩트로 보존                                                        |

세 유스케이스는 **같은 코어 로직(점검-리포트)**을 **다른 트리거로 부르는 것**임에 주목하세요. 도구 본체는 하나이고, 그것을 사람이 직접(UC1) 부르느냐, PR이(UC2) 부르느냐, 스케줄이(UC3) 부르느냐만 다릅니다. 이 구조 덕에 우리는 코어를 잘 만든 뒤 자동화 래퍼만 얹으면 됩니다 — 이것이 ch\_13에서 배운 "헤드리스 인터페이스(종료 코드)를 잘 정해 두면 자동화가 쉬워진다"의 실제 적용입니다.

유스케이스를 적을 때는 **정상 흐름(happy path)**뿐 아니라 **예외 흐름**도 떠올려 보면 좋습니다. 예를 들어 UC1에서 "대상 경로가 존재하지 않으면?", UC2에서 "점검 도중 도구가 죽으면?" 같은 질문은 나중에 종료 코드 설계(ADR4)와 예외 처리로 이어집니다. 분석 단계에서 던진 질문이 설계-구현의 빈틈을 미리 메웁니다.

## ch\_14\_s05. 범위 In/Out 확정

요구사항이 늘어나기 시작하면 "이것도 되면 좋겠다"가 끝없이 붙습니다(이를 **범위 확장, scope creep** 이라 합니다). 이를 막으려면 **무엇을 만들고(In) 무엇을 만들지 않을지(Out)**를 명시적으로 정해야 합니다. 범위를 못 박는 것은 기능을 줄이는 게 아니라, 제한된 시간에 **완성 가능한 산출물**을 보장하는 장치입니다.

### 범위 안(In Scope) — 만든다

- CLI 인자 파싱(대상 경로·임계치·출력 형식)
- 마크다운 로컬 링크·앵커 유효성 검사
- 파일 크기 임계 초과 점검
- 필수 문서(README·LICENSE) 존재 확인
- TODO/FIXME 주석 수집·집계
- 텍스트 요약·JSON 리포트 생성과 종료 코드
- 각 점검 함수 단위 테스트
- GitHub Actions CI와 주간 스케줄 자동화

### 범위 밖(Out of Scope) — 만들지 않는다

| 제외 항목            | 제외 이유                            |
|------------------|----------------------------------|
| 외부 URL 실시간 검사    | 네트워크 의존·비결정성(NFR5 위배). 로컬 링크만 검사 |
| GUI·웹 대시보드       | CLI 한 줄 실행이 목표. 화면 UI는 범위 밖      |
| 데이터베이스·대규모 분산 처리 | 단일 저장소 점검에 불필요한 복잡도              |
| 유료 외부 API 의존     | NFR1(무의존)과 충돌, 비용·키 관리 부담        |
| 자동 수정            | 점검·보고까지만. 수정은 사람이 판단(오작동 위험 회피)  |

특히 마지막 "자동 수정 제외"는 중요한 결정입니다. 도구가 발견을 보고하는 데서 멈추고 고치지는 않습니다 — 무인 자동화에서 코드를 멋대로 고치는 것은 위험하기 때문입니다. 이는 ch\_11의 "관측을 더하되 차단은 신중히"라는 안전 설계 철학과 같은 맥락입니다.

## 흔한 함정: 범위를 안 정하고 시작하기

"일단 만들면서 정하자"는 종합 프로젝트를 미완으로 끝내는 가장 흔한 원인입니다. 외부 URL 검사 하나를 넣는 순간 네트워크·타임아웃·재시도·비결정성까지 줄줄이 달려 와 MVP를 못 끝냅니다. 범위 밖 항목을 **미리 글로 적어** 두면, 구현 중 유혹이 생겨도 "이건 Out으로 정했지"라며 멈출 수 있습니다.

이 범위 정의는 MVP로도 압축됩니다. **MVP**는 "대상 경로를 받아 깨진 로컬 링크와 TODO 수를 세어 텍스트로 보고하고, 문제가 있으면 종료 코드 1을 반환하는 최소 버전"입니다. JSON 리포트·파일 크기·문서 점검·CI·스케줄은 MVP 이후 증분으로 확장합니다 — 이 증분 순서가 다음 챕터(설계)와 ch\_16(구현)의 마일스톤이 됩니다.

## ch\_14\_s06. 추적성: 요구사항 ↔ 단원 개념 매핑

종합 프로젝트의 핵심은 **앞에서 배운 개념을 실제로 통합 적용하는** 것입니다. 그래서 각 요구사항이 어느 단원·개념으로 구현되는지를 명시적으로 잇습니다. 이 **추적성 매핑(traceability mapping)**은 두 방향으로 쓸모가 있습니다. 첫째, 커리큘럼이 빠짐없이 프로젝트에 쓰이는지 확인합니다(커버리지). 둘째, 구현할 때 "이 기능엔 어떤 기술을 쓰지?"의 답을 미리 줍니다.

### 기능 요구사항 ↔ 개념 매핑

| 요구사항             | 사용 단위  | 사용 개념                    |
|------------------|--------|--------------------------|
| FR1 인자 파싱        | u1, u2 | c02 프롬프팅, c03 파일 탐색·편집   |
| FR2 링크 점검        | u2     | c03 파일 탐색·편집             |
| FR3 파일 크기 점검     | u2     | c03 파일 탐색·편집             |
| FR4 필수 문서 확인     | u2     | c03 파일 탐색·편집             |
| FR5 TODO 수집      | u2     | c03 파일 탐색·편집             |
| FR6 텍스트·JSON 리포트 | u2, u4 | c03 파일 탐색·편집, c07 Git·커밋 |
| FR7 종료 코드 판정     | u6     | c13 CI·자동화               |
| FR8 PR 자동 실행     | u6     | c13 CI·자동화               |
| FR9 주간 스케줄       | u6     | c13 CI·자동화               |

## 비기능 요구사항 ↔ 개념 매핑

| 요구사항         | 사용 개념                  |
|--------------|------------------------|
| NFR1 무의존     | c10 MCP 연동, c13 CI·자동화 |
| NFR2 단위 테스트  | c09 서브에이전트 위임          |
| NFR3 안전 게이트  | c11 혹                  |
| NFR4 최소 권한   | c04 권한·allowlist       |
| NFR5 결정론·재현성 | c11 혹, c13 CI·자동화      |
| NFR6 영속 컨텍스트 | c05 CLAUDE.md·컨텍스트     |

## 프로젝트를 만드는 워크플로우 ↔ 개념 매핑

도구의 기능뿐 아니라, 그것을 **Claude Code**로 만드는 과정도 개념을 씁니다. 다음은 만드는 단계와 개념의 연결입니다.

| 만드는 단계                 | 사용 개념         |
|------------------------|---------------|
| 프로젝트 규칙·명령 영속화         | c05 CLAUDE.md |
| 변경 전 단계별 계획 받기         | c06 플랜 모드     |
| 작은 논리 단위 커밋            | c07 Git·커밋    |
| 반복 점검을 /audit 커맨드로 표준화 | c08 슬래시 커맨드   |
| 코드 리뷰·테스트 위임           | c09 서브에이전트    |
| 점검 정책 문서 참조            | c10 MCP       |
| 위험 편집 차단 게이트           | c11 혹은        |
| 조사→구현→리뷰 파이프라인         | c12 오케스트레이션   |
| 헤드리스 CI·스케줄 무인화        | c13 CI·자동화    |

세 표를 합치면 **c01~c13 열세 개 개념이 모두** repo-audit의 어딘가에 쓰입니다. c01(설치·첫 세션)과 c02(프롬프팅)는 모든 단계의 바탕이고, c03~c13은 각 기능·단계에 직접 대응합니다. 이렇게 커리큘럼이 프로젝트로 빠짐 없이 흘러드는 것을 눈으로 확인하는 것이 추적성 매핑의 목적입니다.

추적성 표는 ch\_16(구현)에서 **마일스톤별 사용 개념**으로 다시 등장하고, ch\_17(성찰)에서 "13개 개념을 어떻게 적용했는가"의 회고 자료가 됩니다. 지금 잘 매핑해 두면 프로젝트 끝까지 이 표를 계속 활용합니다.

## ch\_14\_s07. 작성형 연습(2종 1세트): 요구사항 표 채우기·자가체크

이제 직접 분석해 봅니다. 모범 예시를 먼저 보고, 빈 템플릿을 채운 뒤, 자가체크로 점검하는 **2종 1세트** 방식입니다.

### 모범 예시 — 하나의 FR을 완전한 형식으로 작성

요구사항 하나를 "ID·문장·완료 조건·사용 개념"의 네 항목으로 빠짐없이 적은 예시입니다.

| 항목        | 내용                                                              |
|-----------|-----------------------------------------------------------------|
| ID        | FR2                                                             |
| 요구사항(동사로) | 마크다운 파일에서 로컬 링크.앵커를 추출해 깨진 링크를 검출한다                             |
| 완료 조건(검증) | 존재하지 않는 파일을 가리키는 [x](./none.md) 를 error로 보고하고, 외부 URL은 검사하지 않는다 |
| 사용 개념     | c03 파일 탐색과 편집 워크플로우(u2)                                         |

이처럼 **동사로 시작하는 문장 + 판정 가능한 완료 조건 + 매핑된 개념**을 갖추면, 그대로 ch\_02의 작업 지시(목표·맥락·제약·완료 조건)로 옮겨 에이전트에게 줄 수 있습니다.

## 직접 작성 1 — 새 요구사항을 정의하고 개념에 매핑하기

repo-audit에 추가하고 싶은 **새 점검 항목**을 하나 떠올려, 아래 표의 빈칸(\_\_\_\_)을 채워 완전한 요구사항으로 적어 보세요. 예: "빈 디렉터리 검출", "오래된 파일 표시", "중복 파일 탐지" 등.

| 항목             | 내용                             |
|----------------|--------------------------------|
| ID             | FR____                         |
| 요구사항(동사로 시작)   | ____                           |
| 완료 조건(판정 가능하게) | <u>  일 때  </u> 로 보고한다          |
| 범위 In/Out 판단   | 이 항목은 (In / Out)이며, 그 이유는 ____ |
| 사용 단위·개념       | c_ _                           |

작성 힌트:

- 요구사항 문장은 "명사"가 아니라 "동사"로 시작합니다(예: "검출한다", "수집한다").
- 완료 조건은 "됐다/안 됐다"를 분명히 가를 수 있게 적습니다.
- 대부분의 점검 항목은 파일을 읽어 판단하므로 c03(파일 탐색·편집)에 매핑됩니다. 자동화·종료 코드와 연하면 c13도 함께 적습니다.

## 직접 작성 2 — 비기능 요구사항 하나를 근거와 함께 추가하기

repo-audit가 무인 CI에서 안전·견고하게 돌기 위한 **새 비기능 요구사항**을 하나 정해, 빈칸을 채워 보세요. 예: "점검 시간이 일정 한도를 넘지 않는다", "로그를 남겨 추적 가능하다" 등.

| 항목                | 내용                               |
|-------------------|----------------------------------|
| ID                | NFR____                          |
| 비기능 요구사항          | _____                            |
| 왜 필요한가(무인 자동화 관점) | _____                            |
| 어떤 개념으로 보장하나      | c_ _ (예: c11 혹은 c04 권한 / c13 CI) |
| 위배되면 생기는 문제       | _____                            |

작성 힌트:

- 비기능 요구사항은 "무엇을 한다"가 아니라 "어떤 품질을 갖는다"입니다(안전·재현성·성능·일관성 등).
- "위배되면 생기는 문제"를 적어 보면 그 요구사항이 정말 필요한지 스스로 검증됩니다.

## 자가체크

- [ ] 작성한 요구사항 문장이 동사로 시작하고, 완료 조건이 판정 가능한가
- [ ] 각 요구사항을 c01~c13 중 적절한 개념에 매핑했는가(근거 포함)
- [ ] 새 항목이 범위 In/Out 중 어디에 속하는지 이유와 함께 정했는가
- [ ] 비기능 요구사항에 "왜 필요한가"와 "위배 시 문제"를 모두 적었는가
- [ ] 모범 예시(FR2)처럼 ID·문장·완료 조건·사용 개념의 네 항목을 빠짐없이 채웠는가

## 마무리

이번 챕터에서는 종합 프로젝트 repo-audit의 **분석**을 수행했습니다. 드라이빙 퀘스천과 POV로 방향을 다시 잡고, 막연한 아이디어를 **기능 요구사항 9건(FR1~FR9)**과 **비기능 요구사항 6건(NFR1~NFR6)**으로 구체화했으며, **사용자 스토리 4건·유스케이스 3건**으로 사용 흐름을 그렸습니다. 또 **범위 In/Out**을 못 박아 프로젝트가 끝없이 불어나지 않게 통제하고, **요구사항 ↔ 단위 개념 추적성 표**로 c01~c13이 모두 프로젝트에 쓰임을 확인했습니다.

분석이 또렷해진 만큼, 다음 챕터(ch\_15)에서는 이 요구사항을 **어떻게 만들지**를 설계합니다. 점검 결과·리포트의 데이터 구조를 정하고, 기능을 모듈·함수로 분해하며, 다섯 건의 설계 결정을 **ADR 5필드**로 기록하고, 에이전트 오케스트레이션·CI 흐름을 다이어그램으로 표현합니다. 무엇을 만들지(분석)를 끝냈으니, 이제 어떻게 만들지(설계)로 넘어갈 차례입니다.



## 종합 프로젝트: repo-audit — 설계(데이터·모듈·ADR·에이전트 워크플로우)

앞 챕터(ch\_14)에서 repo-audit가 **무엇을 해야 하는지**를 요구사항 9건(FR)·6건(NFR)과 범위로 또렷하게 정했습니다. 이번 챕터는 종합 프로젝트의 두 번째 단계인 **설계(design)**입니다. 같은 질문을 이번에는 "**어떻게** 만들 것인가"로 바꿔, 코드를 짜기 전에 구조를 먼저 그립니다. 점검 결과를 담은 **데이터 구조**를 정하고, 기능을 **모듈·함수**로 나누며, 중요한 갈림길마다 내린 결정을 **ADR**로 기록하고, 이 도구를 만드는 **에이전트 오케스트레이션·CI 흐름**을 다이어그램으로 표현합니다.

설계가 왜 따로 필요할까요? 분석이 "목적지"라면 설계는 "지도"입니다. 지도 없이 출발하면 같은 곳을 빙빙 돌거나, 다 짓고 나서야 구조가 잘못된 걸 발견합니다. 특히 이 도구는 Claude Code로 만들 것이므로, 잘 그린 설계는 곧 ch\_06의 **플랜 모드**에 줄 좋은 입력이 됩니다 — 데이터 구조와 모듈 경계가 분명할수록 에이전트가 세우는 구현 계획도 정확해집니다.

### 이 챕터의 학습 목표

- 점검 결과·리포트의 데이터 구조(finding-report-config)를 설계할 수 있습니다.
- 기능을 책임이 분명한 함수와 모듈로 분해할 수 있습니다.
- 중요한 설계 결정을 ADR 5필드(결정·맥락·선택지·근거·결과)로 기록할 수 있습니다.
- 에이전트 오케스트레이션과 CI 처리 흐름을 다이어그램으로 표현할 수 있습니다.

이 챕터도 종합 프로젝트(capstone) 챕터입니다. 새 Claude Code 기능을 가르치지 않고, ch\_14의 분석을 받아 c01~c13 개념을 적용한 구조 설계를 합니다.

## ch\_15\_s01. 설계 개요: 무엇을 어떻게 나눌까

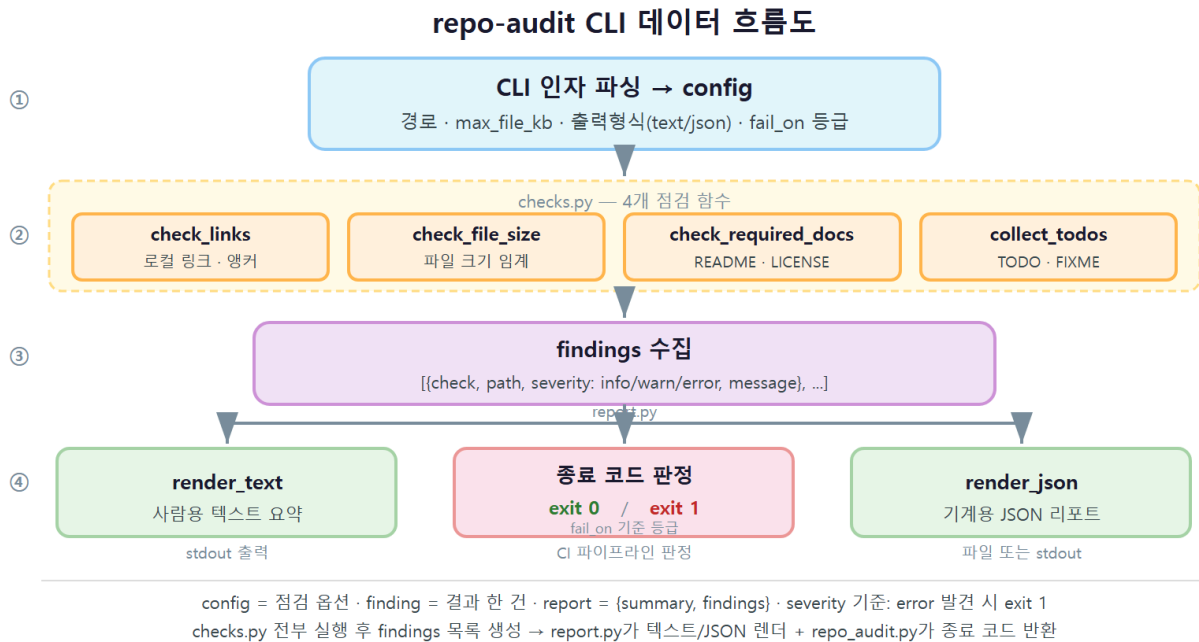
좋은 설계의 출발점은 **관심사 분리(separation of concerns)**입니다. 하나의 큰 스크립트에 모든 걸 욕여넣는 대신, 책임이 다른 일을 다른 자리로 나눕니다. repo-audit는 크게 세 가지 책임으로 나뉩니다.

1. **점검(check)**: 저장소를 읽어 문제를 찾아낸다 — 링크·파일 크기·문서·TODO.
2. **표현(report)**: 찾아낸 문제를 사람용 텍스트와 기계용 JSON으로 보여 준다.
3. **조율(orchestrate)**: 인자를 받아 점검을 호출하고, 결과를 모아 종료 코드를 정한다.

이 세 책임이 그대로 세 모듈(`checks.py` · `report.py` · `repo_audit.py`)이 되고, 점검 함수의 정확성을 검증하는 `tests/`가 더해집니다. 책임을 나누면 ch\_03에서 배운 **외과적 편집**도 쉬워집니다 — 링크 점검을 고칠 때 `checks.py`만 건드리면 되고, 리포트 형식을 바꿀 때 `report.py`만 건드리면 됩니다. 변경의 영향 범위가 좁아지는 것이 모듈화의 가장 큰 이득입니다.

설계의 흐름은 다음 순서로 진행합니다. 먼저 **데이터 구조(s02)**를 정해 "무엇을 주고받을지"의 공통 언어를 만들고, 그 데이터를 다루는 **모듈·함수(s03)**로 책임을 나눈 뒤, 갈림길에서 내린 **결정을 ADR(s04)**로 남기고, 마지막으로 이 모든 걸 만드는 **에이전트·CI 워크플로우(s05)**를 그립니다.

## ch\_15\_s02. 데이터 구조 설계: finding · report · config



모듈을 나누기 전에 먼저 정해야 할 것이 있습니다. 모듈들이 **주고받는 데이터의 모양**입니다. 점검 함수가 무엇을 돌려주고, 리포트 함수가 무엇을 받는지를 합의해 두지 않으면 모듈을 따로 만들 수 없습니다. repo-audit의 데이터는 세 가지입니다 — 점검 결과 한 건(finding), 전체 리포트(report), 점검 옵션(config).

### finding — 점검 결과 한 건

점검에서 발견한 문제 하나를 표현하는 단위입니다. 파이썬 **딕셔너리**로 표현합니다.

```
# 점검 결과 한 건 = finding (딕셔너리)
finding = {
    "check": "links",          # 어느 점검에서 나왔나(links/file_size/required_docs/todos)
    "path": "docs/guide.md",  # 문제가 있는 파일 경로
    "severity": "error",      # 심각도: info / warn / error
    "message": "깨진 로컬 링크: ./none.md (대상 파일 없음)", # 사람이 읽는 설명
}
```

- "check": 어떤 점검에서 나온 발견인지 분류합니다. 리포트에서 점검별로 묶을 때 씁니다.
- "path": 문제가 있는 파일의 상대 경로. 사용자가 바로 찾아갈 수 있게 합니다.
- "severity": info · warn · error 세 등급. 종료 코드 판정(FR7)의 기준이 됩니다.
- "message": 사람이 읽는 설명. 무엇이·왜 문제인지 한 줄로 적습니다.

네 키를 딕셔너리로 묶는 이유는 ADR1에서 결정으로 다룹니다. 핵심은 **JSON 직렬화가 자연스럽게 단순하다**는 점입니다(FR6의 JSON 리포트와 직결).

## report — 전체 리포트

점검을 모두 돌리면 발견(finding)이 여러 건 모입니다. 이를 요약과 함께 묶은 것이 report입니다.

```
# 전체 리포트 = report (딕셔너리)
report = {
  "summary": {"info": 0, "warn": 1, "error": 2}, # 등급별 개수 집계
  "findings": [ # finding 들의 리스트
    {"check": "links", "path": "docs/guide.md",
     "severity": "error", "message": "깨진 로컬 링크: ./none.md"},
    {"check": "file_size", "path": "data/big.bin",
     "severity": "warn", "message": "파일 크기 초과: 700.0KB (임계치: 500KB)"},
    {"check": "required_docs", "path": ".",
     "severity": "error", "message": "필수 문서 없음: LICENSE"},
  ],
}
```

- "summary": 등급별 개수를 집계한 딕셔너리. 텍스트 요약 맨 위에 "error 2, warn 1"처럼 보여 줍니다.
- "findings": 발견을 모두 담은 리스트. 텍스트·JSON 리포트가 모두 이 리스트를 순회해 출력합니다.
- 같은 report 딕셔너리 하나로 텍스트와 JSON을 둘 다 만들 수 있습니다(ADR2). 데이터와 표현이 분리되어 있기 때문입니다.

## config — 점검 옵션

점검 동작을 바꾸는 설정값입니다. CLI 인자(FR1)에서 채워집니다.

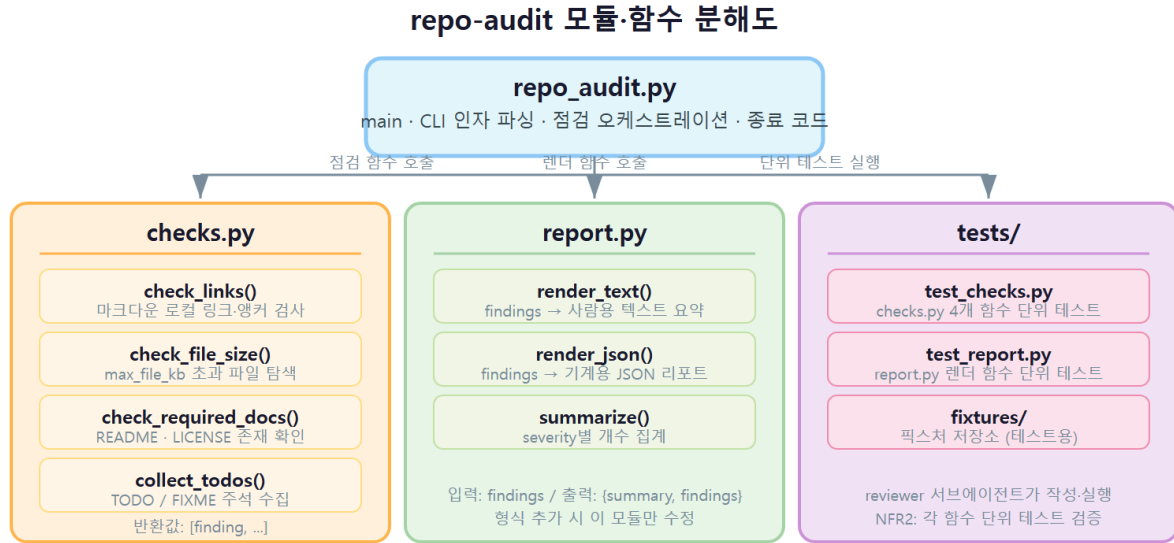
```
# 점검 옵션 = config (딕셔너리)
config = {
  "max_file_kb": 500, # 이 크기(KB)를 넘으면 경고
  "required_docs": ["README", "LICENSE"], # 있어야 하는 필수 문서
  "fail_on": "error", # 이 등급 이상이 있으면 exit 1
}
```

- "max\_file\_kb": 파일 크기 점검(FR3)의 임계. --max-file-kb 인자로 바꿉니다.
- "required\_docs": 필수 문서 점검(FR4)의 목록. 저장소 정책에 맞게 조정합니다.
- "fail\_on": 종료 코드 판정(FR7)의 기준 등급. error 면 error가 하나라도 있을 때 exit 1 입니다.

세 데이터 구조가 모듈 간의 **공통 언어**입니다. 점검 함수는 config를 받아 finding 리스트를 돌려주고, report 함수는 finding 리스트를 받아 텍스트·JSON을 만듭니다. 이 계약을 먼저 못 박았기에 모듈을 따로 만들 수 있습니다.

세 데이터를 모두 딕셔너리로 통일하면 NFR5(결정론·재현성)도 챙기기 쉽습니다. finding 리스트를 (check, path) 기준으로 정렬해 출력하면, 같은 저장소는 항상 같은 순서의 리포트를 냅니다. 파일 탐색 순서가 OS마다 달라도 결과가 흔들리지 않게 하는 작은 장치입니다.

## ch\_15\_s03. 모듈·함수 분해(repo\_audit · checks · report · tests)



관심사 분리: 점검(checks.py) · 표현(report.py) · 조율(repo\_audit.py) · 검증(tests/)  
 변경 영향 범위 최소화 — 링크 점검 수정 시 checks.py만, 리포트 형식 수정 시 report.py만 편집  
 repo\_audit.py = main 진입점 / checks.py·report.py·tests/ 모두 단독 테스트 가능한 독립 모듈

데이터 구조라는 공통 언어가 정해졌으니, 이제 **책임을 모듈로 나눕니다**. s01에서 본 세 책임(점검·표현·조율)에 검증(테스트)을 더해 네 자리로 분해합니다.

| 모듈            | 책임                         | 주요 함수                                                                    |
|---------------|----------------------------|--------------------------------------------------------------------------|
| repo_audit.py | CLI 인자 파싱·점검 오케스트레이션·종료 코드 | main(), parse_args(), run_audit()                                        |
| checks.py     | 개별 점검 함수(finding 리스트 반환)   | check_links(), check_file_size(), check_required_docs(), collect_todos() |
| report.py     | 리포트 렌더와 summary 집계         | render_text(), render_json(), summarize()                                |
| tests/        | 각 점검 함수의 단위 테스트            | test_checks.py (픽스처 저장소 사용)                                              |

### checks.py — 점검 함수의 공통 계약

점검 함수들은 모두 **같은 모양의 계약**을 따릅니다. 입력으로 대상 경로와 config를 받고, 출력으로 finding 리스트를 돌려줍니다. 이 일관된 시그니처 덕에 오케스트레이터가 모든 점검을 똑같이 호출할 수 있습니다.

```
# checks.py - 점검 함수의 공통 시그니처(설계 골격)
def check_links(root: str, config: dict) -> list[dict]:
    """마크다운의 로컬 링크·앵커를 검사해 finding 리스트를 돌려준다."""
    findings = []
    # 1) root 아래 .md 파일을 찾는다(파일 탐색, c03)
    # 2) 각 파일에서 [텍스트](경로) 패턴을 추출한다(re)
    # 3) 외부 URL이 아닌 로컬 경로의 대상 파일 존재를 확인한다(ADR3)
    # 4) 대상 파일이 없으면 finding(severity="error")을 append 한다
    # 5) 파일은 있으나 #앵커가 헤딩에 없으면 severity="warn"으로 보고한다
    return findings

def check_file_size(root: str, config: dict) -> list[dict]:
    """config["max_file_kb"]를 넘는 파일을 warn으로 보고한다."""
    findings = []
    return findings

def check_required_docs(root: str, config: dict) -> list[dict]:
    """config["required_docs"]가 모두 있는지 확인한다."""
    findings = []
    return findings

def collect_todos(root: str, config: dict) -> list[dict]:
    """소스의 주석 태그를 수집한다(TODO=info, FIXME=error로 경중 구분)."""
    findings = []
    return findings
```

- 네 함수 모두 (root, config) -> list[dict] 라는 **같은 계약**을 따릅니다. 오케스트레이터가 함수 목록을 순회하며 똑같이 부를 수 있게 하는 설계입니다.
- 함수 안의 주석은 구현 순서를 미리 적은 것입니다. ch\_16에서 이 골격을 채웁니다.
- check\_links 의 4단계는 ch\_03의 "탐색(찾기)→읽기→판단" 루프를 그대로 따릅니다. 새 기법이 아니라 배운 작업 루프를 코드로 옮긴 것입니다.

## repo\_audit.py — 오케스트레이션과 종료 코드

본체는 인자를 파싱하고, 점검 함수들을 모두 돌려 finding을 모으고, 리포트를 출력한 뒤 종료 코드를 정합니다.

```
# repo_audit.py - 오케스트레이션 골격
import sys
import checks
import report

CHECKS = [
    checks.check_links,
    checks.check_file_size,
    checks.check_required_docs,
    checks.collect_todos,
]

def run_audit(root: str, config: dict) -> dict:
    """모든 점검을 돌려 report 딕셔너리를 만든다."""
    findings = []
    for check in CHECKS:          # 같은 계약이라 똑같이 호출
        findings.extend(check(root, config))
    findings.sort(key=lambda f: (f["check"], f["path"])) # 결정론(NFR5)
    return {"summary": report.summarize(findings), "findings": findings}

def main() -> int:
    """인자 파싱 -> 점검 -> 리포트 -> 종료 코드."""
    root, config, fmt = parse_args(sys.argv[1:])
    result = run_audit(root, config)
    if fmt == "json":
        print(report.render_json(result))
    else:
        print(report.render_text(result))
    # fail_on 등급 이상이 있으면 비정상 종료(FR7)
    if result["summary"].get(config["fail_on"], 0) > 0:
        return 1
    return 0

if __name__ == "__main__":
    sys.exit(main())          # 종료 코드를 셸·CI에 전달
```

- CHECKS 리스트: 점검 함수를 한곳에 모아 둡니다. 새 점검을 추가하려면 이 리스트에 함수만 더하면 되어 확장이 쉽습니다.
- for check in CHECKS: findings.extend(...): 공통 계약 덕에 모든 점검을 같은 방식으로 호출합니다.
- findings.sort(...): 발견을 (check, path) 로 정렬해 같은 입력이 같은 출력을 내게 합니다(NFR5).
- sys.exit(main()): main() 이 돌려준 종료 코드를 셸·CI에 전달합니다. 이것이 헤드리스 판정(c13)의 핵심 인터페이스입니다.

## tests/ — 점검 함수의 단위 테스트

각 점검 함수가 정확히 동작하는지 **픽스처 저장소**(일부러 문제를 심은 작은 가짜 저장소)로 검증합니다. 이 테스트 작성·실행은 ch\_09의 서버에이전트(reviewer/tester)에 위임합니다(NFR2).

```
# tests/test_checks.py - 단위 테스트 골격
import checks

def test_check_links_detects_broken(tmp_path):
    """깨진 로컬 링크를 error로 잡는지 검증한다."""
    md = tmp_path / "a.md"
    md.write_text("[x](./none.md)", encoding="utf-8")
    findings = checks.check_links(str(tmp_path), {})
    assert any(f["severity"] == "error" for f in findings)
```

- `tmp_path`: 임시 디렉터리에 가짜 저장소를 만들어, 외부 환경에 의존하지 않고 결정론적으로 테스트합니다.
- `assert any(...)`: 깨진 링크가 error finding으로 잡혔는지 확인합니다. 점검 함수의 계약이 지켜지는지를 검증하는 것입니다.

이 모듈 분해는 ch\_14의 추적성 매핑과 맞물립니다 — `checks.py` 는 c03(파일 탐색·편집), `tests/` 는 c09(서브에이전트), 종료 코드는 c13(CI-자동화)에 대응합니다. 설계가 분석과 어긋나지 않는지 확인하는 좋은 점검입니다.

## ch\_15\_s04. 설계 결정 기록: ADR 5건

설계 중에는 "둘 중 무엇을 고를까" 하는 갈림길이 여러 번 나옵니다. 그때 무엇을, 왜, 어떤 대안을 제치고 골랐는지를 남기지 않으면, 몇 주 뒤(또는 다른 팀원이) "왜 이렇게 했지?"를 알 수 없습니다. **ADR(Architecture Decision Record)**은 이 결정을 짧게 기록하는 표준 양식입니다. 우리는 **5필드 (결정·맥락·선택지·근거·결과)**로 적습니다.

repo-audit 설계에서 내린 핵심 결정 다섯 건입니다.

### ADR1 — 점검 결과 표현 방식

| 필드  | 내용                                                   |
|-----|------------------------------------------------------|
| 결정  | 점검 결과 한 건을 딕셔너리(finding)로 표현한다                       |
| 맥락  | 결과 한 건은 점검명·경로·등급·메시지를 갖는다                           |
| 선택지 | ① 딕셔너리 ② 데이터클래스 ③ 튜플                                 |
| 근거  | JSON 직렬화(FR6)가 자연스럽고 단순하다 → 딕셔너리(필요 시 dataclass로 확장) |
| 결과  | 키 오타 위험은 상수·테스트(NFR2)로 방어한다                          |

### ADR2 — 리포트 형식

| 필드  | 내용                                        |
|-----|-------------------------------------------|
| 결정  | 텍스트 요약과 JSON을 둘 다 제공한다                    |
| 맥락  | 사람(개발자)과 CI가 결과를 모두 소비한다(US1·UC2)         |
| 선택지 | ① 텍스트만 ② JSON만 ③ 둘 다                      |
| 근거  | 텍스트는 사람용, JSON은 CI·아티팩트 소비용 → 둘 다         |
| 결과  | 렌더러를 report.py로 분리해 형식 추가(예: SARIF)에 유연하다 |

### ADR3 — 외부 URL 검사 포함 여부

| 필드  | 내용                                                  |
|-----|-----------------------------------------------------|
| 결정  | 로컬 링크·앵커만 검사하고 외부 URL은 제외한다                         |
| 맥락  | 외부 URL은 네트워크 의존·비결정적이다(NFR5와 충돌)                    |
| 선택지 | ① 포함 ② 제외 ③ 옵션화                                     |
| 근거  | 결정론·재현성·무의존(NFR1·NFR5)을 우선 → 외부 URL 제외(향후 옵션 확장 여지) |
| 결과  | 외부 깨진 링크는 못 잡지만 CI 안정성이 높아진다                        |

### ADR4 — CI 통과/실패 판정 방식

| 필드  | 내용                                   |
|-----|--------------------------------------|
| 결정  | 심각도 임계(fail_on)에 따른 종료 코드로 판정한다      |
| 맥락  | 헤드리스·CI가 결과를 스스로 판정해야 한다(FR7·UC2)    |
| 선택지 | ① 종료 코드 ② 리포트 파싱 ③ 주석 코멘트            |
| 근거  | 종료 코드가 가장 단순·표준적 → error 발견 시 exit 1 |
| 결과  | 임계 기준을 명확히 문서화해야 오탐·미탐을 통제할 수 있다     |

### ADR5 — Claude Code 워크플로우 구조

| 필드  | 내용                                                         |
|-----|------------------------------------------------------------|
| 결정  | 플랜 모드로 설계→구현, 서브에이전트로 테스트·리뷰, 혹은로 안전 게이트, CI로 무인화한다        |
| 맥락  | 도구를 에이전트 워크플로우로 신뢰성 있게 만들어야 한다(드라이빙 퀘스천)                   |
| 선택지 | ① 단일 세션 수작업 ② 서브에이전트·혹·CI 오케스트레이션                          |
| 근거  | 계획-실행 분리(c06)·전문 위임(c09)·결정론 게이트(c11)·무인화(c13)를 결합해 신뢰성 확보 |
| 결과  | 스캐폴딩(에이전트·혹·CI) 관리 비용이 들지만 재현성·안전성이 높아진다                   |

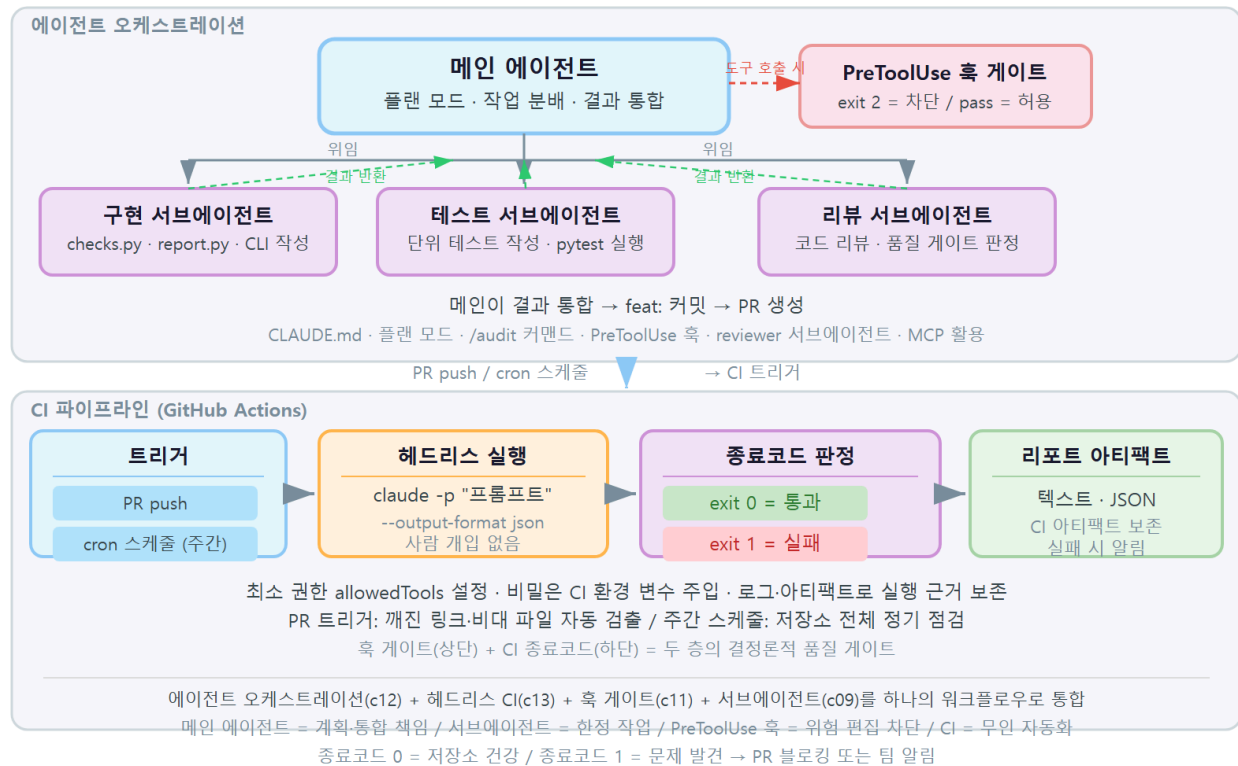
## 좋은 ADR의 특징

- **선택지를 적는다**: "딕셔너리로 했다"만 적으면 왜 데이터클래스를 안 썼는지 알 수 없습니다. 제친 대안을 함께 적어야 결정의 무게가 보입니다.
- **근거를 요구사항에 잇는다**: ADR3의 근거가 NFR5(결정론)를 가리키듯, 결정이 요구사항에 닿으면 "취향"이 아니라 "근거 있는 선택"이 됩니다.
- **결과(트레이드오프)를 솔직히 적는다**: ADR3은 "외부 링크는 못 잡는다"는 손해를 인정합니다. 완벽한 결정은 없고, 무엇을 포기했는지 아는 것이 중요합니다.

이 다섯 ADR이 설계의 핵심 갈림길을 모두 기록합니다. ch\_16(구현)에서 코드를 짜다가 "왜 딕셔너리지?", "왜 외부 링크는 안 잡지?" 같은 질문이 들면 이 ADR을 다시 펼치면 됩니다.

## ch\_15\_s05. 에이전트 오케스트레이션·CI 파이프라인 흐름도

## 에이전트 오케스트레이션 · CI 파이프라인



마지막으로, 이 도구를 **Claude Code**로 어떻게 만들지의 워크플로우를 설계합니다. ADR5에서 결정한 대로, repo-audit는 단일 세션 수작업이 아니라 **여러 에이전트·게이트·자동화를 엮은 오케스트레이션**으로 만듭니다. 이것이 종합 프로젝트가 c01~c13을 통합하는 무대입니다.

## 만드는 과정의 오케스트레이션

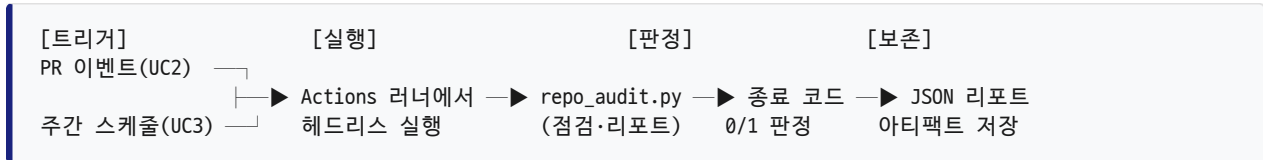
메인 에이전트(오케스트레이터)가 계획·통합을 맡고, 전문 서브에이전트가 한정 작업을 처리하며, 혹이 안전을 강제하는 분업입니다(c12).

| 단계        | 담당               | 사용 개념                    |
|-----------|------------------|--------------------------|
| 1. 셋업·계획  | 메인 + 플랜 모드       | c05 CLAUDE.md, c06 플랜 모드 |
| 2. 구현     | 메인(외과적 편집·커밋)    | c03 편집, c07 커밋           |
| 3. 테스트    | tester 서브에이전트    | c09 서브에이전트               |
| 4. 리뷰     | reviewer 서브에이전트  | c09 서브에이전트               |
| 5. 안전 게이트 | PreToolUse 혹(상시) | c11 혹, c04 권한            |
| 6. 반복 표준화 | /audit 커맨드       | c08 슬래시 커맨드              |
| 통합        | 메인이 결과를 모아 정리    | c12 오케스트레이션              |

흐름으로 풀면 이렇습니다. 메인 에이전트가 CLAUDE.md로 규칙을 읽고(c05) 플랜 모드로 단계 계획을 받아 확정합니다(c06). 그 계획대로 외과적으로 편집·커밋하며 구현하고(c03-c07), 점검 함수가 생길 때마다 tester 서버 에이전트에 단위 테스트를, reviewer 서버에이전트에 코드 리뷰를 위임합니다(c09). 이 모든 편집은 PreToolUse 혹은 위험 경로를 막는 가운데(c11) 진행되고, 반복 점검 실행은 `/audit` 커맨드로 표준화합니다(c08). 메인은 각 서버에이전트의 결과를 통합해 다음 단계를 정합니다(c12).

## CI 파이프라인 흐름

완성된 도구는 헤드리스(-p) 모드로 GitHub Actions에 올라 무인 실행됩니다(c13).



- **트리거**: PR 이벤트(FR8)와 주간 스케줄(FR9)이 같은 파이프라인을 부릅니다. UC2·UC3가 코어를 공유하는 ch\_14의 설계가 그대로 드러납니다.
- **실행**: Actions 러너가 `repo_audit.py` 를 헤드리스로 돌립니다. 표준 라이브러리만 쓰므로(NFR1) 설치 단계가 거의 없어 빠릅니다.
- **판정**: 종료 코드(ADR4·FR7)로 체크를 통과/실패시킵니다. 사람이 보지 않아도 결과가 판정됩니다.
- **보존**: JSON 리포트(ADR2)를 아티팩트로 남겨 이력을 추적합니다.

두 흐름(에이전트 오케스트레이션 + CI 파이프라인)을 합치면 드라이빙 퀘스천의 답이 그려집니다 — **"분석부터 CI 자동화까지 하나의 에이전트 워크플로우"**가 도구를 만드는 과정(오케스트레이션)과 만들어진 도구가 도는 과정(CI)으로 양쪽 모두 실현됩니다.

오케스트레이션은 강력하지만 공짜가 아닙니다. 서버에이전트·혹·CI 스캐폴딩이 늘수록 관리 비용도 늘어납니다(ADR5의 결과). 단순한 변경은 메인 에이전트가 직접 처리하는 게 낫고, 테스트·리뷰처럼 경계가 분명한 작업만 위임합니다 — ch\_12에서 배운 "단순 작업엔 단일 에이전트가 낫다"를 잊지 마세요.

## ch\_15\_s06. 작성형 연습(2종 1세트): ADR 작성·자가체크

이제 직접 설계 결정을 기록해 봅니다. 모범 ADR을 먼저 보고, 빈 템플릿을 채운 뒤, 자가체크로 점검하는 **2종 1세트** 방식입니다.

### 모범 예시 — 한 결정을 ADR 5필드로 완성

본문에 없던 새 결정을 ADR로 적은 예시입니다. "TODO 점검의 심각도를 어떻게 줄까"라는 갈림길을 기록했습니다.

| 필드  | 내용                                                      |
|-----|---------------------------------------------------------|
| 결정  | TODO/FIXME 발견은 info 등급으로 보고하고 종료 코드에 영향을 주지 않는다         |
| 맥락  | TODO는 정상적인 작업 흔적이라 CI를 실패시키면 마찰이 크다                     |
| 선택지 | ① info(실패 안 함) ② warn ③ error(실패시킴)                     |
| 근거  | TODO는 정보 제공이 목적 → info로 보고하되 fail_on=error와 분리(FR5-FR7) |
| 결과  | TODO가 많아도 CI는 통과하며, 필요하면 옵션으로 임계를 올릴 여지를 남긴다            |

다섯 필드가 모두 채워졌고, 근거가 요구사항(FR5-FR7)에 닿으며, 결과에 트레이드오프(옵션 확장 여지)를 남긴 점에 주목하세요.

## 직접 작성 1 — 새 설계 결정을 ADR 5필드로 작성하기

repo-audit를 만들며 마주칠 법한 **새 갈림길**을 하나 골라, 아래 빈칸(\_\_\_\_)을 채워 ADR을 완성하세요. 예시 주제: "리포트 출력 인코딩", "점검 대상에서 제외할 디렉터리(.git,node\_modules)", "JSON 리포트의 스키마 버전 표기" 등.

| 필드  | 내용                              |
|-----|---------------------------------|
| 제목  | ADR_ _                          |
| 결정  | ____ 로 한다                       |
| 맥락  | ____ 한 상황이다                     |
| 선택지 | ① _ ② _ (③ ____)                |
| 근거  | _ 때문에 ① 대신 ②를 고른다 (요구사항 _ 와 연결) |
| 결과  | 이 결정으로 _ 는 좋아지지만 _ 는 포기한다       |

작성 힌트:

- 선택지를 둘 이상 적습니다. 제친 대안이 없으면 결정의 근거가 약합니다.
- 근거를 ch\_14의 요구사항(FR-NFR) 중 하나에 **연결**합니다("취향"이 아니라 "근거"가 되게).
- 결과에 **포기한 것(트레이드오프)**을 솔직히 적습니다.

## 직접 작성 2 — 만드는 워크플로우의 한 단계를 개념에 매핑하기

ch\_15\_s05의 오케스트레이션 표를 참고해, 도구를 만드는 **한 단계**를 골라 어떤 에이전트.게이트가 어떤 개념으로 그 단계를 처리하는지 빈칸을 채워 설계하세요.

| 항목                     | 내용                                  |
|------------------------|-------------------------------------|
| 단계 이름                  | ___ (예: 점검 함수 구현 / 리포트 렌더 / 안전 게이트) |
| 담당(메인/서브에이전트/혹/커맨드)    | ___                                 |
| 사용 개념                  | c_ _                                |
| 왜 이 담당·개념인가            | ___                                 |
| 통합 방법(메인이 결과를 어떻게 모으나) | ___                                 |

작성 힌트:

- 경계가 분명한 작업(테스트·리뷰)은 서브에이전트(c09), 상시 강제할 안전 규칙은 혹(c11), 반복 절차는 커맨드(c08)에 매핑하면 자연스럽습니다.
- 메인 에이전트는 직접 다 하지 않고 **계획·분배·통합**에 집중한다는 c12의 원칙을 떠올리세요.

## 자가체크

- [ ] 작성한 ADR이 5필드(결정·맥락·선택지·근거·결과)를 모두 채웠는가
- [ ] 선택지를 둘 이상 적고, 제친 대안의 이유가 드러나는가
- [ ] 근거가 ch\_14의 요구사항(FR·NFR)에 연결되는가
- [ ] 결과에 포기한 것(트레이드오프)을 솔직히 적었는가
- [ ] 워크플로우 단계의 담당과 사용 개념(c01~c13)이 적절히 매핑됐는가
- [ ] 데이터 구조(finding-report-config)와 모듈 경계가 분석(ch\_14)과 어긋나지 않는가

## 마무리

이번 챕터에서는 종합 프로젝트 repo-audit의 **설계**를 수행했습니다. 모듈 간 공통 언어가 될 **데이터 구조 3종(finding-report-config)**을 디렉터리로 정하고, 기능을 책임이 분명한 **네 모듈(repo-audit·checks-report-tests)**로 분해했으며, 핵심 갈림길의 결정을 **ADR 5건**으로 기록했습니다. 마지막으로 이 도구를 만드는 **에이전트 오케스트레이션**과 도구가 도는 **CI 파이프라인** 흐름을 그려, c01~c13이 어디에 쓰이는지 다시 확인했습니다.

설계도가 준비됐으니, 다음 챕터(ch\_16)에서는 이 구조를 **실제 코드로 구현**합니다. CLAUDE.md-플랜 모드 셋업부터 CLI 골격, 점검 함수, 리포트·종료 코드, 안전 게이트·외부 연동, CI-스케줄 무인화까지 여섯 개의 마일스톤(M1~M6)으로 증분 구현하고, 각 마일스톤이 어떤 단위 개념을 적용하는지 명시하며, 단위 테스트와 실행 결과로 동작을 검증합니다. 어떻게 만들지(설계)를 끝냈으니, 이제 직접 만들(구현) 차례입니다.



## 16장 종합 프로젝트: repo-audit — 구현(마일스톤 증분)

이 장은 종합 프로젝트의 **구현(implementation) 단계**입니다. 14장에서 요구사항(FR/NFR)과 범위를 정하고, 15장에서 데이터 구조·모듈·ADR·에이전트 워크플로우를 설계했습니다. 이제 그 설계를 **실제로 동작하는 코드로 옮깁니다**. 핵심은 한 번에 다 만들지 않고, 검증 가능한 작은 증분(마일스톤)으로 쌓아 올리는 것입니다.

이 장에서는 **새 개념을 배우지 않습니다**. 대신 1~13장에서 따로 익힌 13개 개념을 *하나의 완성을 만드는 과정에 통합* 적용합니다. 각 마일스톤마다 "지금 어떤 단원 개념을 쓰고 있는가"를 명시해, 흩어진 기능들이 어떻게 하나의 신뢰할 수 있는 자동화로 모이는지 직접 확인합니다.

### 이 장의 학습 목표

- CLAUDE.md·플랜부터 CI 무인화까지 증분 마일스톤으로 구현할 수 있습니다.
- 각 마일스톤에서 어떤 단원 개념을 적용했는지 설명할 수 있습니다.
- 단위 테스트·실행 결과로 동작을 확인하고 체크포인트로 검증할 수 있습니다.
- 마일스톤을 통합해 실행 가능한 완성 CLI를 만들 수 있습니다.

### 16.1 구현 전략과 마일스톤 개요

종합 프로젝트의 구현 전략은 "**걷는 해골(walking skeleton)**을 먼저, **살은 나중**에" 입니다. 처음부터 모든 점검을 완벽하게 만들려 하지 말고, 인자를 받아 한 가지 점검을 수행하고 종료 코드를 내는 **가장 작은 동작하는 버전**을 먼저 세웁니다(15장 ADR과 14장 MVP 정의를 기억하세요). 그다음 점검을 하나씩 추가하고, 리포트·안전 게이트·자동화를 증분으로 붙입니다. 이렇게 하면 매 단계가 **항상 실행 가능**하고, 어디서 무엇이 깨졌는지 즉시 알 수 있습니다.

각 마일스톤은 끝에 **체크포인트(checkpoint)**를 둡니다. 체크포인트는 "이 마일스톤이 끝났다"를 객관적으로 판정하는 기준이며, code 트랙이므로 **단위 테스트 통과·실행 결과 확인**으로 검증합니다. 체크포인트를 통과하지 못하면 다음 마일스톤으로 넘어가지 않습니다.

아래 표는 6개 마일스톤과 각 마일스톤이 적용하는 단원·개념을 정리한 것입니다. 이 표가 이 장 전체의 지도입니다.

| 마일스톤              | 산출물                                                     | 적용 단위    | 적용 개념                          | 체크포인트                                |
|-------------------|---------------------------------------------------------|----------|--------------------------------|--------------------------------------|
| M1 셋업·계획          | CLAUDE.md, PLAN.md                                      | u3       | c05(CLAUDE.md), c06(플랜 모드)     | 계획이 검증 가능한 단계로 승인, CLAUDE.md가 명령을 담음 |
| M2 CLI 골격·TODO    | repo_audit.py, checks.collect_todos, 첫 커밋               | u1·u2·u4 | c02(프롬프팅), c03(탐색·편집), c07(커밋) | collect_todos 단위 테스트 통과, feat: 커밋 기록 |
| M3 점검 추가·커맨드      | check_links·check_file_size·check_required_docs, /audit | u2·u4    | c03(편집), c08(슬래시 커맨드)          | 각 점검 함수 테스트 통과, /audit 동작            |
| M4 리포트·종료 코드·리뷰   | report.py, decide_exit_code, reviewer 반영                | u5       | c09(서브에이전트)                    | 텍스트/JSON 일치, 리뷰 반영, exit code 검증     |
| M5 안전 게이트·연동      | guard.py 혹은 settings.json, .mcp.json                    | u2·u5·u6 | c04(권한), c10(MCP), c11(혹)      | 혹이 위반 편집 차단·로깅, 최소 권한 허용             |
| M6 CI·스케줄·오케스트레이션 | claude.yml, /feature 파이프라인                              | u6       | c12(오케스트레이션), c13(CI·자동화)      | CI가 종료 코드로 판정, 스케줄 리포트 아티팩트 보존       |

13개 개념이 6개 마일스톤에 빠짐없이 분산되어 있음을 확인하세요. 이것이 "커리큘럼을 커버하도록 역설계한 프로젝트"의 결과입니다.

**드라이빙 퀘스천(다시 보기).** "실무 개발자가 Claude Code를 페어 프로그래머이자 자동화 엔진으로 삼아, 신뢰할 수 있는 저장소 건강검진 CLI를 분석부터 CI 자동화까지 하나의 에이전트 워크플로우로 어떻게 만들어 낼 수 있을까?" — 이 장은 그 답의 *실행편*입니다. 각 마일스톤이 이 질문의 한 조각을 채웁니다.

## 16.2 M1 — CLAUDE.md·플랜 모드로 셋업·계획 (c05, c06)

적용 단위/개념: u3 — c05(컨텍스트 관리: CLAUDE.md), c06(플랜 모드)

첫 마일스톤은 코드 한 줄을 쓰기 전에 **프로젝트의 헌법을 세우고 계획을 받는** 일입니다. 5장에서 배운 대로, CLAUDE.md는 세션마다 자동으로 읽혀 빌드·테스트 명령·컨벤션·금지사항을 에이전트에 주입합니다. repo-audit는 표준 라이브러리만 쓰고(NFR1) 결정론적이어야 하므로(NFR5), 그 규칙을 CLAUDE.md에 못 박습니다.

```
# repo-audit — 프로젝트 헌법(CLAUDE.md)

## 무엇을 만드는가
저장소를 스캔해 깨진 로컬 링크·비대 파일·누락 문서·TODO를 점검하고
텍스트/JSON 리포트와 종료 코드로 보고하는 CLI.

## 명령
- 실행: `python repo_audit.py <경로> [--format json] [--fail-on error]`
- 테스트: `python -m pytest -q`
- 포맷: `python -m black .`

## 컨벤션
- Python 3.12, 표준 라이브러리만 사용(외부 의존 금지 = NFR1)
- type hint 필수, snake_case, Black 포매팅
- 점검 함수는 finding 딕셔너리 리스트를 반환(설계 ADR1)
- 같은 입력은 같은 출력(결정론 = NFR5): 순회는 정렬, JSON은 키 정렬

## 금지사항
- 외부 네트워크 호출(외부 URL 검사 제외 = ADR3)
- 자동 수정(점검·보고까지만, 수정은 사람 판단)
- secrets/.env 편집(M5에서 흑으로 강제 차단)

## 모듈 구조(설계 15장)
- repo_audit.py: 인자 파싱·점검 오케스트레이션·종료 코드
- checks.py: 개별 점검 함수
- report.py: 리포트 집계·렌더(text/json)
- tests/: 각 점검 함수 단위 테스트
```

### 왜 이렇게 쓰나(c05 적용)

- **명령 섹션**은 에이전트가 "테스트를 어떻게 돌리는지" 매번 묻지 않게 합니다. 체크포인트 검증(pytest)이 여기서 나옵니다.
- **컨벤션·금지사항**은 14장 NFR(표준 라이브러리·결정론)과 15장 ADR(딕셔너리 finding·외부 URL 제외)을 영속 규칙으로 고정합니다. 세션이 바뀌어도 같은 규칙이 적용됩니다.
- **모듈 구조**를 적어 두면, 새 세션에서 "checks에 함수 추가해 줘"라고만 해도 에이전트가 올바른 파일에 작업합니다.

이어서 **플랜 모드(c06)**로 첫 기능의 구현 계획을 받습니다. 코드를 바꾸기 전 읽기 전용으로 계획을 보고, 검토·교정한 뒤 PLAN.md로 확정합니다.

# PLAN.md — repo-audit 구현 계획(플랜 모드 확정본)

## 단계

1. repo\_audit.py에 argparse 인자 파싱 골격 + main() 종료 코드 빼대
2. checks.py에 collect\_todos 구현 + tests/test\_checks.py 단위 테스트
3. check\_links·check\_file\_size·check\_required\_docs 추가(각각 테스트 동반)
4. report.py에 summarize·render\_text·render\_json + decide\_exit\_code 연결
5. PreToolUse 혹은(guard.py)·settings.json·.mcp.json 안전 게이트
6. GitHub Actions(claude.yml) 헤드리스 CI + 주간 스케줄

## 교정 메모(검토 중 바로잡는 가정)

- (초안) finding을 dataclass로 → (교정) JSON 직렬화 단순성 위해 dict 유지(ADR1)
- (초안) 외부 링크도 검사 → (교정) 결정론 위해 로컬 링크만(ADR3)

**체크포인트 M1.** 다음 두 가지를 확인하면 M1 통과입니다. ① CLAUDE.md가 실행·테스트·포맷 명령을 담는가?  
② PLAN.md의 각 단계가 "테스트 통과"처럼 검증 가능한 단위로 나뉘고, 잘못된 가정(dataclass·외부 링크)을 교정했는가? 플랜 모드의 가치는 *대규모 변경 전에 방향을 바로잡는 것*입니다(6장).

## 16.3 M2 — CLI 골격과 TODO 수집·첫 커밋 (c02, c03, c07)

적용 단위/개념: u1·u2·u4 — c02(효과적 프롬프팅), c03(파일 탐색·편집), c07(Git·커밋)

이제 걷는 *해결*을 세웁니다. 인자를 받아 한 가지 점검(collect\_todos)을 수행하고 리포트를 내는 최소 버전입니다. 2장에서 배운 대로 **목표·맥락·제약·완료 조건**을 갖춘 지시로 시작합니다. 예: "checks.py에 collect\_todos(root)를 추가해 줘. 표준 라이브러리만 쓰고, TODO·FIXME가 든 줄을 경로·줄번호와 함께 finding 딕셔너리 리스트로 반환해. **TODO**는 **info**, **FIXME**는 '**지금 고쳐야 할 결함**'이니 **error** 등급으로 구분하고, 바이너리·숨김(.git) 파일은 건너뛰어. 완료 조건은 tests/test\_checks.py의 test\_collect\_todos(TODO→info)와 test\_collect\_fixme(FIXME→error) 통과야."

실무 저장소에서 TODO와 FIXME를 같은 등급으로 뭉뚱그리면 점검이 무뎛습니다. 관례적으로 TODO는 "언젠가 할 일"(정보), FIXME는 "지금 잘못된 것"(결함)을 뜻하므로, 등급을 갈라 두면 CI에서 `--fail-on error` 로 FIXME만 빌드를 막고 TODO는 통과시키는 정교한 게이트를 만들 수 있습니다(FR5·FR7).

먼저 `repo_audit.py`의 골격입니다(3장 외과적 편집·신규 작성 원칙으로 한 파일씩 만듭니다).

```

"""repo-audit: 저장소 건강검진 CLI.

대상 경로를 받아 링크·파일 크기·필수 문서·TODO를 점검하고,
텍스트/JSON 리포트를 출력하며, 심각도 임계 초과 시 종료 코드 1을 반환합니다.
"""

from __future__ import annotations

import argparse
import sys
from pathlib import Path

import checks
import report as report_mod

# 종료 코드 판정 시 각 등급의 심각도 순서(낮음->높음).
_SEVERITY_ORDER = {checks.INFO: 0, checks.WARN: 1, checks.ERROR: 2}

def run_audit(root: Path, config: dict) -> dict:
    """모든 점검을 실행해 통합 리포트를 만듭니다(점검 오케스트레이션)."""
    findings: list[dict[str, str]] = []
    findings += checks.collect_todos(root)
    return report_mod.build_report(findings)

def parse_args(argv: list[str]) -> argparse.Namespace:
    """CLI 인자(대상 경로·임계·출력 형식·판정 기준)를 파싱합니다."""
    parser = argparse.ArgumentParser(description="저장소 건강검진 CLI (repo-audit)")
    parser.add_argument("path", help="점검할 저장소 경로")
    parser.add_argument("--max-file-kb", type=int, default=512, help="비대 파일 임계(KB)")
    parser.add_argument("--format", choices=["text", "json"], default="text", help="출력 형식")
    parser.add_argument("--fail-on", choices=["info", "warn", "error"], default="error",
                        help="이 등급 이상 발견 시 비정상 종료")
    return parser.parse_args(argv)

def main(argv: list[str] | None = None) -> int:
    """엔트리포인트: 점검을 수행하고 리포트를 출력한 뒤 종료 코드를 반환합니다."""
    args = parse_args(sys.argv[1:] if argv is None else argv)
    root = Path(args.path)
    if not root.is_dir():
        sys.stderr.write(f"경로를 찾을 수 없습니다: {root}\n")
        return 2
    config = {
        "max_file_kb": args.max_file_kb,
        "required_docs": ["README", "LICENSE"],
        "fail_on": args.fail_on,
    }
    report = run_audit(root, config)
    print(report_mod.render_text(report))
    return 0

if __name__ == "__main__":
    sys.exit(main())

```

다음은 첫 점검 함수 `collect_todos` 와 등급 상수를 담은 `checks.py` 입니다. finding의 형태는 15장 ADR1에서 정한 디렉터리 `{"check", "path", "severity", "message"}` 입니다.

```

"""repo-audit 개별 점검 함수 모음.

각 함수는 대상 저장소 경로(root)를 받아 finding 딕셔너리 리스트를 돌려줍니다.
finding 한 건의 형태: {"check", "path", "severity", "message"} (설계 ADR1).
"""

from __future__ import annotations

import re
from pathlib import Path

# 점검 결과의 심각도 등급(상수로 고정해 키 오타를 방지 = ADR1 consequences).
INFO = "info"
WARN = "warn"
ERROR = "error"

# TODO/FIXME 주석 추출 정규식(대소문자 무시, 태그를 캡처).
_TODO_RE = re.compile(r"\b(TODO|FIXME)\b", re.IGNORECASE)

def _make(check: str, path: str, severity: str, message: str) -> dict[str, str]:
    """finding 한 건을 표준 형태로 생성합니다."""
    return {"check": check, "path": path, "severity": severity, "message": message}

def collect_todos(root: Path) -> list[dict[str, str]]:
    """소스의 TODO/FIXME 주석을 수집합니다(TODO=info, FIXME=error)."""
    findings: list[dict[str, str]] = []
    # 결정론(NFR5)을 위해 정렬된 순서로 순회합니다.
    for path in sorted(root.rglob("*")):
        if not path.is_file() or ".git" in path.parts:
            continue
        try:
            lines = path.read_text(encoding="utf-8").splitlines()
        except (UnicodeDecodeError, OSError):
            continue # 바이너리·읽기 불가 파일은 건너뜁니다.
        for lineno, line in enumerate(lines, start=1):
            hit = _TODO_RE.search(line)
            if hit:
                # FIXME는 '지금 고쳐야 할 결함'이므로 error, TODO는 info로 구분.
                tag = hit.group(1).upper()
                severity = ERROR if tag == "FIXME" else INFO
                rel = str(path.relative_to(root)).replace("\\", "/")
                findings.append(
                    _make("todos", f"{rel}:{lineno}", severity, line.strip()[:80])
                )
    return findings

```

완료 조건인 단위 테스트입니다. 픽스처 저장소( tmp\_path )에 TODO 한 줄을 심고 결과를 검증합니다.

```

"""checks 모듈의 단위 테스트(픽스처 저장소 사용)."""
from pathlib import Path

import checks

def test_collect_todos(tmp_path: Path) -> None:
    # 준비: TODO 한 줄을 가진 파일을 만든다.
    (tmp_path / "a.py").write_text("x = 1 # TODO 리팩터\n", encoding="utf-8")
    # 실행
    findings = checks.collect_todos(tmp_path)
    # 검증: TODO 한 건이 info 등급으로 수집된다.
    assert len(findings) == 1
    assert findings[0]["severity"] == checks.INFO
    assert findings[0]["check"] == "todos"

def test_collect_fixme(tmp_path: Path) -> None:
    # 준비: FIXME 한 줄을 가진 파일을 만든다.
    (tmp_path / "b.py").write_text("y = 2 # FIXME 빈 입력 처리\n", encoding="utf-8")
    # 실행
    findings = checks.collect_todos(tmp_path)
    # 검증: FIXME는 결함이므로 error 등급으로 수집된다.
    assert len(findings) == 1
    assert findings[0]["severity"] == checks.ERROR

```

테스트를 돌려 체크포인트를 확인합니다.

```

$ python -m pytest -q
.
1 passed in 0.04s
[100%]

```

테스트가 녹색이면 7장에서 배운 대로 작은 논리 단위로 커밋합니다. Conventional Commits 메시지를 씁니다.

```

$ git add repo_audit.py checks.py tests/test_checks.py
$ git commit -m "feat: CLI 골격과 TODO 수집 점검 추가

collect_todos로 TODO/FIXME를 info 등급 finding으로 수집.
argparse 인자 파싱과 main 종료 코드 빼대 포함."

```

**체크포인트 M2.** `test_collect_todos` 가 통과하고, 첫 `feat:` 커밋이 기록되면 M2 통과입니다. 여기서 걷는 해골이 완성됐습니다 — 인자를 받아 한 점검을 하고 리포트를 내는 최소 동작 버전입니다. 이후 마일스톤은 이 골격에 점검과 기능을 외과적으로 덧붙이기만 하면 됩니다(c03).

## 16.4 M3 — 링크·파일 크기·문서 점검과 `/audit` 커맨드 (c03, c08)

적용 단원/개념: u2·u4 — c03(파일 탐색·편집), c08(슬래시 커맨드)

이제 점검을 셋 더 추가합니다(FR2·FR3·FR4). 3장의 **외과적 편집** 원칙대로 `checks.py` 에 함수만 덧붙입니다(기존 함수는 건드리지 않습니다). 아래는 `checks.py` 에 추가되는 세 함수입니다.

```

# checks.py 상단 정규식에 추가: 마크다운 링크 [텍스트](대상)·헤딩
_LINK_RE = re.compile(r"^\[[^\]]*\]\([^\)]+\)\)")
_HEADING_RE = re.compile(r"^(#{1,6})\s+(.+)$", re.MULTILINE)

def _anchors(text: str) -> set[str]:
    """마크다운 헤딩에서 GitHub식 앵커 슬러그를 추출합니다."""
    slugs: set[str] = set()
    for heading in _HEADING_RE.findall(text):
        slug = re.sub(r"^\w\s-", "", heading.strip().lower())
        slugs.add(re.sub(r"\s+", "-", slug).strip("-"))
    return slugs

def check_links(root: Path) -> list[dict[str, str]]:
    """마크다운의 로컬 링크·앵커가 유효한지 점검합니다(외부 URL 제외 = ADR3)."""
    findings: list[dict[str, str]] = []
    for md in sorted(root.rglob("*.md")):
        if ".git" in md.parts:
            continue
        text = md.read_text(encoding="utf-8", errors="replace")
        rel = str(md.relative_to(root)).replace("\\", "/") # 경로 '/' 통일(NFR5)
        for target in _LINK_RE.findall(text):
            target = target.strip()
            # 외부 URL·순수 앵커·메일은 결정론(ADR3)을 위해 검사 제외.
            if target.startswith(("http://", "https://", "#", "mailto:")):
                continue
            local, _, anchor = target.partition("#") # 파일부와 앵커부 분리
            if not local:
                continue
            resolved = (md.parent / local).resolve()
            if not resolved.exists():
                # 대상 파일 자체가 없으면 '깨진 링크' = error.
                findings.append(_make("links", rel, ERROR, f"깨진 로컬 링크: {target}"))
            elif anchor and resolved.suffix == ".md":
                # 파일은 있으나 헤딩 앵커가 없으면 = warn(FR2의 '앵커' 요건).
                if anchor not in _anchors(resolved.read_text(encoding="utf-8", errors="replace")):
                    findings.append(_make("links", rel, WARN, f"잘못된 앵커: #{anchor}"))
    return findings

def check_file_size(root: Path, max_file_kb: int) -> list[dict[str, str]]:
    """임계(KB)를 초과하는 비대 파일을 점검합니다(warn 등급)."""
    findings: list[dict[str, str]] = []
    for path in sorted(root.rglob("*")):
        if not path.is_file() or ".git" in path.parts:
            continue
        size_kb = path.stat().st_size / 1024.0
        if size_kb > max_file_kb:
            rel = str(path.relative_to(root)).replace("\\", "/")
            findings.append(
                _make("file_size", rel, WARN,
                    f"파일 크기 초과: {size_kb:.1f}KB (임계치: {max_file_kb}KB)")
            )
    return findings

def check_required_docs(root: Path, required_docs: list[str]) -> list[dict[str, str]]:
    """필수 문서(README·LICENSE 등)가 있는지 점검합니다(error 등급)."""
    findings: list[dict[str, str]] = []
    existing = [p.name.upper() for p in root.iterdir() if p.is_file()]
    for name in required_docs:
        # 확장자 무관 매칭: 'README' == 'README.md' == 'README.txt'.

```

```

        found = any(n == name.upper() or n.startswith(name.upper() + ".")
                    for n in existing)
        if not found:
            findings.append(_make("required_docs", ".", ERROR, f"필수 문서 없음: {name}"))
    return findings

```

세 함수의 테스트도 함께 추가합니다(완료 조건). 핵심만 보입니다.

```

def test_check_required_docs_missing(tmp_path: Path) -> None:
    # README/LICENSE 없는 빈 저장소 -> 둘 다 error finding.
    findings = checks.check_required_docs(tmp_path, ["README", "LICENSE"])
    assert len(findings) == 2
    assert all(f["severity"] == checks.ERROR for f in findings)

def test_check_file_size_over(tmp_path: Path) -> None:
    # 임계 1KB를 넘는 파일 하나 -> warn finding 한 건.
    (tmp_path / "big.bin").write_bytes(b"x" * 2048)
    findings = checks.check_file_size(tmp_path, max_file_kb=1)
    assert len(findings) == 1
    assert findings[0]["severity"] == checks.WARN

```

그리고 `run_audit` 가 새 점검을 호출하도록 한 줄씩 외과적으로 보강합니다.

```

def run_audit(root: Path, config: dict) -> dict:
    """모든 점검을 실행해 통합 리포트를 만듭니다(점검 오케스트레이션)."""
    findings: list[dict[str, str]] = []
    findings += checks.check_links(root)
    findings += checks.check_file_size(root, config["max_file_kb"])
    findings += checks.check_required_docs(root, config["required_docs"])
    findings += checks.collect_todos(root)
    return report_mod.build_report(findings)

```

이 점검을 매번 손으로 돌리는 대신, 8장의 **커스텀 슬래시 커맨드**로 반복 절차를 표준화합니다.

`.claude/commands/audit.md` 에 인자를 받는 `/audit` 커맨드를 둡니다.

```

---
description: 대상 경로에 repo-audit를 실행하고 결과를 요약한다
---

```

다음 절차로 저장소 건강검진을 수행하세요.

1. ``python repo_audit.py $ARGUMENTS --format text`` 를 실행합니다.
2. error·warn 항목을 우선순위로 묶어 요약합니다.
3. 각 발견에 대해 한 줄 조치 제안을 답니다(수정은 하지 말 것).

이제 `/audit ./myrepo` 한 번으로 점검을 재현할 수 있습니다(`$ARGUMENTS`가 대상 경로로 치환됩니다).

**체크포인트 M3.** `check_links` · `check_file_size` · `check_required_docs` 각각의 단위 테스트가 통과하고, `/audit <경로>` 커맨드가 점검을 실행·요약하면 M3 통과입니다. 점검을 **외과적으로 덧붙였고(c03)** 반복 절차를 **커맨드로 표준화(c08)**한 것이 이 마일스톤의 두 축입니다.

## 16.5 M4 — 리포트·종료 코드와 reviewer 서브에이전트 리뷰 (c09)

---

적용 단위/개념: u5 — c09(서브에이전트 위임)

점검 결과를 **사람용 텍스트**와 **기계용 JSON** 두 형식으로 렌더하고(ADR2), 심각도 임계로 종료 코드를 냅니다(FR7). `report.py` 를 만듭니다.

```

"""repo-audit 리포트 집계·렌더.

finding 리스트를 받아 summary(등급별 개수)를 집계하고,
사람용 텍스트와 기계용 JSON 두 형식으로 렌더합니다(ADR2).
"""

from __future__ import annotations

import json

from checks import INFO, WARN, ERROR


def summarize(findings: list[dict[str, str]]) -> dict[str, int]:
    """finding 리스트에서 등급별 개수를 집계합니다."""
    summary = {INFO: 0, WARN: 0, ERROR: 0}
    for f in findings:
        summary[f["severity"]] = summary.get(f["severity"], 0) + 1
    return summary


def build_report(findings: list[dict[str, str]]) -> dict:
    """summary와 findings를 묶은 전체 리포트 딕셔너리를 만듭니다.

    발견을 (check, path, severity)로 정렬해, 파일 순회 순서(OS마다 다름)와
    무관하게 같은 입력이면 항상 같은 순서로 보고합니다(결정론 = NFR5).
    """
    ordered = sorted(findings, key=lambda f: (f["check"], f["path"], f["severity"]))
    return {"summary": summarize(ordered), "findings": ordered}


# 심각도별 ASCII 마커(이모지 금지)와 정렬 순서(error를 맨 위로).
_MARKER = {ERROR: "[x]", WARN: "[-]", INFO: "[i]"}
_ORDER = {ERROR: 0, WARN: 1, INFO: 2}


def render_text(report: dict) -> str:
    """사람이 읽는 텍스트 요약을 만듭니다(심각도 높은 순)."""
    s = report["summary"]
    sep = "=" * 60
    lines = [sep, "repo-audit 점검 결과", sep,
              f" 오류(error) : {s[ERROR]}건",
              f" 경고(warn)  : {s[WARN]}건",
              f" 정보(info)   : {s[INFO]}건", "-" * 60]
    if not report["findings"]:
        lines.append(" [v] 발견된 문제 없음 - 저장소가 건강합니다.")
    else:
        for f in sorted(report["findings"], key=lambda x: _ORDER[x["severity"]]):
            marker = _MARKER[f["severity"]]
            lines.append(f" {marker} [{f['check']}] {f['path']}: {f['message']}")
        lines.append(sep)
    return "\n".join(lines)


def render_json(report: dict) -> str:
    """기계가 읽는 JSON 문자열을 만듭니다(결정론 = NFR5: 키 정렬)."""
    return json.dumps(report, ensure_ascii=False, indent=2, sort_keys=True)

```

종료 코드 판정 함수를 `repo_audit.py` 에 추가하고 `main` 에 연결합니다(FR7).

```
def decide_exit_code(report: dict, fail_on: str) -> int:
    """summary를 보고 fail_on 임계 이상 등급이 있으면 1, 없으면 0을 돌려줍니다."""
    threshold = _SEVERITY_ORDER[fail_on]
    for severity, count in report["summary"].items():
        if count > 0 and _SEVERITY_ORDER[severity] >= threshold:
            return 1
    return 0

# main() 끝부분을 다음과 같이 보강:
# if args.format == "json":
#     print(report_mod.render_json(report))
# else:
#     print(report_mod.render_text(report))
# return decide_exit_code(report, args.fail_on)
```

동봉한 완성 코드( project/project\_complete/repo\_audit.py )는 같은 판정을 should\_fail(findings, fail\_on) -> bool 로 구현하고 main 에서 return 1 if should\_fail(...) else 0 으로 잇습니다. summary를 보든 findings를 보든 "fail\_on 임계 이상 등급이 하나라도 있으면 실패"라는 규칙은 동일합니다 — 이름과 시그니처는 취향껏 정하되 판정 규칙을 한곳에 모아 테스트하는 것이 핵심입니다.

이제 9장의 **서브에이전트**에 코드 리뷰를 위임합니다. .claude/agents/reviewer.md 에 범위가 좁은 리뷰 전문가를 정의합니다. 메인 세션의 맥락을 오염시키지 않고 독립 컨텍스트에서 리뷰하게 합니다.

```
---
name: reviewer
description: repo-audit 코드의 결정론·에러 처리·테스트 누락을 점검하는 코드 리뷰어
tools: Read, Grep, Glob
model: opus
---
```

당신은 repo-audit 전용 코드 리뷰어입니다. 다음만 점검하고 수정은 하지 마세요.

- 결정론(NFR5): 순회가 정렬되어 있는가, JSON이 키 정렬되는가
- 에러 처리: 바이너리·읽기 불가 파일을 안전하게 건너뛰는가
- 테스트 커버리지: 각 점검 함수에 단위 테스트가 있는가
- ADR 준수: finding이 {check,path,severity,message} 딕셔너리인가

발견을 심각도와 함께 목록으로 보고하세요. 도구는 Read/Grep/Glob만 씁니다(편집 권한 없음).

reviewer의 지적(예: "check\_links에 빈 디렉터리 테스트 누락")을 메인 세션에서 반영합니다. 서브에이전트의 tools 를 읽기 전용으로 좁힌 것이 핵심입니다 — 리뷰어가 코드를 직접 바꿀 수 없게 해, 위임의 경계를 명확히 합니다.

**체크포인트 M4.** ① 같은 입력에 대해 텍스트·JSON 리포트의 내용이 일치하고, ② reviewer가 지적한 항목이 반영됐으며, ③ error 발견 시 decide\_exit\_code 가 1을 반환함을 확인하면 M4 통과입니다. 서브에이전트에 **경계가** 분명한 작업(리뷰) 만 위임한 것이 c09의 정석 적용입니다.

## 16.6 M5 — PreToolUse 흑·권한 게이트와 MCP 연동 (c04, c10, c11)

적용 단위/개념: u2·u5·u6 — c04(권한·allowlist), c10(MCP), c11(흑)

이 도구를 무인으로 돌릴 준비를 합니다. 먼저 11장의 **PreToolUse** 흑으로 위험 경로 편집을 차단합니다(NFR3). `.claude/hooks/guard.py` 는 편집 대상 경로가 보호 목록에 닿으면 종료 코드 2로 차단하고 로그를 남깁니다.

```
#!/usr/bin/env python3
"""PreToolUse 흑: 보호 경로 편집을 차단하고 로깅합니다(fail-open 설계)."""
import json
import sys
from datetime import datetime
from pathlib import Path

# 편집을 막을 보호 경로 조각(NFR3).
PROTECTED = ("secrets/", ".env", ".git/")
LOG_PATH = Path(".claude/guard.log")

def main() -> int:
    try:
        event = json.loads(sys.stdin.read())
    except json.JSONDecodeError:
        return 0 # 입력 파싱 실패 시 통과(fail-open: 정상 작업을 막지 않음).

    file_path = event.get("tool_input", {}).get("file_path", "")
    if any(token in file_path for token in PROTECTED):
        stamp = datetime.now().isoformat(timespec="seconds")
        with LOG_PATH.open("a", encoding="utf-8") as fp:
            fp.write(f"{stamp} BLOCKED edit: {file_path}\n")
        sys.stderr.write(f"보호 경로는 수정할 수 없습니다: {file_path}\n")
        return 2 # PreToolUse 차단.

    return 0 # 정상 통과.

if __name__ == "__main__":
    sys.exit(main())
```

흑을 등록하고, 4장의 **최소 권한 allowlist**(NFR4)를 둡니다. `.claude/settings.json` 에서 점검 실행·테스트만 허용하고 위험 명령은 차단합니다.

```
{
  "permissions": {
    "allow": [
      "Bash(python repo_audit.py:*)",
      "Bash(python -m pytest:*)",
      "Bash(git commit:*)"
    ],
    "deny": [
      "Bash(rm:*)",
      "Bash(curl:*)"
    ]
  },
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [{ "type": "command", "command": "python .claude/hooks/guard.py" }]
      }
    ]
  }
}
```

마지막으로 10장의 **MCP**로 점검 정책 문서를 외부에서 참조합니다(선택). 파일시스템 MCP 서버를 `.mcp.json`에 등록해, 팀 위키의 "저장소 품질 기준" 문서를 에이전트가 읽게 합니다. 비밀(토큰)은 코드가 아니라 환경 변수로 주입합니다.

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem", "./policy-docs"]
    }
  }
}
```

**흔한 함정.** 혹은 너무 뻑뻑하게 만들면(예: 모든 편집 차단) 정상 작업까지 막힙니다. `guard.py`는 *입력 파싱 실패 시 통과(fail-open)* 하도록 설계해, 혹 자체의 버그가 작업 전체를 마비시키지 않게 했습니다. 차단은 *명시적으로 위험한 경로*에만 적용합니다. `allowlist`도 마찬가지로 — `rm`을 `deny`하되 정상 점검 명령은 `allow`해 안전과 생산성을 모두 잡습니다.

**체크포인트 M5.** ① `guard.py`가 `.env` 같은 보호 경로 편집을 실제로 차단하고 `guard.log`에 기록하는가? ② `settings.json`이 점검-테스트-커밋만 허용하고 `rm`·`curl`을 차단하는가? ③ MCP가 외부 정책 문서를 참조하는가? 셋을 확인하면 M5 통과입니다.

## 16.7 M6 — GitHub Actions CI·스케줄 무인화와 오케스트레이션 (c12, c13)

적용 단위/개념: u6 — c12(오케스트레이션), c13(CI·스케줄·자동화)

마지막 마일스톤은 점검을 **사람 없이** 도는 자동화로 내보냅니다. 13장의 헤드리스 실행과 GitHub Actions로, PR 마다(FR8)·매주(FR9) 점검을 무인 실행하고 종료 코드로 판정합니다. `.github/workflows/claude.yml` 입니다.

```
name: repo-audit
on:
  pull_request:
    branches: [main]
  schedule:
    - cron: "0 0 * * 1" # 매주 월요일 00:00 UTC 정기 점검(FR9)

permissions:
  contents: read # 최소 권한(NFR4)

jobs:
  audit:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - name: 단위 테스트
        run: python -m pytest -q
      - name: 헤드리스 점검(JSON 리포트 + 종료 코드 판정)
        run: python repo_audit.py . --format json --fail-on error > audit_report.json
      - name: 리포트 아티팩트 보존
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: audit-report
          path: audit_report.json
```

### 줄별 해설(c13 적용)

- `on: pull_request` + `schedule`: 두 트리거를 모두 겁니다. PR 이벤트(FR8)와 주간 cron 스케줄(FR9)이 같은 점검을 돌립니다.
- `permissions: contents: read`: 무인 환경에서는 권한을 더 좁힙니다(NFR4). 쓰기 권한이 없어 사고 범위가 줄어듭니다.
- `python repo_audit.py . --fail-on error`: 헤드리스 실행. error 발견 시 비정상 종료 코드를 내면 CI가 빨강(실패)으로 판정합니다(FR7).
- `if: always()` + `upload-artifact`: 실패해도 리포트를 보존합니다. **결과를 남기지 않으면 무인 자동화는 신뢰할 수 없습니다(13장)**.

여러 점검을 한꺼번에 늘릴 때는 12장의 **오케스트레이션**으로 작업을 나눕니다. `/feature` 커맨드가 조사→구현→리뷰 파이프라인을 지휘합니다. 독립 조사는 병렬, 구현은 순차, 리뷰는 reviewer 서브에이전트(M4)에 위임, 통합은 메인이 담당합니다.

오케스트레이션 흐름(/feature "보안 점검 추가")

1. 메인: 작업 분해(조사 2건·구현 1건·리뷰 1건)
2. 병렬 조사: researcher A(정규식 패턴) · researcher B(기존 checks 관례)
3. 순차 구현: 메인이 조사 결과를 통합해 `check_secrets` 작성
4. 위임 리뷰: reviewer 서브에이전트가 결정론·테스트 점검
5. 통합: 메인이 리뷰 반영 후 `feat: 커밋`

**체크포인트 M6.** ① CI가 종료 코드로 통과/실패를 판정하는가(error 있으면 빨강)? ② 주간 스케줄 점검의 JSON 리포트가 아티팩트로 보존되는가? ③ 새 점검 추가가 조사(병렬)→구현(순차)→리뷰(위임)→통합(메인) 파이프라인으로 분업되는가? 셋을 확인하면 M6 통과이고, **구현 단계가 완료됩니다.**

## 16.8 통합 실행과 결과 확인 (텍스트·JSON·CI 로그)

M1~M6을 통합하면 repo-audit가 실제 저장소에서 끝까지 동작합니다. 세 가지 산출(텍스트·JSON·CI 로그)을 캡처해 검증합니다.

**(1) 텍스트 리포트** — 사람이 읽는 출력입니다. 동봉한 픽스처 저장소(fixture\_repo/)에는 깨진 링크 2건, 잘못된 헤딩 앵커 1건, FIXME 1건, TODO 1건이 일부러 심겨 있습니다.

```
$ python repo_audit.py fixture_repo --format text --fail-on error
=====
repo-audit 점검 결과
=====
오류(error) : 3건
경고(warn)  : 1건
정보(info)  : 1건
-----
[x] [links] README.md: 깨진 로컬 링크: docs/missing.md
[x] [links] docs/guide.md: 깨진 로컬 링크: ../nonexistent_file.md
[x] [todos] src/main.py:11: # FIXME: 입력 검증이 불완전함 - 빈 문자열을 허용하고 있음
[-] [links] docs/guide.md: 잘못된 앵커: #nonexistent-anchor (대상: ../README.md)
[i] [todos] src/main.py:6: # TODO: 실제 데이터 처리 로직 구현 필요
$ echo $?
1
```

깨진 링크 2건과 FIXME 1건이 error이므로 `--fail-on error` 기준에서 종료 코드 1이 나왔습니다(`echo $?`). FIXME는 error, TODO는 info로 걸린 점, 파일은 있으나 헤딩이 없는 앵커는 warn으로 잡힌 점에 주목하세요. 파일 크기·필수 문서 점검까지 한 번에 보려면 옵션을 더합니다 — `--max-file-kb 1`(작은 임계로 LICENSE를 비대 파일 warn으로) 과 `--required-docs README LICENSE CONTRIBUTING`(없는 CONTRIBUTING을 error로)을 주면 네 점검이 모두 발화합니다.

**(2) JSON 리포트** — 기계가 읽는 출력입니다(결정론: 키 정렬·발견 정렬). 같은 입력이면 OS·실행 순서와 무관하게 바이트까지 동일합니다(NFR5).

```
$ python repo_audit.py fixture_repo --format json --fail-on error
{
  "findings": [
    {
      "check": "links",
      "message": "깨진 로컬 링크: docs/missing.md",
      "path": "README.md",
      "severity": "error"
    },
    ...
  ],
  "summary": { "error": 3, "info": 1, "warn": 1 }
}
```

(지면 관계로 findings 첫 건만 보였습니다. 키가 알파벳순( `check.message.path.severity` )으로 정렬되고 발견이 ( `check, path` ) 순으로 정렬돼 재현성이 보장됩니다.)

(3) CI 로그 — GitHub Actions의 무인 실행 결과입니다.

```
Run python -m pytest -q
..... [100%]
61 passed in 0.13s
Run python repo_audit.py . --format json --fail-on error
Error: Process completed with exit code 1.
```

`tests/` 의 단위 테스트 61건이 모두 통과한 뒤, 헤드리스 점검이 종료 코드 1을 내자 CI가 실패(빨강)로 판정했습니다. PR 작성자는 깨진 링크·FIXME를 고친 뒤 다시 푸시하면 녹색이 됩니다. 같은 입력에 같은 출력이 나오는 지(결정론) 두 번 실행해 `diff` 로 비교하면 NFR5도 검증됩니다.

## 16.9 작성형 연습 (2종 1세트): 빈칸 코드 채우기·자가체크

지금까지의 구현을 직접 손에 익힐 차례입니다. 아래 두 연습은 모범 → 빈칸 템플릿 → 자가체크의 2종 1세트로 구성됩니다.

### 연습 1 — 점검 함수 채우기 (M3 복습: c03)

**모범 참고.** 16.4의 `check_required_docs` 는 필수 문서가 없으면 error finding을 반환했습니다. 같은 골격으로 빈 디렉터리 점검( `check_empty_dirs` )을 만들어 봅시다.

**여러분의 차례.** 빈칸을 채워 빈 디렉터리를 warn 등급으로 보고하는 점검을 완성하세요.

```
def check_empty_dirs(root):
    """내용이 없는 디렉터를 warn 등급으로 점검합니다."""
    findings = []
    for path in sorted(root.rglob("*")):
        if not path.__①__() or ".git" in path.parts: # 디렉터리만 대상
            continue
        # 디렉터리 안에 항목이 하나도 없으면 비어 있음
        if not any(path.iterdir()):
            findings.append(
                _make("empty_dir", str(path), __②__, "빈 디렉터리") # 등급 상수
            )
    return findings
```

```
def test_check_empty_dirs(tmp_path):
    (tmp_path / "empty").__③__() # 빈 하위 디렉터리 생성
    findings = check_empty_dirs(tmp_path)
    assert len(findings) == __④__ # 기대 finding 개수
    assert findings[0]["severity"] == checks.WARN
```

### 자가체크

- [ ] \_\_①\_\_ 는 디렉터리만 거르는 메서드( `is_dir` )인가요?

- [ ] \_\_②\_\_ 는 warn 등급 상수(WARN)인가요?
- [ ] \_\_③\_\_ 는 디렉터리를 만드는 메서드(mkdir)인가요?
- [ ] \_\_④\_\_ 는 빈 디렉터리 한 개에 대응하는 개수(1)인가요?
- [ ] 점검을 *외과적으로* 덧붙였고(c03) 기존 함수는 건드리지 않았나요?
- [ ] run\_audit 에 check\_empty\_dirs 호출을 한 줄 추가했나요?

## 연습 2 — 종료 코드 판정과 CI 트리거 채우기 (M4·M6 복습: c13)

**모범 참고.** 16.5의 decide\_exit\_code 는 fail\_on 임계 이상 등급이 있으면 1을 반환했고, 16.7의 claude.yml은 PR-스케줄 트리거로 무인 실행했습니다.

**여러분의 차례.** 종료 코드 판정과 CI 트리거의 빈칸을 채우세요.

```
def decide_exit_code(report, fail_on):
    """fail_on 임계 이상 등급이 있으면 1, 없으면 0."""
    threshold = _SEVERITY_ORDER[fail_on]
    for severity, count in report["summary"].items():
        if count > 0 and _SEVERITY_ORDER[severity] __⑤__ threshold: # 비교 연산자
            return __⑥__ # 문제 있을 때 종료 코드
    return __⑦__ # 문제 없을 때 종료 코드
```

```
name: repo-audit
on:
    __⑧__: # PR 트리거
        branches: [main]
    schedule:
        - cron: "0 0 * * 1"
permissions:
    contents: __⑨__ # 최소 권한
jobs:
    audit:
        runs-on: ubuntu-latest
        steps:
            - uses: actions/checkout@v4
            - uses: actions/setup-python@v5
              with:
                python-version: "3.12"
            - run: python repo_audit.py . --format json --fail-on __⑩__
```

### 자가체크

- [ ] \_\_⑤\_\_ 는 임계 이상을 뜻하는 비교(>=)인가요?
- [ ] \_\_⑥\_\_ =1(실패), \_\_⑦\_\_ =0(성공)으로 종료 코드를 가렸나요?
- [ ] \_\_⑧\_\_ 는 pull\_request, \_\_⑨\_\_ 는 read(최소 권한)인가요?
- [ ] \_\_⑩\_\_ 는 판정 기준 등급(error)인가요?
- [ ] CI가 종료 코드로 통과/실패를 판정하나요(c13)?
- [ ] 같은 입력에 같은 출력이 나오는지(결정론·NFR5) 두 번 실행해 비교했나요?

## 마무리

이 장에서 우리는 설계를 **6개 증분 마일스톤**으로 구현했습니다. M1에서 CLAUDE.md-플랜으로 헌법과 계획을 세우고(c05-c06), M2에서 걷는 해골과 첫 커밋을 만들고(c02-c03-c07), M3에서 점검을 외과적으로 늘리고 /audit 으로 표준화하고(c03-c08), M4에서 리포트-종료 코드와 서브에이전트 리뷰를 더하고(c09), M5에서 혹·권한·MCP 로 안전 게이트를 세우고(c04-c10-c11), M6에서 CI-스케줄-오케스트레이션으로 무인 자동화를 완성했습니다(c12-c13). 13개 개념이 *하나의 동작하는 도구*로 모였습니다.

무엇보다 각 마일스톤이 **항상 실행 가능하고 단위 테스트로 검증**되었기에, 어디서 무엇이 깨졌는지 즉시 알 수 있었습니다. 다음 장(17장)에서는 이 완성품을 공개 산출물로 **시연**하고, 평가 루브릭으로 점검하며, 13개 개념을 어떻게 적용했는지 **성찰**하고 한계·개선안을 정리합니다.



## 17장 종합 프로젝트: repo-audit — 발표·평가·성찰

이 장은 종합 프로젝트의 마지막 단계인 **발표·평가·성찰(present & reflect)**입니다. 14장에서 분석하고, 15장에서 설계하고, 16장에서 구현한 repo-audit를 이제 **공개 산출물**로 시연하고, 객관적 루브릭으로 평가하며, 1~13장에서 배운 13개 개념을 어떻게 적용했는지 돌아봅니다.

PBL에서 발표·성찰은 "끝났으니 보여 주는 의례"가 아닙니다. **무엇을 왜 그렇게 만들었는지 설명하고, 어떤 개념이 어디로 이어졌는지 메타인지로 정리하며, 한계와 개선안을 스스로 진단하는** 학습의 핵심 단계입니다. 만든 것을 말로 설명할 수 있을 때 비로소 그 개념이 *전이 가능한 자기 지식*이 됩니다.

이 장에서도 **새 개념은 배우지 않습니다**. 배운 것을 통합·회고합니다.

### 이 장의 학습 목표

- 완성한 repo-audit를 공개 산출물로 시연할 수 있습니다.
- 평가 루브릭으로 자신과 동료의 작업을 평가할 수 있습니다.
- 13개 개념을 어떻게 적용했는지 성찰할 수 있습니다.
- 한계와 개선안을 제시할 수 있습니다.

### 17.1 산출물 시연: repo-audit 데모와 CI 녹색

발표의 첫 단계는 **동작하는 산출물을 직접 보여 주는 것**입니다. 슬라이드가 아니라 **실행**이 증거입니다. 데모는 세 장면으로 구성합니다.

**장면 1 — 문제를 잡아내는 모습**. 일부러 문제가 있는 픽스처 저장소에 repo-audit를 돌려, 깨진 링크·누락 문서·비대 파일·TODO를 한 번에 잡아내는 것을 보입니다.

```
$ python repo_audit.py fixture_repo --format text --fail-on error
=====
repo-audit 점검 결과
=====
오류(error) : 3건
경고(warn)  : 1건
정보(info)  : 1건
-----
[x] [links] README.md: 깨진 로컬 링크: docs/missing.md
[x] [links] docs/guide.md: 깨진 로컬 링크: ../nonexistent_file.md
[x] [todos] src/main.py:11: # FIXME: 입력 검증이 불완전함 - 빈 문자열을 허용하고 있음
[-] [links] docs/guide.md: 잘못된 앵커: #nonexistent-anchor (대상: ../README.md)
[i] [todos] src/main.py:6: # TODO: 실제 데이터 처리 로직 구현 필요
$ echo $?
1
```

깨진 링크와 FIXME는 error, 헤딩이 없는 앵커는 warn, 일반 TODO는 info로 등급이 갈린 점을 보입니다 — 도구가 문제의 경중을 구분한다는 설계 의도(FR5-ADR4)가 한눈에 드러납니다. 같은 점검을 `--format json`으로 돌리면 기계가 읽는 리포트가 나옵니다. CI·대시보드가 이 출력을 이어받습니다(ADR2). 키와 발견이 정렬되어 같은 입력이면 *바이트까지 동일한* 출력이 나옵니다(NFR5).

```
$ python repo_audit.py fixture_repo --format json --fail-on error | head -12
{
  "findings": [
    {
      "check": "links",
      "message": "깨진 로컬 링크: docs/missing.md",
      "path": "README.md",
      "severity": "error"
    },
    ...
  ],
  "summary": { "error": 3, "info": 1, "warn": 1 }
}
```

**장면 2 — 고치기 뒤 통과하는 모습.** LICENSE를 추가하고 링크를 고친 *건강한 저장소*에서는 종료 코드 0(통과)이 나옵니다. "문제를 잡고, 고치면 녹색"이라는 점검 도구의 가치를 한눈에 보입니다.

```
$ python repo_audit.py ./healthy-repo --fail-on error
=====
repo-audit 점검 결과
=====
오류(error) : 0건
경고(warn)  : 0건
정보(info)  : 1건
-----
[i] [todos] main.py:10: # TODO 로깅 레벨 환경변수화
=====
$ echo $?
0
```

깨진 링크·FIXME가 없으니 error는 0건이고, 남은 TODO 1건(info)은 `--fail-on error` 기준 아래라 종료 코드 0(통과)입니다. 정보성 TODO는 빌드를 막지 않는다는 등급 설계(FR5-FR7)가 그대로 작동합니다.

**장면 3 — CI 녹색.** GitHub Actions에서 PR마다 자동 점검이 돌고, 종료 코드로 통과/실패가 판정되는 것을 보입니다. *사람이 손대지 않아도 같은 품질의 점검이 영원히 반복된다는* 것이 무인 자동화의 핵심 메시지입니다.

```
GitHub Actions — repo-audit / audit (pull_request)
단위 테스트 ..... 61 passed
헤드리스 점검 ..... exit code 0
리포트 아티팩트 ..... audit-report 보존됨
결과: PASS (녹색 체크)
```

**시연 팁.** 데모는 *실패* → *수정* → *성공*의 서사로 구성하면 설득력이 큼니다. 도구가 "무엇을 막아 주는지"를 보여준 뒤 "고치면 통과한다"로 답습니다. 그리고 텍스트(사람용)·JSON(기계용)·CI 로그(자동화) 세 산출을 모두 보여, 한 도구가 여러 소비자를 만족시킨다는 설계 의도(ADR2)를 드러냅니다.

## 17.2 평가 루브릭

발표를 객관적으로 평가하기 위해 **6개 기준의 가중 루브릭**을 씁니다. 각 기준은 미흡·보통·우수 세 수준을 가지며, 가중치의 합은 1.00입니다. 자기 평가와 동료 평가에 같은 루브릭을 적용해 일관성을 확보합니다.

| 기준                   | 가중치  | 미흡         | 보통                     | 우수                                       |
|----------------------|------|------------|------------------------|------------------------------------------|
| 기능 완성도(점검·리포트·종료 코드) | 0.20 | 일부 점검만 동작  | 링크·TODO 점검과 텍스트 리포트 동작 | 링크·크기·문서·TODO 점검 + 텍스트/JSON + 종료 코드까지 동작 |
| 코드 구조(모듈·함수·테스트)     | 0.15 | 한 덩어리 스크립트 | 점검·리포트 함수 분리           | checks·report 모듈 분리 + 단위 테스트 통과          |
| 안전성(혹·권한)            | 0.20 | 게이트 없음     | 최소 권한 allowlist만       | PreToolUse 혹 + allowlist로 위험 편집 차단·로깅    |
| 자동화(CI·스케줄·오케스트레이션)  | 0.20 | 수동 실행만     | 헤드리스 실행 가능             | PR·스케줄 CI 무인 실행 + 서버에이전트 파이프라인           |
| Claude Code 활용 깊이    | 0.15 | 프롬프트만 사용   | 플랜·커맨드·서브에이전트 일부 활용    | CLAUDE.md·플랜·커맨드·서브에이전트·MCP·혹을 통합 적용     |
| 발표·성찰                | 0.10 | 동작만 보여 줌   | 설계 의도 설명               | ADR 근거·한계·개선안·개념 적용 회고까지 제시              |

점수 계산은 각 기준의 수준(미흡 1·보통 2·우수 3)에 가중치를 곱해 합산합니다. 예를 들어 모든 기준이 우수면  $3 \times (0.20+0.15+0.20+0.20+0.15+0.10) = 3.00$  (만점)입니다.

**루브릭으로 자기 평가하기.** 루브릭은 점수를 매기는 도구이자, *무엇을 더 해야 우수에 이르는지 알려 주는 체크리스트*입니다. 예컨대 "자동화" 기준에서 보통(헤드리스 실행)에 머물렀다면, 우수로 가려면 *PR·스케줄 CI + 서버에이전트 파이프라인*이 필요하다는 것을 루브릭이 알려 줍니다. 평가 전에 루브릭을 먼저 읽고 목표 수준을 정하세요.

**자기 평가 예시(채점된 루브릭).** 아래는 한 학습자가 자신의 repo-audit를 루브릭으로 채점한 예입니다. 수준 점수(미흡 1·보통 2·우수 3)에 가중치를 곱해 합산합니다.

| 기준     | 가중치  | 자평 수준 | 점수 | 가중 점수 | 근거                               |
|--------|------|-------|----|-------|----------------------------------|
| 기능 완성도 | 0.20 | 우수(3) | 3  | 0.60  | 4개 점검 + 텍스트/JSON + 종료 코드 모두 동작   |
| 코드 구조  | 0.15 | 우수(3) | 3  | 0.45  | checks-report 모듈 분리, 5개 테스트 통과   |
| 안전성    | 0.20 | 보통(2) | 2  | 0.40  | allowlist는 있으나 혹 로깅이 미흡          |
| 자동화    | 0.20 | 우수(3) | 3  | 0.60  | PR·스케줄 CI + /feature 파이프라인       |
| 활용 깊이  | 0.15 | 우수(3) | 3  | 0.45  | CLAUDE.md·플랜·커맨드·서브에이전트·MCP·혹 통합 |
| 발표·성찰  | 0.10 | 보통(2) | 2  | 0.20  | 설계 의도는 설명했으나 한계 회고가 얇음           |
| 합계     | 1.00 |       |    | 2.70  | 만점 3.00 대비 우수 영역                 |

이 채점에서 *안전성·발표·성찰*이 보통에 머물렀음이 드러납니다. 루브릭은 즉시 다음 행동을 가리킵니다 — 안전성은 *혹 로깅 보강*, 발표·성찰은 *한계·개선안 구체화*가 우수로 가는 길입니다. 이렇게 루브릭은 점수에 그치지 않고 *개선 로드맵*이 됩니다.

## 17.3 동료 피드백 (갤러리 워크)

**갤러리 워크(gallery walk)** 는 모두의 산출물을 전시하고, 학습자가 서로의 작업을 돌아보며 구조화된 피드백을 남기는 방법입니다. 각자의 repo-audit 저장소(README·리포트 샘플·CI 배지)를 펼쳐 두고, 정해진 형식으로 피드백을 주고받습니다.

피드백은 막연한 칭찬·비난이 아니라 **"2 좋은 점 + 1 개선 질문"** 형식으로 남깁니다. 아래는 동료 피드백 카드의 예시입니다.

| 항목     | 내용(예시)                                                               |
|--------|----------------------------------------------------------------------|
| 좋은 점 1 | <code>decide_exit_code</code> 의 심각도 임계 판정이 명확해 CI 판정 기준을 바로 이해했습니다   |
| 좋은 점 2 | <code>guard.py</code> 가 fail-open으로 설계돼 혹 버그가 작업을 막지 않는 점이 안전합니다     |
| 개선 질문  | <code>check_links</code> 가 심볼릭 링크나 대소문자 차이(macOS/Windows)를 어떻게 다루나요? |

또 다른 피드백 카드 예시입니다. 같은 형식이라 작성자가 받은 피드백을 *비교·종합*하기 쉽습니다.

| 항목     | 내용(예시)                                             |
|--------|----------------------------------------------------|
| 좋은 점 1 | /audit 커맨드 덕에 점검 절차를 외우지 않고 한 줄로 재현할 수 있었습니다       |
| 좋은 점 2 | JSON 리포트의 키 정렬로 두 번 실행해도 diff가 0인 점이 결정론을 잘 보여 줍니다 |
| 개선 질문  | TODO 점검이 .py 외에 .md:.txt의 TODO도 잡나요, 아니면 소스만 보나요?  |

갤러리 워크의 진행은 세 라운드로 나눕니다. ① *둘러보기 라운드* — 말없이 모든 산출물을 5분간 읽고 카드를 채운다. ② *질문 라운드* — 작성자가 받은 개선 질문에 짧게 답한다(방어가 아니라 설명). ③ *반영 라운드* — 가장 가치 있는 질문 한 개를 골라 17.5의 개선안에 옮긴다.

갤러리 워크의 규칙은 세 가지입니다. ① 산출물의 *구체적* 부분을 짚는다(파일·함수·줄), ② 개선은 명령이 아니라 *질문*으로 던진다, ③ 받은 피드백은 다음 절(성찰)에서 한 줄로 반영한다. 이 규칙은 피드백을 공격이 아니라 *개선의 단서*로 만드는 장치입니다 — 같은 도구를 만든 동료의 질문은 종종 작성자 자신이 놓친 경계 조건(심볼릭 링크·대소문자·바이너리 파일)을 정확히 찌릅니다.

**좋은 동료 질문의 조건.** "더 잘 만들 수 있었을 텐데"는 도움이 안 됩니다. "TODO 점검이 주석 안의 'TODO'만 잡나요, 아니면 문자열 리터럴의 'TODO'도 잡나요?"처럼 *구체적이고 검증 가능한* 질문이 작성자에게 진짜 개선 지점을 줍니다. 이런 질문은 17.5의 한계·개선안으로 자연스럽게 이어집니다.

## 17.4 성찰 회고: 13개 개념의 적용

성찰의 핵심은 **메타인지** — "내가 무엇을 배웠고, 그것을 어디에 어떻게 썼는가"를 명시적으로 적는 것입니다. 13개 개념이 repo-audit의 어느 산출물·단계로 이어졌는지 한 줄씩 추적합니다. 아래 표가 그 회고의 모범입니다.

| 개념                 | 단원 | repo-audit에서 어디에 적용됐나                             |
|--------------------|----|---------------------------------------------------|
| c01 설치:첫 세션        | u1 | 첫 세션에서 빈 저장소를 열어 프로젝트를 시작                         |
| c02 효과적 프롬프팅       | u1 | "목표·맥락·제약·완료 조건"을 갖춘 지시로 collect_todos 구현(M2)     |
| c03 파일 탐색·편집       | u2 | checks.py에 점검 함수를 <i>외과적으로</i> 덧붙임(M3)            |
| c04 권한·allowlist   | u2 | 점검·테스트·커밋만 allow, rm-curl을 deny(M5 settings.json) |
| c05 CLAUDE.md·컨텍스트 | u3 | 명령·컨벤션·금지(표준 라이브러리·결정론)를 헌법으로 영속화(M1)             |
| c06 플랜 모드          | u3 | 구현 전 PLAN.md로 단계 확정, dataclass·외부 링크 가정 교정(M1)    |
| c07 Git·커밋         | u4 | 변경을 작은 논리 단위로 feat: 커밋(M2)                        |
| c08 슬래시 커맨드        | u4 | 반복 점검을 /audit 커맨드로 표준화(M3)                        |
| c09 서브에이전트         | u5 | reviewer에 코드 리뷰를 위임(읽기 전용 권한)(M4)                 |
| c10 MCP 연동         | u5 | 파일시스템 MCP로 외부 점검 정책 문서 참조(M5)                     |
| c11 혹              | u6 | PreToolUse guard.py로 보호 경로 편집 차단·로깅(M5)           |
| c12 오케스트레이션        | u6 | /feature로 조사→구현→리뷰 파이프라인 분업(M6)                   |
| c13 CI·자동화         | u6 | claude.yml로 PR·주간 스케줄 무인 실행, 종료 코드 판정(M6)         |

이 표를 채워 보면 *흘어져 배운 13개 기능이 하나의 도구를 만드는 과정에 빠짐없이 동원됐음*을 스스로 확인하게 됩니다. POV의 통찰("기능을 따로 배우는 것과 한 산출물로 엮어 쓰는 것은 다르다")이 바로 이 회고에서 체득됩니다.

이어서 *과정에서 배운 교훈*을 다섯 질문으로 회고합니다. 이 다섯 질문은 종합 프로젝트의 성찰 프롬프트이며, 17.5의 작성형 연습에서 여러분이 직접 답하게 됩니다.

- **개념 → 산출물 추적.** 13개 개념이 각각 어느 마일스톤·산출물로 이어졌는가? 위 표가 그 답이며, *모든 개념이 빠짐없이 동원됐음*을 확인하는 것이 핵심입니다.
- **계획과 실제의 차이.** 플랜 모드로 받은 초기 계획에서 finding을 dataclass로 두려 했지만, JSON 직렬화 단순성을 위해 dict로 바꿨습니다(ADR1). 계획·실행 분리가 *바뀌야 할 가정*을 미리 드러냈습니다.
- **혹 강화의 부작용.** PreToolUse 혹을 더 강하게 하면(예: 모든 편집 차단) 정상 작업까지 막힙니다. 그래서 guard.py를 *명시적 위험 경로에만 차단*하고 fail-open으로 설계했습니다. 더 강화한다면 *차단 사유를 로그로 관측*해 데이터로 튜닝해야 과잉 차단을 피합니다(11장).
- **무인 CI의 안전 위험.** 사람이 막아 줄 안전망이 없는 무인 환경의 가장 큰 위험은 *과도한 권한*입니다. CI에서 권한을 contents: read로 좁히고 비밀을 환경 변수로 주입하며, 모든 실행 결과를 아티팩트로 로깅해 위험을 줄였습니다(13장).
- **다시 만든다면.** 점검 항목으로 *비밀 키 노출 탐지*를 더하고, 리포트에 *심각도별 정렬*과 *이전 실행 대비 추세*를 넣고 싶습니다. 에이전트 워크플로우는 reviewer 외에 *security 서브에이전트*를 병렬로 추가하면 점검 폭이 넓어집니다.

이 다섯 회고가 PBL 성찰의 골격입니다. *무엇을 적용했고(개념 추적), 계획과 무엇이 달랐고(메타인지), 어디가 위험하고(안전), 어떻게 개선할지(전망)* 를 스스로 진단할 수 있을 때 비로소 도구를 만든 경험이 전이 가능한 지식이 됩니다.

## 17.5 작성형 연습 (2종 1세트): 성찰 템플릿·자가체크

이제 여러분 자신의 repo-audit에 대해 직접 회고를 작성할 차례입니다. 아래 두 연습은 *모범* → *빈 템플릿* → *자가체크* 의 2종 1세트입니다.

### 연습 1 — 13개 개념 적용 성찰 템플릿

**모범 참고.** 17.4의 표는 각 개념이 어느 마일스톤 산출물로 이어졌는지 한 줄로 적었습니다. 같은 형식으로 여러분의 작업을 채웁니다.

**여러분의 차례.** 자신의 repo-audit에서 각 개념을 실제로 어디에 적용했는지 구체적으로 적으세요(파일·함수·마일스톤을 짚을 것).

[개념 적용 회고 — 내 repo-audit]

c01 설치·첫 세션 : \_\_\_\_\_  
 c02 프롬프팅 : \_\_\_\_\_  
 c03 탐색·편집 : \_\_\_\_\_  
 c04 권한 : \_\_\_\_\_  
 c05 CLAUDE.md : \_\_\_\_\_  
 c06 플랜 모드 : \_\_\_\_\_  
 c07 Git·커밋 : \_\_\_\_\_  
 c08 슬래시 커맨드 : \_\_\_\_\_  
 c09 서브에이전트 : \_\_\_\_\_  
 c10 MCP : \_\_\_\_\_  
 c11 훅 : \_\_\_\_\_  
 c12 오케스트레이션 : \_\_\_\_\_  
 c13 CI·자동화 : \_\_\_\_\_

[가장 어렵게 통합된 개념 1개와 그 이유]

\_\_\_\_\_

### 자가체크

- [ ] 13개 개념을 *하나도 빠뜨리지 않고* 채웠나요?
- [ ] 각 줄이 "사용했다"가 아니라 *어느 파일·함수·마일스톤인지* 구체적인가요?
- [ ] 적용 못 한 개념이 있다면 *왜인지* 솔직히 적었나요?
- [ ] 가장 어렵게 통합된 개념과 이유를 식별했나요?
- [ ] 계획(PLAN.md)과 실제 구현이 달라진 지점을 하나 이상 짚었나요?

### 연습 2 — 한계·개선안과 위험 회고 템플릿

**모범 참고.** 17.4의 세 교훈(계획과 실제·안전과 생산성·무인 위험)은 *한계를 인정하고 개선 방향을 제시*했습니다. 같은 태도로 여러분 도구의 한계를 진단합니다.

여러분의 차례. 아래 빈칸을 채워 한계·개선안·위험 회고를 완성하세요.

[한계와 개선안 — 내 repo-audit]

1. 현재 한계 (무엇을 못 잡거나 약한가)
  - 예) 외부 URL 링크는 검사하지 않는다(ADR3) → \_\_\_\_\_
  - \_\_\_\_\_
2. 다시 만든다면 (점검 항목·리포트 형식·워크플로우)
  - 점검: \_\_\_\_\_
  - 리포트: \_\_\_\_\_
  - 에이전트 워크플로우: \_\_\_\_\_
3. 무인 CI의 가장 큰 안전 위험과 완화책
  - 위험: \_\_\_\_\_
  - 권한·로깅으로 줄이는 법: \_\_\_\_\_
4. PreToolUse 혹은 더 강하게 할 때의 부작용과 튜닝
  - 부작용: \_\_\_\_\_
  - 튜닝(관측·로깅): \_\_\_\_\_

### 자가체크

- [ ] 한계를 구체적으로(못 잡는 입력·약한 점검) 짚었나요?
- [ ] 개선안이 "더 잘하겠다"가 아니라 **실행 가능한 변경인**가요?
- [ ] 무인 CI의 안전 위험을 식별하고 **권한·로깅** 완화책을 적었나요?
- [ ] 혹은 강화할 때 정상 작업을 막을 위험과 **튜닝 방법**을 적었나요(c11)?
- [ ] 동료 피드백(17.3)에서 받은 개선 질문을 한 줄 이상 반영했나요?
- [ ] 결정론(같은 입력=같은 출력, NFR5)이 깨질 수 있는 지점을 점검했나요?

## 17.6 자주 묻는 질문(FAQ)

**Q. 모든 점검을 다 구현하지 못했는데 발표해도 되나요?** A. 됩니다. 루브릭의 **보통 수준**(링크·TODO 점검 + 텍스트 리포트)만 동작해도 발표 대상입니다. 중요한 것은 **동작하는 부분을 정직하게 시연하고, 못 한 부분을 한계로 명시**하는 것입니다. 발표·성찰 기준은 "완벽한 도구"가 아니라 "설계 의도와 한계를 설명할 수 있는가"를 봅니다.

**Q. 자기 평가 점수가 동료 평가와 다르면 어떻게 하나요?** A. 차이 자체가 학습 자료입니다. 같은 루브릭을 썼는데 점수가 같았다면, **어느 기준에서 근거가 달랐는지** 대화로 맞춰 봅니다. 예컨대 "안전성 우수"를 자평했는데 동료가 "혹 로깅이 안 보인다"고 했다면, 그 지적이 곧 개선 지점입니다.

**Q. 성찰에 13개 개념을 다 적기 어렵습니다. 일부만 써도 되나요?** A. 적용하지 못한 개념이 있다면 **왜 못 했는지**를 적는 것도 훌륭한 성찰입니다. 예: "c10 MCP는 외부 정책 문서가 없어 적용하지 않았다." 빈칸을 비우기보다 **적용/미적용과 이유**를 채우는 것이 메타인지를 키웁니다.

**Q. CI가 빨강(실패)인데 발표해도 되나요?** A. 일부러 문제 있는 저장소에서 빨강을 보이는 것은 **도구가 제대로 작동한다는 증거**입니다. 다만 **건강한 저장소에서는** 녹색이 나오는 것도 함께 보여, 도구가 거짓 양성(false positive)을 남발하지 않음을 입증하세요.

**Q. 다시 만들 시간이 없는데 개선안만 적어도 되나요?** A. 됩니다. 개선안은 *지금 당장 구현*이 아니라 *실행 가능한 변경 제안*이면 충분합니다. "심볼릭 링크를 따라가도록 check\_links를 보강한다"처럼 구체적이면, 그 자체로 도구를 깊이 이해했다는 증거가 됩니다.

---

## 마무리

---

이 장에서 우리는 repo-audit를 **공개 산출물로 시연**하고(실패→수정→성공의 서사와 CI 녹색), **6기준 가중 루브릭**으로 평가하며, **갤러리 워크**로 동료 피드백을 주고받았습니다. 무엇보다 **13개 개념이 어느 마일스톤·산출물로 이어졌는지** 한 줄씩 추적해, 흩어져 배운 기능들이 하나의 신뢰할 수 있는 도구로 모였음을 메타인지로 확인했습니다. 한계(외부 URL 미검사·결정론 제약)와 개선안·무인 위험까지 정직하게 진단한 것이 이 단계의 성과입니다.

이로써 종합 프로젝트 **repo-audit**의 PBL 수명주기(분석 → 설계 → 구현 → 발표·평가·성찰)가 완성되었습니다. 드 라이빙 퀘스천 — "실무 개발자가 Claude Code를 페어 프로그래머이자 자동화 엔진으로 삼아, 신뢰할 수 있는 저장소 건강검진 CLI를 하나의 에이전트 워크플로우로 어떻게 만들어 낼 수 있을까?" — 에 대해, 여러분은 이제 **직접 만든 동작하는 도구**로 답할 수 있습니다. 기능을 따로 배우는 것과 한 산출물로 엮어 쓰는 것은 다릅니다. 그 차이를 손으로 체득한 것이 이 책의 가장 큰 수확입니다.